

설계서

화면 설계서 20

API 설계서 26

데이터 설계서 16

배치 설계서 1

버전 v1.0
작성일 2026-09-21
작성자

대외비

목차

화면 설계서 20건

- UIS-CMN-001 디자인 토큰
- UIS-CMN-002 공통 컴포넌트
- UIS-CMN-003 앱 셀
- UIS-MBR-002 로그인
- UIS-MBR-003 비밀번호 재설정
- UIS-MBR-004 shop_mypage_addresses
- UIS-MBR-005 member_signup
- UIS-MBR-006 member_상세
- UIS-ORD-001 주문 목록
- UIS-ORD-002 주문 상세
- UIS-ORD-003 상품 목록
- UIS-ORD-004 상품 상세
- UIS-ORD-005 장바구니
- UIS-ORD-006 home
- UIS-ORD-007 orders_상세
- UIS-ORD-008 shop
- UIS-ORD-009 shop_products
- UIS-ORD-010 shop_products_상세
- UIS-ORD-011 shop_order
- UIS-ORD-012 shop_cart

API 설계서 26건

- INF-MBR-001 회원가입 인증코드 발송
- INF-MBR-002 가입 요청(인증코드 검증 포함)
- INF-MBR-003 로그인
- INF-MBR-004 로그아웃
- INF-MBR-005 자동 로그인 토큰(리프레시)
- INF-MBR-006 비밀번호 재설정 코드 요청
- INF-MBR-007 비밀번호 재설정 확정
- INF-MBR-008 배송지 CRUD
- INF-MBR-010 api members grades
- INF-ORD-001 회원 목록 조회
- INF-ORD-002 회원 상세 조회
- INF-ORD-003 주문 목록 조회
- INF-ORD-004 주문 상세 조회
- INF-ORD-005 주문 생성
- INF-ORD-006 주문 취소
- INF-ORD-007 주문 배송 목록 조회
- INF-ORD-008 판매중 상품 목록 조회
- INF-ORD-009 상품 단건 조회
- INF-ORD-010 장바구니 담기
- INF-ORD-011 장바구니 조회
- INF-ORD-012 장바구니 수량 변경
- INF-ORD-013 장바구니 품목 삭제
- INF-ORD-014 장바구니 체크아웃
- INF-ORD-015 주문 목록 CSV 내보내기
- INF-ORD-016 api zipcodes
- INF-ORD-017 shop spa static serving

데이터 설계서 16건

SCH-MBR-001 ID_SEQUENCES
SCH-MBR-002 MEMBER_ADDRESSES
SCH-MBR-003 MEMBER_API_KEYS
SCH-MBR-004 MEMBER_LOGIN_ATTEMPTS
SCH-MBR-005 MEMBER_PASSWORD_RESETS
SCH-MBR-006 MEMBER_REFRESH_TOKENS
SCH-MBR-007 MEMBER_SIGNUP_RATE_LIMITS
SCH-MBR-008 MEMBER_SIGNUP_VERIFICATIONS
SCH-MBR-009 MEMBER_PASSWORD_RESET_RATE_LIMITS
SCH-ORD-001 members
SCH-ORD-002 order_delivery
SCH-ORD-003 order_items
SCH-ORD-004 orders
SCH-ORD-005 products
SCH-ORD-006 cart_items
SCH-ORD-007 ZIPCODES

배치 설계서 1건

BAT-MBR-001 비밀번호 재설정 만료 코드 및 상한 카운터 정리

개정 이력

버전	일자	내용	작성자
v1.0	2026-09-21	최초 작성	

화면 설계서

UIS-CMN-001 디자인 토큰

이 화면 설계서에는 화면 그림이 없습니다 — 구성·위젯·연결 API는 소스에서 읽어 채웠습니다.

UIS-CMN-001: 디자인 토큰

> SR-309.1로 신규 도입된 shop-web 전역 디자인 토큰(CSS 커스텀 프로퍼티) + Pretendard 웹폰트 + 스토리북 'Design Tokens' 문서 페이지.

> 화면 라우트가 아니라 전 화면이 참조할 공유 계약이다 — 후속 SR-310(공통 컴포넌트)·SR-311(앱 셀)·SR-322/326(리스크인)이 이 토큰을 참조한다.

1. 개요

정본 파일: ``modules/shop-web/src/styles/tokens.css`` — ``root``에 선언된 CSS 커스텀 프로퍼티 전체가 값의 유일한 정본이다. 다른 파일(카탈로그·스토리)은 이 값을 다시 적지 않고 ``getComputedStyle``로 읽는다(이중관리 금지).

로드 경로: ``src/main.tsx``(앱 진입점)와 ``storybook/preview.ts``(스토리북 프리뷰) 각각 1줄 import — 외부 CDN 없이 Vite가 해시된 정적 자산으로 번들한다.

문서 페이지: ``DesignTokens.stories.tsx``(Storybook, title 'Design Tokens') — 색 견본·배경×본문 명도 대비 표·타이포/간격/radius/그림자 견본을 ``getComputedStyle``로 읽은 실제 적용값으로 렌더한다.

이 SR은 토큰을 **추가만** 한다 — 기존 화면·컴포넌트가 이 토큰을 참조하도록 바꾸는 작업은 범위 밖(SR-310/311/322/326).

2. 색 토큰

CSS 변수	값	용도
<code>`--color-text`</code>	<code>`#101010`</code>	기본 본문 글자색
<code>`--color-text-secondary`</code>	<code>`#767676`</code>	설명·캡션 등 부차 정보
<code>`--color-text-tertiary`</code>	<code>`#aaaaaa`</code>	비활성·플레이스홀더
<code>`--color-surface-1`</code>	<code>`#f2f2f2`</code>	열은 배경면(카드·구분 영역)
<code>`--color-surface-2`</code>	<code>`#f5f5f5`</code>	더 열린 배경면
<code>`--color-surface-3`</code>	<code>`#f9f9f9`</code>	가장 열린 배경면(페이지 바탕 등)
<code>`--color-primary`</code>	<code>`#713fc5`</code>	주요 강조·CTA 텍스트/아이콘
<code>`--color-primary-bg`</code>	<code>`#f2ebff`</code>	포인트 보라의 열린 배경
<code>`--color-link`</code>	<code>`#177bc3`</code>	텍스트 링크
<code>`--color-price`</code>	<code>`#ff5259`</code>	할인율·판매가 강조(배경 제약 — 아래 3절)
<code>`--color-brand`</code>	<code>`#ed1c24`</code>	브랜드 로고·강조 요소(배경 제약 — 아래 3절)
<code>`--color-dim`</code>	<code>`rgba(16, 16, 16, 0.3)`</code>	모달·오버레이 배경 님
<code>`--color-success`</code> / <code>`-bg`</code>	<code>`#136b2f`</code> / <code>`#e9f7ec`</code>	성공 상태(기존 <code>`DeliveryBadge.tsx`</code> 배지색과 정합)
<code>`--color-warning`</code> / <code>`-bg`</code>	<code>`#8a5a00`</code> / <code>`#fff4e5`</code>	경고 상태(기존 <code>`DeliveryBadge.tsx`</code> 배지색과 정합)

`--color-error` / `-bg`	`#b42318` / `#fdecea`	오류 상태(성공·경고와 같은 톤 규칙으로 파생)
`--color-info` / `-bg`	`#0b4ea2` / `#e8f1ff`	정보 상태(성공·경고와 같은 톤 규칙으로 파생)

그라데이션 2종: `--gradient-primary`(파랑→보라, 구매 버튼·헤더 토글) · `--gradient-accent`(보라→파랑, 최근 본 상품 헤더).

3. 명도 대비(WCAG) — 사용 제약

`DesignTokens.stories.tsx`의 대비표가 배경 5종(`#ffffff` · `--color-surface-3` `#f9f9f9` · `--color-surface-2` `#f5f5f5` · `--color-surface-1` `#f2f2f2` · `--color-primary-bg` `#f2ebff`) × 텍스트 톤 전건을 `contrast.ts`(WCAG 상대휘도 공식)로 계산해 렌더한다. QA 4라운드 독립 재계산으로 아래 값이 실측 확정됐다(배경 순서는 위와 동일):

텍스트 톤	#ffffff	surface-3	surface-2	surface-1	primary-bg	판정
`--color-text`	19.03	18.07	17.45	17.00	16.40	본문 AA(4.5:1) 전배경 통과
`--color-text-secondary`	4.54	4.31	4.17	4.06	3.92	`#ffffff`만 AA 통과, 나머지는 큰 글씨(3:1) 전용
`--color-text-tertiary`	2.32	2.21	2.13	2.08	2.00	전배경 AA·큰 글씨 기준 모두 미달 — 본문 사용 금지
`--color-primary`	6.52	6.20	5.98	5.83	5.62	본문 AA 전배경 통과
`--color-link`	4.51	4.28	4.14	4.03	3.89	`#ffffff`만 AA 통과, 나머지는 큰 글씨 전용
`--color-price`	**3.18**	**3.02**	2.92	2.84	2.74	`#ffffff`·`surface-3` 위 **큰 글씨(18.66px 이상 굵게, 또는 24px 이상) 전용** — 그 외 배경은 큰 글씨도 3:1 미달, 본문 사용 금지
`--color-brand`	**4.38**	**4.16**	**4.02**	**3.91**	**3.78**	5배경 전부 큰 글씨(3:1) 통과, AA 본문(4.5:1)은 전배경 미달 — 본문 사용 금지, 큰 글씨 전용
상태색 4종(성공/경고/오류/정보)	6.62~8.00	~5.71~6.89			5.11 최저	5배경 전부 AA 본문 기준 통과

사용 규칙(값의 정본 `tokens.css:44-55` 주석과 스토리 `describeUsage()` 산출 문장이 일치, STEP 5.3 축 B에서 기계 대조 확인됨):

`--color-price`는 흰 배경(`#ffffff`)·`--color-surface-3` 위의 굵은 큰 글씨에만 쓴다. `--color-surface-1`/`-2`·`--color-primary-bg` 위에서는 큰 글씨도 3:1 미달이라 쓰지 않는다 — 그 위의 가격은 `--color-text`를 쓰고, 강조는 별도 배지로 대신한다. `--color-brand`는 5배경 전부에서 큰 글씨로만 쓴다(본문 텍스트에는 쓰지 않는다).

4.5:1 미달 조합은 표에서 "본문 사용 금지"로, 대비 계산 자체가 실패한 조합(비-hex 값 등)은 "계산 불가"로 별도 표기한다(두 사유를 혼동하지 않는다).

4. 타이포그래피

폰트: Pretendard(가변 아님, weight별 개별 `@font-face` 3개) — `Regular`(400)·`SemiBold`(600)·`Bold`(700). 실패 시 시스템 한글 폰트(`-apple-system, BlinkMacSystemFont, 'Malgun Gothic', 'Apple SD Gothic Neo', sans-serif)로 대체(`font-display: swap`).

폰트 파일: `src/assets/fonts/Pretendard-{Regular,SemiBold,Bold}.woff2`(npm 공식 배포 패키지 `pretendard`(OFL-1.1) 원본, 서브셋 미적용 — 경량화는 후속 SR 추적, `## 후속 추적(TODO)` 참조) + `LICENSE-Pretendard.txt`.

크기 스케일 8단(`--text-2xs` 11px ~ `--text-2xl` 24px, 본문 `--text-base` 14px 기준) + 배너 카피용 `--text-display`(40px).

굵기 3종: `--font-weight-regular`(400)·`--font-weight-semibold`(600)·`--font-weight-bold`(700).

****적용 범위 제약****: 이 SR이 바꾼 전역 스타일은 ``body { font-family }`` 선택자 1개뿐이다. 실측(QA 4라운드) 결과 프로덕션 페이지 10개(부품 포함 15개 파일)가 루트 요소에 인라인 ``fontFamily: 'system-ui, sans-serif'``를 갖고 있어 ``body`` 규칙을 가린다 — ****이 토큰 폰트가 프로덕션 화면에 실제로 적용되는 곳은 현재 0곳****이다(스토리북 프리뷰에서는 ``preview.ts`` `import`로 정상 적용·``loaded`` 확인됨). 인라인 제거는 SR-310/SR-311의 선행 조건으로 이월한다(``##`` 후속 추적(TODO) 참조).

5. 간격·radius·그림자·z-index·레이아웃

간격: ``--space-1``(4px) ~ ``--space-8``(40px), 4px 배수 8단.

radius 3종: ``--radius-sm``(4px)·``--radius-md``(8px, 카드 썸네일용)·``--radius-lg``(16px).

그림자 3종: ``--shadow-sm``·``--md``·``--lg``(모두 ``rgba(16,16,16,*)`` 톤).

z-index 층 6단: ``--z-base``(0) < ``--z-dropdown``(100) < ``--z-sticky``(200) < ``--z-overlay``(300) < ``--z-modal``(400) < ``--z-toast``(500).

레이아웃: ``--layout-content-width``(750px, 본문 단일 컬럼) — 값만 정의, 실제 레이아웃 적용은 후속 SR(범위 밖).

6. 참조 무결성(정적 자산 경로)

Vite 자산 파이프라인이 ``src/assets/fonts/*.woff2``를 상대경로 ``url()`` 참조로 해시+베이스를 자동 처리한다(SR-301 #1 함정 회피 — ``public/``에 두지 않음):

``npm run build`` 산출물: ``/shop/assets/Pretendard-*-<hash>.woff2`` (base ``/shop/``)

``npm run build-storybook`` 산출물: ``./Pretendard-*-<hash>.woff2`` (base ``/``)

두 base가 다르므로 후속 SR이 폰트를 서브셋으로 교체할 때 두 경로 모두 재확인해야 한다.

7. 스토리북 문서 페이지

``design-tokens--tokens``(``DesignTokens.stories.tsx``) 1개 — 색 견본(토큰명·값·용도), 배경×본문 명도 대비 표(55행 = 배경 5 × 텍스트 토큰 11), 타이포 스케일 견본, 간격/radius/그림자 견본을 포함한다. 대비 계산은 행 단위로 격리되어(오류 주입 실측 확인) 한 조합의 계산 실패가 페이지 전체를 깨뜨리지 않는다.

8. 범위 밖(후속 SR)

기존 화면·컴포넌트가 이 토큰을 실제로 참조하도록 바꾸는 작업(SR-310 공통 컴포넌트·SR-311 앱 셸·SR-322/326 리스킨).

다크 모드, ``--layout-content-width``의 실제 레이아웃 적용.

Pretendard woff2 서브셋 경량화(SR-291 성능, 현재 원본 3개 합계 약 2.34MB).

변경 이력

<!-- spec-history: machine-managed -->

일자	SR	항목	변경 요약	커밋
2026-09-19	SR-309	#1	신규 생성 — shop-web 전역 디자인 토큰(색·그라데이션·타이포·간격·radius·그림자·z-index·레이아웃 폭)과 Pretendard 웹폰트 계약, 명도 대비 사용 규칙을 코드 기준으로 역생성	shop-web@a4a53fc

화면 상태 1건 — 기준 스냅샷

색

	<code>--color-text</code> — 본문 텍스트 #101010 · 기본 본문 글자색
	<code>--color-text-secondary</code> — 보조 텍스트 #767676 · 설명·캡션 등 부차 정보
	<code>--color-text-tertiary</code> — 3차 텍스트 #aaaaaa · 비활성·플레이스홀더
	<code>--color-surface-1</code> — 면 1 #f2f2f2 · 옅은 배경면(카드·구분 영역)
	<code>--color-surface-2</code> — 면 2 #f5f5f5 · 더 옅은 배경면
	<code>--color-surface-3</code> — 면 3 #f9f9f9 · 가장 옅은 배경면(페이지 바탕 등)
	<code>--color-primary</code> — 포인트 보라 #713fc5 · 주요 강조·CTA 텍스트/아이콘
	<code>--color-primary-bg</code> — 포인트 배경 #f2ebff · 포인트 보라의 옅은 배경
	<code>--color-link</code> — 링크 파랑 #177bc3 · 텍스트 링크
	<code>--color-price</code> — 가격/할인 빨강 #ff5259 · 할인율·판매가 강조
	<code>--color-brand</code> — 브랜드 레드 #ed1c24 · 브랜드 로고·강조 요소
	<code>--color-dim</code> — 딴 처리 rgba(16, 16, 16, .3) · 모달·오버레이 배경 딴(rgba)
	<code>--color-success</code> — 성공 #136b2f · 성공 상태 텍스트(DeliveryBadge 정합)
	<code>--color-success-bg</code> — 성공 배경 #e9f7ec · 성공 배지 배경
	<code>--color-warning</code> — 경고 #8a5a00 · 경고 상태 텍스트(DeliveryBadge 정합)
	<code>--color-warning-bg</code> — 경고 배경 #fff4e5 · 경고 배지 배경
	<code>--color-error</code> — 오류 #b42318 · 오류 상태 텍스트
	<code>--color-error-bg</code> — 오류 배경 #fdecea · 오류 배지 배경
	<code>--color-info</code> — 정보 #0b4ea2 · 정보 상태 텍스트
	<code>--color-info-bg</code> — 정보 배경 #e8f1ff · 정보 배지 배경

그라데이션

	<code>--gradient-primary</code> — 그라데이션 (파랑→보라) linear-gradient(90deg, #177bc3 0%, #713fc5 100%) · 구매 버튼·헤더 토크
	<code>--gradient-accent</code> — 그라데이션 (보라→파랑) linear-gradient(90deg, #713fc5 0%, #177bc3 100%) · 최근 본 상품 헤더

배경×본문 명도 대비 표

판정은 4단계다 — 통과(일반 본문 AA 4.5:1 이상), 큰 글씨 전용(일반 본문 기준은 못 미치지만 WCAG 큰 글씨 기준인 3:1 이상 — 18.66px 이상 굵게, 또는 24px 이상 일반 굵기 로만 사용), 본문 사용 금지(3:1도 못 미침), 계산 불가(토큰 값이 유효한 색상이 아니어서 그 조합만 계산을 건너뛴 — 나머지 표는 그대로 렌더됨). 아래 문장은 이 표와 같은 계산(contrast.ts)으로 매 렌더마다 도출한다 — 표와 문장이 서로 다른 값을 말할 수 없다.

`--color-price`(가격/할인 빨강)는 흰 배경(#####)---`color-surface-3`(#f9f9f9) 위의 굵은 큰 글씨(18.66px 이상 굵게, 또는 24px 이상)에만 쓴다. `--color-surface-2`(#f5f5f5)---`color-surface-1`(#f2f2f2)---`color-primary-bg`(#f2ebff) 위에서는 큰 글씨도 3:1 미달이므로 쓰지 않는다(그 위의 텍스트는 본문색 #101010을 쓰고, 강조는 별도 배지로 대신한다).

`--color-brand`(브랜드 레드)는 흰 배경(#####)---`color-surface-3`(#f9f9f9)---`color-surface-2`(#f5f5f5)---`color-surface-1`(#f2f2f2)---`color-primary-bg`(#f2ebff) 위의 굵은 큰 글씨(18.66px 이상 굵게, 또는 24px 이상)에만 쓴다.

배경	본문색	대비율	판정(AA 4.5:1 / 큰 글씨 3:1)
흰 배경(#####)	<code>--color-text</code> (#101010)	19.03:1	통과
흰 배경(#####)	<code>--color-text-secondary</code> (#767676)	4.54:1	통과
흰 배경(#####)	<code>--color-text-tertiary</code> (#aaaaaa)	2.32:1	본문 사용 금지
흰 배경(#####)	<code>--color-link</code> (#177bc3)	4.51:1	통과
흰 배경(#####)	<code>--color-primary</code> (#713fc5)	6.52:1	통과
흰 배경(#####)	<code>--color-price</code> (#ff5259)	3.18:1	큰 글씨 전용
흰 배경(#####)	<code>--color-brand</code> (#ed1c24)	4.38:1	큰 글씨 전용
흰 배경(#####)	<code>--color-success</code> (#136b2f)	6.62:1	통과
회 배경(#####)	<code>--color-warning</code> (#8a5a00)	5.92:1	통과

흰 배경(#ffffff)	--color-error (#b42318)	6.57:1	통과
흰 배경(#ffffff)	--color-info (#0b4ea2)	8.00:1	통과
--color-surface-3(#f9f9f9)	--color-text (#101010)	18.07:1	통과
--color-surface-3(#f9f9f9)	--color-text-secondary (#767676)	4.31:1	큰 글씨 전용
--color-surface-3(#f9f9f9)	--color-text-tertiary (#aaaaaa)	2.21:1	본문 사용 금지
--color-surface-3(#f9f9f9)	--color-link (#177bc3)	4.28:1	큰 글씨 전용
--color-surface-3(#f9f9f9)	--color-primary (#713fc5)	6.20:1	통과
--color-surface-3(#f9f9f9)	--color-price (#ff5259)	3.02:1	큰 글씨 전용
--color-surface-3(#f9f9f9)	--color-brand (#ed1c24)	4.16:1	큰 글씨 전용
--color-surface-3(#f9f9f9)	--color-success (#136b2f)	6.29:1	통과
--color-surface-3(#f9f9f9)	--color-warning (#8a5a00)	5.63:1	통과
--color-surface-3(#f9f9f9)	--color-error (#b42318)	6.24:1	통과
--color-surface-3(#f9f9f9)	--color-info (#0b4ea2)	7.60:1	통과
--color-surface-2(#f5f5f5)	--color-text (#101010)	17.45:1	통과
--color-surface-2(#f5f5f5)	--color-text-secondary (#767676)	4.17:1	큰 글씨 전용
--color-surface-2(#f5f5f5)	--color-text-tertiary (#aaaaaa)	2.13:1	본문 사용 금지
--color-surface-2(#f5f5f5)	--color-link (#177bc3)	4.14:1	큰 글씨 전용
--color-surface-2(#f5f5f5)	--color-primary (#713fc5)	5.98:1	통과
--color-surface-2(#f5f5f5)	--color-price (#ff5259)	2.92:1	본문 사용 금지
--color-surface-2(#f5f5f5)	--color-brand (#ed1c24)	4.02:1	큰 글씨 전용
--color-surface-2(#f5f5f5)	--color-success (#136b2f)	6.07:1	통과
--color-surface-2(#f5f5f5)	--color-warning (#8a5a00)	5.44:1	통과
--color-surface-2(#f5f5f5)	--color-error (#b42318)	6.03:1	통과
--color-surface-2(#f5f5f5)	--color-info (#0b4ea2)	7.34:1	통과
--color-surface-1(#f2f2f2)	--color-text (#101010)	17.00:1	통과
--color-surface-1(#f2f2f2)	--color-text-secondary (#767676)	4.06:1	큰 글씨 전용
--color-surface-1(#f2f2f2)	--color-text-tertiary (#aaaaaa)	2.08:1	본문 사용 금지
--color-surface-1(#f2f2f2)	--color-link (#177bc3)	4.03:1	큰 글씨 전용
--color-surface-1(#f2f2f2)	--color-primary (#713fc5)	5.83:1	통과
--color-surface-1(#f2f2f2)	--color-price (#ff5259)	2.84:1	본문 사용 금지
--color-surface-1(#f2f2f2)	--color-brand (#ed1c24)	3.91:1	큰 글씨 전용
--color-surface-1(#f2f2f2)	--color-success (#136b2f)	5.91:1	통과
--color-surface-1(#f2f2f2)	--color-warning (#8a5a00)	5.29:1	통과
--color-surface-1(#f2f2f2)	--color-error (#b42318)	5.87:1	통과
--color-surface-1(#f2f2f2)	--color-info (#0b4ea2)	7.14:1	통과
--color-primary-bg(#f2ebff)	--color-text (#101010)	16.40:1	통과
--color-primary-bg(#f2ebff)	--color-text-secondary (#767676)	3.92:1	큰 글씨 전용
--color-primary-bg(#f2ebff)	--color-text-tertiary (#aaaaaa)	2.00:1	본문 사용 금지
--color-primary-bg(#f2ebff)	--color-link (#177bc3)	3.89:1	큰 글씨 전용
--color-primary-bg(#f2ebff)	--color-primary (#713fc5)	5.62:1	통과
--color-primary-bg(#f2ebff)	--color-price (#ff5259)	2.74:1	본문 사용 금지
--color-primary-bg(#f2ebff)	--color-brand (#ed1c24)	3.78:1	큰 글씨 전용
--color-primary-bg(#f2ebff)	--color-success (#136b2f)	5.71:1	통과
--color-primary-bg(#f2ebff)	--color-warning (#8a5a00)	5.11:1	통과
--color-primary-bg(#f2ebff)	--color-error (#b42318)	5.67:1	통과
--color-primary-bg(#f2ebff)	--color-info (#0b4ea2)	6.89:1	통과

타이포 — 크기 스케일

텍스트 2XS — 가나다라 Pretendard 123
--text-2xs = 11px · 가장 작은 보조 정보

텍스트 XS — 가나다라 Pretendard 123
--text-xs = 12px · 캡션·라벨

텍스트 SM — 가나다라 Pretendard 123
--text-sm = 13px · 보조 본문

텍스트 기준(본문) — 가나다라 Pretendard 123
--text-base = 14px · 기본 본문(14px 기준)

텍스트 MD — 가나다라 Pretendard 123
--text-md = 16px · 강조 본문

텍스트 LG — 가나다라 Pretendard 123
--text-lg = 18px · 섹션 소제목

--text-xl = 20px · 섹션 제목

텍스트 XL — 가나다라 Pretendard 123

--text-xl = 20px · 카드/영역 제목

텍스트 2XL — 가나다라 Pretendard 123

--text-2xl = 24px · 페이지 제목

텍스트 디스플레이(40px대) — 가나다라 Pretendard 123

--text-display = 40px · 배너 카피용 굵은 헤드라인

타이포 — 굵기

굵기 Regular(400)(400) — 가나다라 Pretendard

굵기 SemiBold(600)(600) — 가나다라 Pretendard

굵기 Bold(700)(700) — 가나다라 Pretendard

간격(4px 배수)

- space-1 = 4px · 아이콘·텍스트 최소 간격
- space-2 = 8px · 인접 요소 사이
- space-3 = 12px · 컴포넌트 내부 패딩
- space-4 = 16px · 컴포넌트 사이 기본 간격
- space-5 = 20px · 섹션 내부 여백
- space-6 = 24px · 카드 패딩
- space-7 = 32px · 섹션 사이 간격
- space-8 = 40px · 페이지 상하 여백

radius



--radius-sm = 4px · 인풋·배지



--radius-md = 8px · 카드 썸네일



--radius-lg = 16px · 모달·큰 카드

그림자



--shadow-sm = 0 1px 2px rgba(16, 16, 16, .08) · 카드 기본 그림자



--shadow-md = 0 2px 8px rgba(16, 16, 16, .12) · 호버·드롭다운



--shadow-lg = 0 8px 24px rgba(16, 16, 16, .16) · 모달·팝오버

z-index 층

- --z-base = 0 · 평면 콘텐츠
- --z-dropdown = 100 · 드롭다운·셀렉트
- --z-sticky = 200 · 고정 헤더·스티키 요소
- --z-overlay = 300 · 덮 배경
- --z-modal = 400 · 모달·다이얼로그
- --z-toast = 500 · 토스트·알림(최상위)

레이아웃

- --layout-content-width = 750px · 단일 컬럼 본문 최대 폭

Design Tokens · Tokens

UIS-CMN-002 공통 컴포넌트

이 화면 설계서에는 화면 그림이 없습니다 — 구성·위젯·연결 API는 소스에서 읽어 채웠습니다.

UIS-CMN-002: 공통 컴포넌트

> SR-310.1로 신규 도입된 shop-web 공통 UI 컴포넌트 11종 —

Button·TextInput·QuantityStepper·Badge·Tabs·BottomSheet·PopupCarousel·Toast/ToastStack·Skeleton·EmptyState·ErrorState.

> 모두 [[UIS-CMN-001]] 디자인 토큰만 참조하는 신규 추가 컴포넌트다. 이번 SR은 어느 화면에도 조립하지 않는다 — 실제 화면 적용은 리스킨 SR(SR-322-326) 몫.

1. 개요

위치: `modules/shop-web/src/components/common/` (신규 디렉터리). 기존 도메인 컴포넌트(`src/components/`의 Order·DeliveryBadge·LoginForm, `src/features/shop/member/`)와 분리되어 있고 그쪽은 이 SR에서 변경하지 않는다. 컴포넌트마다 상태별 `\*.stories.tsx`가 옆에 있다(`story-per-component` must 규칙).

인증·API 호출 없음(순수 프레젠테이션). `fetch`·`console.\*` 0건.

2. Button

`Button.tsx` — 3종 variant: `primary` (검정 채움) · `secondary` (흰 바탕+회색 테두리) · `purchase` (파랑→보라 그라데이션, `-gradient-primary`). `loading?: boolean`이면 클릭 차단 + 라벨을 "처리 중..."으로 대체 + `aria-busy`. 네이티브 `` 속성을 그대로 확장(`ButtonHTMLAttributes` 상속)한다.

`:hover`/`:focus-visible` 의사클래스는 `Button.css`로 처리 — 이 컴포넌트가 이 코드베이스에서 처음으로 별도 CSS 파일을 쓴 지점이다.

스토리: 3종 × (기본·호버·포커스·비활성·로딩) = 15개.

3. TextInput

`TextInput.tsx` — `label` (필수)·`error?: string | null` 표준 `input` 속성. id 미지정 시 `useId()`로 인스턴스 고유 id를 생성해 `label` htmlFor와 연결한다(같은 label을 가진 입력이 한 화면에 여러이어도 충돌하지 않는다). 오류 시 `aria-invalid` + 빨간 테두리, 비활성 시 회색 배경.

스토리: 기본·포커스·오류·비활성.

4. QuantityStepper

`QuantityStepper.tsx` — `value`·`min`·`max`·`onChange(committed: number)`. `-`/`+` 버튼은 `value===min`/`value===max`에서 `disabled`. 직접 입력은 편집 중 로컬 문자열(draft)만 바꾸고, blur·Enter 커밋 시 `quantityClamp.ts`의 순수함수 `clampQuantity(raw, min, max)`로 경계 보정한 뒤 값이 실제로 바뀌었을 때만 `onChange`를 호출한다.

스토리: 중간값·최솟값·최댓값·직접입력 보정.

5. Badge

`Badge.tsx` — variant별 판별 유니온(`BadgeProps`): `tv` (검정)·`freeShipping`·`installment` (months: number 필수, "무이자 {N}개월")·`live` (countdownSlot?: ReactNode — 실제 초단위 갱신 타이머는 범위 밖, 슬롯만 제공)·`discount` (rate: number 필수, 호출부가 기존 `discountRate.ts`의 `calcDiscountRate`로 계산해 넘긴다 — Badge 안에서 재계산하지 않는다).

스토리: 5종 각 1개.

6. Tabs

`Tabs.tsx` — `tabs: {key,label,badge?:boolean}[]`·`activeKey`·`onChange`·`sticky?: boolean` (스크롤 시 `position: sticky` 고정). 스와이프 전환 판정은 `resolveSwipeTab.ts`의 순수함수 `resolveSwipeTab(startX, endX, thresholdPx, activeIndex, count)`로 분리되어 있고, 컴포넌트는 `onTouchStart/End`로 델타를 계산해 이 함수만 호출한다(스와이프 임계값 40px). 기존 `ProductDetailTabs` (상품 상세 전용 고정 4탭)와는 별개이며 대체하지 않는다.

`badge?: boolean` (SR-311.1 추가) — true인 탭 버튼 안(라벨 위 작은 줄)에 점을 렌더한다. 종전 라운드는 이 표시를 `Tabs` 바깥의 별도 오버레이 ``으로 그렸으나, 활성 탭만 bold인 `cmn-tabs\_\_tab`과 폭 계산이 어긋나 탭 2개 이상에서 점 위치가 밀렸다 — 그래서 `Tabs` 자체의 계약으로 편입해 버튼 내부에서 그린다(위치가 활성 여부·탭 수와 무관해짐).

스토리: 기본·스티키 고정·배지 탭 1개 포함(SR-311.1 추가).

7. 포커스 트랩 공유 로직 — `useFocusTrap`

`useFocusTrap.ts` (hook) — `BottomSheet`·`PopupCarousel`이 공유하는 포커스 트랩·Esc 닫기·닫힌 뒤 트리거 요소로 포커스 복귀 로직. 두 컴포넌트에 각각 구현하지 않고 한 곳에만 두어 한쪽만 고쳐지는 사고를 막는다.

초기 포커스 이동: "닫힘→열림" 전이당 최초 1회만(`wasOpenRef`로 가드, `[open]` 단독 의존) — React 19 StrictMode 이중 이펙트에서 이미 옮겨진 포커스를 "직전 포커스"로 잘못 캡처하는 것을 방지.

Esc/Tab 리스너: 최신 `onClose`를 `onCloseRef`로 읽어 effect 의존성에서 `onClose`를 제외(`[open]` 단독 의존) — 부모가 인라인 화살표 `onClose`를 넘기고 재렌더해도 시트/팝업 내부 포커스가 매번 첫 포커스 가능 요소로 튀지 않는다.

포커스 복귀: `open`이 실제로 `false`로 떨어질 때(effect cleanup)만 트리거 요소로 되돌린다(Esc 키다운 시점의 동기 처리가 아니다).

8. BottomSheet

`BottomSheet.tsx` — 구매 시트 용도. `open`·`onClose`·`children`·`ariaLabel?: string` (기본 "구매 시트"). controlled(오버레이 + 패널) 프레젠테이션, `useFocusTrap` 사용. `open=false`이면 아무것도 렌더하지 않는다(스토리북 '닫힘' 스토리는 `tags: ['renders-nothing']`로 표시된 정답 상태).

스토리: 열림·닫힘.

9. PopupCarousel

`PopupCarousel.tsx` — 이미지 팝업 캐러셀. `open`·`slides: {id, background}[]` (실제 이미지가 아니라 배경색 블록 — 팝업 운영 이미지 데이터는 범위 밖(SR-339), Storybook 단독 렌더에서 `` 404를 원천 차단)

단)·`onClose`·`onDismissToday`·`initialIndex?` (기본 0)·`autoAdvanceMs?` (기본 4000, 0이면 정지 — 스토리에서 시간 의존성 제거용). "오늘은 그만 보기"는 로컬 체크 표시(`dismissChecked`) 후 `onClose()` 호출, **그 뒤에** 별도로

`onDismissToday()`를 호출한다(닫기와 억제 기록을 독립적으로 재작업할 수 있도록 하나로 묶지 않는다). `useFocusTrap` 사용.

스토리: 첫번째(1/5)·마지막·오늘은그만보기.

10. Toast / ToastStack

`Toast.tsx` — 토스트 1개 프레젠테이션(`message`·`variant: 'default'|'success'|'error'`·`onDismiss`).

`ToastStack.tsx` — `toasts: ToastItem[]`·`onDismiss`·`max?: number` (기본 3). `toastQueue.ts`의 순수함수 `capToasts(toasts, max)`로 **자체적으로** 최대 개수를 강제한다(초과 시 가장 오래된 것부터 제거) — 호출부가 상한 없이 넘겨도 방어된다.

스토리: 1개·3개·초과 시 제거.

11. Skeleton

`Skeleton.tsx` — `variant: 'card'|'list'`. 기존 `ProductDetailSkeleton`(상품 상세 고정형)과 별개.

스토리: 카드·리스트.

12. EmptyState

`EmptyState.tsx` — 가운데 라인 아이콘(인라인 SVG) + `message`(한 줄 안내) + `actionLabel`/`onAction`(내부적으로 `Button variant="secondary"` 재사용) + `railSlot?: ReactNode`(하단 추천 레일 자리, 벤치마크 bench/screenshot-10 빈 장바구니 구성 — 실제 추천 데이터는 범위 밖). 기존 `CartEmptyState`(장바구니 전용)와 별개.

스토리: 기본·추천레일포함.

13. ErrorState

`ErrorState.tsx` — 원형 느낌표 아이콘(인라인 SVG) + 고정 문구 "이용에 불편을 드려 죄송합니다." + `subMessage`(호출부가 고정 카피만 전달 — 서버 원문을 이 컴포넌트가 조합하지 않는다) + `onHome`(`Button variant="secondary"`, "홈으로") + `onBack`(`Button variant="primary"`, "돌아가기"). 벤치마크 bench/screenshot-11 404 구성과 동일.

스토리: 기본.

14. 범위 밖(후속 SR)

이 11개 컴포넌트를 기존 화면에 실제로 조립하는 작업(리스킨 SR-322-326).

PopupCarousel의 실제 이미지."오늘은 그만 보기" 영속화(localStorage/DB) — `onDismissToday` 콜백만 제공, 실제 저장은 SR-339.

Badge `live`의 실제 초 단위 카운트다운 갱신 타이머 — 이 배지를 쓰는 화면 SR 뒀.

Tabs/EmptyState/ErrorState/Skeleton이 기존 `ProductDetailTabs`/`CartEmptyState`/`ProductDetailSkeleton`을 대체하는 마 이그레이션.

변경 이력

<!-- spec-history: machine-managed -->

일자	SR	항목	변경 요약	커밋
2026-09-19	SR-310	#1	신규 생성 — 공통 컴포넌트 11종 (Button/TextInput/QuantityStepper/Badge/Tabs/BottomSheet/PopupCarousel/Toast·ToastStack/Skeleton/EmptyState/ErrorState)과 상태별 스토리·단위/RTL 테스트를 코드 기준으로 역생성	shop-web@7d32de0
2026-09-19	SR-311	#1	Tabs에 badge?:boolean 계약 추가(탭 안 빨간 점 표시)	shop-web@0acc411

UIS-CMN-003 앱 셀

이 화면 설계서에는 화면 그림이 없습니다 — 구성·위젯·연결 API는 소스에서 읽어 채웠습니다.

UIS-CMN-003: 앱 셀

- > SR-311.1로 신규 도입된 shop-web 공통 레이아웃 셀(AppShell) — [[UIS-CMN-002]](공통 컴포넌트)와 같은 급의
- > "화면이 아닌 공통 조립 지점" 문서다. 쇼핑 라우트(`/shop/\*\*`) 8개 화면 전부를 감싸며, 화면 자신의
- > 비즈니스 로직·상태·라우트는 바꾸지 않는다(리스킨은 SR-322·326 몫). SR-309 디자인 토큰·SR-310 공통
- > 컴포넌트([[UIS-CMN-002]]의 `Tabs`·`BottomSheet`)를 재사용한다. 4라운드 QA(round1 CONCERNS → round2
- > CONCERNS → round3 FAIL → round4 CONCERNS/납품 가능)를 거친 **최종 구현**만을 기술한다.

1. 개요

`AppShell.tsx`가 조립 지점이다. 8개 페이지(`ShopHomePage`·`ProductListPage`·`ProductDetailPage`·`CartPage`·`OrderPage`·`MyAddressesPage`·`LoginPage`·`PasswordResetPage`)가 각자의 본문을
`<AppShell>{children}</AppShell>`로 감싼다. `App.tsx`의 라우트 목록·부팅 리다이렉트(`/shop` → `/shop`)는 이 SR에서 무변경이며, 기존 주문 목록 라우트(`/`, `/orders/\*` — `OrderListPage`/`OrderDetailPage`)는 셀 밖 그대로다.
배치 순서: `Gnb` (헤더, 무변경 재사용) → `GnbTabs` (가로 GNB 탭) → 브레이크포인트별 본문 영역 → `ShopFooter` (무변경 재사용) → (모바일만) `BottomTabBar`.

2. 브레이크포인트 계약

경계 판정은 순수함수 `classifyBreakpoint(width)`(`breakpoint.ts`)가 정본이고, `useBreakpoint()`가
`window.innerWidth` + `resize` 리스너로 이를 구동한다(`window.matchMedia`는 의도적으로 쓰지 않는다 — jsdom이 미구현이라 쓰면 8개 페이지 테스트 전부가 마운트 단계에서 하드 크래시한다).

폭	상태	본문	퀵바/레일	하단 탭바
< 750px	`isMobile`	전폭(`100%`)	없음	있음(고정, 하단)
750~1199px	`isTablet`	`var(--layout-content-width)` (750px) 고정, 가운데	없음	없음
≥ 1200px	`isDesktopRail`	`var(--layout-content-width)` (750px), 3열 그리드 가운데 칸	있음	없음

- > **태블릿(750~1199px) 대역은 AC·확정문답 어디에도 규정이 없다** — "퀵바·레일·하단탭바 전부 없는
- > 단일 컬럼"은 `breakpoint.ts` 주석에 적힌 구현 쪽의 보수적 선택이며, 이 코드 주석만이 유일한 정본이다.
- > 후속 SR에서 재검토 시 이 사실을 전제로 삼을 것.

`AppShell`은 테스트/스토리 전용 `forceBreakpoint?: 'mobile' | 'tablet' | 'desktop'` prop을 받아

`useBreakpoint()`의 실측값 대신 상태를 결정적으로 고정할 수 있다(기본 `undefined` = 실측 사용).

3. 헤더(Gnb) — 무변경 재사용 + 검색 아이콘

`Gnb.tsx` 자체(로그·카테고리 버튼·검색 입력·장바구니 수량·로그인/마이 영역)는 SR-311 이전 그대로다.

이번 SR은 검색 입력 옆에 돋보기 아이콘 버튼(`aria-label`="검색 화면으로 이동")만 추가했다. 기존
`aria-label`="상품 검색" 입력(직접 타이핑·Enter 제출)은 그대로 남아 있어 `ShopHomePage.test.tsx`/
`ProductListPage.test.tsx`의 기존 검색 회귀가 없다.

검색 아이콘 클릭(`handleSearchIconClick`) 동작(3분기, round4 최종):

1. ****검색어가 있으면**** — `onSearchSubmit()`을 호출한다(각 페이지가 이미 가진 제출 핸들러, Enter와 완전히 같은 채널). 새 검색 구현이 아니다.
2. ****검색어가 없고 이미 `/shop/products` (목록 화면) 안이면**** — 이동할 곳이 없으므로 같은
`onSearchSubmit()`을 호출해 페이지가 스스로 "검색어 없음"으로 정리하게 한다.
3. ****검색어가 없고 목록 화면 밖이면**** — `navigate('/shop/products', { state: { focusSearch: true } })`로 이동만 한다. ****`Gnb`는 클릭 시점에 스스로 포커스를 걸지 않는다.****

포커스 인계(round3 FAIL → round4 해소): 라우트 전환 시 현재 `Gnb` 인스턴스는 곧 언마운트되므로
클릭 시점에 `focus()`를 걸면 사라질 인스턴스에 거는 것이라 운영에서 동작하지 않는다(round3 QA 실측:
`document.activeElement`가 `BODY`). 대신 `Gnb`는 `autoFocusSearch?: boolean` prop을 받아 ****마운트
이펙트****(`useEffect(..., [])`)에서만 자기 자신의 검색 입력에 포커스한다. 목적지 화면(`ProductListPage`)이
마운트 시 `location.state.focusSearch`를 읽어 `focusSearchOnMount`를 계산하고, 이를 `AppShell` → `Gnb`의
`autoFocusSearch`로 내려준 뒤 ****같은 마운트에서**** `navigate(location.pathname + location.search,
{ replace: true })`로 state를 즉시 소거한다(뒤로가기로 돌아와도 다시 포커스되지 않음, 같은 라우트로의
`replace`라 재마운트 없음). 이 왕복 계약을 갖는 곳은 현재 `ProductListPage` 하나뿐이다(검색 아이콘의
빈 검색어 이동 목적지가 그 화면 하나이므로).

4. GNB 탭 (GnbTabs)

`GnbTabs.tsx`는 SR-310 공통 `Tabs`([[UIS-CMN-002]] §6)를 그대로 쓴다(재구현 없음). 탭 목록은
`shopStatic.ts`의 `GNB\_TAB\_CONFIG` (설정값 한 곳) — 요구사항 배경의 9개 라벨(편성표·TV쇼핑·특가쇼·홈·
주말엔보너스·브랜드관·랭킹·VIP라운지·혜택/이벤트) 순서를 유지하되, ****지금만 '홈'만 `implemented:true`
(`path`:`/shop`)이고 나머지 8개는 `path:null`로 GNB에서 완전히 빠진다****(도착 화면 없는 탭 숨김, AC).
`activeKey`는 `useLocation().pathname`을 `visibleTabs[].path`와 매칭해 계산한다. ****일치하는 탭이
없으면(`/shop/cart`·`/login` 등) 존재하지 않는 key(`""`)를 넘겨 어떤 탭도 `aria-selected`되지
않는다**** — round1에서는 `?? visibleTabs[0]`로 임의로 '홈'이 강조됐던 결함이 round2에서 해소됐다.

"탭 위 빨간 라벨"은 `Tabs`의 `TabItem.badge?: boolean` 계약(SR-311이 [[UIS-CMN-002]]에 새로 연 확장, 탭 버튼 내부에 점을 그려 활성 여부·노출 탭 수와 무관하게 정렬 — round1의 별도 오버레이 방식은 탭이 2개 이상일 때 어긋나 round2에서 전삭제되고 이 방식으로 대체됐다)로 그린다. **현재 `GNB\_TAB\_CONFIG`의 어느 탭도 `badge:true`가 아니라 실제로 화면에 점이 뜨는 경로는 없다** — 설정값만 켜면 바로 표시된다.

`onChange`는 `implemented:true`(`path` 있는) 탭만 `navigate(path)`한다.

5. 좌측 퀵바 / 하단 탭바 (QuickBar / BottomTabBar)

두 컴포넌트는 `shopStatic.ts`의 `QUICK\_NAV\_ITEMS`(설정값 한 곳, 홈·ON AIR·카테고리·마이)를 공유한다 — 하단 탭바가 퀵바와 같은 설정을 쓰는 것은 AC가 명시하지 않아 구현이 추론한 것이고, 사람이 구현 계획 확인 단계에서 승인했다(STORY "사람 확인 사항" 1).

key	라벨	`implemented`	비고
home	홈	true	`#/shop`
onAir	ON AIR	**false**	방송중 기능(SR-319) 전까지 숨김
category	카테고리	**false**	서랍(SR-237) 전까지 숨김
my	마이	true	session 유무로 목적지 분기(아래)

QuickBar(>1200px, `features/shop/QuickBar.tsx`): 세로 배치. `QUICK\_NAV\_ITEMS.filter(implemented)`

로 홈·마이만 노출 + QR·TOP은 목록과 무관하게 항상 노출(4항목 실질 표시: 홈·마이·QR·TOP).

마이: `session`이 있으면 `#/shop/mypage/addresses`, 없으면

`#/login?redirect=` + `encodeURIComponent(location.pathname + location.search)`.

QR: 내부 `useState`로 `QrPopup` 오픈(\$7).

TOP: `window.scrollTo({ top: 0, behavior: 'smooth' })`.

BottomTabBar(<750px, `features/shop/BottomTabBar.tsx`): 가로 고정(`position:fixed; bottom:0`),

`QUICK\_NAV\_ITEMS.filter(implemented)`로 홈·마이만 노출(QR·TOP은 AC 원문에 없어 하단 탭바에 없다).

'마이' 분기 로직은 QuickBar와 동일.

리다이렉트 파라미터는 **`location.pathname + location.search`**를 싣는다(round1: `pathname`만 실어

`?keyword=가방` 같은 쿼리가 로그인 후 유실되던 결함, round2에서 해소).

6. 우측 레일 (RightRail)

>1200px 전용(`features/shop/RightRail.tsx`). 두 카드 모두 "값 없으면 숨김"이고, **둘 다 없으면

컴포넌트 자체가 `null`을 반환**한다(그 경우에도 AppShell의 레일 "칸" 자체는 \$8 그리드 계약에 따라

DOM에 남는다 — 카드만 없을 뿐 트랙은 유지).

최근 본 상품 카드: `recentlyViewedStorage.loadRecentlyViewedSkus()`(localStorage)로 sku 목록을

얻고, 자체 `useEffect` + `inFlightRef` (StrictMode 이중 마운트 가드)로 기존 `fetchProducts()`(신규

엔드포인트 아님, `api.ts`의 기존 export)를 1회 호출해 sku→상품명을 붙인다. 최대 3개, 그라데이션
 (`--gradient-accent`) 헤더 카드. **sku가 0개면 `fetchProducts()` 자체를 부르지 않는다**(round1:
 카드가 어차피 숨는데도 GET이 먼저 나가던 결함 — `LoginPage`/`PasswordResetPage`처럼 원래 상품 API를
 안 부르던 화면에서도 불필요한 호출이 생겼었다. round2에서 `if (loadRecentlyViewedSkus().length ===
 0) return` 가드로 해소). 화면 본문이 같은 API를 자기 목적으로 이미 호출하는 것과는 별개 호출이라
 화면당 GET이 1회 더 생기는 것 자체는 범위 밖으로 남겨졌다(\$10).

****보유 쿠폰/주문 내역 수 카드****: `entitlementCounts?: { coupons: number; orders: number }` prop —
 undefined거나 둘 다 0이면 렌더하지 않는다. **이번 SR은 이 prop을 아무도 채우지 않는다**(쿠폰
 데이터는 SR-276 확정 제외, 주문 수는 기존 주문 도메인 뒀) — "값 없으면 숨김" AC는 항상-숨김으로
 충족되고, 호출부가 값을 채우는 순간(SR-276 이후) 그대로 뜨는 계약만 열려 있다.

7. QR 팝업 (QrPopup)

`features/shop/QrPopup.tsx` — SR-310 공통 `BottomSheet` (무변경 재사용, `ariaLabel="QR 코드"`)로 감싸고
 `qrcode.react`의 ``로 **로컬에서** SVG를 그린다(외부 QR 생성 API 없음 → 네트워크 요청 자체가
 없다 — 실패 사례집 SR-306 #1 대조로 의도적으로 고른 방식). `url`은 열릴 때 `window.location.href`를
 그대로 담는다(HashRouter라 현재 해시 경로까지 포함, AC "현재 페이지 URL").

8. ≥1200px 3열 그리드 — 본문 정렬 계약

```
display: grid;
grid-template-columns: minmax(0, 1fr) var(--layout-content-width) minmax(0, 1fr);
```

좌측 칸(`appshell-quickbar-slot`, `QuickBar`, `justify-content:flex-end`) · 가운데 칸(``<main>``, 750px) ·
 우측 칸(`appshell-rail-slot`, `RightRail`, `justify-content:flex-start`) — **레일-퀵바 칸은 `isDesktopRail`
 에만 조건부이지 카드 콘텐츠 유무에 조건부가 아니다**, 즉 최근 본 상품이 0개라 `RightRail`이 `null`을
 반환해도 그 칸(래퍼 ``<div>`')은 DOM에 그대로 남는다.

`minmax(0, 1fr)`(round4 최종, round3 FAIL의 필수수정)이 핵심이다 — 단순 `1fr`은 CSS에서
 `minmax(auto, 1fr)`이라 트랙이 칸 안 콘텐츠의 min-content(레일 220px·`flexShrink:0`, 퀵바 64px) 아래로
 줄지 않는다. 1200~1310px처럼 좌우 여유가 440px 미만인 대역에서 레일 칸만 220px 하한에 눌러 좌우가
 비대칭이 되고, 레일이 카드 없이 `null`이면(min-content 0) 다시 대칭이 되는 식으로 **본문이 사용자의
 localStorage 상태(최근 본 상품 유무)에 따라 최대 51px(1200px 기준, 1240px 31px·1280px 11px·1366px
 이상 0px) 움직이는 결함**이 round2~round3에 걸쳐 있었다. `minmax(0, 1fr)`은 그 min-content 하한을
 0으로 강제해 **좌우 트랙이 콘텐츠 유무·뷰포트 폭과 무관하게 항상 동일하게 계산**되고, 가운데 750px

본문은 컨테이너 정중앙에 고정된다.

****round4 QA가 1200/1240/1280/1310/1366/1920px × 레일 있음/없음 12조합을 playwright 실측으로 검증**:**

`<main>` 중심이 전 폭에서 캔버스 중심과 오차 0px, 레일 카드 유무 간 차이도 전 폭 0px. `AppShell`.

stories.tsx의 `최근본상품\_3개`/`최근본상품\_0개` 두 스토리 모두 `play`(`assertMainCentered`, 허용오차 2px)로 이를 고정한다.

> ****트레이드오프(결함 아님, 알려진 제약 §10)**:** `minmax(0, 1fr)`가 트랙 하한을 0으로 만든 대가로,

> 1200~1237px 대역에서 레일(220px 고정폭)이 칸보다 좁아지면 ****레일이 칸 밖으로 흘러나가 가로 스크롤이**

> 생긴다******(본문 좌표 자체는 흔들리지 않음). round3 QA 수정안이 제시하고 사람이 그대로 승인한 절충이다.

9. 푸터

`ShopFooter`(무변경 재사용) — 사업자 정보 자리만 마련되어 있고 실제 내용은 SR-329 몫. 모바일에서는

`BottomTabBar`(고정 하단)가 푸터를 가리지 않도록, 하단 여백(`paddingBottom: isMobile ? 56 : 0`)을

`<main>`이 아니라 셀 최외곽 컨테이너(`

있어 푸터 마지막 줄이 하단 탭바에 가려지던 결함, round2에서 해소).

10. 알려진 제약 (결함 은폐 금지)

****[SR-322 추적] 1200~1237px에서 우측 레일이 뷰포트 밖으로 최대 19px 넘친다****(가로 스크롤 발생) —

§8의 `minmax(0,1fr)` 절충이 낳은 트레이드오프. 본문(`

않는다(0px 실측). round3 QA 수정안·사람 지시가 이미 승인한 결과라 round4 QA는 결함으로 올리지

않았다. SR-322(기존 화면 리스킨)에서 함께 재검토 예정.

****`ProductListPage`의 `location.state.focusSearch` 소거(`replace`) 로직에 테스트 가드가 없다**** —

그 분기를 무력화해도 41 suites / 312 tests가 그대로 통과한다(round4 QA 실측). 동작 자체는 QA가

임시 테스트로 정상 확인했으나(소거 후 `focusSearch`가 false로 남음), 조용한 되돌림을 스위트가 못

잡는 커버리지 구멍으로 남아 있다.

****태블릿(750~1199px) 대역은 AC·확정문답에 규정이 없다**** — "퀵바·레일·하단탭바 없는 단일 컬럼"은

구현(`breakpoint.ts` 주석)만이 유일한 정본이다(§2).

****GNB 탭이 2개 이상 열리면(SR-237/SR-319 이후) "선택 없음" 상태의 좌우 스와이프가 첫 탭이 아니라**

두 번째 탭으로 간다******(`GnbTabs.tsx`가 불일치 시 `activeKey=""`를 넘기는데 `Tabs.tsx`는 기존 인덱스를

`Math.max(0, findIndex)`로 0으로 잡는다) — 지금은 노출 탭이 '홈' 1개뿐이라 무해하며, 사람이 "이번

범위 대상 아님"으로 명시해 후속 TODO로 보류됐다.

****목록 화면(`/shop/products`) 안에서 빈 검색어로 검색 아이콘을 누르면 URL의 `?keyword=`가 stale로**

남는다****** — `ProductListPage`가 마운트 시 1회만 URL을 읽는 기존 설계(SR-303)와 얹힌 것으로 round3

19 / 218

신규 결함은 아니며, 사람이 "이번 범위 대상 아님"으로 명시해 후속 TODO로 보류됐다.

**\$8의 `assertMainCentered` 실좌표 검증(storybook `play`)은 `project.env`에
`STORYBOOK\_TEST\_CMD`가 설정돼야 AIDD 측E에서 실행된다** — 없으면 측E는 콘솔 오류만 보는 내장
렌더러로 폴백해 `play`를 돌리지 않는다. 이 SR의 QA round4 이후 사람 지시로
`modules/shop-web/scripts/test-storybook-ci.cjs` + `project.env`의 `STORYBOOK\_TEST\_CMD` 설정이
추가되어 이후 라운드부터는 측E가 이 검증을 자동으로 포함한다.

11. 데이터 흐름·보안

새 인증·인가 경로 없음. 유일한 자격 분기는 \$5의 '마이' 목적지이고 각 페이지가 이미 `loadSession()`으로
판정한 `session` prop을 표시용으로 재사용할 뿐이다. 신규 데이터 호출은 \$6의 `fetchProducts()`(기존
공개 상품 목록 엔드포인트, `api.ts` 경유 — 규칙 `web-fetch-only-in-api` 준수, 부품 직접 `fetch`·
`console.log` 0건)뿐이고 백엔드·DB 변경은 없다. QR(\$7)에 담기는 `window.location.href`에 토큰류 없음.

12. 적용 화면 (8개, 렌더 트리만 교체)

`ShopHomePage`·`ProductListPage`·`ProductDetailPage`·`CartPage`·`OrderPage`·`MyAddressesPage`·
`LoginPage`·`PasswordResetPage` — 각 페이지의 ``<AppShell>{기존 본문}</AppShell>`로 교체했다(비즈니스 로직·상태·effect는 무변경). `LoginPage`·
`PasswordResetPage`는 이 SR 이전에는 헤더 자체가 없던 화면이라 `session`/`cartItemCount`/`searchValue`
상태를 새로 추가했다. **손대지 않은 것**: `App.tsx` (라우트 목록·부팅 리다이렉트 전부 무변경),
`OrderListPage`/`OrderDetailPage`(`#/`, `#/orders/\*` — 제외 대상), `Gnb.tsx`의 기존 부분·`ShopFooter`·
`RecentlyViewed` (전부 무변경 재사용).

13. 범위 밖 (후속 SR)

카테고리 서랍 내용(SR-237) · 검색 동작 고도화(자동완성 등, SR-239) · 쿠폰 수 데이터 연동(SR-276) ·
헤더 '기프트쇼' 토글 · 기존 주문 목록(`#/`, `#/orders/\*`) 셀 적용 · 백엔드 변경 · 푸터 실제 내용
(SR-329) · 기존 화면 내부 리스킨(SR-322-326) · \$10의 1200~1237px 레일 오버플로 완화(SR-322) ·
`RightRail`↔페이지 간 `fetchProducts` 중복 호출 최적화.

변경 이력

<!-- spec-history: machine-managed -->

일자	SR	항 목	변경 요약	커밋
2026-09-19	SR-311	#1	앱 셸(AppShell) 신규 — shop-web 쇼핑 라우트 전체에 공통 레이아웃(퀵바/GNB탭/레일/하단탭바) 적용	shop-web@b1c7c6f5850d, shop-web@0acc411

UIS-MBR-002 로그인

이 화면 설계서에는 화면 그림이 없습니다 — 구성·위젯·연결 API는 소스에서 읽어 채웠습니다.

> [반영: FUNC-member-004] 2026-09-13

UIS-MBR-002: 로그인

- > **근거 소스(권위):** `modules/shop-web/src/pages/LoginPage.tsx` + `src/components/LoginForm.tsx` + `src/App.tsx` (부팅 시 무음 리프레시) + `src/redirectTarget.ts` + `src/session.ts` + `src/api.ts`.
- > 소스폴백 모드 — DOM 스냅샷·스크린샷 없음(AIDD 작업 중 `npm run dev`를 켜지 않는다는 제약, HMR OOM 회피). `LoginForm.stories.tsx`의 6개 상태(기본·입력됨·제출중·실패401·잠금429·비밀번호토글)를
- > \$5 표시조건의 1급 증거로 사용했다.
- > **라우트 참고:** `HashRouter` 사용(`App.tsx:45-51`) — 브라우저 주소창 상 실제 경로는 `/#/login`이다.
- > `redirect` 쿼리 파라미터는 `/#/login?redirect=<encodeURIComponent(path)>` 형태의 **프론트 전용
- > 관례**(서버와 무관한 라우팅 계약, `LoginPage.tsx:24-27`) — 없으면 로그인 성공 후 기본 `/`로 이동한다.

1. 화면 목적

이메일+비밀번호로 로그인해 세션(리프레시 토큰 30일 + API 키)을 발급받고, 로그인 성공 시 원래 가려던 경로(`redirect` 파라미터) 또는 기본 `/`로 이동하는 화면이다. 잠금·존재·비밀번호 대조는 서버(INF-MBR-003)가 이미 판정하며, 이 화면은 그 결과(코드·문구·`retryAfterSeconds`)를 그대로 표시만 한다 — 회원 없음/탈퇴/비밀번호 오류를 프론트가 구분해 노출하지 않는다(존재 오라클 방지).

2. 주요 작업 시나리오

****시나리오: 로그인****

1. 이메일(`email`)·비밀번호(`password`)를 입력한다(`LoginForm.tsx`).
2. 필요하면 [보기] 버튼으로 비밀번호를 평문으로 확인한다(로컬 상태 토글, 서버 호출 없음).
3. [로그인] 버튼을 눌러 제출 → `LoginPage.handleSubmit` → POST `/api/members/login(INF-MBR-003)`.
4. ****성공****: 응답(`SessionResult`)을 `saveSession`으로 localStorage 단일 키(`sl.member.session`)에 원자적으로 저장 → `resolveRedirectTarget(params.get('redirect'))`가 정한 경로로 이동(기본 `/`, 절대 URL·프로토콜 상대·역슬래시 변형은 오픈 리다이렉트로 판단해 무시하고 `/`로 보냄).
5. ****실패 401****(`MBR-4011`, "...($n/5$)" 카운트 포함 문구): 오류 배너에 서버 문구를 그대로 표시, 재입력·재제출 가능.
6. ****실패 429****(`MBR-4291`, 5회 연속 실패 도달): 오류 배너에 서버의 완곡 문구("...잠시 후 다시 시도해 주세요")를 그대로 표시하고, `retryAfterSeconds`가 다 지날 때까지(내부 타이머) 로그인

버튼이 비활성된다(재제출 자체가 막힌다) — **정확한 초는 화면에 노출하지 않는다**(round2 QA

재작업: 정확 초 병기가 "잠시 후"로 완곡화하라는 요건과 상충했다).

부가 시나리오: 부팅 시 자동 재로그인(사용자 조작 아님, 세션 생명주기 참고)

1. 앱이 뜰 때(`App.tsx`) 저장된 세션에 `refreshToken`이 있으면, 이 화면과 무관하게 무음으로

POST /api/members/sessions/refresh(INF-MBR-005)를 1회 호출해 세션을 30일 롤링 갱신한다

(`refreshOnce.ts`의 모듈 스코프 in-flight 가드로 StrictMode 이중 발사를 막는다).

2. 응답이 오면 무조건 `saveSession`(회전 API라 구 토큰은 이미 폐기됨).

3. 401(`MBR-4012`: 미존재/만료/폐기/탈퇴 4가지 사유 동일 응답)만 저장된 세션을 조용히 지운다.

그 외 오류(5xx·네트워크 장애)는 세션을 보존하고 다음 부팅 때 재시도한다.

3. 화면 구성 (블록)

마커	블록	역할	주요 위젯	소스 근거
①	로그인 폼	이메일/비밀번호 입력 + 제출	email input, password input, 보기/숨기기 토글, 로그인 버튼	`LoginForm.tsx:72-101`
②	오류 배너	401/429 오류 표시(조건부)	`role=alert` div, 서버 문구 그대로(정확 초 없음)	`LoginForm.tsx:90-94`

4. 위젯·액션

① 로그인 폼

번호	위젯	타입	레이블	동작	연결 API(raw)	결과
(1)	email input	input(email)	이메일	입력값 갱신 (`onChange`)	—	—
(2)	password input	input(password/text)	비밀번호	입력값 갱신, `showPassword`에 따라 `type` 전환	—	—
(3)	보기/숨기기 버튼	button(type=button)	보기 / 숨기기	`showPassword` 로컬 토글(서버 호출 없음)	—	password ↔ text 전환
(4)	로그인 버튼	button(type=submit)	로그인 / 로그인 중...	폼 제출 → `LoginPage.handleSubmit`	POST /api/members/login	성공: 세션 저장+이동 / 401: 오류 배너 표시 / 429: 오류 배너(완곡 문구), 버튼 비활성(정확 초는 미노출)

② 오류 배너

번호	위젯	타입	레이블	동작	연결 API(raw)	결과
----	----	----	-----	----	-------------	----

(5)	오류 메시지	div[role=alert]	(서버 `message` 그대로)	표시 전용, 재해석 없음	—	`MBR-4011`(401):`MBR-4291`(429):`MBR-5000`(500) 문구 렌더
-----	--------	-----------------	--------------------	---------------	---	---

5. 접근 권한·표시 조건

요소	표시 조건	근거	스토리
로그인 버튼 비활성(`disabled`)	`busy`(제출 중) 이거나 `locked && remaining > 0`(429 내부 타이머 진행 중, 화면에 초는 안 보임)	`LoginForm.tsx:70,95`, stories: ./src/components/LoginForm.stories.tsx	[제출중](story:회원-로그인--제출중)
오류 배너 표시	`error`가 non-null	`LoginForm.tsx:90-94`, stories: ./src/components/LoginForm.stories.tsx	[실패401](story:회원-로그인--실패401)
(제거됨, round2 QA 재작업) 정확 초 카운트다운 문구	종전에 "` — N초 후 다시 시도`"를 병기했으나, 서버의 "잠시 후" 완곡화 요건과 상충해 **표시를 제거**했다. `retryAfterSeconds`(`error?.retryAfterSeconds`)는 여전히 `remaining` 내부 타이머로 남아 버튼 비활성 유지에만 쓰인다	`LoginForm.tsx:38-39,55-68`	[잠금429](story:회원-로그인--잠금429)
비밀번호 평문 표시	`showPassword === true`(보기 버튼 클릭)	`LoginForm.tsx:37,81`, play 인터랙션으로 검증	[비밀번호토글](story:회원-로그인--비밀번호토글)
기본(빈 입력) 상태	초깃값(`email:"", password:"")	`LoginForm.tsx`	[기본](story:회원-로그인--기본)
입력된 상태	이메일·비밀번호 모두 채워짐, 오류 없음	`LoginForm.tsx`	[입력됨](story:회원-로그인--입력됨)

6. 팝업·연계 화면

없음(팝업/모달 트리거 없음). 로그인 성공 시 `useNavigate`로 같은 SPA 내 다른 라우트(`redirect`

파라미터가 가리키는 경로, 기본 `/`)로 이동한다 — 팝업이 아닌 페이지 이동(`LoginPage.tsx:28`).

7. 데이터 출처·연결

****연결 API(raw → INF):****

`POST /api/members/login` — INF-MBR-003. 이 화면(로그인 폼)의 유일한 사용자 트리거 호출.

`POST /api/members/sessions/refresh` — INF-MBR-005. ****이 화면이 직접 호출하지 않는다**** — `App.tsx`의

부팅 시 무음 리프레시(`useSilentRefresh`)가 호출하는 것으로, 로그인 폼 자체의 액션은 아니지만

"세션 생명주기의 반대편"으로서 이 화면의 스펙에 참고 기록한다.

****`INF-MBR-004`(로그아웃)는 이 화면이 FUNC이 호출하지 않는다**** — `api.ts`에 `logout` 함수 자체가

없다(직접 확인). 로그아웃 UI/트리거는 아직 shop-web 어디에도 없다(§8 참조).

****참조 테이블(SCH):**** 이 화면 자체는 DB를 직접 호출하지 않는다(모두 INF-MBR-003/005 경유). 참조

테이블은 해당 INF 문서가 정보 — `MEMBERS`, `MEMBER\_LOGIN\_ATTEMPTS`, `MEMBER\_REFRESH\_TOKENS`, `MEMBER\_API\_KEYS`.

8. 미확인 사항

로그아웃 UI/트리거(헤더·계정메뉴)는 이 화면(UIS-MBR-002)의 범위가 아니다 — 백엔드(INF-MBR-004, FUNC-member-006)는 Done이지만 이를 호출할 컴포넌트가 shop-web에 아직 없다. **범위 밖(후속 FUNC)** — STORY-FUNC-member-004 "범위 밖" 절에서 명시적으로 확정된 사항이며 미확인이 아니다.

보호 라우트(비로그인 시 접근 차단 + `?redirect=` 자동 채움)는 아직 없다 — 이 화면은 `redirect` 파라미터를 "읽어서 돌아가는" 쪽만 구현하고, 채워 보내는 진입점(가드)은 shop-web의 어떤 화면에도 없다. **범위 밖(후속 FUNC)**.

`/login`에 이미 로그인된 상태로 접근했을 때 자동 리다이렉트하는 동작은 없다(AC에 없는 엣지케이스, STORY에 범위 밖으로 명시).

로그인 성공 후 발급받은 `apiKey`를 이후 인증이 필요한 API 호출 헤더에 실제로 붙이는 동작은 아직 없다 — `api.ts`의 현재 함수(`get`/`fetchOrders`/`fetchOrder`) 중 인증이 필요한 엔드포인트를 호출하는 것이 없다. 이 화면은 발급·저장까지만 담당한다. **범위 밖(그 엔드포인트를 쓰는 후속 FUNC의 책임)**.

`refreshToken`을 localStorage에 저장하는 것은 XSS로 탈취 가능한 노출 위험이 있으나 랩 규모로 수용된 임시 결정이다(`session.ts:10-12`, 사람 결정 3) — httpOnly 쿠키 전환은 백엔드 계약 변경이 선행돼야 하는 **후속 SR 대상**으로 이미 이월 확정됨.

변경 이력

<!-- spec-history: machine-managed -->

일자	SR	항목	변경 요약	커밋
2026-09-15	QUICK-20260915-1	#1	잠금429 카운트다운 정확 초 노출 제거 — 완곡화 요건 반영, 라인 앵커 갱신	

화면 상태 6건 — 기준 스냅샷

이메일

비밀번호

보
기

로그인

로그인 · 기본

이메일
user@example.com

비밀번호
.....

보
기

로그인

로그인 · 입력됨

이메일
user@example.com

비밀번호
.....

보
기

로그인 중...

로그인 · 제출중

이메일
user@example.com

비밀번호
.....

보
기

이메일 또는 비밀번호가 올바르지 않습니다 (3/5)

로그인

로그인 · 실패401

이메일
user@example.com

비밀번호
.....

보
기

로그인 시도 횟수를 초과했습니다. 잠시 후 다시 시도해 주세요

로그인

로그인 · 잠금429

이메일

비밀번호
abcd1234

숨
기

로그인

로그인 · 비밀번호토글

UIS-MBR-003 비밀번호 재설정

이 화면 설계서에는 화면 그림이 없습니다 — 구성·위젯·연결 API는 소스에서 읽어 채웠습니다.

> [반영: FUNC-member-007] 2026-09-13

UIS-MBR-003: 비밀번호 재설정

- > ****근거 소스(권위):**** `modules/shop-web/src/pages/PasswordResetPage.tsx` (컨테이너 — 상태기계·응답코드)
- > 매핑 + `src/features/member/PasswordReset{Request,Code,Password,Done}Step.tsx` (4개 순수 프레젠테이션)
- > 부품 + `src/features/member/requestCodeOnce.ts` (단일 비행 가드) + `src/pages/LoginPage.tsx` (진입)
- > 링크 + `src/App.tsx` (라우트) + `src/api.ts` (API 호출). 소스폴백 모드 — DOM 스냅샷·스크린샷 없음
- > (AIDD 작업 중 `npm run dev`를 켜지 않는다는 제약, HMR OOM 회피). 4개 부품 각각의 `.stories.tsx`
- > (기본·입력됨·제출중·형식오류·재전송쿨다운끝·코드오류·만료·시도초과·비밀번호규칙오류 등 총 12개
- > 상태)를 \$5 표시조건의 1급 증거로 사용했다. 응답코드→화면전이 매핑은
- > `docs/00\_FUNC/stories/STORY-FUNC-member-007.md`의 "AC5". "구현 계획" 순서·보안"에서 사람이 확정한
- > 표를 그대로 옮겼다(재해석 금지).
- > ****라우트 참고:**** `HashRouter` 사용(`App.tsx:47-54`, UIS-MBR-002와 동일) — 브라우저 주소창 상 실제
- > 경로는 `/#/password-reset`이다. 이 화면 안에서는 라우트 전이가 없다 — 4단계 전부 같은 라우트에서
- > 컨테이너의 로컬 상태(`step`)로 스위치 렌더된다(\$3 참조, 탭이 아니라 상태기계).

1. 화면 목적

비밀번호를 잊은 회원이 이메일 또는 휴대폰번호로 받은 6자리 코드를 확인하고 새 비밀번호를 설정하는 3단계(+완료) 화면이다. 코드 발급·검증·비밀번호 반영의 실제 판정은 전부 서버(INF-MBR-006/007)가 하며, 이 화면은 그 결과(응답 코드·문구·`expiresInSeconds`)를 상태기계로 옮기고 표시만 한다 — 계정 존재 여부를 프론트가 판단해 노출하지 않는다(요청 API는 형식 오류가 아닌 한 항상 202, 존재 오라클 방지). 재설정이 완료되면 그 회원의 모든 기기가 로그아웃된다(서버 사이드이펙트).

2. 주요 작업 시나리오

****시나리오: 비밀번호 재설정****

1. 로그인 화면의 [비밀번호를 잊으셨나요?] 링크로 진입한다(`LoginPage.tsx:45-47`) — 또는 주소창에 직접 `/#/password-reset`으로 진입.
2. ****1단계(계정 확인):**** 이메일 또는 휴대폰번호를 입력하고 [코드 요청]을 누른다 → `PasswordResetPage.handleRequestSubmit` → `requestCodeOnce` (단일 비행 가드) →

POST /api/members/password-resets/codes(INF-MBR-006). 휴대폰은 하이픈·공백을 제거해 숫자만, 이메일은 입력 그대로 전송한다(`normalizeTargetForSend`).

****202 성공****: 서버가 돌려준 `target`(정규화됨)·`channel`·`expiresInSeconds`를 저장하고 2단계로 전이. 유효시간 카운트다운과 재전송 60초 쿨다운을 벽시계(epoch ms) 기준으로 시작.

****400 MBR-4100****(형식 오류): 1단계 유지, 인라인 오류 표시.

3. ****2단계(코드 확인)****: 발송 채널·대상(`target`)과 남은 유효시간을 보여준다. 6자리 숫자를 입력하면 (로컬 형식 검증만, 서버 호출 없음) [다음]이 활성화되어 3단계로 넘어간다. [재전송](60초 쿨다운 경과 후 활성화)을 누르면 코드를 다시 요청하고 남은 유효시간·쿨다운을 재시작하며 옛 코드 입력·오류 문구를 지운다.

4. ****3단계(새 비밀번호)****: 새 비밀번호를 입력하고 [비밀번호 변경] 제출 →

`PasswordResetPage.handlePasswordSubmit` → POST /api/members/password-resets/confirmations (INF-MBR-007, `target`+`code`+`newPassword`를 원자 전송 — "코드 확인"은 별도 API가 아니라 이 호출 안에서 함께 검증된다).

****204 성공****: `clearSession()`으로 이 기기의 로컬 세션(apiKey/refreshToken)을 즉시 지우고 4단계(완료)로 전이.

****410 MBR-4101****(코드 만료) · ****409 MBR-4103****(시도 초과): 2단계로 되돌리고 "다시 요청" 버튼만 보여준다(코드 입력 UI는 숨김). [다시 요청]을 누르면 전체 상태를 초기화하고 1단계로 돌아간다.

****409 MBR-4102****(코드 불일치): 2단계로 되돌리고 코드 인라인 오류를 표시하되 ****입력한 새 비밀번호 값은 상태에 유지****(코드만 고치면 3단계 재진입 시 다시 칠 필요 없음).

****400 MBR-4001****(비밀번호 규칙 오류): 3단계 유지, 비밀번호 필드 인라인 오류.

****400 MBR-4100****(방어적 — 정상 흐름에서는 도달하지 않음): 1단계로, 계정 필드 인라인 오류.

****그 외(미정의·네트워크·5xx, 예 MBR-5000)****: 3단계를 유지한 채 재시도 가능한 인라인 오류만 표시(1단계로 되돌리지 않는다 — 입력한 코드·새 비밀번호를 보존).

5. ****4단계(완료)****: "비밀번호가 변경되었습니다 — 모든 기기에서 로그아웃되었습니다" 안내 + [로그인으로 이동] 링크.

****부가 시나리오: 재전송·만료·시도초과 되돌이표****

1. 2단계에서 유효시간이 다 되기 전에 [재전송]을 누르면 같은 target으로 코드를 다시 받아 카운트다운을 재시작한다(\$2-3).

2. 카운트다운이 남아 있어도 3단계에서 confirm이 410/409(4103)을 돌려주면 그 즉시 2단계 "만료/시도초과" 화면으로 강제 전환된다 — 여기서부터는 [다시 요청]으로 1단계부터 다시 시작하는 것 외엔 진행할 수 없다.

3. 화면 구성 (블록)

- > 4단계는 서로 다른 라우트가 아니라 **한 라우트(`/password-reset`) 안의 상태기계**다(컨테이너의
- > `step` state로 스위치 렌더, `PasswordResetPage.tsx:199-225`). 탭이 아니라 배타적 화면 상태이므로
- > 마커 ②~⑤는 "동시에 보이지 않는" 블록이다.

마커	블록	역할	주요 위젯	소스 근거
①	단계 헤더	제목 + 진행 표시 (1/3·2/3·3/3·완료)	`<h1>`, 단계 라벨 텍스트	`PasswordResetPage.tsx:192-202`
②	1단계: 계정 확인 폼	이메일/휴대폰 입력 + 코드 요청	target input, 코드 요청 버튼	`PasswordResetRequestStep.tsx`
③	2단계: 코드 확인 폼	6자리 코드 입력 + 유효시간/재전송	code input, 다음 버튼, 재전송 버튼, 다시요청 버튼(오류 시)	`PasswordResetCodeStep.tsx`
④	3단계: 새 비밀번호 폼	새 비밀번호 입력 + 제출	password input, 비밀번호 변경 버튼	`PasswordResetPasswordStep.tsx`
⑤	4단계: 완료	안내 문구 + 로그인 이동	안내 텍스트, 로그인 이동 링크	`PasswordResetDoneStep.tsx`

4. 위젯·액션

① 단계 헤더

번호	위젯	타입	레이블	동작	연결 API(raw)	결과
(1)	단계 라벨	text	"1/3 · 계정 확인" 등 4종	`step` state에 따라 표시 전용	—	—

② 1단계: 계정 확인 폼

번호	위젯	타입	레이블	동작	연결 API(raw)	결과
(2)	target input	input(text)	이메일 또는 휴대폰번호	입력값 갱신(`onChange`)	—	—
(3)	코드 요청 버튼	button(type=submit)	코드 요청 / 요청 중...	폼 제출 → `handleRequestSubmit`(내부에서 `normalizeTargetForSend`로 정규화 후 `requestCodeOnce` 단일비행 가드 경유)	POST /api/members/password-resets/codes	202: 2단계로 전이 (target-channel-expiresInSeconds 저장, 카운트다운 시작) / 400 MBR-4100: 폼 유지+인라인 오류

③ 2단계: 코드 확인 폼

번호	위젯	타입	레이블	동작	연결 API(raw)	결과
(4)	안내 문구	text	"(이메일휴대폰) (target)로 발송된 6	표시 전용	—	—

			자리 코드를 입력하세요"			
(5)	남은 유효시간 문구	text	"남은 유효시간 N초"	`remainingSeconds`(백시계 파생) 표시, 만료/시도 초과 오류 시 숨김	—	—
(6)	code input	input(numeric, maxLength=6)	인증코드	숫자만 필터링해 입력값 갱신, 6자리 숫자 형식이면 로컬 유효	—	—
(7)	다음 버튼	button	다음	로컬 형식 검증(6자리 숫자) 통과 시에만 활성화 → 클릭 시 `handleCodeNext`	— (서버 호출 없음)	3단계로 전이(코드 실제 검증은 3단계 confirm 호출에 포함)
(8)	재전송 버튼	button	재전송 / 재전송 (N초)	`resendRemainingSeconds`가 0일 때만 활성화 → 클릭 시 `handleResend`	POST /api/members/password-resets/codes	성공: 카운트다운. 쿨다운 재시작 + 옛 코드/오류 초기화 / 실패: 조용히 무시(기존 상태 유지)
(9)	다시 요청 버튼(오류 시)	button	다시 요청	만료(MBR-4101)-시도 초과(MBR-4103) 오류일 때만 표시 → `handleRestart`	—	전체 상태 초기화 후 1단계로
(10)	코드 오류 인라인 (role=alert)	div	(서버 message)	`error.code === 'MBR-4102'`일 때만 표시, 입력은 유지	—	—

****④ 3단계: 새 비밀번호 폼****

번호	위젯	타입	레이블	동작	연결 API(raw)	결과
(11)	password input	input(password)	새 비밀번호	입력값 갱신 (`onChange`)	—	—
(12)	비밀번호 변경 버튼	button (type=submit)	비밀번호 변경 / 변경 중...	폼 제출 → `handlePasswordSubmit` (target+code+newPassword 원자 전송)	POST /api/members/password-resets/confirmations	\$2 응답코드 매핑 참조(204→완료 / 410-409-4103→2단계 다시요청 / 409-4102→2단계 코드오류 (비번유지) / 400-4001→3단계 유지+인라인 / 400-4100→1단계(방어적) / 그외→3단계 유지+인라인
(13)	비밀번호 오류 인라인 (role=alert)	div	(서버 message)	`error`가 non-null일 때 표시(규칙오류·방어적 400-미정의 오류 공용)	—	—

****⑤ 4단계: 완료****

번호	위젯	타입	레이블	동작	연결 API(raw)	결과
----	----	----	-----	----	-------------	----

					w)	
(14)	안내 문구	text	"비밀번호가 변경되었습니다 — 모든 기기에서 로그아웃되었습니다"	표시 전용	—	—
(15)	로그인으로 이동	a href="#/login"	로그인으로 이동	순수 앵커 이동(react-router `Link`가 아니라 ` <a>` — Storybook에서 Router 컨텍스트 없이도 렌더되게 하려는 의도적 선택)	—	`#/login`으로 이동

5. 접근 권한·표시 조건

> 4개 부품 각각의 `stories.tsx`가 이 화면의 상태를 1급으로 남긴다(총 12개 상태). 한 행 = 한 스토리.

요소	표시 조건	근거	스토리
1단계 기본(빈 입력)	초깃값(`target: ""`)	`PasswordResetRequestStep.tsx`, stories: ./src/features/member/PasswordResetRequestStep.stories.tsx	[기본](story:회원-비밀번호-재설정-1단계-요청--기본)
1단계 입력됨	target에 값이 채워짐, 오류 없음	위와 동일	[입력됨](story:회원-비밀번호-재설정-1단계-요청--입력됨)
1단계 제출중(버튼 비활성)	`busy === true`	`PasswordResetRequestStep.tsx:29,41`	[제출중](story:회원-비밀번호-재설정-1단계-요청--제출중)
1단계 형식오류 인라인	`error?.code === 'MBR-4100'`	`PasswordResetRequestStep.tsx:38-40`	[형식오류](story:회원-비밀번호-재설정-1단계-요청--형식오류)
2단계 기본(카운트다운-쿨다운 진행중)	`remainingSeconds > 0`, `resendRemainingSeconds > 0`, `error` 없음	`PasswordResetCodeStep.tsx:55-88`	[기본](story:회원-비밀번호-재설정-2단계-코드--기본)
2단계 재전송 활성화	`resendRemainingSeconds === 0`	`PasswordResetCodeStep.tsx:82-87`	[재전송쿨다운끝](story:회원-비밀번호-재설정-2단계-코드--재전송쿨다운끝)
2단계 코드오류 (입력 유지)	`error?.code === 'MBR-4102'`	`PasswordResetCodeStep.tsx:73-75`	[코드오류](story:회원-비밀번호-재설정-2단계-코드--코드오류)
2단계 만료(다시 요청만 표시)	`error?.code === 'MBR-4101'`	`PasswordResetCodeStep.tsx:48,58-66`	[만료](story:회원-비밀번호-재설정-2단계-코드--만료)
2단계 시도초과 (다시 요청만 표시)	`error?.code === 'MBR-4103'`	`PasswordResetCodeStep.tsx:48,58-66`	[시도초과](story:회원-비밀번호-재설정-2단계-코드--시도초과)
3단계 기본	초깃값(`value: ""`)	`PasswordResetPasswordStep.tsx`	[기본](story:회원-비밀번호-재설정-3단계-새-비밀번호--기본)
3단계 제출중(버튼 비활성)	`busy === true`	`PasswordResetPasswordStep.tsx:30,42`	[제출중](story:회원-비밀번호-재설정-3단계-새-비밀번호--제출중)
3단계 비밀번호 규칙오류 인라인	`error?.code === 'MBR-4001'`(그 외 미정의 코드도 같은 인라인 자리에 표시됨)	`PasswordResetPasswordStep.tsx:39-41`	[비밀번호규칙오류](story:회원-비밀번호-재설정-3단계-새-비밀번호--비밀번호규칙오류)

4단계 완료	confirm 204 수신 직후('step === 'done')	'PasswordResetDoneStep.tsx', 'PasswordResetPage.tsx:157-158'	[기본](story:회원-비밀번호-재설정-4 단계-완료--기본)
--------	-------------------------------------	---	-------------------------------------

6. 팝업·연계 화면

팝업/모달 없음. 다만 이 화면과 다른 화면 사이의 SPA 내 이동이 둘 있다:

트리거 위젯	연계 화면	연결 API/화면(raw)	용도
로그인 화면의 `Link to="/password-reset"``	이 화면(1단계)	GET /#/password-reset (INF 없음 — 화면 전환)	비밀번호를 잊은 회원의 진입점 ('LoginPage.tsx:45-47')
4단계 ` <a href="#/login">`	UIS-MBR-002(로그인)	GET /#/login (INF 없음 — 화면 전환)	재설정 완료 후 재로그인 유도

7. 데이터 출처·연결

****연결 API(raw → INF):****

'POST /api/members/password-resets/codes' — INF-MBR-006. 1단계 제출 + 2단계 재전송이 공유 호출

('requestCodeOnce' 단일 비행 가드로 같은 target의 동시 호출을 중복 발사하지 않는다).

'POST /api/members/password-resets/confirmations' — INF-MBR-007. 3단계 제출의 유일한 호출.

코드 검증과 비밀번호 반응을 원자 처리한다(별도의 "코드 확인 API"는 없다).

이 화면은 두 API의 요청/응답 계약을 그대로 소비만 한다 — INF-MBR-006/007 본문이 정본이며 여기서 재정의하지 않는다.

****로컬 부수효과(API 아님):**** confirm 204 성공 직후 'clearSession()'('src/session.ts', FUNC-member-004 소유)을 호출해 이 기기의 localStorage 세션(apiKey/refreshToken)을 지운다 — 서버가 이미 모든 기기의 리프레시 토큰/API 키를 폐기했으므로, 이 화면의 로컬 상태를 그 사실과 맞추기 위한 자기 정리다.

****참조 테이블(SCH):**** 이 화면 자체는 DB를 직접 호출하지 않는다(모두 INF-MBR-006/007 경유). 참조 테이블은 해당 INF 문서가 정본 — 'MEMBER_PASSWORD_RESETS', 'MEMBERS'.

8. 미확인 사항

****완료(4단계) 화면에서도 카운트다운 인터벌이 계속 돈다**** — 'PasswordResetPage.tsx:76-81'의

'useEffect'가 'requestSeq > 0'인 한 'step'과 무관하게 매초 'setNow'를 갱신한다(재작업 지시 1로

"단계와 무관하게 흐르게" 고친 부작용). 언마운트 시 cleanup되어 누수는 아니고 무의미한 리렌더

비용만 있다 — QA가 남긴 후속 TODO(low, 게이트 비차단, 'STORY-FUNC-member-007.md' "후속 TODO" 2번).

****'clearSession()'이 계정을 가리지 않는다**** — 로그인 상태(계정 A)에서 다른 계정(B)의 비밀번호를

재설정하면 A의 로컬 세션까지 지워진다. 세션에 email이 없어 'normalizedTarget'과 대조할 수단이

현재 계약엔 없다. QA 판단: "더 로그아웃"이라 안전 방향이며 재로그인으로 즉시 복구됨(low, 후속 TODO 3번).

****실제 이메일/SMS 발송 게이트웨이 연동****은 서버가 로그 시뮬레이션으로 대체하고 있다 — 프론트가
 관여할 부분이 없다(STORY "범위 밖").

****비밀번호 재설정 링크(URL 클릭형) 방식****은 구현되지 않았다 — 이번 SR은 "코드" 방식만 채택했다
 (STORY "범위 밖", INF-MBR-006/007도 코드 기반).

보호되지 않은 라우트(비로그인 전용 화면이라 로그인 여부와 무관하게 접근 가능)의 접근 제어는 이
 FUNC 범위가 아니다(STORY에 명시된 확정 사항).

화면 상태 13건 — 기준 스냅샷

이메일(user@example.com)로 발송된 6자리 코드를 입력하세요.

남은 유효시간 540초

인증코드

다음

재전송 (42초)

2단계 코드 · 기본

이메일(user@example.com)로 발송된 6자리 코드를 입력하세요.

남은 유효시간 480초

인증코드

다음

재전송

2단계 코드 · 재전송쿨다운끝

이메일(user@example.com)로 발송된 6자리 코드를 입력하세요.

남은 유효시간 300초

인증코드

코드가 올바르지 않습니다

다음

재전송

2단계 코드 · 코드오류

이메일(user@example.com)로 발송된 6자리 코드를 입력하세요.

인증코드가 만료되었습니다. 다시 요청해 주세요

다시 요청

2단계 코드 · 만료

이메일(user@example.com)로 발송된 6자리 코드를 입력하세요.

코드 확인 시도 횟수를 초과했습니다. 다시 요청해 주세요

다시 요청

2단계 코드 · 시도초과

비밀번호가 변경되었습니다 — 모든 기기에서 로그아웃되었습니다.

[로그인으로 이동](#)

4단계 완료 · 기본

새 비밀번호

비밀번호 변경

3단계 새 비밀번호 · 기본

새 비밀번호

변경 중...

3단계 새 비밀번호 · 제출중

새 비밀번호

비밀번호 형식이 올바르지 않습니다

비밀번호 변경

3단계 새 비밀번호 · 비밀번호규칙오류

이메일 또는 휴대폰번호

코드 요청

1단계 요청 · 기본

이메일 또는 휴대폰번호

코드 요청

1단계 요청 · 입력됨

이메일 또는 휴대폰번호

요청 중...

1단계 요청 · 제출중

이메일 또는 휴대폰번호

이메일 또는 휴대폰번호 형식이 올바르지 않습니다

코드 요청

1단계 요청 · 형식오류

UIS-MBR-004 shop_mypage_addresses

이 화면 설계서에는 화면 그림이 없습니다 — 구성·위젯·연결 API는 소스에서 읽어 채웠습니다.

UIS-MBR-004: shop_mypage_addresses

> [반영: SR-235.1] — 코드(라우트) 기준 재동기화 골격 + 소스 기준 본문 보강(ddd-ui-agent). 근거 소스(권위):

> `modules/shop-web/src/pages/MyAddressesPage.tsx` 외 `features/member/\*`의 `api.ts`·`types.ts`.

> 캡처/DOM 스냅샷 없음(Storybook 미기동) — **소스폴백 모드**로 작성, 0(화면 미리보기) 절 생략.

1. 화면 개요

라우트: `/shop/mypage/addresses` (spa-route, `modules/shop-web/src/App.tsx:113`)

목적: 회원이 자신의 배송지를 조회·등록·수정·삭제하고 기본 배송지를 지정하는 마이페이지 화면. 회원당

최대 10개, 기본 배송지는 항상 1개(등록 시 최초 배송지는 서버가 기본으로 강제, 기본 배송지 삭제 시

최근 사용 순 다음 배송지가 자동 승계). 백엔드(INF-MBR-008 배송지 CRUD, INF-MBR-009 우편번호 검색)는

이번 SR 이전에 이미 구현되어 있고, 이 화면은 그 위에 얹는 순수 프론트(HTTP 클라이언트)다.

진입 경로: 쇼핑 GNB의 "마이페이지 > 배송지 관리" 링크(로그인 상태에서만 노출, `Gnb.tsx:54-57`) 또는

URL 직접 접근. 마이페이지 허브 화면은 없다 — 이 라우트 1개가 유일한 진입점.

비로그인 접근: 세션 없으면 즉시 `/login?redirect=%2Fshop%2Fmypage%2Faddresses`로 리다이렉트하고,

로그인 성공 후 기존 `redirectTarget` 규약으로 이 화면에 복귀한다(`MyAddressesPage.tsx:61-65`).

구현 근거: SR-235.1, `docs/변경관리/SR-235/STORY-1.md`(STORY-FUNC-member-010).

2. 화면 구성 (블록)

마커	블록	역할	주요 위젯	소스 근거
①	GNB(공통)	로그인 상태 표시·검색·장바구니 배지·마이페이지 링크·로그아웃	`Gnb`	`Gnb.tsx:54-57`
②	배송지 목록	조회 결과(최근 사용 순, 서버 정렬 그대로 렌더) + 등록 진입	헤더(건수)[추가]·행(기본배지/수령인·연락처/주소/[기본으로 설정]/[수정]/[삭제])	`AddressList.tsx`
③	등록·수정 폼 (목록 하단 인라인)	배송지 입력/수정	수령인·연락처·우편번호(읽기전용+찾기버튼)·도로명(읽기전용)·상세주소·공동현관출입방법·배송요청사항·기본설정 체크박스(등록만)·저장/취소	`AddressForm.tsx`, `MyAddressesPage.tsx`: 264-271`
④	우편번호 검색 모달(오버레이)	도로명/건물명으로 검색해 우편번호·도로명주소 선택	검색어 입력+검색, 결과 목록/0건 안내, 닫기	`ZipcodeSearchModal.tsx` (기존 부품 재사용, 수정 없음)

⑤	삭제 확인 다이얼로그(오버레이)	삭제 전 확인 + 기본배송지 승계 안내	대상 요약(수령인·도로명주소)·[취소]·[삭제]	`AddressDeleteConfirm.tsx`
---	-------------------	-----------------------	----------------------------	----------------------------

목록·폼·모달·삭제확인 4개 블록은 전부 fetch를 직접 하지 않는 표시 전용 부품이며(규칙

`web-fetch-only-in-api`), 실제 API 호출은 전부 컨테이너 `MyAddressesPage.tsx`가 오케스트레이션한다.

3. 입력·검증 규칙

****등록/수정 폼(③) 필드****

필드	필수	입력 방식	비고
수령인 (recipient)	O	텍스트 입력	미입력 시 "수령인을 입력해 주세요"
연락처 (phone)	O	텍스트 입력	미입력 시 "연락처를 입력해 주세요"
우편번호 (zipcode) + 도로명주소 (roadAddresses)	O(한 쌍)	**읽기전용** — [우편번호 찾기] 클릭 → 모달(④)에서 선택해야만 채워짐, 직접 타이핑 불가	둘 중 하나라도 비면 "우편번호 찾기로 주소를 선택해 주세요"(zipcode 필드 오류로 통합)
상세주소 (detailAddress)	O	텍스트 입력	미입력 시 "상세주소를 입력해 주세요"
공동현관 출입 방법 (entranceMethod)	선택	텍스트 입력	검증 없음
배송 요청사항 (deliveryMemo)	선택	텍스트 입력	검증 없음
기본 배송지로 설정 (isDefault)	선택, **등록 모드에서만 노출**	체크박스	수정 모드는 렌더 자체를 하지 않음(기본 지정은 전용 API `setDefaultAddress`만 담당, `AddressForm.tsx:100-108`). 현재 목록이 0건(최초 배송지)이면 `disabled + checked` 강제 표시 + "첫 배송지는 자동으로 기본 배송지로 설정됩니다" 안내(서버가 여차피 강제하므로 사용자가 끌 수 없음을 미리 고지, `isFirstAddress` prop)

****클라이언트 우선 검증****: [저장] 클릭 시 `validateDeliveryAddress`(`deliveryAddressValidation.ts`, 주문서와

공유하는 순수 함수)가 수령인·연락처·zipcode·상세주소를 검사하고, 하나라도 실패하면 API를 호출하지 않고

필드별 인라인 오류만 표시한다(`MyAddressesPage.tsx:125-131`).

****서버 최종 검증****(클라이언트가 못 잡는 형식 오류의 최종 방어선): 필수값 누락 `400 MBR-4200` — 서버는

필드별로 나누지 않고 문장 하나만 주므로 폼 하단 일반 오류 영역에 그대로 표시(`saveError`, 필드 매핑 없음).

****등록 상한****: 11번째 등록 시도 → `409 MBR-4201`("배송지는 최대 10개") 인라인 표시. 목록이 10건이면

애초에 [추가] 버튼이 비활성화되어 폼 진입 자체가 막힌다(\$5).

****존재/소유 오류****: 수정·삭제 대상이 없거나 남의 배송지면 `404 MBR-4041` — 두 경우를 구분하지 않고 서버 메시지를 그대로 표시(존재 오라클 방지, 서버 `selectOwned` 단일 SQL이 판정).

****우편번호 검색어****: 2자 미만이면 `400 MBR-4202`("검색어" 관련 안내) — `searchZipcodes` 호출 결과로 받아 모달 내 인라인 오류(`error` prop)로 표시.

4. 호출 API

- > 인증: 아래 배송지(`/api/members/me/addresses`) 5개 호출은 전부 `X-API-Key: session.apiKey` (회원 자신의 키)를 명시 헤더로 싣는다. `memberId`는 쿼리로 넘기지 않는다 — `ApiKeyAuthFilter`의 `SCOPE\_TO\_SELF`가
- > 헤더의 키로 회원을 강제한다(`api.ts:288-309`). 우편번호 검색은 인증 불요(공개 API).

트리거	메서드/엔드포인트(raw)	함수 (`api.ts`)	결과
화면 진입(세션 확인 후 자동)	GET /api/members/me/addresses	`fetchAddresses`	`{items}` 봉투를 풀어 배송지 배열 반환(0건도 200, 최근 사용 순은 서버 정렬)
[저장](등록 모드)	POST /api/members/me/addresses	`registerAddress`	201 + 생성된 배송지. 로컬 목록 맨 앞에 추가(재조회 없음), 기본이 됐으면 다른 행의 배치를 로컬에서 내림
[저장](수정 모드)	PUT /api/members/me/addresses/{addressId}	`updateAddress`	200 + 수정된 배송지. 로컬에서 해당 행 치환. `isDefault`는 서버가 무시
[삭제] 확인	DELETE /api/members/me/addresses/{addressId}	`deleteAddress`	204(본문 없음). 삭제 대상이 **기본이 아니었으면** 로컬 목록에서 filter, **기본이었으면** 승계된 새 기본을 서버가 알려주지 않으므로 `fetchAddresses`로 **재조회**
[기본으로 설정]	PUT /api/members/me/addresses/{addressId}/default	`setDefaultAddress`	200 + 갱신된 배송지. 응답 행으로 갱신 + 로컬에서 이전 기본 행의 `isDefault`를 `N`으로 내림(이미 기본인 행은 버튼 자체를 숨겨 호출을 만들지 않음)
[우편번호 찾기] → 검색	GET /api/zipcodes?q={검색어}	`searchZipcodes`	`{items}` 봉투. 결과 있음/0건 모두 200, 검색어 2자 미만은 400 MBR-4202

연타 방지: 삭제·기본설정은 `inFlightAddressIdRef` (단일 ref, addressId 보관)로 같은 대상에 대한 중복 호출을 막는다(`MyAddressesPage.tsx:57,175,200`). 목록 로드는 `inFlightKeyRef` + `requestIdRef`로 StrictMode 이중 실행에 안전하다.

5. 표시 조건(상태)

- > 스토리북 스토리가 있으면 `[이름](story:ID)`로 연결(`.speclinker/storybook\_index.json` 기준). `ZipcodeSearchModal`은
- > 기존 재사용 부품이라 스토리가 `UIS-ORD-011` (주문서)에 연결돼 있지만 이 화면도 수정 없이 그대로 쓴다.

요소	표시 조건	근거	스토리
----	-------	----	-----

로그인 게이트	`session.memberId` 없음 → `/login?redirect=...`로 즉시 이동, 목록 조회조차 안 함	`MyAddressesPage.tsx:61-65`	(컨테이너, 스토리 대상 아님)
목록 0건	`addresses.length === 0` → "등록된 배송지가 없습니다" + [추가] 활성화	`AddressList.tsx:56-59`	[목록 0건](story:마이페이지-배송지-목록--목록-0건)
목록 1건 이상	각 행에 수령인·연락처·주소, `isDefault===`Y`인 행만 "기본" 배지	`AddressList.tsx:67-71`	[목록 1건](story:마이페이지-배송지-목록--목록-1건)
목록 10건 상한	`addresses.length >= 10` → [추가] disabled + "배송지는 최대 10개까지 등록할 수 있습니다" 안내	`AddressList.tsx:38,44-54`	[목록 10건상한](story:마이페이지-배송지-목록--목록-10건상한)
기본 배송지 전환 중	해당 행이 `pendingAddressId`와 일치 → "처리 중..." 표시 + 그 행 버튼 전부 disabled	`AddressList.tsx:63,74-89`	[기본배송지전환중](story:마이페이지-배송지-목록--기본배송지전환중)
폼 — 신규	`formMode.kind==='add`, 빈 값에서 시작, isDefault 체크박스 노출	`MyAddressesPage.tsx:102-107`, `AddressForm.tsx:101-113`	[신규](story:마이페이지-배송지-폼--신규)
폼 — 수정	`formMode.kind==='edit`, 기존 값 prefill, isDefault 체크박스 숨김	`MyAddressesPage.tsx:109-120`, `AddressForm.tsx:100`	[수정](story:마이페이지-배송지-폼--수정)
폼 — 검증 오류	필수값 누락 필드마다 인라인 오류(수령인/연락처/우편번호/상세주소)	`deliveryAddressValidation.ts:18-25`, `AddressForm.tsx` errors 표시부	[검증오류](story:마이페이지-배송지-폼--검증오류)
우편번호 검색 — 결과 있음	`results.length > 0` → 클릭 가능한 목록	`ZipcodeSearchModal.tsx:56-69`	결과있음(주문서 스토리 재사용, UIS-ORD-011)
우편번호 검색 — 0건	`results.length === 0` → "검색 결과가 없습니다"	`ZipcodeSearchModal.tsx:70-72`	결과없음(주문서 스토리 재사용, UIS-ORD-011)
삭제 확인 — 일반	`deleteTarget !== null`, `isDefault === 'N'` → 대상 요약만	`AddressDeleteConfirm.tsx:33-37`	[일반삭제확인](story:마이페이지-배송지-삭제-확인--일반삭제확인)
삭제 확인 — 기본 배송지	`deleteTarget.isDefault === 'Y'` → "삭제하면 다음 배송지가 자동으로 기본이 됩니다" 안내 추가	`AddressDeleteConfirm.tsx:38-42`	[기본배송지삭제확인](story:마이페이지-배송지-삭제-확인--기본배송지삭제확인)
API 오류(공통 인라인)	목록 로드 실패(`loadError`)·행 액션 실패(`actionError`)·저장 실패(`saveError`) — 코드별 문구 그대로(MBR-4200/4041/4201/4202/5000)	`MyAddressesPage.tsx` 각 `.catch(toDisplayMessage)`	(정적 props 스토리 대상 아님 — 통합테스트로 커버)

> 보강: `/sl-sync --apply --kind=uis` 또는 ddd-ui-agent

변경 이력

<!-- spec-history: machine-managed -->

일자	SR	항목	변경 요약	커밋
2026-09-19	SR-235	#1	마이페이지 배송지 관리 화면 신규 구현(목록·등록/수정 폼·우편번호 검색 모달·삭제 확인·기본배송지 전환) — 화면설계서 본문 코드 기준 보강	shop-api@1c82e8c, shop-web@8e1e3e3

2026-09-19	SR-235	#2	본문 보강(ddd-ui-agent, 소스폴백 — 화면개요/구성/검증규칙/호출API/표시조건)	shop-api@1c82e8c, shop-web@8e1e3e3
------------	--------	----	---	------------------------------------

화면 상태 7건 — 기준 스냅샷

수령인

연락처

우편번호

우편번호 찾기

상세주소

공동현관 출입 방법

배송 요청사항

☐ 기본 배송지로 설정

저장

취소

배송지 품 · 신규

수령인

홍길동

연락처

010-1234-5678

우편번호

06236

우편번호 찾기

서울특별시 강남구 테헤란로 123

상세주소

101동 202호

공동현관 출입 방법

공동현관 비밀번호 1234

배송 요청사항

부재 시 경비실에 맡겨주세요

저장

취소

배송지 품 · 수정

수령인

수령인을 입력해 주세요

연락처

연락처를 입력해 주세요

우편번호

우편번호 찾기

우편번호 찾기로 주소를 선택해 주세요

상세주소

상세주소를 입력해 주세요

공동현관 출입 방법

배송 요청사항

☐ 기본 배송지로 설정

저장

취소

배송지 품 · 검증오류

배송지 관리 (0)

추가

등록된 배송지가 없습니다

배송지 목록 · 목록 0건

배송지 관리 (1)

추가

기본

수령인1 · 010-1234-1001
서울 강남구 테헤란로 11동 1호

수정

삭제

배송지 목록 · 목록 1건

배송지 관리 (10)

추가

배송지는 최대 10개까지 등록할 수 있습니다

기본

수령인1 · 010-1234-1001 서울 강남구 테헤란로 11동 1호	수정	삭제	
수령인2 · 010-1234-1002 서울 강남구 테헤란로 2 2동 2호	기본으로 설정	수정	삭제
수령인3 · 010-1234-1003 서울 강남구 테헤란로 3 3동 3호	기본으로 설정	수정	삭제
수령인4 · 010-1234-1004 서울 강남구 테헤란로 4 4동 4호	기본으로 설정	수정	삭제
수령인5 · 010-1234-1005 서울 강남구 테헤란로 5 5동 5호	기본으로 설정	수정	삭제
수령인6 · 010-1234-1006 서울 강남구 테헤란로 6 6동 6호	기본으로 설정	수정	삭제
수령인7 · 010-1234-1007 서울 강남구 테헤란로 7 7동 7호	기본으로 설정	수정	삭제
수령인8 · 010-1234-1008 서울 강남구 테헤란로 8 8동 8호	기본으로 설정	수정	삭제
수령인9 · 010-1234-1009 서울 강남구 테헤란로 9 9동 9호	기본으로 설정	수정	삭제
수령인10 · 010-1234-1010 서울 강남구 테헤란로 10 10동 10호	기본으로 설정	수정	삭제

배송지 목록 · 목록 10건상한

배송지 관리 (2)

추가

기본

수령인1 · 010-1234-1001

서울 강남구 테헤란로 11동 1호

수령인2 · 010-1234-1002

서울 강남구 테헤란로 2 2동 2호

처리 중...

기본으로 설정

수정

삭제

배송지 목록 · 기본배송지전환중

UIS-MBR-005 member_signup

이 화면 설계서에는 화면 그림이 없습니다 — 구성·위젯·연결 API는 소스에서 읽어 채웠습니다.

UIS-MBR-005: member_signup

> 코드(라우트) 기준 재동기화 골격(zero-LLM) — 화면 구성·상태·검증 규칙은 보강 대상

1. 화면 개요

라우트: `/member/signup` (server-view, `modules/shop-api/src/main/java/com/sm/lab/shop/controller/MemberSignupViewController.java:44`)
목적: [TBD]

2. 화면 구성

[TBD]

3. 입력·검증 규칙

[TBD]

4. 호출 API

[TBD]

5. 표시 조건(상태)

[TBD] — 스토리북 스토리가 있으면 `[이름](story:ID)`로 연결

> 보강: `/sl-sync --apply --kind=uis` 또는 ddd-ui-agent

UIS-MBR-006 member_상세

이 화면 설계서에는 화면 그림이 없습니다 — 구성·위젯·연결 API는 소스에서 읽어 채웠습니다.

UIS-MBR-006: member_상세

> 코드(라우트) 기준 재동기화 골격(zero-LLM) — 화면 구성·상태·검증 규칙은 보강 대상

1. 화면 개요

라우트: `/member/{\*}` (server-view, `modules/shop-api/src/main/java/com/sm/lab/shop/controller/MemberViewController.java:51`)
목적: [TBD]

2. 화면 구성

[TBD]

3. 입력·검증 규칙

[TBD]

4. 호출 API

[TBD]

5. 표시 조건(상태)

[TBD] — 스토리북 스토리가 있으면 `[이름](story:ID)`로 연결

> 보강: `/sl-sync --apply --kind=uis` 또는 ddd-ui-agent

UIS-ORD-001 주문 목록

주문 목록

회원 ID 주문상태 검색

주문번호	회원	상태	총액	주문일시
260817-0003	이도그 (M-0002)	CANCELED	450,000원	2026-08-17 09:40
260817-0001	박무당 (M-0003)	PLACED	129,000원	2026-08-17 09:00
260816-0003	김של용 (M-0001)	PARTIAL_SHIPPED	579,000원	2026-08-16 16:05
260816-0001	이도그 (M-0002)	PAID	390,000원	2026-08-16 14:30
260815-0001	김של용 (M-0001)	DONE	519,000원	2026-08-15 10:12

전체 5건

UIS-ORD-001: 주문 목록

> **근거 소스(권위):** `src/main/resources/templates/order/list.html` +

> `src/main/java/com/sm/lab/shop/controller/OrderViewController.java`. 소스폴백 모드(스크린샷 없음).

1. 화면 목적

회원 ID·주문상태·조회기간(시작일~종료일)으로 주문 내역을 검색하고, 주문번호·회원·상태·총액·주문일시·배송상태를 목록으로 조회하는 화면이다. 각 주문번호는 주문 상세 화면으로 연결된다.

> **배송상태(6번째 열, SR-210 추가):** 값은 해당 주문의 **최신 배송 이력 상태**이며, 배송 이력이 없으면 "-"로 표시한다. INF-ORD-003(GET /api/orders) 응답 계약은 바뀌지 않는다(확정 문답 scope_freeze, api_compat) — 이 값은 REST API 응답이 아니라 화면 조회 경로(`OrderViewController.orderList` → `OrderService.latestDeliveryStatesByOrderNo`)에서 별도로 조합된다. "최신"은 배송 이력이 가장 최근에 **생성된** 건을 의미하며(배송 이력 생성 순서 = `delivery\_no` DESC 단독 기준 — `ORDER\_DELIVERY`에 별도 등록시각 컬럼이 없어 배송번호 발번 순서를 대리 지표로 사용), 출고일시(`shipped\_at`) 기준이 아니다. 이 정의는 FUNC-order-006(주문 상세 화면의 배송 이력 정렬)과 다르다 — 그 화면은 출고일시(`shipped\_at`) 기준 정렬이 목적에 맞지만, 이 화면은 미출고(READY, shipped_at NULL) 건이 더 나중에 생성됐어도 항상 뒤로 밀리는 결함이 있어 별도로 재정의했다.

2. 주요 작업 시나리오

시나리오: 주문 목록 조회

1. 회원 ID(`memberId`), 주문상태(`orderState`), 시작일(`startDate`), 종료일(`endDate`)을 필요한 만큼 입력한다(모두 선택 입력).
2. [검색] 버튼(`#btnSearch`)을 눌러 `GET /order/list`로 재요청한다 — 입력한 조건은 모두 AND로 결합된다.
3. 조회 결과가 표에 채워지고, 총 건수가 하단에 표시된다. 결과가 0건이면 "조회 결과 없음" 안내가 표시된다.
4. 조회 구간이 역전(시작일 > 종료일) 등으로 서버가 4xx를 반환하면, 화면 상단에 "조회 실패: {사유}" 오류 배너만 표시되고 주문목록 그리드 "조회 결과 없음" 안내 "전체 N건" 총건수 라벨은 모두 숨겨진다(5xx는 그대로 전파되어 이 화면에서 흡수하지 않음).
5. 목록의 주문번호 링크를 클릭하면 `/order/{orderNo}` 주문 상세 화면으로 이동한다.

3. 화면 구성 (블록)

마커	블록	역할	주요 위젯	소스 근거
①	오류 배너	4xx 조회 실패 안내(조건부 표시)	안내 문구	`list.html:25-27`
②	검색조건	회원ID·주문상태·조회기간 필터	memberId, orderState, startDate, endDate, 검색 버튼	`list.html:28-44`
③	주문목록 그리드	조회결과 표시(4xx 시 숨김)	주문번호 링크, 회원, 상태, 총액, 주문일시, **배송상태 (SR-210)**	`list.html:51-65`
④	결과 요약	빈 결과 안내 + 전체 건수(4xx 시 숨김)	안내 문구, 전체 건수	`list.html:68-70`

4. 위젯·액션

② 검색조건

번호	위젯	타입	레이블	동작	연결 API(raw)	결과
(1)	`memberId`	input(text)	회원 ID	검색 파라미터 입력	—	—
(2)	`#selState` (`orderState`)	select	주문 상태	전체/PLACED/PAID/SHIPPED/PARTIAL_SHIPPED/CANCELLED/DONE 중 선택	—	—
(3)	`#startDate`	input(date)	시작 일	조회기간 시작	—	—
(4)	`endDate`	input(date)	종료 일	조회기간 종료	—	—
(5)	`#btnSearch`	button(submit)	검색	폼 제출 → 목록 재조회	GET /order/list	목록 갱신 또는 오류 배너 표시

③ 주문목록 그리드

번호	위젯	타입	레이블	동작	연결 API(raw)	결과
(6)	주문번호 링크	a	주문 번호	클릭 시 상세 이동	GET /order/{orderNo}	주문 상세 화면(UIS 미생성) 이동
(7)	배송상태 열 (SR-210)	td(text)	배송 상태	표시 전용(액션 없음)	— (화면 조회 경로 전용 조합값, INF-ORD-003 응답 아님)	해당 주문의 최신 배송 이력 상태, 이력 없으면 "-"

5. 접근 권한·표시 조건

요소	표시 조건	근거	스토리
----	-------	----	-----

오류 배너	`searchError`가 null이 아닐 때(4xx 조회 거부)	`list.html:25-27`, `OrderViewController.java:53-64`	
주문목록 그리드	`searchError`가 null일 때만 렌더(4xx 시 숨김)	`list.html:51`	
배송상태 열(SR- 210)	그리드가 보이는 동안 각 행에 항상 표시 — 신규 예외 조건 없음. 값은 `deliveryStatusByOrderNo.get(o.orderNo)`, 키가 없으면 (이력 없음) "-"	`list.html:62`, `OrderViewController.java:74`, `OrderService.latestDeliveryStatesBy OrderNo`	
"조회 결 과 없음" 문구	`searchError`가 null이고 `result['items']`가 빈 목록일 때 (4xx 시 숨김)	`list.html:68`	
"전체 N 건" 라벨	`searchError`가 null일 때만 표시(4xx 시 숨김)	`list.html:70`	
출고대기	(스토리 `주문/배송상태 배지`의 상태 — 표시 조건은 실 물 참조)	stories: ./src/components/DeliveryBadge.st ories.tsx	[출고대기](story:주문-배송 상태-배지--출고대기)
출고	(스토리 `주문/배송상태 배지`의 상태 — 표시 조건은 실 물 참조)	stories: ./src/components/DeliveryBadge.st ories.tsx	[출고](story:주문-배송상 태-배지--출고)
배송완료	(스토리 `주문/배송상태 배지`의 상태 — 표시 조건은 실 물 참조)	stories: ./src/components/DeliveryBadge.st ories.tsx	[배송완료](story:주문-배송 상태-배지--배송완료)
이력없음	(스토리 `주문/배송상태 배지`의 상태 — 표시 조건은 실 물 참조)	stories: ./src/components/DeliveryBadge.st ories.tsx	[이력없음](story:주문-배송 상태-배지--이력없음)
빈조건	(스토리 `주문/검색 조건`의 상태 — 표시 조건은 실물 참 조)	stories: ./src/components/OrderFilters.stori es.tsx	[빈조건](story:주문-검색- 조건--빈조건)
조건입력	(스토리 `주문/검색 조건`의 상태 — 표시 조건은 실물 참 조)	stories: ./src/components/OrderFilters.stori es.tsx	[조건입력](story:주문-검 색-조건--조건입력)
기간역전	(스토리 `주문/검색 조건`의 상태 — 표시 조건은 실물 참 조)	stories: ./src/components/OrderFilters.stori es.tsx	[기간역전](story:주문-검 색-조건--기간역전)
조회중	(스토리 `주문/검색 조건`의 상태 — 표시 조건은 실물 참 조)	stories: ./src/components/OrderFilters.stori es.tsx	[조회중](story:주문-검색- 조건--조회중)
집계있음	(스토리 `주문/주문 요약 카드`의 상태 — 표시 조건은 실 물 참조)	stories: ./src/components/OrderSummaryC ard.stories.tsx	[집계있음](story:주문-주 문-요약-카드--집계있음)
결과없음	(스토리 `주문/주문 요약 카드`의 상태 — 표시 조건은 실 물 참조)	stories: ./src/components/OrderSummaryC ard.stories.tsx	[결과없음](story:주문-주 문-요약-카드--결과없음)

조회실패 숨김	(스토리 '주문/주문 요약 카드'의 상태 — 표시 조건은 실물 참조)	stories: ./src/components/OrderSummaryCard.stories.tsx	[조회실패숨김](story:주문-주문-요약-카드--조회실패숨김)
목록있음	(스토리 '주문/주문 목록 그리드'의 상태 — 표시 조건은 실물 참조)	stories: ./src/components/OrderTable.stories.tsx	[목록있음](story:주문-주문-목록-그리드--목록있음)
결과없음	(스토리 '주문/주문 목록 그리드'의 상태 — 표시 조건은 실물 참조)	stories: ./src/components/OrderTable.stories.tsx	[결과없음](story:주문-주문-목록-그리드--결과없음)
조회중	(스토리 '주문/주문 목록 그리드'의 상태 — 표시 조건은 실물 참조)	stories: ./src/components/OrderTable.stories.tsx	[조회중](story:주문-주문-목록-그리드--조회중)
조회실패	(스토리 '주문/주문 목록 그리드'의 상태 — 표시 조건은 실물 참조)	stories: ./src/components/OrderTable.stories.tsx	[조회실패](story:주문-주문-목록-그리드--조회실패)
탈퇴회원 포함	(스토리 '주문/주문 목록 그리드'의 상태 — 표시 조건은 실물 참조)	stories: ./src/components/OrderTable.stories.tsx	[탈퇴회원포함](story:주문-주문-목록-그리드--탈퇴회원포함)
미지의배송코드	(스토리 '주문/주문 목록 그리드'의 상태 — 표시 조건은 실물 참조)	stories: ./src/components/OrderTable.stories.tsx	[미지의배송코드](story:주문-주문-목록-그리드--미지의배송코드)
선택행강조	(스토리 '주문/주문 목록 그리드'의 상태 — 표시 조건은 실물 참조)	stories: ./src/components/OrderTable.stories.tsx	[선택행강조](story:주문-주문-목록-그리드--선택행강조)

6. 팝업·연계 화면

없음(팝업/모달 트리거 없음). 주문번호 링크는 팝업이 아닌 페이지 이동(`/order/{orderNo}`)이다.

7. 데이터 출처·연결

****연결 API(raw → INF):**** GET /order/list — link_uis_inf가 INF-ID 매핑

****참조 테이블(SCH):**** [미확인 — INF 매핑 후 확정]

8. 미확인 사항

조회기간 미입력 시 서버(OrderService.list)가 적용하는 "최근 30일" 기본값이 입력창에 자동으로 채워지는지 여부는 소스 주석상 "자동으로 채우지 않는다"로 명시([미상] 최소가정, SR-205 근거).

`/order/{orderNo}` 주문 상세 화면(UIS)은 아직 생성되지 않음.

화면 상태 17건 — 기준 스냅샷

출고 대기중

배송상태 배지 · 출고대기

출고

배송상태 배지 · 출고

배송완료

배송상태 배지 · 배송완료

배송상태 배지 · 이력없음

회원 ID

예: m001

주문상태

전체

시작일

yyyy-mm-dd

종료일

yyyy-mm-dd

검색

검색 조건 · 빈조건

회원 ID

m001

주문상태

전체

시작일

2026-08-01

종료일

2026-08-31

검색

검색 조건 · 조건입력

회원 ID

예: m001

주문상태

전체

시작일

2026-08-31

종료일

2026-08-01

검색

시작일이 종료일보다 늦습니다

검색 조건 · 기간역전

회원 ID

예: m001

주문상태

전체

시작일

yyyy-mm-dd

종료일

yyyy-mm-dd

조회 중...

검색 조건 · 조회중

3건

총 155,400원

PAID 2 · SHIPPING 1

주문 요약 카드 · 집계있음

집계할 주문이 없습니다

주문 요약 카드 · 결과없음

주문번호	회원	상태	총액	주문일시	배송상태
20260816-0001	김주문	DONE	128,000원	2026-08-16 10:22	배송완료
20260816-0002	이배송	SHIPPING	43,900원	2026-08-16 11:03	출고 대기중
20260816-0003	(탈퇴)	PAID	7,900원	2026-08-16 12:41	-

전체 3건

주문 목록 그리드 · 목록있음

조회 결과 없음

주문 목록 그리드 · 결과없음

조회 중...

주문 목록 그리드 · 조회중

주문을 조회하지 못했습니다 — 500 Internal Server Error

주문 목록 그리드 · 조회실패

주문번호	회원	상태	총액	주문일시	배송상태
20260816-0003	(탈퇴)	PAID	7,900원	2026-08-16 12:41	-

전체 1건

주문 목록 그리드 · 탈퇴회원포함

주문번호	회원	상태	총액	주문일시	배송상태
20260816-0001	김주문	DONE	128,000원	2026-08-16 10:22	RETURNED

전체 1건

주문 목록 그리드 · 미지의배송코드

주문번호	회원	상태	총액	주문일시	배송상태
20260816-0001	김주문	DONE	128,000원	2026-08-16 10:22	배송완료
20260816-0002	이배송	SHIPPING	43,900원	2026-08-16 11:03	출고 대기중
20260816-0003	(탈퇴)	PAID	7,900원	2026-08-16 12:41	-

전체 3건

주문 목록 그리드 · 선택행강조

UIS-ORD-002 주문 상세

UIS-ORD-002: 주문 상세

> **근거 소스(권위):** `src/main/resources/templates/order/detail.html` (Thymeleaf 서버 렌더) +

> `src/main/java/com/sm/lab/shop/controller/OrderViewController.java`. 스크린샷은 보조.

0. 화면 미리보기

[← 주문 목록](#)

주문 상세 — 20260817-0002

회원	이도그 (M-0002)
상태	CANCELED
총액	450,000원
주문일시	2026-08-17 09:40

주문 상품

라인	SKU	상품명	수량	단가
1	SKU-1003	4K 모니터	1	450,000원

배송

배송번호	상태	송장	출고일시
배송 없음			

[주문 취소](#)

> 원 안 번호 = 「4. 위젯.액션」 표의 번호와 1:1.

1. 화면 목적

주문 1건(`orderNo`)의 상태·회원·총액·주문일시와 주문 상품 라인, 배송 이력을 한 화면에서 조회한다.

주문 상태와 무관하게 화면 하단의 「주문 취소」 버튼으로 즉시 취소 요청을 보낼 수 있다.

2. 주요 작업 시나리오

시나리오: 주문 상세 조회

- 주문 목록 화면(`/order/list`)에서 특정 주문을 클릭하거나, `/order/{orderNo}` 라우트로 직접 진입한다.
- 서버(`OrderViewController#orderDetail`)가 `orderService.detail(orderNo)`로 주문 1건을 조회해 화면에 렌더한다(GET 시점에 이미 데이터 확정 — 화면 내 별도 조회 액션 없음).
- 상단 표에서 회원·상태·총액·주문일시를 확인한다.
- 「주문 상품」 표에서 라인별 SKU·상품명·수량·단가를 확인한다.
- 「배송」 표에서 배송 이력(배송번호·상태·송장·출고일시)을 확인한다. 배송 이력이 없으면 "배송 없음" 1행이 표시된다.
- 목록으로 돌아가려면 상단 「← 주문 목록」 링크로 이동한다.

시나리오: 주문 취소

- 화면 하단 「주문 취소」 버튼(①)을 클릭한다.

2. `PATCH /api/orders/{orderNo}/cancel` 호출 → 완료 시 `location.reload()`로 화면을 재조회해 갱신된 상태(CANCELED 등)를 반영한다.

3. 별도 확인(confirm) 다이얼로그 없이 즉시 요청이 전송된다(소스에 confirm 없음 — 8. 미확인 사항 참고).

3. 화면 구성 (블록)

블록	역할	주요 위젯	소스 근거
상단 네비게이션	목록 화면으로 복귀	`← 주문 목록` 링크(href)	`detail.html:18`
주문 요약 표	회원/상태/총액/주문일시 표시	정적 테이블(위젯 없음)	`detail.html:21-26`
주문 상품 그리드	라인별 SKU·상품명·수량·단가	반복 테이블(`th:each`)	`detail.html:28-40`
배송 이력 표	배송번호·상태·송장·출고일시	반복 테이블 + 빈 배열 시 안내행	`detail.html:42-54`
액션 영역	주문 취소	`#btnCancel`	`detail.html:56-59`

4. 위젯·액션

> 버튼/필드 + **동작 → API(raw) → 결과**. 번호 = 「0. 화면 미리보기」 마커 번호와 1:1.

번호	위젯	타입	레이블	동작	연결 API(raw)	결과
(1)	`#btnCancel`	button	주문 취소	현재 주문(`orderNo`) 취소 요청	PATCH /api/orders/{orderNo}/cancel	성공 시 `location.reload()`로 화면 재조회·상태 갱신

> 「← 주문 목록」 링크(`href="/order/list"`, 정적 앵커)와 상단/상품/배송 표는 인터랙션 위젯(id/onclick)이

> 아니므로 `select\_tab\_widgets.py` 전수 목록에 포함되지 않는다 — 3번 「화면 구성」에 별도 기록.

5. 접근 권한·표시 조건

요소	표시 조건	근거	스토리
화면 전체	권한 분기 없음 — `/order/{orderNo}` 진입 시 무조건 렌더	`OrderViewController.java:31-35`	
「주문 취소」 버튼	항상 노출 — 현재 주문 상태(CANCELED 등)에 따른 disabled 분기 없음	`detail.html:56-59`	
기본	(스토리 `주문/주문 상세`의 상태 — 표시 조건은 실물 참조)	stories: ./src/components/OrderDetailCard.stories.tsx	[기본](story:주문-주문-상세--기본)
미출고취임	(스토리 `주문/주문 상세`의 상태 — 표시 조건은 실물 참조)	stories: ./src/components/OrderDetailCard.stories.tsx	[미출고취임](story:주문-주문-상세--미출고취임)
배송이력없음	(스토리 `주문/주문 상세`의 상태 — 표시 조건은 실물 참조)	stories: ./src/components/OrderDetailCard.stories.tsx	[배송이력없음](story:주문-주문-상세--배송이력없음)

품목없음	(스토리 `주문/주문 상세`의 상태 — 표시 조건은 실물 참조)	stories: ./src/components/OrderDetailCard.stories.tsx	[품목없음](story:주문-주문-상세--품목없음)
탈퇴회원	(스토리 `주문/주문 상세`의 상태 — 표시 조건은 실물 참조)	stories: ./src/components/OrderDetailCard.stories.tsx	[탈퇴회원](story:주문-주문-상세--탈퇴회원)
로딩	(스토리 `주문/주문 상세`의 상태 — 표시 조건은 실물 참조)	stories: ./src/components/OrderDetailCard.stories.tsx	[로딩](story:주문-주문-상세--로딩)
조회실패	(스토리 `주문/주문 상세`의 상태 — 표시 조건은 실물 참조)	stories: ./src/components/OrderDetailCard.stories.tsx	[조회실패](story:주문-주문-상세--조회실패)
주문없음	(스토리 `주문/주문 상세`의 상태 — 표시 조건은 실물 참조)	stories: ./src/components/OrderDetailCard.stories.tsx	[주문없음](story:주문-주문-상세--주문없음)
조회실패다시 시도	(스토리 `주문/주문 상세`의 상태 — 표시 조건은 실물 참조)	stories: ./src/components/OrderDetailCard.stories.tsx	[조회실패다시 시도](story:주문-주문-상세--조회실패다시 시도)

6. 팝업·연계 화면

팝업 없음(섹션 생략 대상이나 이동 대상만 기록).

트리거 위젯	팝업/연계 화면	연결 API/화면 (raw → INF)	용도
`← 주문 목록` 링크	주문 목록 화면	GET /order/list ← 화면 이동(INF 없음)	목록으로 복귀

7. 데이터 출처·연결

****연결 API(raw → INF):****

`GET /order/{orderNo}` — 화면 자체 라우트(서버 렌더 진입점). INF 대상 아님(view 반환, JSON API 아님).

`PATCH /api/orders/{orderNo}/cancel` → INF-ORD-006

****참조 테이블(SCH):**** ORDERS, MEMBERS, ORDER_ITEMS, PRODUCTS, ORDER_DELIVERY (INF-ORD-004/006 기준 추정 — 화면 자체는 컨트롤러가 `orderService.detail()` 1회 호출로 전 데이터를 서버 렌더하며, 별도 REST API 조회는 없음)

8. 미확인 사항

「주문 취소」 버튼에 확인(confirm) 다이얼로그가 없어, 클릭 즉시 PATCH 요청이 전송된다 — 의도된 UX인지 확인 필요.

화면 소스(`detail.html`)에는 주문 상태(CANCELED 등)에 따라 「주문 취소」 버튼을 비활성화하는 분기가 없다 — 이미 취소된 주문에서도 버튼이 그대로 노출된다(실제 캡처 상태=CANCELED 화면에서도 버튼 활성화 확인). 서버측

(`/api/orders/{orderNo}/cancel`) 상태 검증 여부는 이 화면 소스만으로는 미확인.

`GET /order/{orderNo}` 자체는 화면 라우트라 INF 매칭 대상이 아니며, 실제 데이터 조회 API(`orderService.detail()`) 내부 구현)는 이 슬라이스 범위 밖.

화면 상태 9건 — 기준 스냅샷

주문 20260816-0001

회원	김주문 (m001)
상태	DONE
총액	128,000원
주문일시	2026-08-16 10:22

주문 품목 2건

상품	수량	단가	금액
무선 마우스	2	24,000원	48,000원
기계식 키보드	1	80,000원	80,000원

배송 이력

- 배송완료 D-002 — 2026-08-17 09:10
- 출고 D-001 — 2026-08-16 18:40

주문 상세 · 기본

주문 20260816-0001

회원	김주문 (m001)
상태	DONE
총액	128,000원
주문일시	2026-08-16 10:22

주문 품목 2건

상품	수량	단가	금액
무선 마우스	2	24,000원	48,000원
기계식 키보드	1	80,000원	80,000원

배송 이력

- 배송완료 D-002 — 2026-08-17 09:10
- 출고 D-001 — 2026-08-16 18:40
- 출고 대기중 D-003 — 미출고

주문 상세 · 미출고취임

주문 20260816-0001

회원	김주문 (m001)
상태	DONE
총액	128,000원
주문일시	2026-08-16 10:22

주문 품목 2건

상품	수량	단가	금액
무선 마우스	2	24,000원	48,000원
기계식 키보드	1	80,000원	80,000원

배송 이력

배송 이력이 없습니다.

주문 상세 · 배송이력없음

주문 20260816-0001

회원	김주문 (m001)
상태	DONE
총액	128,000원
주문일시	2026-08-16 10:22

주문 품목 0건
품목이 없습니다.

배송 이력

- 배송완료 D-002 — 2026-08-17 09:10
- 출고 D-001 — 2026-08-16 18:40

주문 상세 · 품목없음

주문 20260816-0001

회원	(탈퇴) (m001)
상태	DONE
총액	128,000원
주문일시	2026-08-16 10:22

주문 품목 2건

상품	수량	단가	금액
무선 마우스	2	24,000원	48,000원
기계식 키보드	1	80,000원	80,000원

배송 이력

- 배송완료 D-002 — 2026-08-17 09:10
- 출고 D-001 — 2026-08-16 18:40

주문 상세 · 탈퇴회원

불러오는 중...

주문 상세 · 로딩

주문을 불러오지 못했습니다 — 404 Not Found

주문 상세 · 조회실패

주문을 찾을 수 없습니다.

주문 상세 · 주문없음

주문을 불러오지 못했습니다 — 503 Service Unavailable

다시 시도

주문 상세 · 조회실패다시시도

UIS-ORD-003 상품 목록

UIS-ORD-003: 상품 목록

- > **근거 소스(권위):** `src/main/resources/templates/product/list.html` (Thymeleaf 서버 렌더) +
- > `src/main/java/com/sm/lab/shop/controller/ProductViewController.java`. 스크린샷은 보조.
- > SR-201로 신규 구현된 화면(FUNC-order-009, Done). > [반영: FUNC-order-009] SR-214(정렬 선택트).
- > SR-224(빈 목록 문구 통일) 반영(2026-09-09).

0. 화면 미리보기

상품 목록

상품명 키워드 검색

SKU	상품명	가격	재고	상태
SKU-1001	원딩 데스크	390,000원	12	판매중
SKU-1002	계식 키보드	129,000원	40	판매중
SKU-1003	유 모니터	450,000원	7	판매중

- > 원 안 번호 = 「4. 위젯.액션」 표의 번호와 1:1.

1. 화면 목적

판매중 상품 목록을 상품명 키워드로 검색·조회하는 화면이다. 각 행의 상품명 링크를 클릭하면 해당 SKU의 상품 상세 화면(`/product/{sku}`, FUNC-order-010)으로 이동한다.

2. 주요 작업 시나리오

****시나리오: 상품 조회****

- 「상품명」 입력란에 검색어를 입력한다(선택 — 비우면 전체 판매중 상품 대상). (1)
 - 「검색」 버튼을 클릭한다. 폼(`#searchForm`, method=GET)이 `GET /product/list?keyword=...`로 재요청되며, 서버가 `ProductService.list(keyword)`로 상품명 부분일치(LIKE) 조회를 수행해 동일 화면을 재렌더한다. (2)
 - 조회 결과 목록에서 원하는 상품의 상품명 링크를 클릭하면 `GET /product/{sku}`로 이동해 상품 상세 화면을 확인할 수 있다. (3~5)
- (별도의 등록/수정/삭제 시나리오 없음 — 이 화면은 조회·상세이동 전용 카탈로그 화면이다.)

****시나리오: 정렬 변경 (SR-214)**** — [반영: FUNC-order-009]

1. 「정렬」 셀렉트(`selSort`)에서 기본순(기본값)·가격 낮은순·가격 높은순 중 하나를 고르면 `onchange`로 폼이 즉시 `GET /product/list?sort=...`로 재제출된다(별도 버튼 클릭 불요). (6)
2. `sort` 값은 URL 파라미터로 남아 새로고침·뒤로가기에도 선택이 유지된다. 화이트리스트(`/latest`/`priceAsc`/`priceDesc`) 밖의 값은 오류 화면 없이 서버가 조용히 `latest`로 되돌린다. "기본순" 라벨은 등록일시 컬럼 부재로 `ORDER BY p.sku ASC`(기존 등록순)를 그대로 쓰는 실제 동작에 맞춘 표기다 — 참 최신순이 아니다.

3. 화면 구성 (블록)

블록	역할	주요 위젯	소스 근거
검색조건	상품명 키워드로 목록 필터링 + 정렬 선택(SR-214)	`keyword` 입력, `selSort` 셀렉트, `btnSearch` 버튼	`list.html:24-39`
상품목록 테이블	조회 결과 표시(SKU/상품명/가격/재고/상태) + 상세 이동 링크	상품명 링크 N건(현재 페이지 기준)	`list.html:40-53`
빈 목록 안내	조회 결과 0건일 때 안내 문구(SR-224 — 검색 조건 유무로 문구 구분)	keyword 없음: "조회 결과가 없습니다" / keyword 있음: "조건에 맞는 결과가 없습니다"	`list.html:55-56`

4. 위젯·액션

- > 번호 = 「0. 화면 미리보기」 마커 번호와 1:1. **(6)은 SR-214로 추가된 위젯이라 기존
- > `preview\_annotated.png`에는 마커가 없다** — 화면 재캡처(`/sl-recon-uis` 재실행) 전까지는
- > 아래 표만 정본으로 본다.

번호	위젯	타입	레이블	동작	연결 API(raw)	결과
(1)	`keyword`	input(text)	상품명	검색 조건 입력(부분일치 검색어)	—	폼 파라미터로 (2) 검색 시 전송
(2)	`btnSearch`	button(submit)	검색	폼(`#searchForm`, method=GET) 제출	GET /product/list	동일 화면 재렌더(keyword LIKE 필터 적용된 목록 갱신)
(3)	상품명 링크(예: 스탠딩 데스크, SKU-1001)	a	{상품명}	해당 SKU 상세로 이동	GET /product/{sku}	상품 상세 화면(FUNC-order-010) 이동
(4)	상품명 링크(예: 기계식 키보드, SKU-1002)	a	{상품명}	해당 SKU 상세로 이동	GET /product/{sku}	상품 상세 화면(FUNC-order-010) 이동
(5)	상품명 링크(예: 4K 모니터, SKU-1003)	a	{상품명}	해당 SKU 상세로 이동	GET /product/{sku}	상품 상세 화면(FUNC-order-010) 이동

(6)	`selSort`	select	정렬	옵션 변경 시 `onchange="this.form.submit()"`로 폼 즉시 재제출(SR-214)	GET /product/ list	동일 화면 재렌더(선택 정렬 기준 적용된 목록, `sort` URL 파라미터 유지)
-----	-----------	--------	----	---	--------------------------	--

- > (3)~(5)는 조회 결과 행 수만큼 반복 렌더되는 ****동일 패턴의 링크****(`list.html:34-40`),
- > `th:each="p : \${products}"`. 캡처 시점 조회 결과가 3건이라 3개로 나타났을 뿐, 표는 위젯
- > ****패턴****을 설명한다. `상태` 컬럼(판매중/품질)은 `p.stockQty == 0` 여부로 클라이언트 개입 없이
- > 서버에서 계산되는 표시 전용 값(액션 없음, 마커 대상 아님).

5. 접근 권한·표시 조건

요소	표시 조건	근거
검색조건·목록·상세이동 링크 전체	조건 없음(항상 표시) — 소스에 `auth:` 슬롯·권한 분기 없음	`list.html` 전체, `ProductViewController.java:32-37`

7. 데이터 출처·연결

****연결 API(raw → INF):**** `GET /product/list` (화면 자기갱신), `GET /product/{sku}` (상세 화면 이동) — 둘 다 Thymeleaf 서버 렌더 라우트(`kind:form`)이며, 이 화면에서 INF로 치환되는 API 호출은 없다(정상 — INF 없음).

****데이터 원천(서버사이드, api_hints 아님):**** 이 화면은 브라우저에서 `GET /api/products`를 호출하지 않는다 — `ProductViewController.productList()`가 `ProductService`를 서버 내부에서

****직접 주입·호출****해 렌더링 데이터를 만든다(`ProductViewController.java:32-37`). 다만 그 내부 로직(판매중 상시필터 + `keyword` 부분일치, `ProductService.list(keyword)`)은 [[INF-ORD-008]] (`GET /api/products`)와 ****동일한 서비스 메서드를 공유****하므로, INF-ORD-008을 이 화면의 데이터 출처(사양상 원천 참고)로 명시한다. 두 계층은 진입 경로가 다를 뿐(REST vs 서버렌더) 조회 규칙은 같다 — INF-ORD-008의 `screens:`에는 이 화면을 실제 호출자로 등재하지 않는다(호출 사실이 없으므로).

****참조 테이블(SCH):**** [[SCH-ORD-005]] (`PRODUCTS`) — [[INF-ORD-008]] 참조 테이블과 동일.

8. 미확인 사항

목록 페이징 UI(다음 페이지 등)는 뷰·컨트롤러 어디에도 존재하지 않는다 — 전체(또는 keyword 필터) 결과가 페이징 없이 한 번에 렌더된다.

자동 마커 선택 스크립트(`select\_tab\_widgets.py`, 기본 `--toolbar-y 90`)는 이 화면에서 상품명 링크(순수 `href`만 가진 ``, `id`/`onclick` 없음)를 필터 조건 `id` 있음 OR onclick 있음

미충족으로 제외했고, `keyword` 입력(input 태그)도 대상 태그(`button`/`a`)가 아니라 제외했다.

`--toolbar-y 30`으로 낮춰 `btnSearch`(`y=64`)만 추가 확보(1건) 후, 나머지 4건(입력 1 + 링크 3)은

DOM 스냅샷 전수를 에이전트가 수동으로 `preview\_widgets.json`에 반영해 생성했다(UIS-ORD-001과

동일 사례) — 누락 없음을 확인했으나, 순수 href 링크 input 태그 화면에 대한 스크립트 개선이 필요하다.

UIS-ORD-004 상품 상세

> [변경: SR-202] 2026-08-22

> [변경: SR-202] 2026-08-23 — 담기 품 반영(FUNC-order-010 r6~r7 구현 실측)

UIS-ORD-004: 상품 상세

> **근거 소스(권위):** `src/main/resources/templates/product/detail.html` (Thymeleaf 서버 렌더) +

> `src/main/java/com/sm/lab/shop/controller/ProductViewController.java` +

> `src/main/java/com/sm/lab/shop/service/CartService.java`. 스크린샷은 보조.

0. 화면 미리보기

[← 목록으로](#)

상품 상세 — SKU-1001

SKU	SKU-1001
상품명	스탠딩 데스크
가격	390,000원
재고	12
상태	판매중
회원	김철수 (M-0001) ▼ 수량 1 담기

> 이 화면은 `select\_tab\_widgets.py` 기준 마킹 대상 인터랙션 위젯(id-onclick 보유 button/a) 0건이다

> (재검증 완료). 담기 품의 `select`/`input`/`button`은 실재하지만 소스에 `id`·`onclick` 속성이

> 없어(name 속성만 사용) 결정론 스캐너의 선택 대상이 아니다 — **결함이 아니라 소스 실측**이며,

> 아래 「4. 위젯·액션」에 소스 근거로 전수 기록했다. 유일한 링크(← 목록으로)는 상단 공통영역이라

> 3번 「화면 구성」에 기록. 따라서 마커 번호 없음, annotate 생략.

1. 화면 목적

상품 1건(`sku`)의 SKU·상품명·가격·재고·상태를 표시하고, 재고가 있으면 같은 화면에서 회원을 선택해 장바구니에 담을 수 있다(SR-202). 담기는 서버 렌더 POST(PRG)로 처리되며, 담기 자체는 재고를 차감하지 않는다(보관 전용). 존재하지 않는 SKU로 접근하면 예외 페이지 대신 "상품을 찾을 수 없습니다" 안내 문구를 보여준다.

2. 주요 작업 시나리오

시나리오: 상품 상세 조회 (정상 SKU)

1. 상품 목록 화면(`/product/list`, UIS-ORD-003)에서 상품명을 클릭하거나, `/product/{sku}` 라우트로

직접 진입한다.

2. 서버(`ProductViewController#productDetail` → `loadProduct`)가 `productService.get(sku)`를 호출해 상품 1건을 조회한다(GET 시점에 이미 데이터 확정 — 화면 내 별도 조회 액션 없음).
3. 표에서 SKU·상품명·가격(`#,#,##원` 포맷)·재고·상태(재고 0이면 "품절", 그 외 "판매중")를 확인한다.
4. 목록으로 돌아가려면 상단 「← 목록으로」 링크로 이동한다.

****시나리오: 존재하지 않는 SKU로 접근****

1. 등록되지 않은 `sku`로 `/product/{sku}`에 접근한다.
2. `productService.get(sku)`가 던지는 404(`ResponseStatusException`)를 `loadProduct`가 흡수하고 `product=null`로 모델에 담아 같은 뷰(`/product/detail`)를 그대로 반환한다(예외 스택 트레이스 노출 없음).
3. 화면에는 "상품을 찾을 수 없습니다" 안내 문구만 표시된다(상세 표·담기 폼 없음).
4. 「← 목록으로」 링크는 이 상태에서도 동일하게 노출되어 목록으로 복귀할 수 있다.

****시나리오: 장바구니 담기 (재고 있음, SR-202 실측)****

1. 재고(`stockQty`) > 0인 상품 상세 화면 하단의 담기 폼에서 회원을 선택한다(`memberService.list()`)가 채운 기존 회원 목록 선택).
2. 수량을 입력한다(입력값 기본 1, `min="1"` — 클라이언트 힌트일 뿐 서버가 실검증한다).
3. [담기] 버튼을 클릭하면 `POST /product/{sku}/cart` {memberId, qty}가 제출된다. 컨트롤러는 회원·수량·재고 판정을 직접 수행하지 않고 `CartService.addItem(memberId, sku, qty)`를 ****자바 메서드로 직접 호출****한다(HTTP 아님, REST `POST /api/cart/items`([[INF-ORD-010]])와는 별도 경로 — 같은 서비스 계층만 공유).
4. 같은 회원·같은 상품 재담기는 `CartService`가 DB 원자 UPSERT로 수량을 합산한다(D2, `CART\_ITEMS` PK(member_id, sku)).
5. ****성공****: 서버가 `redirect:/product/{sku}`로 302 리다이렉트하며 `RedirectAttributes` 플래시에 `addToCartSuccess=true`·`addedMemberId`를 담는다. 이어지는 GET 렌더에서 "장바구니에 담았습니다. 장바구니로 이동" 문구 + `/cart?memberId=...` 링크(UIS-ORD-005, 채번만)가 표시된다. 플래시는 1회성이라 그 다음 새로고침(F5)에서는 문구가 사라진다.
6. ****거부****(400 수량<1 / 404 회원·상품 없음 / 409 재고초과·품절): 동일하게 302 리다이렉트 + `addToCartError=<CartService 사유 메시지>` 플래시 → 재렌더 시 사유 문구만 표시(재검증 로직 없이 서비스 메시지 그대로).
7. 두 경우 모두 F5(새로고침)는 리다이렉트된 ****GET****을 재요청할 뿐이라 `CartService.addItem`이

중복 호출되지 않는다(PRG — QA r6→r7 결정, 아래 §8).

****시나리오: 품질 상품****

1. 재고(stockQty) == 0인 상품 상세 화면에서는 담기 폼 자체가 렌더되지 않고, "품질 상품은 담을 수 없습니다" 안내만 표시된다(품을 통한 우회 제출은 불가 — 품이 아예 없음).

3. 화면 구성 (블록)

블록	역할	주요 위젯	소스 근거
상단 네비게이션	목록 화면으로 복귀	← 목록으로 링크(href, 정상/미존재 양쪽 공통)	`detail.html:20`
상품 상세 표 (정상 SKU)	SKU·상품명·가격·재고·상태 5행 표시	정적 테이블(위젯 없음)	`detail.html:29-37`
안내 영역 (미존재 SKU)	"상품을 찾을 수 없습니다" 문구	정적 텍스트	`detail.html:23-26`
담기 성공 안내	담기 완료 확인 + 장바구니 이동 링크 (PRG 플래시 1회성)	안내 텍스트 + `/cart?memberId=...` 링크	`detail.html:41-45`
담기 거부 사유 안내	400/404/409 사유 메시지 표시 (PRG 플래시 1회성)	안내 텍스트	`detail.html:49-51`
장바구니 담기 폼 (재고 있음)	회원 선택 + 수량 입력 후 담기	회원 선택 셀렉트, 수량 input(기본1/min1), [담기] 버튼	`detail.html:54-65`
품질 안내 (재고 0)	담기 폼 대신 안내 문구, 폼 미노출	정적 텍스트	`detail.html:68-70`

4. 위젯·액션

> ****소스 실측****: 담기 폼의 `select`/`input`/`button`은 `id`·`onclick` 속성이 없어(name만 사용)

> `select\_tab\_widgets.py` 전수 스캔에서 마커 대상이 되지 않는다(0건, 결함 아님 — 위 0번 참고).

> 아래 표는 소스(`detail.html`)를 직접 읽어 도출한 ****전수**** 기록이다. 마커 번호는 없다.

위젯	타입	레이블	동작	연결 API(raw)	결과
회원 선택 (`name="memberId"`)	select	회원	`members`(기존 회원 목록) 중 선택 — 옵션 0개(회원 0명)여도 폼은 렌더됨	—	선택값 = memberId, 제출 시 ""이면 CartService 404
수량 입력 (`name="qty"`)	input(number, min=1, value=1)	수량	서버가 `@RequestParam(defaultValue="1")`로도 기본 1 고정(이중 방어). `min=1`은 클라이언트 힌트일 뿐, 1 미만 제출은 서버(CartService)가 400으로 거부	—	입력값 = qty
[담기] 버튼	button(submit)	담기	폼 제출 → `POST /product/{sku}/cart`	POST /product/{sku}/cart	성공: 302 redirect + flash(성공 문구/장바구니 링크). 거부: 302 redirect + flash(사유). 5xx: 그대로 rethrow(흡수 안 함)

담기 성공 링크 ('장바구니로 이동')	a	장바구니로 이동	성공 플래시가 있을 때만 렌더, 클릭 시 이동	GET /cart	UIS-ORD-005(장바구니, 채번만·화면 본문 미저작)로 이동
거부 사유 텍스트	text(조건부)	—	`addToCartError != null`일 때만 노출, `CartService`가 던진 `ResponseStatusException.reason`을 그대로 출력(재검증·가공 없음)	—	400/404/409 사유 메시지 표시
품절 안내 텍스트	text(조건부)	—	`stockQty == 0`일 때만 노출(품 자체가 함께 미노출)	—	"품절 상품은 담을 수 없습니다"

5. 접근 권한·표시 조건

요소	표시 조건	근거
화면 전체 (GET/POST)	권한 분기 없음 — `/product/{sku}` GET·`/product/{sku}/cart` POST 모두 무조건 처리	`ProductViewController.java:56-96`
상세 표(5행)	`product != null` (정상 조회된 SKU)일 때만 렌더	`detail.html:28`
"상품을 찾을 수 없습니다" 문구	`product == null` (404 흡수)일 때만 렌더 — POST에서도 이 경우 `addItem` 호출 없이 즉시 이 문구로 응답(조기 반환)	`detail.html:23`, `ProductViewController.java:78-82`
담기 성공/거부 안내	각각 `addToCartSuccess` / `addToCartError != null` — PRG 플래시로 **직후 GET 1회만** 채워짐(그 다음 새로고침엔 사라짐)	`detail.html:41,49`
장바구니 담기 폼	`product != null && product.stockQty > 0`일 때만 렌더	`detail.html:54`
품절 안내	`product != null && product.stockQty == 0`일 때만 렌더(품과 상호배타)	`detail.html:68`
「← 목록으로」 링크	위 모든 분기와 무관하게 항상 노출	`detail.html:20`

6. 팝업·연계 화면

팝업 없음(화면 이동 대상만 기록).

트리거 위젯	팝업/연계 화면	연결 API/화면 (raw → INF)	용도
「← 목록으로」 링크	상품 목록 화면(UIS-ORD-003)	GET /product/list ← 화면 이동 (INF 없음)	목록으로 복귀
담기 성공 링크('장바구니로 이동')	장바구니 화면(UIS-ORD-005, 채번만 — 화면 본문 미저작)	GET /cart ← 화면 이동 (INF 없음)	방금 담은 장바구니 확인

7. 데이터 출처·연결

연결 API(raw → INF):

`GET /product/{sku}` — 화면 자체 라우트(서버 렌더 진입점). INF 대상 아님(view 반환, JSON API

아님). 내부적으로 `ProductService.get(sku)`를 **자바 메서드로 직접 호출**한다(HTTP 호출 아님) —

이 메서드는 REST API 컨트롤러([INF-ORD-009]) `GET /api/products/{sku}`와 **동일 서비스

인스턴스를 공유**하지만, 상세 화면은 그 API를 HTTP로 부르지 않고 같은 서비스 계층을 재사용한다.

`POST /product/{sku}/cart` — 담기 품 제출(서버 렌더 PRG). INF 대상 아님(리다이렉트 응답, JSON

API 아님). 내부적으로 `CartService.addItem(memberId, sku, qty)`를 **자바 메서드로 직접

호출**한다(HTTP 호출 아님) — REST API인 [[INF-ORD-010]](`POST /api/cart/items`,

`CartController`)와 **동일 `CartService` 인스턴스를 공유**하지만, 이 화면은 그 REST API를

직접 호출하지 않는다(서버사이드 직접 호출로 대체 — 명시적으로 구분해 둔다).

`GET /product/list` — 목록 화면 이동. INF 대상 아님.

`GET /cart` — 담기 성공 후 이동 링크(장바구니 화면). INF 대상 아님.

****참조 테이블(SCH):****

PRODUCTS ([[SCH-ORD-005]], INF-ORD-009 기준) — `ProductService.get(sku)` 경유.

CART_ITEMS — `CartService.addItem`이 `CartDao.upsertMergeQty`로 갱신(REST 측 [[INF-ORD-010]]과

동일 테이블). ****이 화면 기준 SCH 문서는 아직 없음**** — 아래 §8 참조.

MEMBERS(간접) — 담기 품의 회원 선택이 `memberService.list()`로 조회.

8. 미확인 사항

****미존재 SKU 응답 코드 불일치 (SR-201 QA 결정 수용, r7까지 유지):**** 이 화면은 존재하지 않는

SKU에 대해 ****HTTP 200 + "상품을 찾을 수 없습니다" 안내 문구****를 렌더한다(GET·POST 조기 반환

양쪽 동일). 반면 같은 자원을 다루는 API [[INF-ORD-009]](`GET /api/products/{sku}`)는 ****404****를

반환한다 — 두 계층의 "미존재" 의미가 일부러 다르게 설계됐다. STORY-FUNC-order-010 QA(r1

CONCERNS 권고3, r2 재게이트 PASS 시 잔존·수용, r7 재게이트에서도 잔존·수용으로 재확인)에서

"AC가 상태 코드를 규정하지 않고, 화면단은 사용자 안내가 목적이라 200 렌더가 화면 테스트·구현상

더 단순하다"는 이유로 ****수용(waive)**** 결정됨 — 결함이 아니라 화면/API 계층 간 의도된 차이로 기록.

(근거: `docs/00\_FUNC/stories/STORY-FUNC-order-010.md` QA Gate r1 권고3 / r7 잔존 권고1)

****등록일 미표시 (SR-201 의사결정 D1, 변동 없음):**** 요구사항 원문은 등록일 표시를 언급하나,

`PRODUCTS` 테이블에 등록일 컬럼이 없어(DB 실측: sku-product_name-price-stock_qty-sale_yn

5개뿐) 이번 범위에서 ****제외****했다. "DB 변경 없음" 원칙 유지가 이유이며, 등록일은 상품 등록

기능이 도입되는 추후 SR에서 컬럼과 함께 재검토한다. (근거:

`docs/변경관리/SR-201/inputs/\_decisions.md` D1)

****F5 중복 담기 방지 — PRG 채택 근거 (SR-202, QA r6→r7):**** r6 QA는 최초 구현이 담기 성공/거부를

같은 뷰(`product/detail`)로 ****재렌더****(200, redirect 없음)해 F5 재제출 시 `cartService.addItem`이

2회 호출되고 수량이 조용히 누적됨을 프로브로 실측했다(`CART\_ITEMS`는 PK UPSERT 합산이라 행

중복은 없었으나 수량 누적은 사용자 의도와 무관하게 발생). 기존 `CartViewController`(수량변경·

삭제)가 이미 `redirect:/cart?memberId=...` 관례를 쓰고 있어 이 화면만 예외였던 점도 지적됨(medium

회귀/UX, 권고2). r7에서 성공·거부 양쪽 모두 `RedirectAttributes` 플래시 + `redirect:/product/{sku}` (PRG)로 전환 — 재제출 대상이 POST가 아니라 멍등 GET이 되어 중복 호출이 구조적으로 제거됐다.

r7 QA 재게이트가 프로브로 재확인: 성공/거부 각각 `addItem` 총 호출 1회, F5 후 플래시 소멸 →

****PASS****. (근거: `docs/00\_FUNC/stories/STORY-FUNC-order-010.md` QA Gate r6 CONCERNS 권고2 /

r7 재게이트 "권고2(PRG) — 해소")

****CART_ITEMS의 SCH/추가 INF 문서 미생성(7-N 원칙, D5):**** 담기 품은 구현·QA 완료됐지만, 이 화면이

경유하는 `CART\_ITEMS` 테이블의 SCH 문서와 `POST/GET/PATCH/DELETE /api/cart/...` 4종(D3)의 INF

중 `POST /api/cart/items`([[INF-ORD-010]])만 이미 존재하고 나머지는 아직 `/sl-recon-inf`·

`/sl-recon-sch`가 소스에서 역생성하지 않았다. 이 화면 자체는 REST가 아니라 `CartService`를

직접 호출하므로 INF 신설 대상은 아니나, §7의 `CART\_ITEMS` SCH 링크가 비어 있는 이유는 이

잔여 recon 미실행 때문이다.

****경계 케이스(잔존, 변동 없음):**** 경로 변수 경계 케이스(`/product/` 빈 sku, `/` 포함 sku)는

이 화면의 친화 안내(200+문구) 대상이 아니라 Spring 기본 404로 빠진다(스택 트레이스 노출 없음) —

실 SKU 형식(`SKU-####`)에는 해당 없어 조치 불요(QA r2 잔존 권고2, r7까지 무변화). 회원 0명이면

옵션 0개인 빈 셀렉트 + 살아있는 [담기] 버튼이 렌더되고 제출 시 404("회원 없음")가 표시되는 것,

폼 우회 요청에서 `memberId` 누락 시 스프링 기본 400이 되는 것도 QA가 실측했으나 랩 데이터상

실사고 없어 인지 사항으로만 유지된다(QA r6 권고5-6, r7까지 무변화).

UIS-ORD-005 장바구니

UIS-ORD-005: 장바구니

> [변경: SR-224] 2026-09-09 — 빈 목록 안내 문구 통일 반영(구현 완료 후 STEP 5.5 재동기화)

> **근거 소스(권위):** `src/main/resources/templates/cart/list.html` (Thymeleaf 서버 렌더) +

> `src/main/java/com/sm/lab/shop/controller/CartViewController.java`. 스크린샷은 보조.

0. 화면 미리보기

장바구니

회원

상품명	단가	수량	품목합계	관리
가계식 키보드	129,000원	2 변경	258,000원	삭제

총 합계 258,000원

[상품 목록으로](#)

> 재캡처 시점(FUNC-order-012 구현 완료 후, 장바구니 1품목 상태) 위젯 7개 중 `#selMember`·`#btnSelect`·

> `#btnCheckout` 3개는 `id`를 갖지만, 행별 수량 input[변경][삭제] 버튼 4개는 여전히 `id`·`onclick`이

> 없는 순수 HTML ``<form method="post"><button type="submit">`` 네이티브 제출 방식이다(JS 바인딩 없음).

> `select\_tab\_widgets.py`(`id`/`onclick` 보유 위젯만 선택) 기준으로 3개만 걸러 전수 조건을 만족하지

> 못하므로, 이번에도 자동 마커·annotate를 생략하고 **DOM 스냅샷의 위젯 번호(`n`, 1~7)** 를 그대로

> 참조 번호로 사용해 전수 문서화한다(위젯 개수 7 = 아래 4번 표 행 수, 누락 없음).

1. 화면 목적

회원별로 서버에 보관된 장바구니(품목·수량·합계)를 조회하고, 그 자리에서 수량을 바꾸거나 품목을 삭제하며, 담긴 품목을 그대로 주문으로 전환(체크아웃)한다. 로그인이 없어 상단 셀렉트로 조회 대상 회원을 고른다. 담기 자체(신규 추가)는 이 화면의 책임이 아니다 — 상품 상세 화면(UIS-ORD-004)에서 담고, 여기서는 이미 담긴 품목만 다루고 주문으로 마무리한다.

2. 주요 작업 시나리오

시나리오: 회원별 장바구니 조회

1. `/cart` 또는 `/cart?memberId=`로 진입한다. `memberId`를 안 주면 서버가 `MemberService.list()`의 첫 회원을 기본 선택한다(`CartViewController#cart`).

2. 상단 회원 선택(1)에서 다른 회원을 고르면 `onchange="this.form.submit()"`으로 ****즉시 재조회****된다
(`GET /cart?memberId=...`). [조회] 버튼(2)은 선택을 안 바꾼 상태에서 폼을 다시 제출하고 싶을 때의
보조 수단이다(JS로 이미 자동 제출되므로 평소엔 클릭할 일이 없다).

3. 품목 표에서 상품명·단가·수량·품목합계를 확인하고, 표 아래 총합계(모든 품목의 `lineTotal` 합)를
확인한다.

****시나리오: 품목 수량 변경****

1. 해당 행의 수량 입력칸(3)을 원하는 값으로 고친다(`min="1"`).
2. 같은 행의 [변경] 버튼(4)을 누르면 `POST /cart/items/{sku}/update`가 `memberId` (hidden)와 `qty`를
실어 제출된다.
3. 서버(`CartViewController#updateQty` → `CartService.updateQty`)가 처리 후 `redirect:/cart?memberId=...`
로 돌아온다(PRG 패턴 — 새로고침 시 중복 제출 없음).
4. 재조회된 화면에서 바뀐 수량·품목합계·총합계를 확인한다. `qty<1`이면 400, 재고 초과면 409로 거부되며
두 경우 모두 사유 문구는 응답 본문에 담기지만 ****이 화면 자체로 재렌더되지 않고 오류 페이지로 이탈****한다
(8번 참조 — 알려진 한계).

****시나리오: 품목 삭제****

1. 해당 행의 [삭제] 버튼(5)을 누른다. ****수량을 0으로 낮춰 사라지게 하는 방식이 아니라 삭제는 반드시
이 버튼으로만**** 한다(SR-202 D4 — 수량 1 미만은 거부하고 삭제 유도는 별도 버튼으로).
2. `POST /cart/items/{sku}/delete`가 `memberId` (hidden)를 실어 제출된다.
3. 서버(`CartViewController#delete` → `CartService.delete`)가 처리 후 `redirect:/cart?memberId=...`.
4. 재조회된 화면에서 해당 행이 사라지고 총합계가 다시 계산된다. 마지막 품목을 지우면 빈 상태로
전환된다.

****시나리오: 빈 장바구니 → 상품 목록으로 이동****

1. 선택된 회원의 장바구니에 품목이 없으면 표 총합계[주문하기] 버튼 대신 "조회 결과가 없습니다"
문구(SR-224 요건① — 화면마다 섞여 있던 빈 목록 문구를 통일. 회원 선택트는 검색조건이 아니라
조회 대상 식별용이라 "조건에 맞는 결과가 없습니다" 변형은 쓰지 않는다, 요건②)와 "상품 목록으로"
링크가 노출된다(체크아웃 폼은 D6 요구사항대로 품목이 있을 때만 렌더되므로 빈 상태에서는 애초에
노출되지 않는다).
2. 빈 상태에서 직전 체크아웃 실패(`checkoutError` 플래시)가 함께 있으면 — 예: 이미 빈 장바구니에서
주문하기를 재제출(중복 submit·뒤로가기 등)한 경우 — "조회 결과가 없습니다" 문구는 ****숨고**** 상단
오류 배너("체크아웃 실패: {사유}")만 빈 상태 사유를 알린다. "상품 목록으로" 링크는 이 경우에도

그대로 노출되어 화면이 막다른 상태(탈출 경로 없음)가 되지 않는다(SR-224 요건③ — 오류로 인한 빈 목록에는 안내 문구를 쓰지 않는다; `list.html:94-95`).

3. 링크를 클릭하면 `GET /product/list`(UIS-ORD-003)로 이동해 담을 상품을 고른다.

4. 품목이 있는 상태에서는 표:총합계.[주문하기] 버튼 아래에 같은 "상품 목록으로" 링크(7)가 별도로 한 번 더 노출된다(`list.html:97`) — 추가 담기를 위해 목록으로 돌아가는 용도로 항상 존재.

****시나리오: 체크아웃(주문하기)****

1. 품목이 있는 상태에서만 [주문하기] 버튼(6)이 노출된다(`list.html:77`,

`th:if="\${!#lists.isEmpty(cart['items'])}"`). 빈 장바구니면 이 폼 자체가 렌더되지 않아 버튼을 누를 수 없다.

2. 버튼(6)을 누르면 `POST /cart/checkout`가 `memberId`(hidden)를 실어 제출된다(PRG 패턴).

3. 서버(`CartViewController#checkout` → `CartService#checkout`)가 해당 회원 장바구니를

`SELECT ... FOR UPDATE`로 잠근 뒤 전 품목을 사전 스윕(상품없음/판매중지/재고부족)한다. 문제 없으면 `OrderService.create`로 위임해 재고 차감 + 주문 생성(`ORDERS`/`ORDER\_ITEMS` INSERT)을 같은 트랜잭션에서 처리하고, 트랜잭션 확정 후 장바구니를 비운 뒤 `redirect:/order/{orderNo}` (기존 주문 상세 화면)로 이동한다. 재조회하면 장바구니는 빈 상태다.

4. 실패(400 빈 장바구니 · 404 회원 없음 · 409 재고부족/판매중지/상품없음 — 사전 스윕 또는

`OrderService.create`의 실제 차감 실패로 판정, 전량 거부)면 컨트롤러가 ****4xx만 흡수****해

`redirect:/cart?memberId=...` + flash `checkoutError`로 돌아온다. 화면 상단에 "체크아웃 실패:

{사유}" 문구가 노출되고(`list.html:28-30`) 장바구니는 그대로 보존된다(트랜잭션 롤백). 409 사유는

품목별로 모아 `"재고 부족: SKU-1001(가용1/요청2), SKU-1002(판매중지)"` 형태로 한 번에 전달된다

(D9 — 재고부족+판매중지+미존재 상품을 포괄하는 단일 계열, 접두어는 현행 "재고 부족:"로 통일 전).

5xx는 흡수하지 않고 그대로 전파된다(장애 은폐 금지).

3. 화면 구성 (블록)

블록	역할	주요 위젯	소스 근거
회원 선택 영역	조회 대상(장바구니 소유자) 전환	(1) 회원 셀렉트, (2) 조회 버튼	`list.html:33-42`
품목 표 (품목 있음)	상품명·단가·수량·품목합계 표시 + 행별 수량변경/삭제	(3) 수량 input, (4) 변경, (5) 삭제 (행마다 반복)	`list.html:44-70`
총합계	전 품목 합계 표시	정적 텍스트 (`cart.totalAmount`)	`list.html:71-73`

주문하기 영역 (품목 있음)	장바구니 전체를 주문으로 전환(체크아웃)	(6) 주문하기 버튼/품	`list.html:75-80`
빈 상태 안내 (품목 없음)	"조회 결과가 없습니다"(SR-224 통일 문구, `checkoutError`가 있으면 숨김) + 이동 링크 (항상 노출)	정적 텍스트 + `a`	cart/list.html 빈 상태 블록(라인 앵커는 편집마다 밀려 서술형으로 대체 — round9)
하단 네비게이션	상품 목록으로 복귀(품목 있을 때도 항상 노출)	(7) `a` 링크	`list.html:97`
체크아웃 실패 안내 (실패 후 재진입, 품목 유무 무관)	직전 체크아웃 실패 사유 표시 — 빈 장바구니에서도 노출 가능(SR-224 요건③으로 빈 상태 안내와 배타)	정적 텍스트 (`checkoutError` flash)	`list.html:27-30`

4. 위젯·액션

> 번호는 「0. 화면 미리보기」에서 설명한 대로 **DOM 스냅샷 위젯 번호(`n`)** 를 그대로 쓴다(annotate 마커

> 아님 — id/onclick 보유 위젯이 3/7뿐이라 자동 선택은 전수를 만족 못 함).

번호	위젯	타입	레이블	동작	연결 API(raw)	결과
(1)	`#selMember`	select	회원 (이름 (ID) 목록)	회원 전환 시 `onchange`로 자동 제출	GET /cart	선택 회원의 장바구니로 재조회
(2)	`#btnSelect`	button(submit)	조회	회원 셀렉트 품 수동 재제출(백업 수단)	GET /cart	그리드 갱신(현재 선택 회원 기준)
(3)	qty input (행별)	input(number, min=1)	수량	값 수정	— (같은 행 (4)의 파라미터로 제출)	—
(4)	변경 버튼 (행별)	button(submit)	변경	수량변경 품 제출 (hidden memberId + qty)	POST /cart/items/{sku}/update	redirect:/cart → 수량·품목합계·총합계 갱신. 실패: 400(qty<1)/409(재고초과)
(5)	삭제 버튼 (행별)	button(submit)	삭제	명시적 삭제(D4 — 수량 0 유도 금지)	POST /cart/items/{sku}/delete	redirect:/cart → 해당 행 제거, 총합계 갱신
(6)	`#btnCheckout`	button(submit)	주문하기	체크아웃 품 제출 (hidden memberId)	POST /cart/checkout	성공: redirect:/order/{orderNo}(장바구니 비워짐). 실패(400/404/409): redirect:/cart+flash `checkoutError`, 장바구니 보존
(7)	`a[href=/product/list]`	a	상품 목록으로	화면 이동	GET /product/list	상품 목록(UIS-ORD-003)으로 이동, 담기 재진입

5. 접근 권한·표시 조건

요소	표시 조건	근거
회원 셀렉트 옵션	`MemberService.list()` 전체 회원(권한 분기 없음)	`list.html:36-38`, `CartViewController.java:42`

품목 표 + 총합계	`cart.items`가 비어있지 않을 때만 렌더	`list.html:44,71`
[주문하기] 버튼/폼(6)	`cart.items`가 비어있지 않을 때만 렌더(D6 — 빈 장바구니는 동작하지 않는다)	`list.html:77`
빈 상태 안내("조회 결과가 없습니다")	`cart.items`가 비어있고 **`checkoutError`가 없을 때만** 렌더(SR-224 요건③ — 오류로 인한 빈 목록에는 이 문구를 쓰지 않는다)	cart/list.html 빈 상태 블록(서술형 참조, round9)
빈 상태 "상품 목록으로" 링크	`cart.items`가 비어있으면 `checkoutError` 유무와 무관하게 항상 렌더(round8 재작업 — 탈출 링크 소멸 방지)	위와 동일 위치
하단 "상품 목록으로" 링크((7))	`cart.items`가 비어있지 않을 때 별도로 다시 노출(빈 상태 링크와 별개 위치)	`list.html:97`
각 행의 [변경]/[삭제] 폼	품목이 존재하는 행마다 항상 노출(권한·상태 분기 없음)	`list.html:52-66`
"체크아웃 실패: {사유}" 안내	직전 요청의 `checkoutError` 플래시 속성이 있을 때만 렌더(1회성, `cart.items` 유무 무관)	`list.html:27-30`

6. 팝업·연계 화면

팝업 없음. 이동 대상만 기록.

트리거 위젯	팝업/연계 화면	연결 API/화면 (raw → INF)	용도
(7) `a[href=/product/list]` (빈 상태·품목 있음 상태 공통)	상품 목록 화면(UIS-ORD-003)	GET /product/list ← 화면 이동(INF 없음)	목록에서 추가 상품 담기 (UIS-ORD-004로 이어짐)
(6) `#btnCheckout` (성공 시)	주문 상세 화면(기존, `/order/{orderNo}`)	POST /cart/checkout ← 화면 이동(INF 없음, 대응 REST는 [[INF-ORD-014]])	체크아웃 성공 후 생성된 주문 확인

7. 데이터 출처·연결

****연결 API(raw → INF):****

`GET /cart` — 화면 자체 라우트(서버 렌더 진입점). INF 대상 아님(view 반환). 내부적으로

`CartService.get(memberId)`를 ****자바 메서드로 직접 호출****한다(HTTP 아님).

`POST /cart/items/{sku}/update`, `POST /cart/items/{sku}/delete` — 서버렌더 폼 제출(PRG,

redirect 반환). JSON API가 아니라 INF 대상 아님. 내부적으로 `CartService.updateQty`/`CartService.delete`를

자바 메서드로 직접 호출하며, 이는 ****REST API 컨트롤러(`CartController`)**와 동일 서비스 인스턴스를

공유******한다 — 다만 이 화면은 그 REST API를 HTTP로 부르지 않는다. 같은 서비스가 노출하는 대응 REST

엔드포인트: 담기 [[INF-ORD-010]](`POST /api/cart/items`), 조회 [[INF-ORD-011]](`GET /api/cart/`),

수량변경 [[INF-ORD-012]](`PATCH /api/cart/items/{sku}`), 삭제 [[INF-ORD-013]](`DELETE /api/cart/items/{sku}`).

`POST /cart/checkout` — 서버렌더 폼 제출(PRG, redirect 반환). JSON API가 아니라 INF 대상 아님.

내부적으로 `CartService.checkout(memberId)`를 자바 메서드로 직접 호출하며, `OrderService.create`를

그대로 재사용해 재고 차감·주문 생성을 같은 트랜잭션으로 처리한다(SR-203 D6-D7, 새 주문 규칙 복제

금지). 같은 비즈니스 로직을 노출하는 대응 REST 엔드포인트: 체크아웃 [[INF-ORD-014]]

(`POST /api/cart/checkout`) — SR-203에서 신설, 409 응답에 품목별 사유 목록을 담아 전량 거부한다(D9).

`GET /product/list` — 목록 화면 이동. INF 대상 아님.

****참조 테이블(SCH):**** `CART\_ITEMS`, `PRODUCTS`, `MEMBERS`, `ORDERS`, `ORDER\_ITEMS`(INF-ORD-010~014

`tables` 기준) — 단 ****CART_ITEMS의 SCH 문서는 아직 생성되지 않았다****(`docs/05\_설계서/order/SCH/`에

SCH-ORD-001~006만 존재, CART_ITEMS 언급 0건) — `/sl-recon-sch` 재실행 대상. `PRODUCTS`는 [[SCH-ORD-005]].

8. 미확인 사항

****REST API(`/api/cart`)와 화면 폼(`/cart/\*`)은 별개 경로다.**** 이 화면의 수량변경·삭제·체크아웃은

브라우저가 직접 REST API(`PATCH`/`DELETE /api/cart/items/{sku}`, `POST /api/cart/checkout`)를

호출하는 게 아니라, 서버렌더 폼이 `CartViewController`의 자체 라우트로 POST하고 그 안에서

`CartService`를 자바 메서드로 호출한다. REST API(담기 [[INF-ORD-010]], 조회 [[INF-ORD-011]], 수량변경

[[INF-ORD-012]], 삭제 [[INF-ORD-013]], 체크아웃 [[INF-ORD-014]])는 별도 클라이언트(외부 연동 등)를

위한 것으로 보이며 같은 비즈니스 로직을 공유할 뿐 이 화면이 호출하는 경로가 아니다 — 두 계층을

혼동하지 않도록 api_hints도 화면 경로(`/cart/\*`) 위주로 담았고, 체크아웃 REST 대응만 SR-203 지시에

따라 예외적으로 `POST /api/cart/checkout`을 api_hints에 병기했다(INF-ORD-014 역참조 연결용).

****폼 오류 시 Whitelabel 이탈(QA r1~r3 CONCERNS, 잔존 이월 — 수량변경/삭제 한정):**** `qty<1`(400) 또는

재고 초과(409)로 수량변경이 실패하면 이 화면으로 재렌더되지 않고 Spring 기본 오류 페이지

(Whitelabel)로 이동한다. 사유 문구 자체는 응답 본문에 정상 전달되지만(`server.error.include-message:

always`), 장바구니 화면 컨텍스트는 유지되지 않는다. ****체크아웃(SR-203)은 이 문제를 겪지 않는다**** —

`CartViewController#checkout`이 4xx를 명시적으로 흡수해 `redirect:/cart` + flash로 화면 컨텍스트를

보존하도록 새로 구현됐다. QA에서 권고로 남기고 수용됨(`STORY-FUNC-order-011.md` 후속 추적 TODO) —

수량변경/삭제까지 화면 재렌더+사유 표시로 확장하는 것은 이번 범위 밖.

****CSRF 미방어(D3/QA 결정 수용):**** `/cart/items/{sku}/update`·`/cart/items/{sku}/delete`·

`/cart/checkout`은 앱 전역에 spring-security가 없어 CSRF 토큰이 없다. 사람 코멘트로 이번 SR 범위에서

명시 수용됐고 후속 SR/KNOWN_LIMITATIONS 등재가 후속 추적 TODO로 남아 있다(`STORY-FUNC-order-011.md`

참조).

****수량 1 미만은 거부만, 삭제 유도 없음(D4):**** `qty<1` 입력 시 자동으로 삭제 처리되지 않고 400으로

거부된다 — 사용자가 품목을 없애려면 반드시 (5) [삭제] 버튼을 눌러야 한다. 소스 주석

(`list.html:57` 대응 원칙)에도 이 원칙이 명시돼 있다.

****`CART\_ITEMS` SCH 미생성:**** 7번 참조 — 이번 UIS 재작성 시점에도 아직 없어 테이블명만 기록했다.

`/sl-recon-sch` 재실행 대상.

****409 메시지 접두어 통일 미완(D9):**** 체크아웃 409는 재고부족·판매중지·미존재 상품을 포괄하는 단일 사유 계열로 확정됐으나, 현재 구현은 세 사유 모두 "재고 부족:" 접두어를 그대로 쓴다(예: 판매중지 품목도 "재고 부족: SKU-x(판매중지)"로 표기). 메시지 접두어 자체를 통일하는 것은 후속 추적(비차단, D9).

UIS-ORD-006 home

이 화면 설계서에는 화면 그림이 없습니다 — 구성·위젯·연결 API는 소스에서 읽어 채웠습니다.

UIS-ORD-006: home

> 코드(라우트) 기준 재동기화 골격(zero-LLM) — 화면 구성·상태·검증 규칙은 보강 대상

1. 화면 개요

라우트: `/` (spa-route, `modules/shop-web/src/App.tsx:49`)

목적: [TBD]

2. 화면 구성

[TBD]

3. 입력·검증 규칙

[TBD]

4. 호출 API

[TBD]

5. 표시 조건(상태)

[TBD] — 스토리북 스토리가 있으면 `[이름](story:ID)`로 연결

> 보강: `/sl-sync --apply --kind=uis` 또는 ddd-ui-agent

UIS-ORD-007 orders_상세

이 화면 설계서에는 화면 그림이 없습니다 — 구성·위젯·연결 API는 소스에서 읽어 채웠습니다.

UIS-ORD-007: orders_상세

> 코드(라우트) 기준 재동기화 골격(zero-LLM) — 화면 구성·상태·검증 규칙은 보강 대상

1. 화면 개요

라우트: ``/orders/{*}`` (spa-route, ``modules/shop-web/src/App.tsx:50``)

목적: [TBD]

2. 화면 구성

[TBD]

3. 입력·검증 규칙

[TBD]

4. 호출 API

[TBD]

5. 표시 조건(상태)

[TBD] — 스토리북 스토리가 있으면 ``[이름](story:ID)``로 연결

> 보강: ``/sl-sync --apply --kind=uis`` 또는 `ddd-ui-agent`

UIS-ORD-008 shop

이 화면 설계서에는 화면 그림이 없습니다 — 구성·위젯·연결 API는 소스에서 읽어 채웠습니다.

UIS-ORD-008: shop

> [반영: SR-302.1] — 코드(라우트) 기준 재동기화 골격. 화면 구성·상태·검증 규칙은 보강 대상

1. 화면 개요

라우트: `/shop` (spa-route, `modules/shop-web/src/App.tsx:93`)

목적: 커머스 홈(메인) — GNB·히어로 배너 캐러셀·카테고리 숏컷·추천 상품 그리드·랭킹(탭 전환)·최근 본 상품 (localStorage)·푸터. 로그인 없이 접근 가능(SR-302).

진입/라우팅 (SR-302.1 round 3 확정값)

`/shop`·`/shop/` (서버 경로, `INF-ORD-017`) 아래로 **해시 없이 최초 진입**할 때만 `#/shop` (이 화면)으로 1회 이동한다 — `applyShopBootRedirect()` (`modules/shop-web/src/App.tsx:75-79`, 호출부 `modules/shop-web/src/main.tsx:10`, 라우터 렌더 이전 모듈 최상위 1회).

그 뒤 `#/`는 **언제나** 주문 목록(`UIS-ORD-001`)이다 — `/` 라우트는 리다이렉트를 거치지 않고 무조건 `OrderListPage`를 렌더한다 (`modules/shop-web/src/App.tsx:86`).

`/shop#/` (빈 해시로 재진입)와 `/shop#/shop` (이 화면)은 서로 다른 경로다 — 전자는 주문 목록, 후자만 이 화면.

2. 화면 구성

위에서부터(부품 경로는 `modules/shop-web/src/features/shop/`):

영역	부품	내용
상단 GNB	`Gnb.tsx`	로고, 카테고리 메뉴, 검색 입력, 장바구니 아이콘(담긴 수량 배지), 로그인 상태에 따라 [로그인] 또는 [로그아웃]
히어로 배너	`HeroBannerCarousel.tsx`	자동 넘김·좌우 이동·현재 위치 표시. 배너는 화면 정적 데이터(`shopStatic.ts`)
카테고리 숏컷	`CategoryShortcuts.tsx`	아이콘+이름 가로 배열(카테고리 API가 없어 정적 표시)
추천 상품	`ProductGrid.tsx` + `ProductCard.tsx`	상품 카드 그리드(상단 8개). 카드: 썸네일(없으면 이니셜 대체), 상품명, 판매가, 정가 취소선, 할인을 배지, 품질 배지
랭킹	`RankingSection.tsx`	탭으로 기준 전환, 순위 숫자와 상위 5개
최근 본 상품	`RecentlyViewed.tsx`	브라우저 저장(`recentlyViewedStorage.ts`, 최대 8개), 없으면 안내 문구
푸터	`ShopFooter.tsx`	사업자 정보·고객센터·약관 링크 영역

3. 입력·검증 규칙

검색 입력: 제출하면 검색어를 `?keyword=`로 실어 상품 목록 화면(`UIS-ORD-009`, `/shop/products`)으로 이동한다

— [반영: SR-303.1] 2026-09-17. 값 검증 없음(빈 검색어 제출도 이동은 함, 목록 화면이 전체를 보여줌).

카테고리 섷트 클릭: 파라미터 없이 상품 목록 화면(`UIS-ORD-009`)으로 이동(전체 목록) — [반영: SR-303.1]

2026-09-17. 카테고리 데이터 필터 API가 없어 라벨을 검색어로 대체하지 않는다(대체 시도가 시드 상품명과 겹치지 않아 항상 빈 결과가 되는 결함이었음, round2에서 확인 후 되돌림). 실제 카테고리 필터링은 후속 SR.

수량·금액 입력 없음 — 이 화면은 읽기 전용이다.
정가·할인율: 정가가 없거나 판매가 이상이면 취소선·할인 배지를 그리지 않는다. 할인율은 내림해 정수 %로 쓴다.
최근 본 상품: 같은 상품은 하나로 합치고(중복 제거) 8개를 넘으면 오래된 것부터 지운다.

4. 호출 API

용도	API	비고
상품 목록	`GET /api/products` (INF-ORD-008)	화면 전체가 이 응답 하나를 공유한다(섹션마다 다시 부르지 않는다). 무인 증 공개(SR-307)
장바구니 수량	`GET /api/cart` (INF-ORD-011)	로그인 상태에서만 부른다 — 배지 숫자용
로그아웃	`POST /api/members/sessions/logout` (INF-MBR-004)	서버 토큰 폐기 후 로컬 세션 정리(실패해도 로컬은 정리)
이미지	`GET /images/products/{파일}` (SR-306)	앱이 서빙하는 정적 자산. 없으면 이니셜 대체 영역

5. 표시 조건(상태)

상태	표시
로딩	상품 영역 골격 표시
목록 있음	상품 카드 그리드·랭킹·최근 본 상품
빈 목록	"표시할 상품이 없습니다" 안내
조회 실패	사유 문구 + [다시 시도](연타해도 요청은 한 번)
상품 품절	카드에 품절 배지
이미지 없음·로드 실패	상품명 이니셜 대체 영역
정가 없음·할인 0%	취소선·할인 배지 없음
비로그인 / 로그인	GNB 우측이 [로그인] / [로그아웃]+장바구니 수량
최근 본 상품 없음	"최근 본 상품이 없습니다"

상태별 실물은 스토리북 스토리로 남아 있다(`features/shop/\*.stories.tsx` — 부품 8개).

> 보강: `sl-sync --apply --kind=uis` 또는 ddd-ui-agent

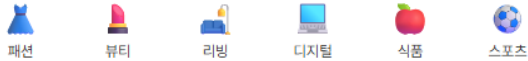
변경 이력

<!-- spec-history: machine-managed -->

일자	SR	항목	변경 요약	커밋
----	----	----	-------	----

2026-09-17	SR-302	#1	쇼핑 홈(메인) 신규 구현 — GNB·히어로배너·카테고리·추천상품그리드·랭킹·최근본상품·푸터·진입/라우팅 규칙(round3 확정) 포함	shop-web@efd0e7c
2026-09-17	SR-306	#1	상품 카드에 정가 취소선·할인을 배지(정수 연산, 내림)·이미지 대체 영역 표시 규칙 반영	shop-web@cbe9816
2026-09-17	SR-303	#1	GNB 검색 제출·카테고리 섷트 진입 경로를 상품 목록 화면(/shop/products)으로 연결(round1~3 확정, 카테고리는 파라미터 없이 전체 목록)	shop-web@59f2008

화면 상태 23건 — 기준 스냅샷



카테고리 섷트 · 기본

SL Shop 카테고리 로그인

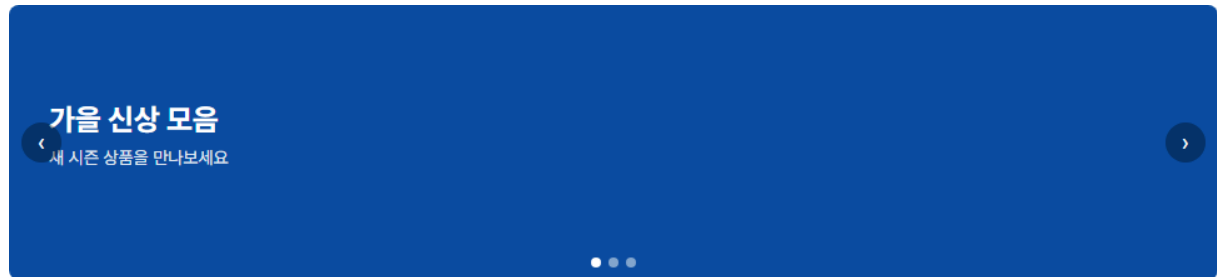
GNB · 비로그인

SL Shop 카테고리 홍길동님 마이페이지, 배송지 관리

GNB · 로그인

SL Shop 카테고리 홍길동님 마이페이지, 배송지 관리

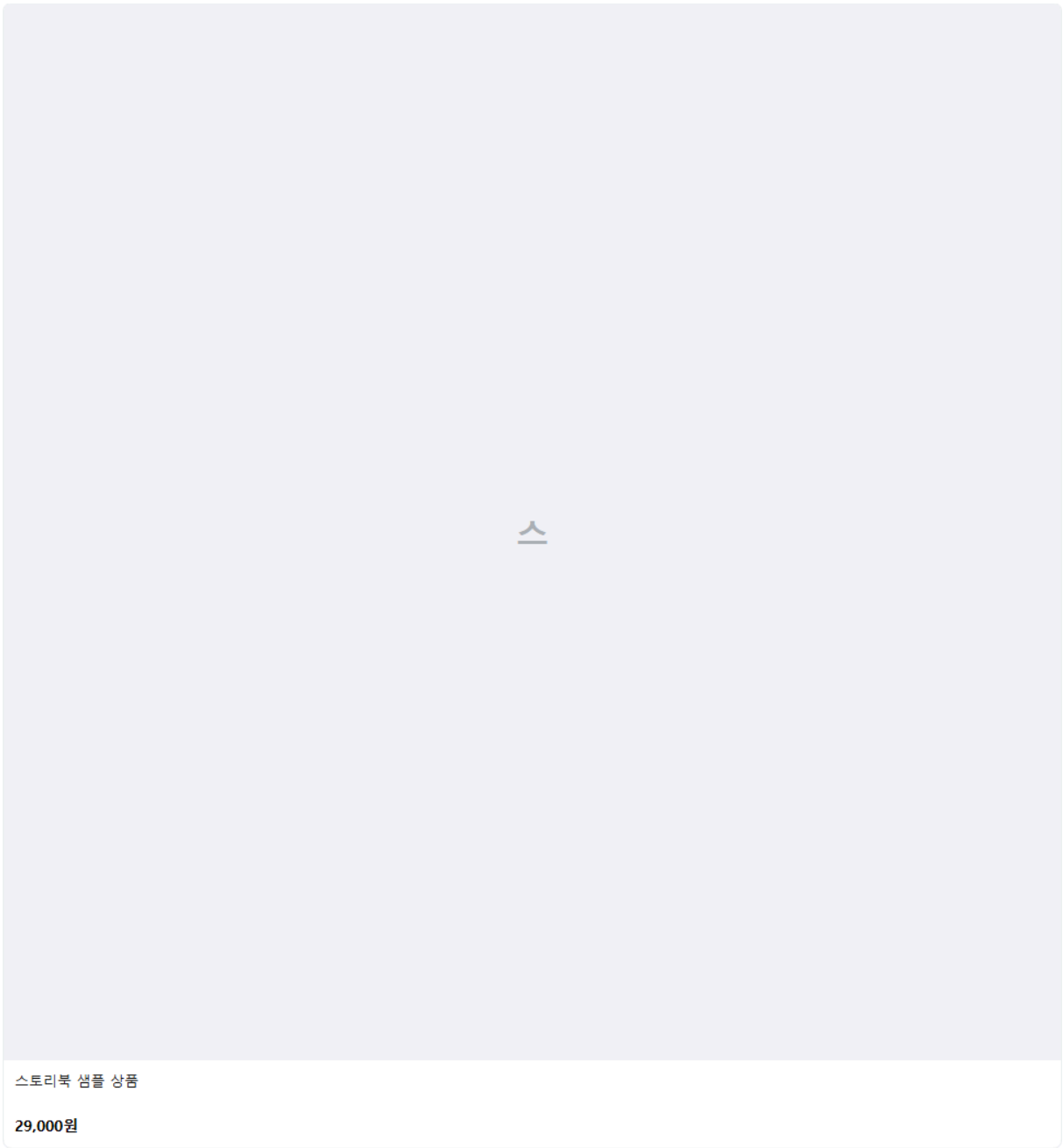
GNB · 장바구니담김



히어로 배너 · 여러장



히어로 배너 · 한장



상품 카드 · 기본

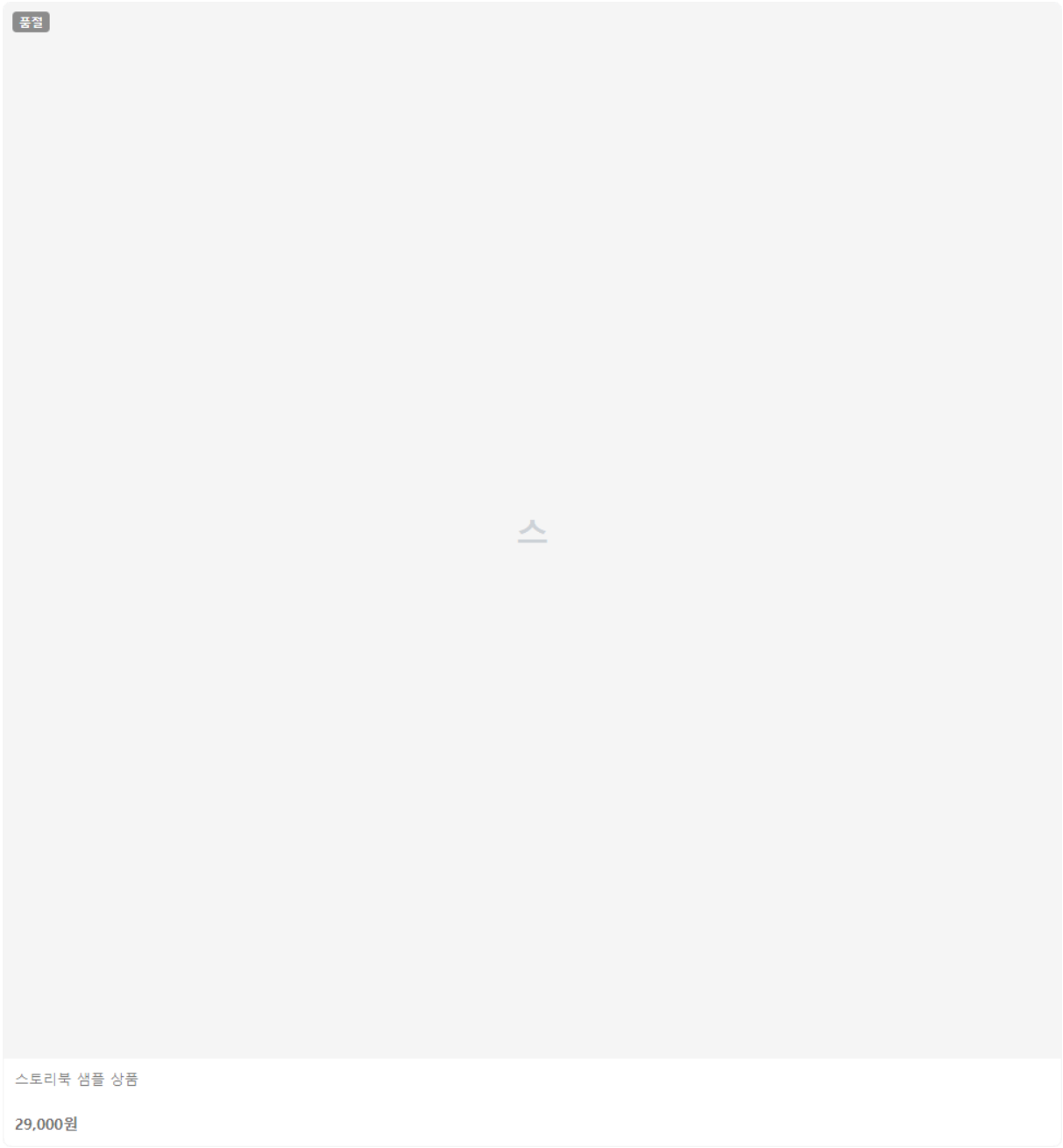
스탠딩 데스크

SKU-1001

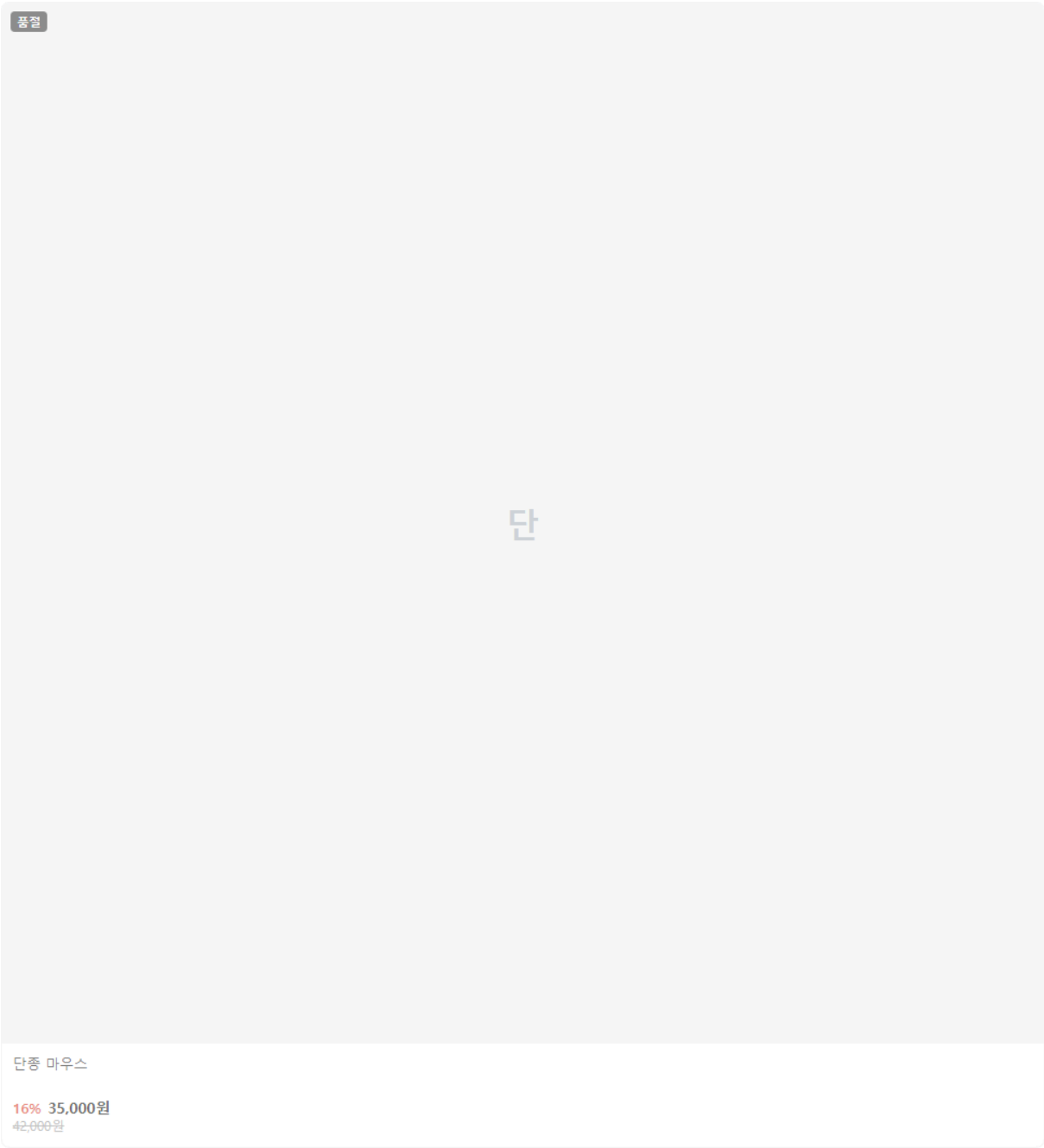
스탠딩 데스크

13% 390,000원
450,000원

상품 카드 · 할인



상품 카드 · 품질



상품 카드 · 이미지없음

기계식 키보드

SKU-1002

기계식 키보드

129,000원

상품 카드 · 정가없음

기계식 키보드

SKU-1002

기계식 키보드

129,000원

상품 카드 · 이미지있음

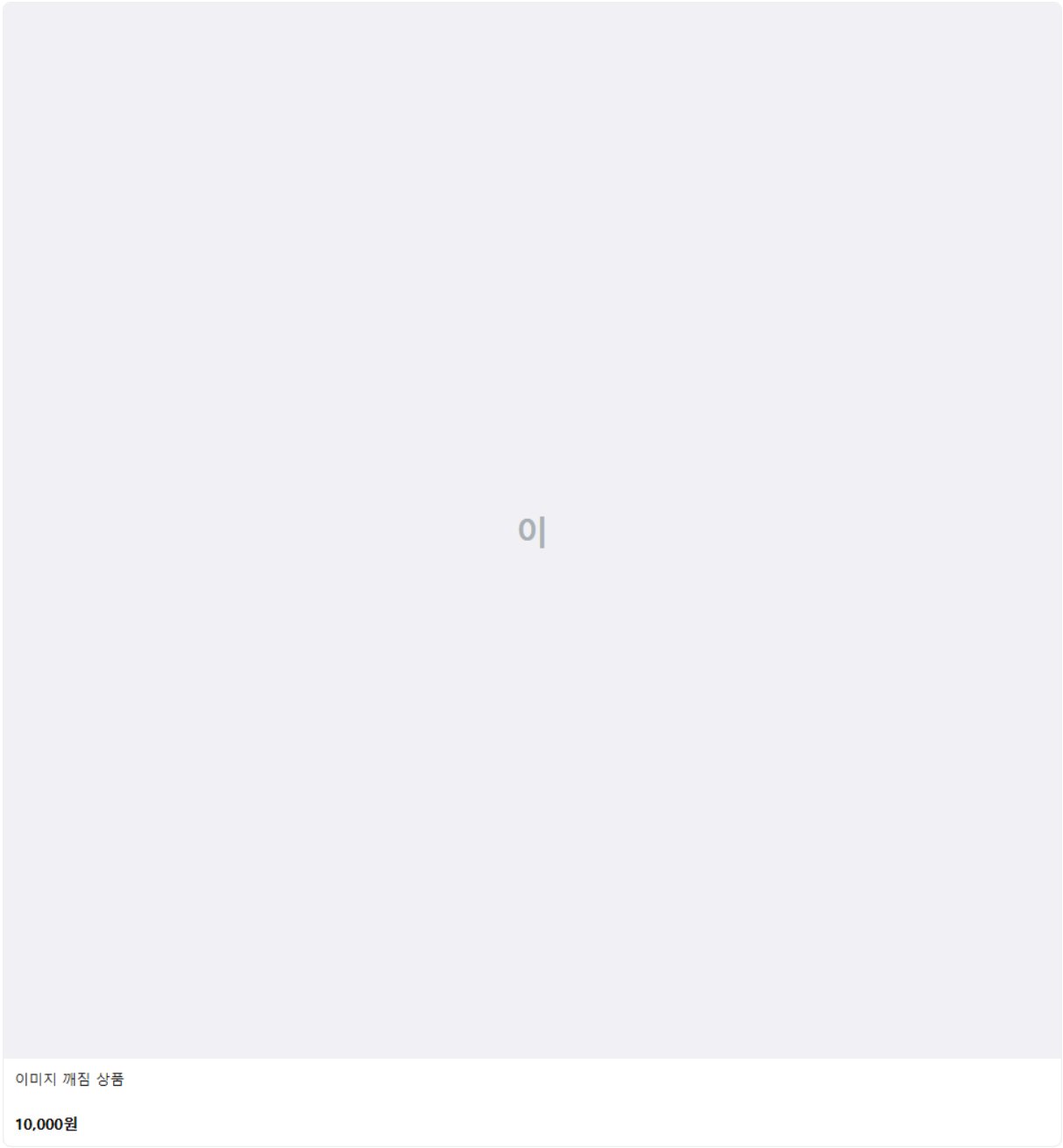
4K 모니터

SKU-1003

4K 모니터

450,000원

상품 카드 · 할인0퍼센트



상품 카드 · 이미지로드실패

스탠딩 데스크
SKU-1001

스탠딩 데스크
13% 390,000원
450,000원

기계식 키보드
SKU-1002

기계식 키보드
129,000원

4K 모니터
SKU-1003

4K 모니터
450,000원

품절

단

단종 마우스
16% 35,000원
42,000원

상

상품 5
10,000원

상

상품 6
11,500원

상

상품 7
13,000원

품절

상

상품 8
14,500원

추천 상품 그리드 · 목록있음

표시할 상품이 없습니다
위 카테고리에서 다른 상품을 찾아보세요

추천 상품 그리드 · 빈목록

불러오는 중...

추천 상품 그리드 · 로딩

불러오지 못했습니다 — 500 Internal Server Error

다시 시도

추천 상품 그리드 · 조회실패

랭킹

인기

가격

1

인

인기 1위 상품
30,000원

2

인

인기 2위 상품
12,000원

3

인

인기 3위 상품
45,000원

4

인

인기 4위 상품(최저가)
8,000원

5

인

인기 5위 상품
21,000원

랭킹 · 인기탭

랭킹

인기

가격

1

응

응답순서 6번째(랭킹 미표시)

5,000원

2

인

인기 4위 상품(최저가)

8,000원

3

인

인기 2위 상품

12,000원

4

인

인기 5위 상품

21,000원

5

인

인기 1위 상품

30,000원

랭킹 · 가격탭전환

최근 본 상품

최

최근 본 상품 A

15,000원

품절

최

최근 본 상품 B

22,000원

최근 본 상품 · 있음

최근 본 상품

최근 본 상품이 없습니다

최근 본 상품 · 없음

SL Shop(테스트베드) · 대표 홍길동 · 사업자등록번호 000-00-00000
주소 서울특별시 어딘가 1길 1 · 통신판매업신고 제0000-서울-00000호
고객센터 1544-0000(평일 09:00~18:00) · 이용약관 · 개인정보처리방침

푸터 · 기본

UIS-ORD-009 shop_products

이 화면 설계서에는 화면 그림이 없습니다 — 구성·위젯·연결 API는 소스에서 읽어 채웠습니다.

UIS-ORD-009: shop_products

> [반영: SR-303.1] — 상품 목록(검색·필터) 화면 신규. 홈(UIS-ORD-008)의 GNB 검색·카테고리 숏컷에서 진입.

1. 화면 개요

라우트: `/shop/products` (spa-route, `modules/shop-web/src/App.tsx:97`)

목적: 키워드 검색·재고 필터·정렬·페이지 이동으로 상품을 찾는 화면. 정렬·페이지네이션·가격대 필터는 전부

클라이언트 계산(서버 API 계약 변경 없음, `GET /api/products`의 `keyword`·`inStock`만 사용).

로그인 없이 접근 가능(`GET /api/products`는 SR-307에서 공개 경로로 확정).

진입 경로 (round1~3 확정)

홈(UIS-ORD-008) GNB 검색 제출 → `?keyword={검색어}`를 실어 이 화면으로 이동, 초기 검색어로 반영·입력창에 표시.

홈 카테고리 숏컷 클릭 → **파라미터 없이** 이 화면으로 이동(전체 목록). 카테고리 데이터·필터 API가 없어

라벨을 검색어로 대체하지 않는다(round2에서 "라벨→keyword"를 시도했다가 시드 상품명과 전혀 겹치지 않아

6/6 빈 결과가 되는 결함으로 판명, round3에서 원복). 실제 카테고리 필터링은 서버에 카테고리 데이터가

생긴 뒤 후속 SR에서 다룬다.

이 화면 안에서의 재검색·재필터는 URL을 갱신하지 않는다(`useSearchParams`를 초기값 읽기 전용으로만 사용) —

`?keyword=A`로 들어와 화면 안에서 `B`로 재검색한 뒤 새로고침·북마크·공유하면 `A` 결과가 열린다(의도된 동작).

2. 화면 구성

위에서부터(부품 경로는 `modules/shop-web/src/features/shop/`):

영역	부품	내용
상단 GNB	`Gnb.tsx` (홈과 공유)	로그·카테고리 메뉴·검색 입력(제출 시 이 화면으로 이동)·장바구니 아이콘·로그인 상태 표시
필터바	`ProductFilterBar.tsx`	재고 있는 상품만 체크박스, 가격대 min/max 입력, 정렬 select(추천순/낮은가격순/높은가격순), 결과 개수 텍스트. fetch 없음(전부 controlled)
상품 그리드	`ProductListGrid.tsx`	로딩(스켈레톤)/결과없음(검색어·필터·일반)/조회실패/결과있음 렌더. 카드 자체는 홈과 같은 `ProductCard` 재사용
페이지네이션	`ProductPagination.tsx`	이전/다음 버튼 + "페이지 N / M". 첫 페이지 이전 비활성, 마지막 페이지 다음 비활성

컨테이너 `ProductListPage.tsx` (페이지, 스토리 대상 아님)가 검색·필터·정렬·페이지 상태를 총괄하고 위 4개 부품을 조립한다.

3. 입력·검증 규칙

서버 반영 상태(`appliedKeyword`, `inStockOnly`)가 바뀔 때만 `GET /api/products` 재호출. 나머지(가격대, 정렬, 페이지)는 클라이언트 전용 — 재요청 없음.

파생 순서(고정): 서버 응답 → 가격대 필터(`filterByPriceRange`) → 정렬(`sortProducts`) → 페이지 slice.

결과 개수 텍스트는 가격 필터까지 적용하고 정렬·페이지 적용 **전**의 배열 길이.

`PAGE\_SIZE = 12` — 확정 문답에 페이지당 개수 규정이 없어 계획 확인 게이트에서 사람이 정한 값(상수, `productListFilters.ts`).

`appliedKeyword`/`inStockOnly`/`priceMin`/`priceMax`/`sortKey` 중 하나라도 바뀌면 `page`를 1로 리셋. 재시도(조회 실패 후 [다시 시도]) 연타는 같은 파라미터 요청을 1회로 억제(`inFlightKeyRef`). 서로 다른 요청이

겹칠 때 응답이 역전되면 세대 카운터(`requestIdRef`)로 최신 요청의 응답만 반영.

같은 검색어를 다시 제출하면 React state가 바뀌지 않아 재조회가 일어나지 않는다(알려진 동작, 후속 개선 후보).

4. 호출 API

용도	API	비고
상품 목록 조회	`GET /api/products` ('INF-ORD-008')	`keyword`(검색어)·`inStock`(재고 필터)만 사용. 정렬·페이지·가격대는 서버에 보내지 않는다. 무인증 공개(SR-307)

5. 표시 조건(상태)

상태	표시
로딩	`ProductListGrid`가 스켈레톤(`PAGE_SIZE` 개수 파생) 표시
결과 있음	상품 카드 그리드(홈과 동일 `ProductCard`)
결과 없음(검색어)	"검색 결과가 없습니다" + [검색어 지우기]
결과 없음(필터)	"조건에 맞는 상품이 없습니다" + [필터 초기화](재고·가격대 리셋, 검색어는 유지)
결과 없음(일반)	안내 문구만(버튼 없음)
조회 실패	사유 문구 + [다시 시도](연타해도 실제 요청은 1회)
첫 페이지	[이전] 비활성
마지막 페이지	[다음] 비활성
정렬 변경	추천순(서버 응답 순서)·낮은가격순·높은가격순으로 카드 순서 재배열
재고 필터 켜	`inStock=true`로 재조회, 품절 상품 제외

상태별 실물은 스토리북 스토리로 남아 있다(`ProductFilterBar.stories.tsx`·`ProductListGrid.stories.tsx`·

`ProductPagination.stories.tsx` — 부품 3개, 확정문답 8상태 전부 커버).

> 보강: `sl-sync --apply --kind=uis` 또는 ddd-ui-agent

변경 이력

```
<!-- spec-history: machine-managed -->
```

일자	SR	항목	변경 요약	커밋
2026-09-17	SR-303	#1	상품 목록(검색·필터) 화면 신규 — /shop/products, GET /api/products의 keyword-inStock 사용, 정렬·페이지·가격대는 클라이언트 계산	shop-web@59f2008

화면 상태 11건 — 기준 스냅샷

☐ 재고 있는 상품만

최소 가격

0

최대 가격

제한 없음

정렬

추천순

▼

총 24개

상품 목록 필터바 · 기본

☐ 재고 있는 상품만

최소 가격

0

최대 가격

제한 없음

정렬

낮은 가격순

▼

총 24개

상품 목록 필터바 · 정렬변경

☒ 재고 있는 상품만

최소 가격

0

최대 가격

제한 없음

정렬

추천순

▼

총 17개

상품 목록 필터바 · 재고필터컴

스

스탠딩 데스크

13% 390,000원

450,000원

기

기계식 키보드

129,000원

4

4K 모니터

450,000원

품절

단

단종 마우스

16% 35,000원

42,000원

상품 목록 그리드 · 결과있음

검색 결과가 없습니다

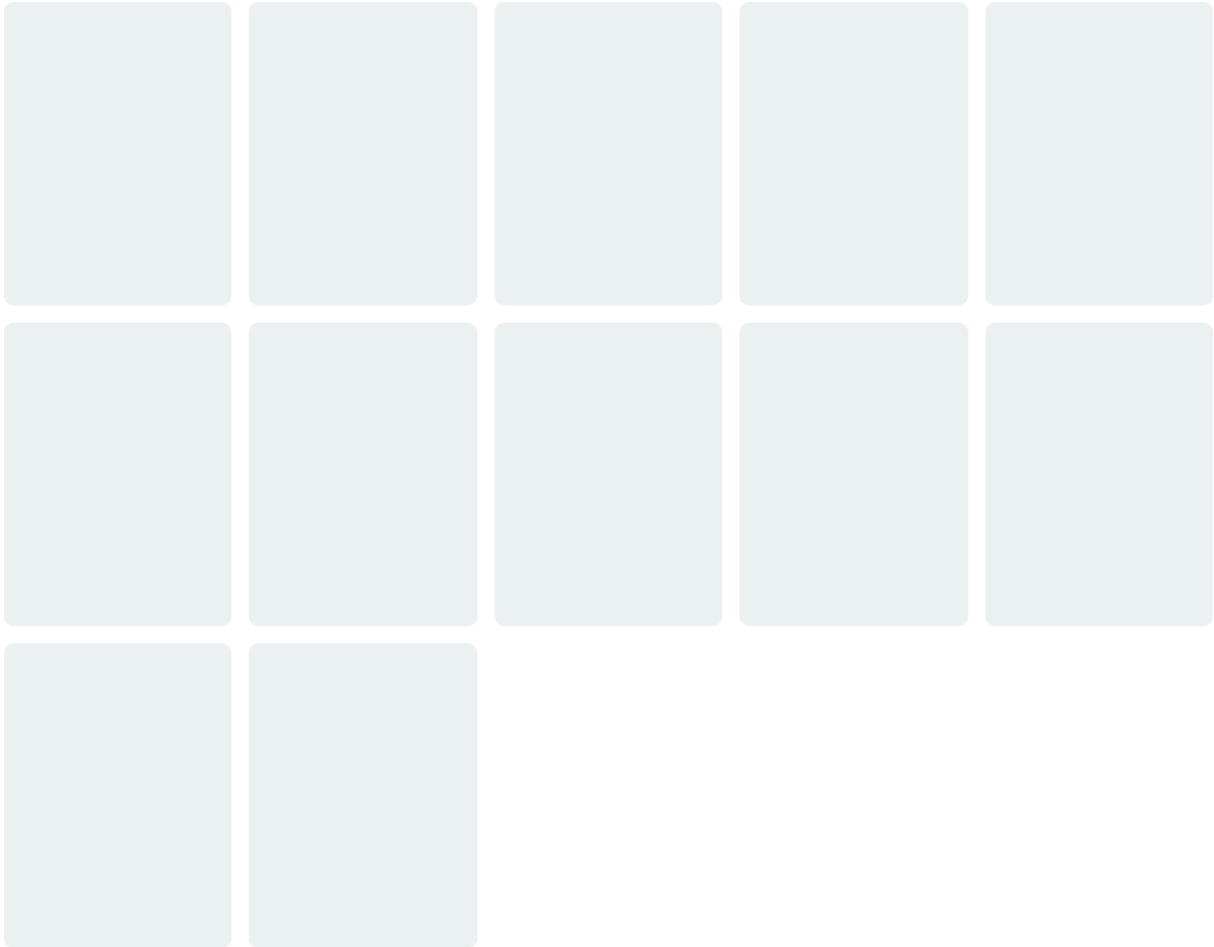
검색어 지우기

상품 목록 그리드 · 결과없음 검색어

조건에 맞는 상품이 없습니다

필터 초기화

상품 목록 그리드 · 결과없음 필터



상품 목록 그리드 · 로딩

불러오지 못했습니다 — 500 Internal Server Error

다시 시도

상품 목록 그리드 · 조회실패

이전

페이지 3 / 5

다음

상품 목록 페이지네이션 · 기본

이전

페이지 5 / 5

다음

상품 목록 페이지네이션 · 마지막페이지

이전

페이지 1 / 5

다음

상품 목록 페이지네이션 · 첫페이지

UIS-ORD-010 shop_products_상세

이 화면 설계서에는 화면 그림이 없습니다 — 구성·위젯·연결 API는 소스에서 읽어 채웠습니다.

UIS-ORD-010: shop_products_상세

> **근거 소스(권위):** `modules/shop-web/src/pages/ProductDetailPage.tsx` 외 `features/shop/\*` (SR-304.1

> 신규 구현). DOM 스냅샷 없음(스토리북/앱 미기동, 소스폴백 모드) — 소스 슬라이스 + STORY-1.md

> (확정 답변·QA 결과) + `speclinker/storybook\_index.json`의 스토리 21건을 근거로 작성.

1. 화면 개요

라우트: `/shop/products/:sku` (spa-route, `modules/shop-web/src/App.tsx:102`)

목적: 쇼핑몰 상품 상세 화면(SR-304 신규). 상품 목록(SR-303, `/shop/products`)에서 상품을 선택하면

이 경로로 진입해 대표 이미지, 가격(판매가·정가·할인율), 재고 상태를 확인하고 수량을 선택해

장바구니에 담거나 바로 구매를 시도한다. 그 아래로 상세정보/구매정보/상품평/상품문의 탭,

상품정보제공고시 표, 함께 보면 좋은 상품 레일을 제공한다. 옵션(색상·사이즈 등)은 없음(단일 SKU

기준, 확정 제외) — 결제 연동도 이 화면 범위 밖이다.

진입 경로: `ProductListPage.handleSelect`의 `navigate('/shop/products/' + sku)` (SR-303이 남기고

SR-304가 라우트를 닫아 도달 가능해짐, `App.tsx:99-102`) · `RelatedProducts` 카드 선택으로 다른

sku 상세에 재진입(`ProductDetailPage.tsx:184-185`).

접근 권한: 화면 열람 자체는 인증 불요(공개 조회). 담기/바로 구매만 세션(memberId) 필요 — 별도

인가(auth) 게이팅은 없다(자세한 조건은 `access\_control` frontmatter 및 §5 참조).

2. 화면 구성

블록	역할	구성 요소	소스 근거
GNB	검색·세션 표시·장바구니 수량 배지·로그아웃(기존 화면과 공통 재사용, 신규 아님)	`Gnb`	`ProductDetailPage.tsx:9,155-156`
로딩 스켈레톤	최초 상품 조회 응답 전 표시	`ProductDetailSkeleton`	`ProductDetailSkeleton.tsx`
조회 실패 안내	상품 없음(404)·조회 실패(그 외) 시 이미지/정보/탭/고지표/관련상품 전체를 대체	`ProductNotFoundNotice`	`ProductNotFoundNotice.tsx`, `ProductDetailPage.tsx:160-163`
이미지 갤러리	대표 이미지 + 썸네일(같은 이미지 1장 재사용, 여러 장으로 지어내지 않음) / 이미지 없음·로드 실패 시 이니셜 대체 영역	`ProductImageGallery`	`ProductImageGallery.tsx`
상품 정보 패널	상품명·판매가·정가·할인율·재고상태·수량 선택·[장바구니 담기]/[바로 구매]·진행 상태 표시	`ProductInfoPanel`	`ProductInfoPanel.tsx`

상세 탭	상세정보/구매정보(고정 정적 문구):상품평/상품문의("준비 중") 4탭 전환	`ProductDetailTabs`	`ProductDetailTabs.tsx`
상품정보 제공고시	전자상거래 표준 고지 항목 6중, 값은 전항목 '-'(값 소스 없음)	`ProductNoticeTable` + `productNoticeItems.ts`	`ProductNoticeTable.tsx`
함께 보면 좋은 상품	현재 상품 제외, 앞 8개 가로 스크롤 레일 — 추천 알고리즘 아님. 후보 없으면 섹션 자체 숨김	`RelatedProducts` + `relatedProductsPicker.ts`	`RelatedProducts.tsx`, `relatedProductsPicker.ts`

> 이미지 갤러리·상품 정보 패널·상세 탭·고지표·관련상품 블록은 상품 로드 성공 시에만 렌더된다 —

> 로딩 중엔 스켈레톤만, 실패 시엔 안내만 렌더한다(셋이 상호 배타, `ProductDetailPage.tsx:158-188`).

3. 입력·검증 규칙

****sku(경로 파라미터)**:** `useParams<{sku}>()`로 취득. 프론트 측 형식 검증 없음 — 존재 여부는 서버

응답(200/404)으로만 판정한다(`ProductDetailPage.tsx:36,60-80`).

****수량(qty)**:** 기본값 1. `handleQtyChange`가 `1 ~ product.stockQty` 범위로 클램프하고

(`Math.min(Math.max(1, safe), max)`), 비정수·NaN 입력은 1로 대체한다(`ProductDetailPage.tsx:117-122`).

`<input min=1 max=stockQty>`는 브라우저 힌트일 뿐 실제 강제는 이 클램프 함수가 한다. 재고가

0이면 수량 선택 UI 자체가 렌더되지 않는다(`ProductInfoPanel.tsx:74`).

****담기/구매 가능 조건****(`disabledReason`, `ProductDetailPage.tsx:145-149`):

1. 세션 없음 → `"로그인이 필요합니다"`

2. 재고 0(`stockQty<=0`) → `"품절된 상품입니다"`

3. 위 두 사유가 모두 없을 때만 버튼 활성화. `addStatus===`'pending'`이면 위 사유와 무관하게 버튼만 추가로 비활성(연타 방지, 별도 사유 문구 없음).

****연타 방지**:** `inFlightRef` 동기 플래그 — 클릭 핸들러 진입 시점에 즉시 잠그고 요청 종료(`finally`)

시 해제한다. React state 반영 지연에 의존하지 않는다(`ProductDetailPage.tsx:124-127,142`).

****담기 요청 파라미터**:** `{memberId, sku, qty}`(`addCartItem` 호출부, `ProductDetailPage.tsx:130`).

`memberId`는 세션에서만 취득하고 프론트가 게스트용 임의 값을 만들지 않는다(신규 신원 확인 경로 없음).

4. 호출 API

트리거	API	용도	성공 처리	실패 처리
화면 진입 / sku 변경	`GET /api/products/{sku}` (INF-ORD-009, `fetchProduct`)	상품 단건 조회	`product` 상태 세팅, `qty=1`·`addStatus='idle'`로 초기화	404 → `errorKind='notFound'`, 그 외(네트워크/5xx) → `errorKind='fetchError'`(두 사유를 하나의 default로 뭉치지 않음)

화면 진입(1회)	`GET /api/products` (INF-ORD-008, `fetchProducts()`, 무인자, 기존 API 재사용)	관련상품 레일 후보 확보(전용 API 없음)	`allProducts` 세팅 → `pickRelatedProducts`로 현재 sku 제외 앞 8개 산출	실패 시 조용히 빈 배열
화면 진입(세션 있을 때 만)	`GET /api/cart?memberId=...` (INF- ORD-011, `fetchCartItemCount`, 기 존 API 재사용)	GNB 장바구니 수량 배지	아이템 `qty` 합산 표시	실패 시 조용히 0
[장바구니 담기] 클릭	`POST /api/cart/items` (INF-ORD-010, `addCartItem`)	담기 처리, body `{memberId, sku, qty}`	응답 성공 확인 → 로컬 `cartItemCount += qty` → `addStatus='success'`, "장바구니에 담 았습니다" 표시(순서 고정, 낙관적 처리 금지)	400/404/409는 사유 문구 매핑, 그 외는 포괄 문구(서버 원문 미노출) — `qty` 선택 상태는 초기화하지 않 음
[바로 구매] 클릭	`POST /api/cart/items` (동일 엔드포인트, `lastAction='buyNow'`)	담기와 동일 API 호출 — 결제 연 동 없음(SR 범위 제외)	성공 시 **이동 없이** "바로 구매 대신 장바구니에 담았습니다" + 비활성 [장 바구니 보기](title="준비 중", `/shop/cart`는 SR-305 뒷)	담기와 동일
관련상 품 카드 선택	(API 아님) `navigate('/shop/product s/'+sku)`	다른 상품 상세 로 재진입	라우트 이동 후 위 진입 API들이 새 sku로 재요청	해당 없음
[목록으 로] / [다 시 시도]	(API 아님)	오류 화면의 복 구 동작	목록으로: `navigate('/shop/products')` / 다시 시도: `load(sku)` 재호출	해당 없음

담기 실패 문구 매핑(`mapAddErrorMessage`, `ProductDetailPage.tsx:26-33`): `400` = "수량을 확인해

주세요" · `404` = "상품 또는 회원 정보를 찾을 수 없습니다" · `409` = "재고가 부족하거나 판매중지된

상품입니다" · 그 외 = "일시적 오류입니다. 다시 시도해 주세요".

> 이 화면이 신규로 만든 서버 API는 없다 — 전부 기존 `ProductController`/`CartController` 엔드포인트

> 재사용(구현 범위는 `shop-web` 프론트엔드로 한정, STORY-1.md "API 사전 확인" 절). 프론트 전용 오류

> 타입 `OrderHttpError`(HTTP status만 보관, `code` 없음, `api.ts:181-187`)는 서버 계약이 아니므로

> INF 문서에 새 오류 봉투로 반영하지 않는다.

5. 표시 조건(상태)

> 표시 조건은 이 화면(및 하위 부품) 각각의 `.stories.tsx`로 전부 남겨져 있다(`.speclinker/storybook\_index.json`

> 기준 21개). 한 행 = 한 스토리.

요소	표시 조건	근거	스토리
로딩 스켈레톤	최초 상품 조회 응답 전(`loading===true`)	`ProductDetailPage.tsx:158-159`, `ProductDetailSkeleton.tsx`	[로딩](story:쇼핑상세-로딩-스켈레톤--로딩)

상세 탭 — 기본	상품 로드 성공, 초기 탭 = '상세정보'(고정 정적 문구)	`ProductDetailTabs.tsx:20-21,33`	[기본](story:쇼핑상세-상세-탭--기본)
상세 탭 — 상품평	'상품평' 탭 선택 시 "준비 중입니다."	`ProductDetailTabs.tsx:23`	[상품평](story:쇼핑상세-상세-탭--상품평)
상세 탭 — 상품문의	'상품문의' 탭 선택 시 "준비 중입니다."	`ProductDetailTabs.tsx:24`	[상품문의](story:쇼핑상세-상세-탭--상품문의)
상세 탭 — 탭 전환	탭 4개를 실제 클릭으로 순회 — 매번 `aria-selected`·`tabpanel` 문구가 정확히 바뀜(play 함수 검증)	`ProductDetailTabs.tsx:38-45`	[탭전환](story:쇼핑상세-상세-탭--탭전환)
이미지 갤러리 — 기본	`imageUrl` 있음 → 대표 이미지 + 같은 이미지 재사용 썸네일 1개	`ProductImageGallery.tsx:44-56`	[기본](story:쇼핑상세-이미지-갤러리--기본)
이미지 갤러리 — 이미지없음	`imageUrl=null` → 이니셜 대체 영역, 썸네일 자체 없음	`ProductImageGallery.tsx:34-42`	[이미지없음](story:쇼핑상세-이미지-갤러리--이미지없음)
이미지 갤러리 — 로드실패	`imageUrl`은 있으나 로드 실패(`onError`) → 이미지없음과 동일하게 이니셜 대체 영역으로 폴백	`ProductImageGallery.tsx:27,32,47`	[로드실패](story:쇼핑상세-이미지-갤러리--로드실패)
상품 정보 패널 — 기본	정상 재고, 할인 없음(`listPrice`가 `price`보다 크지 않음)	`ProductInfoPanel.tsx:56-72`	[기본](story:쇼핑상세-상품-정보-패널--기본)
상품 정보 패널 — 할인있음	`listPrice > price` → 할인을 배지 + 정가 취소선 (`discountRate.ts` 재사용, 재계산 없음)	`ProductInfoPanel.tsx:51,61,64-68`	[할인있음](story:쇼핑상세-상품-정보-패널--할인있음)
상품 정보 패널 — 품절	`stockQty<=0` → 재고 표시 "품절"(텍스트, 배지 아님 — 후속 TODO), 담기/구매 비활성 + 사유, 수량 선택 UI 숨김	`ProductInfoPanel.tsx:50,70-71,74,94-107`	[품절](story:쇼핑상세-상품-정보-패널--품절)
상품 정보 패널 — 담기실패	`addStatus='error'` → `role=alert` 사유 문구 노출, `qty` 선택 상태 유지, 버튼 재활성	`ProductInfoPanel.tsx:109-111`	[담기실패](story:쇼핑상세-상품-정보-패널--담기실패)
상품 정보 패널 — 담기중	`addStatus='pending'` → 버튼 문구 "담는 중..." + 비활성 (연타 방지)	`ProductInfoPanel.tsx:53-54,101,105`	[담기중](story:쇼핑상세-상품-정보-패널--담기중)
상품 정보 패널 — 담기성공	`addStatus='success'`, `lastAction='cart'` → "장바구니에 담았습니다"	`ProductInfoPanel.tsx:113-119`	[담기성공](story:쇼핑상세-상품-정보-패널--담기성공)
상품 정보 패널 — 바로구매성공	`addStatus='success'`, `lastAction='buyNow'` → "바로 구매 대신 장바구니에 담았습니다" + 비활성 [장바구니 보기](title="준비 중")	`ProductInfoPanel.tsx:118-127`	[바로구매성공](story:쇼핑상세-상품-정보-패널--바로구매성공)
상품 정보 패널 — 로그인필요	세션 없음(`disabledReason`) → 담기/구매 비활성 + "로그인이 필요합니다"	`ProductInfoPanel.tsx:94-95`, `ProductDetailPage.tsx:145-146`	[로그인필요](story:쇼핑상세-상품-정보-패널--로그인필요)
조회 실패 안내 — 상품없음	404 → "상품을 찾을 수 없습니다" + [목록으로]	`ProductNotFoundNotice.tsx:24-32`	[상품없음](story:쇼핑상세-조회-실패-안내--상품없음)
조회 실패 안내 — 조회실패	네트워크/5xx → "불러오지 못했습니다" + [다시 시도]	`ProductNotFoundNotice.tsx:34-41`	[조회실패](story:쇼핑상세-조회-실패-안내--조회실패)
상품정보제공 고시 — 기본	항상 표시, 6항목(품명/모델명·제조자/수입자·제조국·크기·색상재질·A/S 책임자·소비자상담전화) 전부 값 "-"	`ProductNoticeTable.tsx`, `productNoticeItems.ts`	[기본](story:쇼핑상세-상품-정보제공고시--기본)

함께 보면 좋은 상품 — 기본	관련상품 후보(현재 sku 제외 앞 8개) 1개 이상 → 가로 레일 렌더	`RelatedProducts.tsx:15-30`	[기본](story:쇼핑상세-함께-보면-좋은-상품--기본)
함께 보면 좋은 상품 — 비어있음	후보 0개(현재 상품이 유일) → 섹션 자체를 렌더하지 않음(`renders-nothing`)	`RelatedProducts.tsx:16`	[비어있음](story:쇼핑상세-함께-보면-좋은-상품--비어있음)

> 미확정/후속 TODO(\$6 참조): 품질 표기가 `ProductCard`의 배지가 아니라 텍스트 한 줄(사람이 이번 SR

> 범위 밖으로 명시, STORY-1.md QA round1 권고5/round2 후속TODO3).

6. 미확인 사항

****품질 표기 형태****: 확정 답변은 "품질 → 배지"였으나 구현은 텍스트 한 줄(`aria-label="재고 상태"`, 빨간색 '품질')이다. 사람이 STORY-1.md QA round1에서 이번 SR 범위 밖으로 명시적으로 미룸(후속 SR 또는 이 UIS 확정 시 처리) — 결함이 아니라 알려진 이월 항목이다.

****상세정보/구매정보 탭 문구, 상품정보제공고시 항목명 목록****: `Product`에 대응 필드가 없어 확정 문답 밖 해석으로 정적 문구/항목을 둔 것 — STEP 3-0 게이트에서 사람이 승인했다(STORY-1.md "데이터" 절, "사람 수정" 절). 실제 상품 설명·배송정책·법정 고지 데이터가 백엔드에 추가되면 이 화면과 스펙 모두 갱신이 필요하다.

****"바로 구매"의 `/shop/cart` 이동****: SR-305(장바구니 화면, 별도 진행)가 완료되기 전까지는 이동하지 않고 인라인 알림 + 비활성 [장바구니 보기]로 대체된다(사람 결정, STORY-1.md "사람 수정" 절). SR-305 병합 후 이 동작이 바뀔 수 있다 — 이 UIS는 그 변경의 영향범위 후보다.

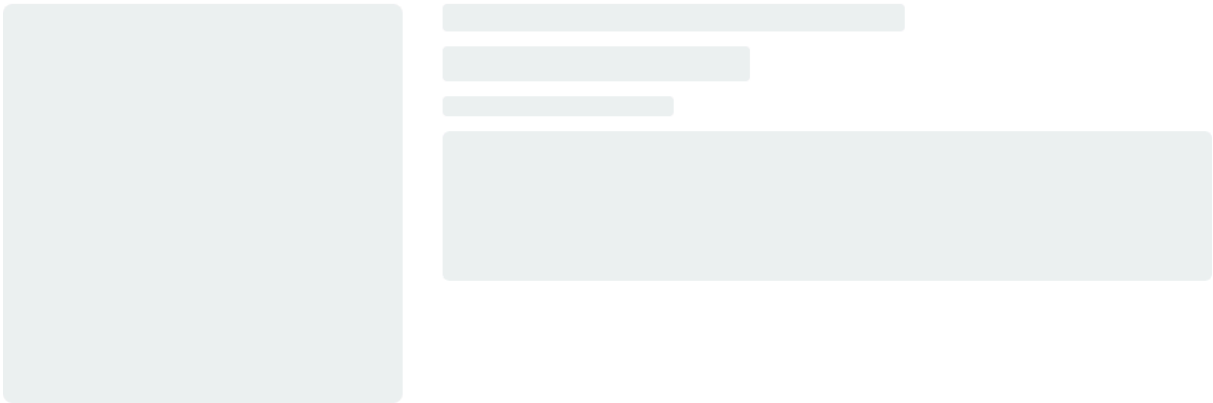
****Gnb 장바구니 아이콘 자체의 활성화****: 현재 "준비 중" 비활성(SR-305 뒀) — 이 화면은 건드리지 않음. 그 외 화면 구조·API·표시 조건은 소스(`ProductDetailPage.tsx` 외 `features/shop/\*`)와 STORY-1.md 확정 답변·QA 결과로 전부 확인됨(미확인 없음).

변경 이력

<!-- spec-history: machine-managed -->

일자	SR	항목	변경 요약	커밋
2026-09-17	SR-304	#1	쇼핑 상품 상세 화면(`/shop/products/{sku}`) 신규 구현 — 이미지·정보패널·탭·고지표·관련상품, GET `/api/products/{sku}`·POST `/api/cart/items` 재사용, 코드 기준 소스로 역생성	shop-web@4ebe84e

화면 상태 20건 — 기준 스냅샷



로딩 스켈레톤 · 로딩

상세정보

구매정보

상품평

상품문의

등록된 상세 설명이 없습니다.

상세 탭 · 기본

상세정보

구매정보

상품평

상품문의

준비 중입니다.

상세 탭 · 상품평

상세정보

구매정보

상품평

상품문의

준비 중입니다.

상세 탭 · 상품문의

상세정보

구매정보

상품평

상품문의

등록된 상세 설명이 없습니다.

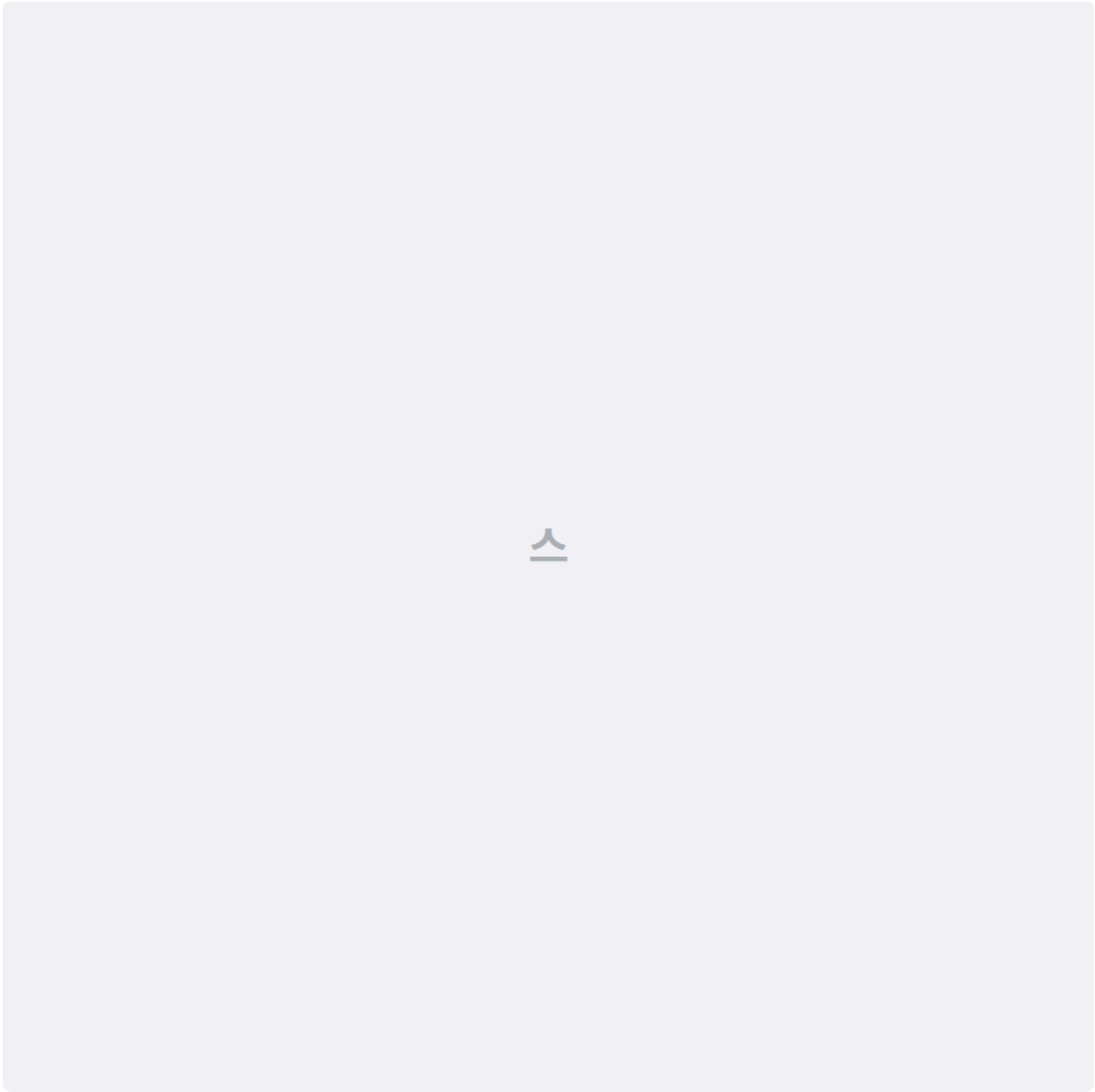
상세 탭 · 탭전환

스탠딩 데스크

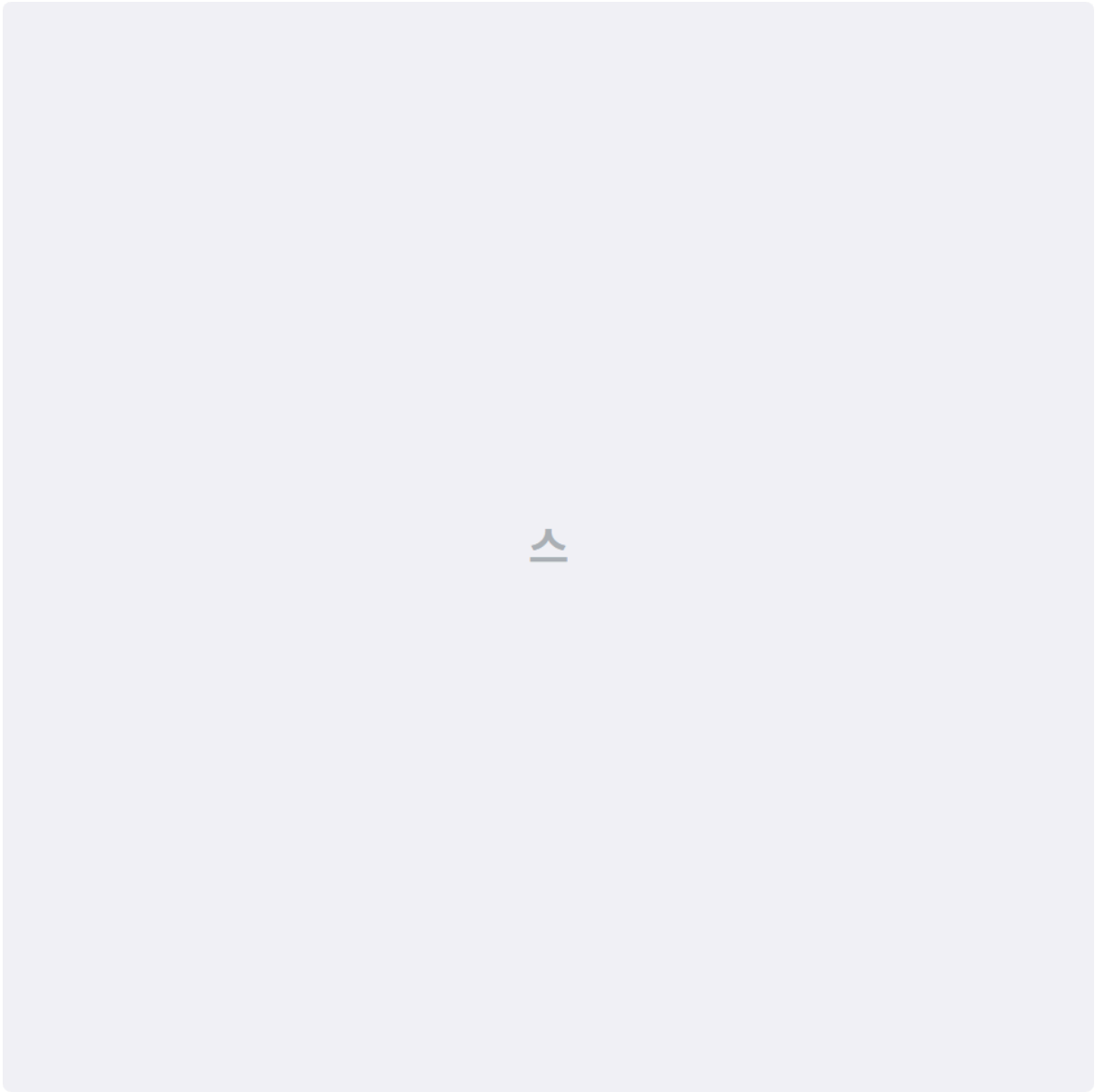
SKU-1001



이미지 갤러리 · 기본



이미지 갤러리 · 이미지없음



스

이미지 갤러리 · 로드실패

기계식 키보드

129,000원

재고 20개

수량

장바구니 담기

바로 구매

상품 정보 패널 · 기본

스탠딩 데스크

13% 390,000원

450,000원

재고 12개

수량

장바구니 담기

바로 구매

상품 정보 패널 · 할인있음

기계식 키보드

129,000원

품질

품질된 상품입니다

장바구니 담기

바로 구매

상품 정보 패널 · 품질

기계식 키보드

129,000원

재고 20개

수량

-

1

+

장바구니 담기

바로 구매

재고가 부족하거나 판매중지된 상품입니다

상품 정보 패널 · 담기실패

기계식 키보드

129,000원

재고 20개

수량

-

1

+

담는 중...

담는 중...

상품 정보 패널 · 담기중

기계식 키보드

129,000원

재고 20개

수량

-

1

+

장바구니 담기

바로 구매

장바구니에 담았습니다

상품 정보 패널 · 담기성공

기계식 키보드

129,000원

재고 20개

수량

-

1

+

장바구니 담기

바로 구매

바로 구매 대신 장바구니에 담았습니다 장바구니 보기

상품 정보 패널 · 바로구매성공

기계식 키보드

129,000원

재고 20개

수량

-

1

+

로그인이 필요합니다

장바구니 담기

바로 구매

상품 정보 패널 · 로그인필요

상품을 찾을 수 없습니다

목록으로

조회 실패 안내 · 상품없음

불러오지 못했습니다

다시 시도

조회 실패 안내 · 조회실패

상품정보제공고시

품명 및 모델명	-
제조자/수입자	-
제조국	-
크기, 색상, 재질 등	-
A/S 책임자와 전화번호	-
소비자상담 관련 전화번호	-

상품정보제공고시 · 기본

함께 보면 좋은 상품

스	기	4
스탠딩 데스크	기계식 키보드	4K 모니터
13% 390,000원 450,000원	129,000원	450,000원

함께 보면 좋은 상품 · 기본

UIS-ORD-011 shop_order

이 화면 설계서에는 화면 그림이 없습니다 — 구성·위젯·연결 API는 소스에서 읽어 채웠습니다.

UIS-ORD-011: shop_order

- > **근거 소스(권위):** `modules/shop-web/src/pages/OrderPage.tsx` 외 `features/shop/\*` (SR-305.1
- > 신규 구현). DOM 스냅샷 없음(소스폴백 모드) — 소스 슬라이스 + `docs/변경관리/SR-305/STORY-1.md`
- > (확정 답변·구현 계획·QA round1~3) + `speclinker/storybook\_index.json`의 스토리 15건을 근거로 작성.

1. 화면 개요

라우트: `/shop/order` (spa-route, `modules/shop-web/src/App.tsx:109`)

목적: 쇼핑 장바구니(`/shop/cart`, UIS-ORD-012)에서 [주문하기]로 진입하는 **주문서·주문완료 화면**

(SR-305 신규, 한 라우트 안에서 두 상태를 오간다). 배송지를 입력하고 우편번호를 검색해 채운 뒤,

결제 수단(표시만)을 확인하고 [결제하기]를 누르면 기존 `POST /api/cart/checkout`을 호출해 주문을

생성한다. 성공하면 같은 라우트 안에서 완료 상태로 전환되어 주문번호와 다음 행동(주문 내역

보기/쇼핑 계속하기)을 보여준다. 실제 결제 연동·쿠폰·적립금·주문 취소·환불은 이번 SR 범위 밖이다.

진입 경로: 장바구니 화면(`/shop/cart`)의 [주문하기](전체선택 상태에서만 활성화) → `navigate('/shop/order')`.

라우터 state에는 의존하지 않는다 — **이 화면은 진입 시 자체적으로 `GET /api/cart`를 재조회**한다

(새로고침·직접 URL 진입에도 견고, `OrderPage.tsx:1-4,62-86`).

접근 권한: 세션(memberId) 없으면 `GET /api/cart` 자체를 호출하지 않고 "로그인이 필요합니다" +

로그인 링크만 표시한다(신규 신원확인 경로 없음, `OrderPage.tsx:84,162-166`). 별도 인가(auth)

게이팅은 없다.

2. 화면 구성

블록	역할	구성 요소	소스 근거
GNB	검색·세션 표시·장바구니 수량 배지·로그아웃(기존 화면과 공통 재사용)	`Gnb`	`OrderPage.tsx:10,157-158`
비로그인 안내	세션 없을 때 화면 전체를 대체	인라인 안내 + 로그인 링크	`OrderPage.tsx:162-166`
로딩 표시	`GET /api/cart` 응답 전	`role="status"` 인라인	`OrderPage.tsx:167-168`
로드 실패 안내	`GET /api/cart` 실패 시	`role="alert"` + [다시 시도]	`OrderPage.tsx:169-178`
빈 장바구니 안내	조회 결과 0건(직접 URL 진입·이미 주문 완료 후 재진입 등)	인라인 안내 + `/shop/cart` 링크	`OrderPage.tsx:182-186`

주문 상품 요약	현재 장바구니 전량(부분 선택 체크아웃 미지원이라 화면·서버가 항상 일치)	`OrderItemsSummary`	`OrderItemsSummary.tsx`
배송지 입력	수령인·연락처·우편번호(읽기전용)·도로명주소(읽기전용)·상세주소 + 필드별 오류	`DeliveryAddressForm`	`DeliveryAddressForm.tsx`
우편번호 검색 모달	[우편번호 찾기] 클릭 시 오픈, 검색어 입력 → 목록 → 선택	`ZipcodeSearchModal`	`ZipcodeSearchModal.tsx`
결제 수단 선택	라디오 3종(표시 전용, 항상 기본값 선택돼 있어 빈 상태 없음)	`PaymentMethodSelect`	`PaymentMethodSelect.tsx`
결제 합계	상품금액/배송비/결제예정금액 3줄	`<aside aria-label="결제 합계">`	`OrderPage.tsx:201-207`
결제 실패 안내	체크아웃 실패 시	`OrderFailureNotice`	`OrderFailureNotice.tsx`
[결제하기] 버튼	배송지 검증 → 체크아웃 실행	`<button>`	`OrderPage.tsx:213-218`
주문 완료 안내	체크아웃 성공 시 위 전체를 대체	`OrderCompleteNotice`	`OrderCompleteNotice.tsx`

> 주문 상품 요약~[결제하기]까지는 세션 있음 + 로드 성공 + 장바구니 비어있지 않음 + 체크아웃 미완료

> 상태에서만 렌더된다. 비로그인/로딩/로드실패/빈장바구니/주문완료는 서로 배타적으로 화면 전체를

> 대체한다(`OrderPage.tsx:162-220`의 분기 순서 그대로).

3. 입력·검증 규칙

****배송지 필드(`deliveryAddressValidation.ts`)**** — [결제하기] 클릭 시 부모(`OrderPage`)가 먼저

호출하는 순수 함수, API 호출 없음:

1. 수령인 미입력 → "수령인을 입력해 주세요"
2. 연락처 미입력 → "연락처를 입력해 주세요"
3. 우편번호 또는 도로명주소 미입력 → "우편번호 찾기로 주소를 선택해 주세요"(둘은 `ZipcodeSearchModal` 선택으로만 함께 채워지는 한 쌍이라 하나의 오류로 묶는다 — 직접 타이핑 입력란이 아니다)
4. 상세주소 미입력 → "상세주소를 입력해 주세요"

검증 실패 시 필드별 오류만 세팅하고 `checkoutCart`는 호출되지 않는다. ****입력값은 그대로 유지된다****

(재입력 방지, `deliveryAddressValidation.ts:17-25`, `OrderPage.tsx:117-121`).

****우편번호·도로명주소는 읽기전용****이다 — 직접 타이핑 불가, `[우편번호 찾기]` → `ZipcodeSearchModal` 선택(`handleZipcodeSelect`)으로만 채워진다(`DeliveryAddressForm.tsx:46-58`, `OrderPage.tsx:102-107`).

선택 시 zipcode 오류는 즉시 지워진다(`OrderPage.tsx:106`).

****우편번호 검색어****: 자유 입력(`zipcodeQuery`), 형식 검증은 프론트에 없음 — 서버(`GET /api/zipcodes`, 400 검색어 미달/초과)가 판정한다(`OrderPage.tsx:93-100`, `api.ts:276-284`).

****연타 방지****: `checkoutInFlightRef` 동기 플래그로 [결제하기] 클릭 시점에 즉시 잠그고 요청 종료

(`finally`) 시 해제한다. React state 반영 지연에 의존하지 않는다(`OrderPage.tsx:60,114-115,138`).

****체크아웃 요청 파라미터는 `{memberId}`뿐**** — 배송지 입력값·결제수단 선택값은 **서버로 전송되지

않는다**. 화면에 보이는 배송지/결제수단은 표시·클라이언트 검증 목적일 뿐이며, 우편번호 검색

자체는 서버 API(`GET /api/zipcodes`)를 재사용하지만 그 검색 결과(선택한 주소)가 서버에 저장되는

것은 아니다(`api.ts:261-274`, `PaymentMethodSelect.tsx:1-3`). 이 사실은 화면 문구로도 숨기지 않는다

(`PaymentMethodSelect.tsx:2-3`의 "실제 결제 연동은 하지 않습니다" 안내).

4. 호출 API

트리거	API	용도	성공 처리	실패 처리
화면 진입 (세션 있을 때만)	`GET /api/cart` (`fetchCart`)	장바구니 재조회 — 라우터 state 미의존	`cartRows` 세팅 → 0건이면 빈 장바구니 안내, 1건 이상이면 주문서 렌더	`toDisplayMessage`로 일반화한 오류 문구 + [다시 시도](`OrderPage.tsx:62-86,169-178`)
[우편번호 찾기] → 검색	`GET /api/zipcodes?q=` (`searchZipcodes`)	도로명·지번 검색	결과 목록 렌더(0건도 200 — "검색 결과가 없습니다")	`toDisplayMessage`로 일반화한 오류 문구(`OrderPage.tsx:93-100`, `ZipcodeSearchModal.tsx:70-73`)
[결제하기] 클릭	(API 호출 없음) `validateDeliveryAddress`	배송지 클라이언트 검증 먼저	통과 시에만 다음 단계(`checkoutCart`) 진행	실패 시 필드별 안내, 입력값 유지, API 미호출(`OrderPage.tsx:117-121`)
[결제하기] 클릭(검증 통과 시)	`POST /api/cart/checkout` (`checkoutCart`, 요청 바디 `{memberId}` 뿐)	주문 생성	`checkoutResult` 세팅 → 완료 상태 전환 + `cartRows`를 빈 배열로 (GNB 배지 갱신)	400/404/409는 서버 `message` 그대로 노출, 그 외(계약 밖 5xx 등)는 일반 문구로 대체(\$5, `OrderPage.tsx:124-138`, `api.ts:261-274`)
로드 실패 시 [다시 시도]	`GET /api/cart` 재호출	복구	위 진입 API와 동일	동일
[로그인 하러 가기]	(API 아님) ` <a href="#/login">>`	로그인 화면 이동	해당 없음	해당 없음
[장바구니로 이동] (빈 장바구니)	(API 아님) ` <a href="#/shop/cart">>`	장바구니 화면 이동	해당 없음	해당 없음
[주문 내역 보기](완료 후)	(API 아님) ` <a >`<="" href="#/" td=""><td>**기존 주문 목록 화면**으로 이동 — 상세(`/orders/:orderNo`) 아님(확정 답변)</td><td>해당 없음</td><td>해당 없음</td>	**기존 주문 목록 화면**으로 이동 — 상세(`/orders/:orderNo`) 아님(확정 답변)	해당 없음	해당 없음
[쇼핑 계속하기](완료 후)	(API 아님) ` <a href="#/shop/products">>`	상품 목록 이동	해당 없음	해당 없음

> 이 화면이 신규로 만든 서버 API는 없다 — `GET /api/cart`·`GET /api/zipcodes`·`POST /api/cart/checkout`

- > 전부 기존 엔드포인트 재사용(하위호환 확정 답변, `STORY-1.md` "계약" 절). `OrderHttpError`(HTTP
- > status만 보관, `code` 없음)는 서버 계약이 아니므로 INF 문서에 새 오류 봉투로 반영하지 않는다.

4.1 결제 합계 — 주문서·주문완료 3줄 일치 (중요, QA round1~3 재확인 대상)

주문서(결제예정금액)와 주문완료 화면은 **같은 3줄 구성** (상품금액/배송비/결제금액)을 같은 계산 로직으로 보여준다. 화면이 다르지만 서로 다른 숫자를 말하면 사용자가 "결제 전/후 금액이 다르다"고 느끼는 결함이 되므로(QA round2 FAIL 필수2 원인) 이 일치를 명시한다.

주문서(체크아웃 전):** `cartTotals.calcCartTotals(cartRows 전체, 전체 sku)` → 상품금액은

`lineTotal` 합산(서버 값 재사용, 클라이언트 재계산 금지), 배송비는 `SHIPPING\_FEE\_FLAT`(선택

항목 1개 이상이면 고정 부과), 결제예정금액 = 상품금액+배송비(`OrderPage.tsx:88-91,201-207`).

주문완료(체크아웃 후):** `OrderCompleteNotice`가 `cartTotals.calcTotalsFromProductAmount`

(`checkoutResult.totalAmount`)로 **같은 배송비 규칙**을 적용해 같은 3줄을 재구성한다

(`OrderCompleteNotice.tsx:12,24`, `cartTotals.ts:42-45`).

서버 `checkout` 응답의 `totalAmount`는 상품금액에만 대응한다** — `CartService.checkout`에는

배송비 개념이 없다(서버 정보 필드가 아니다). 두 화면이 "상품금액" 줄에 이 값을 매핑하고,

"배송비"."결제금액" 줄은 **클라이언트가 계산해 붙인다**(`CheckoutResult.totalAmount` 필드

자체 정의는 `types.ts:114-118` 참조).

5. 표시 조건(상태)

- > 표시 조건은 이 화면 하위 부품 각각의 `.stories.tsx`로 남겨져 있다(`.speclinker/storybook\_index.json`

- > 기준, 이 화면(UIS-ORD-011)에 연결된 스토리 15건). 한 행 = 한 스토리.

요소	표시 조건	근거	스토리
배송지 입력 — 기본	초기 상태(전 필드 공란, 오류 없음)	`DeliveryAddressForm.tsx:25-68`	[기본](story:주문서-배송지-입력--기본)
배송지 입력 — 입력완료	전 필드 유효값 채움(우편번호/도로명주소는 모달 선택 결과)	`DeliveryAddressForm.tsx:46-58`	[입력완료](story:주문서-배송지-입력--입력완료)
배송지 입력 — 배송지오류	`[결제하기]` 클릭 시 검증 실패 — 필드별 `role="alert"` 오류 문구	`deliveryAddressValidation.ts:18-24`, `DeliveryAddressForm.tsx:35,42,57,64`	[배송지오류](story:주문서-배송지-입력--배송지오류)
우편번호 검색 — 검색전	모달 오픈 직후, 검색어 미입력 결과 없음	`ZipcodeSearchModal.tsx:56-73`	[검색전](story:주문서-우편번호-검색--검색전)
우편번호 검색 — 결과있음	검색 성공, 결과 1건 이상 → 선택 가능한 목록	`ZipcodeSearchModal.tsx:57-69`	[결과있음](story:주문서-우편번호-검색--결과있음)

우편번호 검색 — 결과없음	검색 성공, 결과 0건 → "검색 결과가 없습니다"	`ZipcodeSearchModal.tsx:71`	[결과없음](story:주문서-우편번호-검색--결과없음)
우편번호 검색 — 검색중	`zipcodeLoading===true` → `role="status"` "검색 중..."	`ZipcodeSearchModal.tsx:53`, `OrderPage.tsx:94,99`	[검색중](story:주문서-우편번호-검색--검색중)
우편번호 검색 — 조회실패	검색 API 실패 → `role="alert"` 오류 문구(내부 상세 문자열이면 일반 문구로 대체)	`ZipcodeSearchModal.tsx:54`, `OrderPage.tsx:98`, `httpErrorMessage.ts`	[조회실패](story:주문서-우편번호-검색--조회실패)
결제 수단 — 기본선택	초기값 `CARD`(신용/체크카드) 선택	`PaymentMethodSelect.tsx:4,11-15,44`, `OrderPage.tsx:44`	[기본선택](story:주문서-결제-수단--기본선택)
결제 수단 — 계좌이체선택	`BANK_TRANSFER` 선택 상태(표시만, 서버 미전송)	`PaymentMethodSelect.tsx:17-35`	[계좌이체선택](story:주문서-결제-수단--계좌이체선택)
상품 요약 — 주문서기본	현재 장바구니 전량 읽기전용 리스트	`OrderItemsSummary.tsx:12-26`	[주문서기본](story:주문서-상품-요약--주문서기본)
결제 실패 안내 — 빈장바구니 400	체크아웃 400(장바구니가 비어 있음) — 서버 message 그대로	`OrderFailureNotice.tsx:31-41`, `httpErrorMessage.ts:24`	[빈장바구니400](story:주문서-결제-실패-안내--빈장바구니400)
결제 실패 안내 — 회원없음404	체크아웃 404(회원 없음) — 서버 message 그대로	상동	[회원없음404](story:주문서-결제-실패-안내--회원없음404)
결제 실패 안내 — 재고부족409	체크아웃 409(재고 부족) — 서버 message 그대로	상동	[재고부족409](story:주문서-결제-실패-안내--재고부족409)
결제 실패 안내 — 일시적오류	계약 밖 상태(허용목록 400/404/409 밖, 예 5xx) 또는 내부 상세 문자열 혼입 — 일반 문구로 대체	`httpErrorMessage.ts:24,28-30,38-47`, `OrderFailureNotice.tsx:32-34`	[일시적오류](story:주문서-결제-실패-안내--일시적오류)
주문 완료 — 주문완료	체크아웃 200 성공 → 주문번호·상품금액/배송비/결제금액 3줄[주문 내역 보기][쇼핑 계속하기]	`OrderCompleteNotice.tsx:23-52`	[주문완료](story:주문서-주문-완료--주문완료)

> `OrderPage` 자체(컨테이너)는 규칙 `story-per-component` 예외(`src/pages/`)라 스토리 대상이 아니다

> — 위 상태 조합은 `OrderPage.test.tsx`(정상 진입/빈 장바구니/배송지 미입력/우편번호 선택 반영/

> 결제 성공/연타 방지/로드·검색 실패 메시지 필터/금액 표기 일치/결제 실패 3종)로 통합 검증된다.

6. 미확인 사항

****배송비 정책 미확정(SHIPPING_FEE_FLAT = 3,000원)****: 명세·용어집·기존 코드 어디에도 배송비

계산 규칙의 근거가 없다. "선택 항목 1개 이상이면 고정 3,000원, 없으면 0원(무료배송 기준금액

없음)"은 ****사람이 STEP 3-0 게이트에서 임시 확정된 값****이며, 실제 정책(기본요금·무료배송

기준금액·정보 위치)은 후속 SR-308에서 확정한다(`cartTotals.ts:7-11`, `STORY-1.md` "사람 수정".

"후속 SR" 절). 정책이 바뀌면 이 화면과 스펙 모두 갱신이 필요하다.

****배송지 서버 미저장****: 입력한 배송지(수량인·연락처·우편번호·도로명주소·상세주소)는 화면 표시·

클라이언트 검증 전용이며 ****어디에도 저장되지 않는다**** — 체크아웃 요청 바디는 `{memberId}`뿐이고,

`MemberAddressController`(배송지 CRUD, SR-235)는 이 SR이 쓰지 않는다. 실제로 배송지를 주문에 반영하려면 `POST /api/cart/checkout` 계약 자체를 바꿔야 하는 **후속 SR 대상**이다
(`STORY-1.md` "범위 밖" 절).

****부분선택 체크아웃 미지원****: 이 화면은 항상 장바구니 전량을 주문한다(`CartService.checkout`이 선택 여부와 무관하게 회원 장바구니 전체를 전환) — 개별 상품만 골라 결제하는 기능은 서버 계약 변경이 필요해 이번 범위 밖이다(부분선택 차단 UI 자체는 장바구니 화면 `UIS-ORD-012` 쪽 책임).

****in-flight 중 blur 커밋 유실(장바구니 화면 쪽 후속 TODO)****: 이 화면 자체의 결함은 아니지만 같은 SR의 자매 화면(`/shop/cart`)에 남은 알려진 이월 항목이라 참고용으로만 남긴다
(`STORY-1.md` "후속 추적(TODO)" 절).

변경 이력

<!-- spec-history: machine-managed -->

일자	SR	항목	변경 요약	커밋
2026-09-17	SR-305	#1	장바구니·주문서 화면 신규(주문서/주문완료 부분) — 결제예정금액·완료금액 3줄 일치, checkout 바디는 {memberId}뿐, 배송지·결제수단은 표시·검증 전용(서버 미전송), 배송비 SHIPPING_FEE_FLAT=3000 임시가정(SR-308 이월)	shop-web@ca3335c

화면 상태 11건 — 기준 스냅샷

수령인

연락처

우편번호

상세주소

배송지 입력 · 기본

수령인

연락처

우편번호

상세주소

배송지 입력 · 입력완료

수령인

수령인을 입력해 주세요

연락처

연락처를 입력해 주세요

우편번호

우편번호 찾기

우편번호 찾기로 주소를 선택해 주세요

상세주소

상세주소를 입력해 주세요

배송지 입력 · 배송지오류

주문이 완료되었습니다

주문번호 20260917-000001

2개 상품

상품금액	55,000원
배송비	3,000원
결제금액	58,000원

주문 내역 보기

쇼핑 계속하기

주문 완료 · 주문완료

장바구니가 비어 있습니다

결제 실패 안내 · 빈장바구니400

회원 없음: m-1

결제 실패 안내 · 회원없음404

재고 부족: sku-1(가용0/요청2)

결제 실패 안내 · 재고부족409

일시적 오류입니다. 다시 시도해 주세요

결제 실패 안내 · 일시적오류

주문 상품 (2개)

테스트 상품A × 2

30,000원

테스트 상품B × 1

8,000원

상품 요약 · 주문서기본

결제 수단

이번 SR에서는 표시만 하고 실제 결제 연동은 하지 않습니다.

- ☒ 신용/체크카드
- ☐ 실시간 계좌이체
- ☐ 가상계좌

결제 수단 · 기본선택

결제 수단

이번 SR에서는 표시만 하고 실제 결제 연동은 하지 않습니다.

- ☐ 신용/체크카드
- ☒ 실시간 계좌이체
- ☐ 가상계좌

결제 수단 · 계좌이체선택

UIS-ORD-012 shop_cart

이 화면 설계서에는 화면 그림이 없습니다 — 구성·위젯·연결 API는 소스에서 읽어 채웠습니다.

UIS-ORD-012: shop_cart

- > **근거 소스(권위):** `modules/shop-web/src/pages/CartPage.tsx` 외
`features/shop/{CartLineItem,CartEmptyState,CartSummary,cardTotals,httpErrorMessage}.\*(SR-305`
- > 신규 구현). DOM 스냅샷 없음(소스폴백 모드) — 소스 슬라이스 + `docs/변경관리/SR-305/STORY-1.md`
- > (확정 요건 문답·구현 계획-QA round1~3 결과) + `speclinker/storybook\_index.json`의 스토리 10건을
- > 근거로 작성.

1. 화면 개요

라우트: `/shop/cart` (spa-route, `modules/shop-web/src/App.tsx:108`)
 목적: 쇼핑몰 장바구니 화면(SR-305 신규). 로그인 회원이 담아둔 상품을 조회해 수량을 조절하거나
 삭제하고, 주문할 상품을 선택해 합계(상품금액/배송비/결제예정금액)를 확인한 뒤 주문서
 (`/shop/order`, UIS-ORD-011)로 넘어가는 진입 화면이다. 결제 자체는 이 화면 범위 밖(주문서에서
 처리)이고, 이 화면은 조회·조작·합계 미리보기·주문서 이동까지만 담당한다.
 진입 경로: `Gnb`의 장바구니 아이콘(`[, `Gnb.tsx:40`\) — 쇼핑홈·상품목록·
 상품상세 세 화면이 공유하는 유일한 진입점\(SR-305에서 기존 `title="준비 중"` 비활성 아이콘을
 실제 링크로 전환해 닫음, 사례집 "신규 화면 진입점 누락" 재발 방지\). 그 외 직접 URL 진입.
 접근 권한: 화면 라우트 자체는 인증 게이팅이 없으나\(라우트 진입은 항상 가능\), 세션\(memberId\)이
 없으면 `GET /api/cart` 호출 자체를 하지 않고 "로그인이 필요합니다" 안내만 보여준다\(\\$5, \\$4 참조\).](#/shop/cart)

2. 화면 구성

블록	역할	구성 요소	소스 근거
GNB	검색·세션 표시·장바구니 수량 배지(`cartItemCount`)· 로그아웃(기존 화면과 공통 재사용)	`Gnb`	`CartPage.tsx:9,181-182`
로그인 필 요 안내	세션 없을 때 목록 대신 표시	안내 문구 + `/login` 링크	`CartPage.tsx:186-190`
로딩 표시	GET 응답 전	`role="status"` "불러오는 중..."	`CartPage.tsx:192`
로드 실패 안내	GET 실패(네트워크/5xx 등)	`role="alert"` 오류 문구 + [다시 시 도]	`CartPage.tsx:193-202`
빈 장바구니 안내	담긴 상품 0건	`CartEmptyState`("담긴 상품이 없습 니다" + [쇼핑 계속하기])	`CartEmptyState.tsx`

장바구니 목록	라인아이템 반복 렌더(체크박스·썸네일·상품명·단가·수량 스테퍼/직접입력·라인합계·삭제)	`CartLineItem` × N	`CartPage.tsx:207-215`, `CartLineItem.tsx`
합계 패널	선택 항목 기준 상품금액/배송비/결제예정금액 + [주문하기]	`CartSummary`	`CartPage.tsx:217-219`, `CartSummary.tsx`

> 로그인 필요 / 로딩 / 로드 실패 / 빈 장바구니 / (목록+합계) 다섯 상태는 상호 배타이며 이 순서로

> 우선순위가 매겨진다(`CartPage.tsx:186-221`의 조건부 렌더 체인).

3. 입력·검증 규칙

****memberId****: 세션(`loadSession()`)에서만 취득 — 프론트가 게스트용 임의 값을 만들지 않는다.

세션이 없으면 `GET /api/cart` 자체를 호출하지 않는다(`CartPage.tsx:76-79`, STORY "순서·보안" 1 — 신규 신원확인 경로를 만들지 않는다).

****선택 상태(selected)****: 초기값은 응답으로 받은 전체 sku(전체선택 기본값, `CartPage.tsx:64`).

체크박스 토글로 개별 증감(`toggleSelected`, `CartPage.tsx:94-100`).

****수량 변경 클램프(1~stockQty)****: `handleQtyChange`가 요청값을 `1 ~ Math.max(1, stockQty)` 범위로 자르고(`Math.min(Math.max(1, safeRaw), max)`), 비정수/`NaN`은 직전 확인 수량으로 대체한다(`CartPage.tsx:108-121`). 클램프 결과가 현재 수량과 같으면(경계 시도) PATCH를 호출하지 않고 즉시 경고만 표시한다 — "1개 미만으로는 변경할 수 없습니다" / "최대 수량은 {max}개입니다"(`CartPage.tsx:117-121`).

****수량 직접입력은 로컬 draft 상태로만 유지되다 커밋 시에만 PATCH가 나간다****: `CartLineItem`의 입력란은 `qtyDraft`(문자열, `null`=편집 중 아님)를 별도로 두고, `onChange`마다 draft 문자열만 바꾼다 — ****연속 키 입력마다 커밋하지 않는다****(`CartLineItem.tsx:74-85,112-113`). 커밋은 `onBlur` 또는 `Enter`(`→` `blur()`) 위임에서만 일어난다(`CartLineItem.tsx:113-116`). 스테퍼(±버튼)는 클릭 즉시 커밋(draft 경우 없음, `CartLineItem.tsx:107-121`). 이는 round1~2 QA가 실측 지정한 결함의 재작업 결과다 — 이전 구현(`value={qty}` 완전 제어 + 매 키 입력마다 커밋)은 재고 20·현재 2인 줄에서 "15"를 치려던 사용자가 "1" 시점에 PATCH{qty:1}이 먼저 확정되고 입력란이 비활성화돼 "5"를 받지 못하는 결함이 있었다(`CartLineItem.tsx:9-30` 주석). 빈 값/비숫자로 blur하면 직전 확인 수량으로 되돌리고 PATCH를 호출하지 않는다(`CartLineItem.tsx:82`). 커밋값이 직전 확인 수량과 같아도 PATCH를 호출하지 않는다(`CartLineItem.tsx:83`).

****표시값은 항상 서버가 마지막으로 확인해 준 수량****: 낙관적 갱신 없음. PATCH 진행 중/실패 시에도 별도로 "되돌릴" 로컬 값을 두지 않는다 — 그 값이 곧 직전 확인 수량이다(`CartPage.tsx:102-107,130-133`).

****연타 방지(품목별)**:** `inFlightSkusRef`(Set) — 같은 sku에 PATCH/DELETE가 진행 중이면 새 요청을

무시한다(`CartPage.tsx:110,125,135,142-143,155`). [주문하기]는 별도 동기 ref

(`orderNavigateRef`)로 잠근다(`CartPage.tsx:161-165`).

****알려진 미해결 결함(회귀 아님, 후속 추적)**:** 어떤 sku의 PATCH가 in-flight인 동안 같은 줄

입력란에서 blur 커밋이 들어오면 `inFlightSkusRef` 가드에서 ****안내 없이 조용히 버려진다****

(`CartPage.tsx:108-110` 진입부 가드). QA round2 CONCERNS 권고3-round3 재작업 지시로

식별되었으나 "이번에는 고치지 않음"으로 이월(STORY-1.md "후속 추적(TODO)" 절, SR-308 이전

범위 밖 — §6 참조).

4. 호출 API

트리거	API	용도	성공 처리	실패 처리
화면 진입 (세션 있을 때만)	GET /api/cart (`fetchCart`)	장바구니 라인 조회 (`{items,totalAmount}`에서 items만 사용)	`cartRows` 세팅, 전체 sku를 `selected`로 초기화(전체선택 기본)	그 외 상태 → `loadError`에 표시 문구, [다시 시도] 버튼으로 재호출(`load`)
화면 진입(1회)	GET /api/products (`fetchProducts`, 무인자, 기존 API 재사용)	라인별 재고 (`stockQty`)-이미지(`imageUrl`) join(신규 API 아님)	sku로 매칭해 라인아이템 구성. 매칭 실패 시 재고 상한을 현재 수량으로 두어 조작을 부당하게 막지 않음(`CartPage.tsx:82-89`)	실패 시 `Promise.all`이 함께 실패 → 위 로드 실패 안내로 합류
수량 ± 클릭 / 직접 입력 blur-Enter 커밋	PATCH /api/cart/items/{sku} (`updateCartItemQty`)	클램프 통과한 수량으로 변경	응답의 `qty`,`lineTotal`로 그 행을 다시 그림(서버 응답이 정본)	400(qty)/404(품목없음)/409(재고초과) → 서버 message를 그 줄 오류로 표시 + 표시 값은 직전 확인 수량 유지. 5xx 등 계약 밖 → 일반 문구(\$5-httpErrorMessage.ts)
삭제 클릭	DELETE /api/cart/items/{sku} (`deleteCartItem`)	즉시 삭제(확인 다이얼로그 없음)	그 행 제거 + 선택·경고·오류 상태 정리	404 등 → 그 줄 오류 표시(행은 유지)
[주문하기] 클릭(전체 선택일 때만 활성화)	(API 아님) `navigate('/shop/order')`	주문서(UIS-ORD-011)로 이동	라우트 이동, 상태 전달 없음(주문서가 자체적으로 GET /api/cart 재조회)	해당 없음
[쇼핑 계속하기](빈 장바구니)	(API 아님) ` <a href="#/shop/products">`	상품 목록으로 이동	라우트 이동	해당 없음
[로그인 하러 가기](세션 없음)	(API 아님) ` <a href="#/login">`	로그인 화면 이동	라우트 이동	해당 없음

- > 이 화면이 신규로 만든 서버 API는 없다 — 기존 GET/PATCH/DELETE `/api/cart\*`를 그대로 소비만
- > 한다(구현 범위는 `shop-web` 프론트엔드로 한정, STORY-1.md "확정된 요건 문답" 절). 프론트
- > 전용 오류 타입 `OrderHttpError` (HTTP status만 보관, `code` 없음, `api.ts:184-190`)는 서버
- > 계약이 아니므로 새 오류 봉투로 반영하지 않는다.

내부 상세 문자열 노출 방지 — 상태코드 허용목록이 주 방어선

서버 message를 그대로 화면에 보여줄지는 ****상태코드 허용목록****(`ALLOWED\_MESSAGE\_STATUSES = [400, 404, 409]`, `httpErrorMessage.ts:24`)이 1차로 판정한다 — 이 목록에 없는 상태(5xx 등 어떤 컨트롤러도 명시 처리하지 않는 상태)는 메시지가 아무리 정상 문구처럼 보여도 무조건 일반 문구 ("일시적 오류입니다. 다시 시도해 주세요")로 대체한다. 허용목록을 통과한 메시지에는 보조로 정규식 거부목록(`looksLikeInternalDetail`, 자바 예외 클래스명·스택프레임·파일경로 패턴)을 추가로 적용한다(`httpErrorMessage.ts:38-47`). 라인 오류·장바구니 로드 실패 두 표시 지점 모두 같은 함수(`toDisplayMessage`) 하나만 쓴다(round2 QA FAIL 필수3 재작업 — 이전엔 필터가 `OrderFailureNotice` 한 곳에만 있어 이 두 지점이 서버 원문을 무필터로 노출했다).

5. 표시 조건(상태)

- > 표시 조건은 이 화면(및 하위 부품) 각각의 `stories.tsx`로 남겨져 있다(`.speclinker/storybook\_index.json`
- > 기준 10개). 한 행 = 한 스토리.
- >
- > ▲ ****스토리 태그 불일치(미확인 사항 §6 참조)****: 아래 스토리들은 실제로 이 화면(`CartLineItem`/`CartEmptyState`/`CartSummary`)의 상태이지만, `stories.tsx` 파일의 `tags`는 `UIS-ORD-011`로
- > 박혀 있다(STORY-1.md 작성 시점의 예약 ID가 그대로 남음 — 실제로 `UIS-ORD-011`은 주문서
- > `shop\_order` 화면에 배정됨). 컴포넌트 경로(`features/shop/Cart\*.tsx`)로 대조해 이 화면 것으로
- > 확인했다.

요소	표시 조건	근거	스토리
라인아이템 — 기본	선택됨, 조작 가능한 정상 상태	`CartLineItem.tsx`	[기본](story:장바구니-라인아이템--기본)
라인아이템 — 선택해제	체크박스만 꺼짐	`CartLineItem.tsx:90`	[선택해제](story:장바구니-라인아이템--선택해제)
라인아이템 — 수량변경 중	`pending=true` → 조작 버튼 비활성 + "변경 중..." 표시	`CartLineItem.tsx:107,117,122,134`	[수량변경중](story:장바구니-라인아이템--수량변경중)

라인아이템 — 재고초과 안내	클라이언트 클램프가 최대 수량을 즉시 안내(API 호출 없음)	`CartPage.tsx:118`, `CartLineItem.tsx:125-127`	[재고초과안내](story:장바구니-라인아이템--재고초과안내)
라인아이템 — 최소수량 안내	1 미만으로 내리려는 시도를 즉시 안내(API 호출 없음)	`CartPage.tsx:118`	[최소수량안내](story:장바구니-라인아이템--최소수량안내)
라인아이템 — 수량변경 실패	PATCH 400/409 응답 → 서버 message 그대로 그 줄에 노출('shows-error')	`CartPage.tsx:133`, `CartLineItem.tsx:128-130`	[수량변경실패](story:장바구니-라인아이템--수량변경실패)
빈 장바구니	담긴상품 0건	`CartPage.tsx:203-204`, `CartEmptyState.tsx`	[빈장바구니](story:장바구니-빈-장바구니--빈장바구니)
합계 — 전체 선택	selectedCount === totalCount → [주문하기] 활성화	`CartSummary.tsx:28,31`	[전체선택](story:장바구니-합계--전체선택)
합계 — 부분 선택	0 < selectedCount < totalCount → [주문하기] 비활성 + "부분 선택 주문은 지원하지 않습니다 — 제외할 상품은 삭제해 주세요"(aria-describedby로 버튼과 연결)	`CartSummary.tsx:29,33-34,47-54`	[부분선택](story:장바구니-합계--부분선택)
합계 — 선택 없음	selectedCount === 0(totalCount>0) → [주문하기] 비활성 + "주문할 상품을 선택해 주세요"(같은 aria-describedby id 재사용, 부분선택과 상호 배타)	`CartSummary.tsx:30,35-37`	[선택없음](story:장바구니-합계--선택없음)

- > 문서화되지 않은 추가 상태(스토리 없음, 소스 근거만): 로그인 필요(`CartPage.tsx:186-190`) ·
- > 로딩 중(`CartPage.tsx:192`) · 장바구니 로드 실패(`CartPage.tsx:193-202`, [다시 시도] 포함) —
- > 이 셋은 `CartPage`(컨테이너, 스토리 대상 아님, 규칙 `story-per-component` 제외 대상)가 직접
- > 렌더하는 상태라 부품 스토리로 남지 않는다.

6. 합계 규칙 — 배송비는 임시 가정(정책 미확정)

`cartTotals.ts`의 `calcCartTotals`가 선택된 라인의 `lineTotal`만 합산해 상품금액을 구하고

(서버가 이미 계산한 `price×qty`를 그대로 쓴다 — 클라이언트가 재계산해 서버 응답을 덮지 않음),

결제예정금액 = 상품금액 + 배송비다.

- > △ **배송비 규칙(`SHIPPING\_FEE\_FLAT = 3000`, `cartTotals.ts:11`)은 서버 정보가 아니라 임시
- > 가정이다.** 선택 항목이 1개 이상이면 고정 3,000원, 0건이면 0원(무료배송 기준금액 없음) —
- > 명세·용어집·기존 코드 어디에도 이 규칙의 근거가 없고, 사람이 STEP 3-0 게이트 회신에서
- > "계획대로 진행"으로 임시 확정된 값이다(STORY-1.md "데이터" 절, "사람 수정" 절). **정책의
- > 실제 확정(기본요금·무료배송 기준금액·정보 위치)은 후속 SR-308 대상**이며, 이 화면과
- > `OrderPage`/`OrderCompleteNotice`가 이 상수 하나에 함께 의존한다(배송비만큼 두 화면이 서로
- > 어긋나던 결함의 재발 방지, `calcTotalsFromProductAmount` 참고).

7. 부분선택 체크아웃 미지원 — 서버 계약과의 정합

서버(``CartService.checkout``, `POST /api/cart/checkout`)는 ****선택 여부와 무관하게 회원의 장바구니 전체를 주문으로 전환****한다(요청 바디는 `{memberId}` 뿐, 계약 변경 금지 확정답변). 화면이 "선택 항목만 주문"하는 것처럼 보이는 상태로 그대로 다음 화면(주문서)에 넘기면, 화면 표시(선택분)와 실제 서버 동작(전체)이 어긋난다.

이를 막기 위해 이 화면은 ****[주문하기]를 선택 개수 === 전체 개수(전체선택)일 때만 활성화****하고, 부분선택이면 안내 문구로 막는다(\$5 "합계 — 부분선택"). 제외하고 싶은 상품은 개별 [삭제]로 치우는 것이 대체 경로다(개별 삭제 API는 이미 있음). 합계(상품금액/배송비/결제예정금액) 자체는 선택 상태 기준으로 계속 미리보기를 보여준다(전체선택 강제와 무관하게).

8. 미확인 사항

****배송비 정책 미확정****: \$6에서 명시한 대로 `SHIPPING_FEE_FLAT=3000` 규칙은 명세·용어집·기존 코드 어디에도 근거가 없는 임시 가정이다. 실제 기본요금·무료배송 기준금액·정본 위치 확정은 ****SR-308****(후속) 대상 — 이 SR(SR-305) 및 이 화면 범위 밖.

****부분선택 체크아웃 미지원**(서버 계약 변경 필요, 범위 밖)**: `CartService.checkout`이 품목 단위 주문을 지원하지 않아 이 화면은 "전체선택 아니면 [주문하기] 비활성"으로 우회했다. 품목 단위 체크아웃이 필요해지면 `POST /api/cart/checkout`에 `skus` 파라미터를 추가하는 계약 변경이 필요하며, 이는 새 API/필드 추가 없음이라는 SR-305 확정답변을 벗어나므로 ****후속 SR 대상****(STORY-1.md "범위 밖" 절).

****스토리 태그(tags) 불일치****: \$5의 10개 스토리는 실제로 이 화면(UIS-ORD-012)의 상태이지만 `.stories.tsx`의 `tags`는 STORY 작성 시점의 예약 ID `UIS-ORD-011`을 그대로 갖고 있다(실제 `UIS-ORD-011`은 주문서 `shop_order` 화면에 배정됨, STORY-1.md QA round1 코멘트 "예약 ID 그대로 — 사례집 줄 45 재발 주의" 참조). `story_link.py sync --uis UIS-ORD-012 --apply`로 태그를 교정하는 것을 권고 — 현재는 컴포넌트 경로 대조로 수동 확인해 연결했다.

****in-flight 중 blur 커밋이 안내 없이 버려지는 결함****(\$3 참조): QA round2 CONCERNS 권고3·round3 재작업 지시로 식별됐으나 "이번에는 고치지 않음"으로 명시 이월(STORY-1.md "후속 추적(TODO)" 절) — 결함이지만 이번 SR 범위에서 의도적으로 미해결.

그 외 화면 구조·API·표시 조건·클램프/커밋 규칙은 소스(`CartPage.tsx` 외 `features/shop/Cart`)와 STORY-1.md 확정 답변·QA 결과로 전부 확인됨.

변경 이력

<!-- spec-history: machine-managed -->

일자	SR	항목	변경 요약	커밋
2026-09-17	SR-305	#1	장바구니 화면(/shop/cart) 신규 — 수량 직접입력 draft/blur 커밋, 부분선택 체크아웃 차단(서버 전 체전환 계약), 내부오류 상태코드 허용목록, 배송비 SHIPPING_FEE_FLAT=3000 임시가정(SR-308 이월)	shop-web@ca3335c
2026-09-17	SR-305	#2	소스 기준 SOP급 본문 보강(화면개요·구성·입력검증·API·표시조건·배송비 임시가정·부분선택 미지원·미확인사항)	shop-web@ca3335c

화면 상태 10건 — 기준 스냅샷

담긴 상품이 없습니다

쇼핑 계속하기

빈 장바구니 · 빈장바구니

☒

테

테스트 상품
15,000원

- 2 +

30,000원

삭제

라인아이템 · 기본

☐

테

테스트 상품
15,000원

- 2 +

30,000원

삭제

라인아이템 · 선택해제

☒

테

테스트 상품
15,000원

- 2 + 변경 중...

30,000원

삭제

라인아이템 · 수량변경중

☒

테

테스트 상품
15,000원

- 5 +

최대 수량은 5개입니다

30,000원

삭제

라인아이템 · 재고초과안내

☒

테

테스트 상품
15,000원

- 1 +

1개 미만으로는 변경할 수 없습니다

30,000원

삭제

라인아이템 · 최소수량안내

☒

테

테스트 상품
15,000원

-

1

+

재고 초과: 가용 1, 요청 5

30,000원

삭제

라인아이템 · 수량변경실패

상품금액	55,000원
배송비	3,000원
결제예정금액	58,000원
주문하기	

합계 · 전체선택

상품금액	30,000원
배송비	3,000원
결제예정금액	33,000원
부분 선택 주문은 지원하지 않습니다 — 제외할 상품은 삭제해 주세요	
주문하기	

합계 · 부분선택

상품금액	0원
배송비	0원
결제예정금액	0원
주문할 상품을 선택해 주세요	
주문하기	

합계 · 선택없음

API 설계서

INF-MBR-001 회원가입 인증코드 발송

INF-MBR-001: POST /api/members/signup/verification-codes — 회원가입 인증코드 발송

> ****개요:**** 이메일 또는 휴대폰번호를 받아 6자리 인증코드를 생성·저장(5분 유효)하고 "발송"한다(이 랩은 실제 발송 게이트웨이가 없어 로그로 대체). 인증코드 확인 + 회원 생성(가입완료)은 별도 API(INF-MBR-002, FUNC-member-003)의 책임이며 이 API 범위 밖이다.

> ****근거 소스:**** `modules/shop-api/src/main/java/com/sm/lab/shop/controller/MemberSignupController.java:38-42`

요청

Method: POST

Path: /api/members/signup/verification-codes

Content-Type: application/json

인증: 없음(무인증) — 신규 가입자는 아직 API 키가 없으므로 `ApiKeyAuthFilter` 화이트리스트(`MEMBER\_SIGNUP\_VERIFICATION\_CODE\_PATH`)에 등록되어 있다.

파라미터	위치	타입	필수	설명
target	Body	string	Y	인증코드를 받을 대상 — 이메일 주소 또는 휴대폰번호(하이픈 없이 숫자만). 최대 100자(DB 컬럼 `target VARCHAR(100)` 정합). 형식으로 채널(EMAIL/SMS)을 자동 판별한다

응답 (200 OK)

```
{
  "channel": "EMAIL",
  "target": "user@example.com",
  "expiresInSeconds": 300
}
```

`channel`: 판별된 발송 채널 — `EMAIL` 또는 `SMS`

`target`: 요청받은 값의 에코(trim 처리됨) — 개인정보 누출 아님(요청자 본인이 보낸 값)

`expiresInSeconds`: 코드 유효시간(고정값 300) — 코드 자체는 응답에 담기지 않는다(로그에도 마스킹)

비즈니스 규칙

인증코드는 6자리 숫자, 발급 시점부터 5분(300초) 유효(`MemberSignupService.EXPIRES\_IN\_SECONDS`)

채널 자동판별: 이메일 정규식 매칭 → `EMAIL`, 국내 휴대폰번호 패턴(`01[016789]` + 7~8자리, 하이픈 없음) 매칭 → `SMS`, 둘 다 아니면 400

레이트리밋(동일 `target` 기준, 채널 무관): 60초 쿨다운 + 하루 5회 상한. ****거부된 요청은 quota를 소비하지 않는다**** —

원자 UPSERT의 조건부 갱신이라 `daily\_count`/`last\_token`이 갱신되지 않으면 그 요청은 카운트에 반영되지 않는다

판정 방식(round5/round6, 사람 결정): 요청마다 생성한 UUID 토큰을 조건부로 `last\_token`에 기록한 뒤 재조회한 값이 자기 토큰과 같으면 허용 — JDBC affected-rows나 datasource 전역 설정(`useAffectedRows`)에 더 이상 의존하지 않는다(그 설정이 이 FUNC 밖 `ProductDao.decreaseStock`의 반환값 시맨틱까지 바꿔 `POST /api/orders`를 200→409로 회귀시킨 적이 있어 제거됨)

발송은 시뮬레이션(로그 1줄, 개인정보 마스킹 — 앞 2자 제외 `\*\*` 처리) — 실제 이메일/SMS 게이트웨이 미연동. 코드 값은 응답·로그 어디에도 노출하지 않는다

레거시 주의: `MEMBER\_SIGNUP\_VERIFICATIONS.requested\_at`/`previous\_requested\_at`/`daily\_count` 컬럼은 round3 잔재(deprecated) — round4부터 앱이 더 이상 쓰지 않는다(카운터는 `MEMBER\_SIGNUP\_RATE\_LIMITS`로 완전히 이전됨).

재발송 시 시도 횟수·소비 표시 리셋(SR-295): 같은 채널·타겟 행에 재발송하면 새 6자리 코드·만료시각과 함께

`attempt\_count=0`, `consumed\_at=NULL`, `verified\_at=NULL`로 함께 리셋한다 — 잠긴(`attempt\_count` $\geq$ 5) target이 재발송을 받으면 새 코드로 정상 검증 가능해진다(INF-MBR-002 `MBR-4093` 회복 경로).

`attempt\_count`/`consumed\_at`/`verified\_at`의 판정·소비는 여전히 INF-MBR-002(FUNC-member-003)가 소유한다 — 이 API는 재발송 시 초기화만 한다.

만료행 정리(purge)는 이 요청 경로에 없다 — `MemberSignupMaintenanceScheduler`(`@Scheduled`, 10분 주기 배치)가 `purgeExpiredCodes`/`purgeOldRows`를 별도로 호출한다

트랜잭션 순서

1. `MEMBER\_SIGNUP\_RATE\_LIMITS`에 원자 UPSERT(`touchRateLimit`) — 쿨다운·일일상한 조건을 동시에 만족할 때만 `daily\_count`/`last\_requested\_at`/`last\_token`을 이번 요청 토큰으로 갱신(단일 문장, autocommit — `@Transactional` 없음. round4에서 의도적으로 제거해 `purgeExpired`와의 락 경합·데드락을 없앴다)

2. 같은 `target`/`day\_key`를 재조회(`selectRateLimit`)해 `last\_token`이 내 토큰과 같은지 확인 — 다르면 여기서 429로 즉시 응답하고 코드는 생성하지 않는다

3. 판정을 통과한 경우에만 `MEMBER\_SIGNUP\_VERIFICATIONS`에 원자 UPSERT(`writeCode`)로 6자리 코드·만료시각을 기록한다(1단계와 별개의 autocommit 문 — 하나의 트랜잭션으로 묶여 있지 않다). 재발송(기존 행 갱신)이면 같은 문장에서 `attempt\_count=0`, `consumed\_at=NULL`, `verified\_at=NULL`도 함께 리셋한다(SR-295) — 신규 행(INSERT)은 DDL 기본값이 이미 이 값들이라 리셋 대상이 없다

오류 응답

응답 봉투는 모두 `{ "code": "...", "message": "..." }` 형태(`MemberSignupExceptionHandler`).

코드	사유	발생 조건
400	MEMBER_TARGET_INVALID	`target`이 비었거나 100자를 초과, 또는 이메일·휴대폰번호 형식에 둘 다 불일치
429	MEMBER_VERIFY_COOLDOWN	동일 `target` 재발송을 60초 이내에 재요청(쿨다운 미경과)
429	MEMBER_VERIFY_DAILY_LIMIT_EXCEEDED	동일 `target`이 오늘 이미 5회 요청을 완료
500	MBR-5000	DB 계층 예외(`DataAccessException`) — 정제된 메시지만 응답, 경로·SQL·커넥션 정보는 노출하지 않으며 원본은 서버 로그에만 남는다(무인증 엔드포인트라 정보 노출 위험이 특히 큼)

참조 테이블

`MEMBER\_SIGNUP\_VERIFICATIONS`

`MEMBER\_SIGNUP\_RATE\_LIMITS`

curl 예시

```
curl -X POST /api/members/signup/verification-codes \
```

```
-H "Content-Type: application/json" \  
-d '{"target": "user@example.com"}'
```

변경 이력

<!-- spec-history: machine-managed -->

일자	SR	항 목	변경 요약	커밋
2026-09-15	SR-295	#1	재발송(writeCode) 시 attempt_count/consumed_at/verified_at을 함께 리셋하도록 UPSERT SET절 확장 — MBR-4093 회복 경로	shop-api@45e21e8

INF-MBR-002 가입 요청(인증코드 검증 포함)

> [반영: FUNC-member-003] 2026-09-12

INF-MBR-002: POST /api/members/signup — 가입 요청 (인증코드 검증 포함)

> ****개요:**** 이메일 또는 휴대폰번호 + 인증코드(6자리) + 비밀번호·이름·마케팅 수신 동의를 받아 회원을 생성한다. 인증코드 "발송"은 별도 API(INF-MBR-001)의 책임이지만, 코드 ****검증****(대조·소비)은 발송 쪽이 아니라 이 API 자신이 수행한다 — `MEMBER\_SIGNUP\_VERIFICATIONS` 행은 INF-MBR-001이 써 둔 것을 이 API가 조건부 UPDATE로 대조·소비할 뿐, 그 테이블을 소유한 파일은 건드리지 않는다. round1~4에 걸친 QA 재작업으로 "① 코드 검증(비트랜잭션) → ② 존재 판정(비트랜잭션, 안내용) → ③ 가입(트랜잭션, 별도 스프링 빈)"의 3단계 순서가 사람이 직접 지정한 것으로 고정됐다 — 이 순서 자체가 보안 요건(존재 오라클 방지)이므로 임의로 바꾸면 안 된다.

> ****근거 소스:**** `modules/shop-api/src/main/java/com/sm/lab/shop/controller/MemberRegistrationController.java:34-40`

요청

Method: POST

Path: /api/members/signup

Content-Type: application/json

인증: 없음(무인증) — 신규 가입자는 아직 API 키가 없으므로 `ApiKeyAuthFilter` 화이트리스트

(`MEMBER\_SIGNUP\_REQUEST\_PATH`, 정확 일치)에 등록되어 있다. ****레이트리미트 없음****(아래 오류 표·비즈니스 규칙 참고).

파라미터	위치	타입	필수	설명
target	Body	string	Y	인증코드를 받은 대상 — 이메일 주소 또는 휴대폰번호(하이픈 없이 숫자만). 값 자체로 채널(EMAIL/SMS)을 자동 판별(INF-MBR-001과 동일 정규식을 이 서비스에 복제해 사용)
code	Body	string	Y	INF-MBR-001이 발송한 6자리 인증코드. 이 API가 직접 대조·소비한다
password	Body	string	Y	8~64자, 영문+숫자 포함(정규식 위반 시 400)
name	Body	string	Y	회원 이름, 최대 50자(빈 값·공백만·50자 초과 시 400 — `MEMBERS.member_name VARCHAR(50)` 정합)
marketingOptIn	Body	boolean	N	마케팅 수신 동의. 생략(null) 시 `false`로 처리

****서버 내부 검증 순서(그대로 구현된 순서 — 응답 코드가 이 순서에 의존한다):****

1. `target` 공백/누락 → 409 `MBR-4091` (형식이 성립 않으면 인증 기록도 존재할 수 없다는 논리로 "인증 필요"에 수렴)
2. `target` 형식이 이메일·휴대폰 정규식 둘 다 불일치 → 409 `MBR-4091` (동일 논리)
3. `name` 공백/누락/50자 초과 → 400 `MBR-4001`
4. `password` 규칙 위반 → 400 `MBR-4001`
5. (STEP 1) 코드 검증 실패 → 409 `MBR-4091` 또는 `MBR-4093`

6. (STEP 0) 코드 검증 성공 후에만 중복 판정 → 409 `MBR-4092`(`+login\_url`) 또는 `MBR-4094`

7. BCrypt 해시 계산 → (STEP 2) 가입 트랜잭션 → 성공 시 201

응답 (201 Created)

```
{
  "memberId": "M-0053",
  "channel": "EMAIL",
  "target": "user@example.com"
}
```

`memberId`: 신규 채번된 회원 ID, `M-` + `%04d` 포맷(`ID\_SEQUENCES` 원자 채번, `MEMBER\_ID` 시퀀스)

`channel`: 판별된 채널 — `EMAIL` 또는 `SMS`

`target`: 요청받은 값의 에코(trim 처리됨)

응답에 이메일·비밀번호 해시·마케팅 동의값 등은 포함하지 않는다

비즈니스 규칙

****3단계 흐름과 트랜잭션 경계(사람이 직접 지정, round3~4 최종 확정 — 임의 변경 금지):****

****STEP 1 — 코드 검증****(`MemberRegistrationService#verifyCode`, 비트랜잭션): `markVerifiedIfCodeMatches` 조건부 UPDATE(`channel`+`target`+`code`+미만료+`consumed\_at` IS NULL+`verified\_at` IS NULL+`attempt\_count` < 5)로 검증. 0행이면 `incrementAttemptCount`(`SR-298` — `channel`+`target`+`consumed\_at` IS NULL+`verified\_at` IS NULL+`attempt\_count` < 5` 조건부 UPDATE로 증가와 상한판정을 한 문장에 원자화, read-modify-write 아님)를 시도한다. 이 원자 UPDATE가 1행이면 정상 증가 후 409 `MBR-4091`. **\*\*0행이면(사유가 상한 도달 하나가 아니다 — 이미 소비·이미 인증된 행이거나 매치 행 자체가 없어도 0행이다) 그 순간만 `selectAttemptCount`로 현재 값을 읽어 원인만 재분기\*\***한다: `attempt_count` >= 5(상한 도달)면 409 `MBR-4093`, 그 밖의 모든 0행 사유는 409 `MBR-4091` (재증가하지 않는다 — 증가는 이미 원자 UPDATE에서 끝났다). 이 읽기는 경합 판정이 아니라 오류 코드 표시용일 뿐이며, "5회 도달 후 카운터가 더 늘지 않는다"는 성질은 이제 이 UPDATE의 `attempt_count` < 5` 조건절 자체가 보장한다(select→분기→update 패턴 아님). 각 UPDATE는 autocommit 개별 문장 — 이후 어떤 예외로도 롤백되지 않는다.

****STEP 0 — 존재 판정****(`precheckDuplicate`, 비트랜잭션 읽기, ****STEP 1 성공 뒤에만 호출****): `MEMBERS`를 `email`/`phone\_norm`으로 직접 조회해 이미 있으면 이메일→409 `MBR-4092`(`+login\_url`), 휴대폰→409 `MBR-4094`. 안내용일 뿐 최종 보장은 STEP 2의 UNIQUE 위반 캐치다.

****STEP 2 — 가입****(`MemberSignupCompletionWriter#completeSignup`, `@Transactional`, 이 메서드만 갖는 별도 스프링 빈): ①`ID\_SEQUENCES` 원자 채번(`touchMemberIdSeq`+`selectLastMemberIdSeq`, 같은 커넥션 필수 — `LAST\_INSERT\_ID()` 세션 스코프) ②`MEMBERS` INSERT ③`MEMBER\_SIGNUP\_VERIFICATIONS.consumed\_at` UPDATE. ****이 세 문장만**** — 별도 빈으로 분리한 이유는 self-invocation(같은 클래스 안 호출은 프록시를 거치지 않아 `@Transactional`이 적용되지 않는 Spring AOP 함정)을 원천 차단하기 위함(과거 라운드에서 이 함정으로 시도횟수 카운터가 롤백되는 결함이 있었다).

`MemberRegistrationService#signUp` 자체는 `@Transactional`이 전혀 없는 순수 오케스트레이션 메서드다.

****존재 오라클 방지(사람이 명시 승인한 보안 우선 트레이드오프):**** STEP 1(코드 검증)이 STEP 0(존재 판정)보다 항상 먼저 실행된다. 코드 검증에 실패하면(오답·만료·미요청·이미 소비/인증됨·시도상한 도달 등 ****어떤 사유든****) STEP 0은 호출조차 되지 않는다 — 즉 ****코드 검증 실패는 그 target의 가입 여부와 무관하게 항상 동일한 409 `MBR-4091`****(같은 문구, 존재 여부에 따른 추가 조회가 없어 응답 시간대도 갈리지 않음)이다. 코드를 받지 못한 요청자는 이 target의 가입 여부를 알아낼 방법이 없다.

****부작용(사람이 명시 수용, round4 — SR-295로 발생 조건 축소):**** "이미 가입 완료된 target"이 ****재발송 없이**** 예전 코드로 재시도하면(그 코드의 `consumed\_at`이 이미 채워져 있으므로) STEP 1 자체가 실패해 409 `MBR-4092`가 아니라 ****409 `MBR-4091`****을 받는다. 자가치유는 있다 — 코드 만료(5분) + 정리배치(`MemberSignupMaintenanceScheduler`, 10분 주기)가 행을 지우거나, ****재발송을 받으면 즉시****(SR-295, `consumed\_at`이 리셋되므로) 다음 시도에서 정상적으로 `MBR-4092`(`+login\_url`)가 나온다. 실사용 영향은 "가입 직후 같은 날, 재발송 없이 재시도" 구간으로 한정된다.

****코드값 의미**:**

`attempt\_count` >= 5: 해당 (channel,target) 코드는 잠금 상태 — 정답이어도 더 이상 매치되지 않음(409 `MBR-4093`). **회복 경로(SR-295)**: 재발송(`writeCode`, INF-MBR-001 소유)이 같은 행의 `attempt\_count`를 0으로, `consumed\_at`/`verified\_at`을 `NULL`로 리셋하므로, 상한에 걸린 target도 재발송을 받으면 새 코드로 STEP 1 조건 (`attempt\_count` < 5)을 다시 만족해 정상 검증 가능해진다. 이 STEP(STEP 1 SQL-조건식:순서)은 순대지 않았고, 선행 상태만 바뀐다.

병렬 버스트 우회 차단(SR-298): `incrementAttemptCount`의 `attempt\_count` < 5` 조건절이 단일 PK(`channel`,`target`) 행 락 위에서 재평가되므로, 동시에 도착한 오답 요청들도 이 한 문장의 짧은 수명 동안만 직렬화된다 — 상한(5)을 넘겨 증가할 수 없다(이전에는 "현재값 읽기 → 상한 미만이면 별도 증가" 순서라 두 문장 사이 레이스로 5를 초과해 증가할 수 있었다). 0행 반환 시의 `selectAttemptCount` 읽기는 이 보장과 무관하다 — 그 시점엔 증가 여부가 이미 원자적으로 확정된 뒤이며, 오류 코드(4091 vs 4093) 표시에만 쓰인다. 부가로, 이 읽기가 있을 때(verification 행 없음) 3문장 / 없을 때(행 있고 1행 매치) 2문장으로 문장 수가 같리지만, 어느 쪽도 `MEMBERS` 테이블을 조회하지 않아 존재 오라클 방지에는 영향이 없다(관측 가능한 응답 시간대 차이는 verification 테이블 PK 조회 1회 수준 — 회원 존재 신호를 만들지 않는다).
 `consumed\_at` IS NOT NULL: 코드가 이미 가입에 사용됨 — 같은 코드로 재검증 시도는 항상 0행(재사용 차단).
 `verified\_at`: STEP 1이 세팅하는 검증 시각. STEP 0에서 예외가 나도(트랜잭션이 아니므로) 롤백되지 않고 커밋된 채 남는다.
 login_url 필드: 이 랩에는 실제 로그인 화면 라우트가 없다(grep 0건) — `"/login"`은 자리표시(placeholder) 값이며 실제하지 않는 경로다. MBR-4092 응답에만 채워진다.

비밀번호 해시 시점: BCrypt 계산은 STEP 1(코드 검증) 이후, STEP 2(가입 트랜잭션) 진입 직전에 수행한다 — 무인증 엔드포인트에서 시도 상한에 걸리는 요청까지 매번 BCrypt(수십~백 ms)를 돌리지 않기 위함.

미분류(PK) 충돌 재시도: STEP 2에서 `DuplicateKeyException` 발생 시 메시지에 실린 제약 이름으로 `uq\_members\_email`→4092, `uq\_members\_phone\_norm`→4094로 분기하고, 둘 다 아니면(주로 PK 충돌) 완전히 새 트랜잭션으로 1회만 재채번 재시도한다 — 재시도도 실패하면 rethrow하여 일반 500 `MBR-5000`으로 떨어뜨린다(업무 409로 오분류하지 않음).

레거시/미해결 주의: ① `MemberSignupCompletionDao#selectVerifiedAt`/`#selectConsumedAt`는 테스트/조회 전용이며 운영 경로에서 호출되지 않는 죽은 코드로 남아 있다. ② 탈퇴 회원(`del\_yn`='Y')의 email/phone_norm이 UNIQUE 인덱스를 계속 점유해 재가입이 영구 불가하다(사전 판정도 `del\_yn`을 보지 않아 UNIQUE와 판정을 의도적으로 일치시킴) — 이 API 범위 밖, 백로그. ③ 이 엔드포인트에는 어떤 레이트리밋도 없다(존재 오라클은 막혔지만, 제3자가 "가입 진행 중인" target의 시도 5회를 대신 소진시켜 그 target의 가입을 일시 방해하는 경로는 남아 있다 — 일일 발송 상한 5회로 영향은 제한적).

트랜잭션 순서

1. (비트랜잭션, autocommit) STEP 1 — `MEMBER\_SIGNUP\_VERIFICATIONS` 조건부 UPDATE(`markVerifiedIfCodeMatches`). 실패 시 원자 조건부 UPDATE(`incrementAttemptCount`, `attempt\_count` < 5` 포함 — SR-298 원자화)로 증가+상한판정을 한 문장에서 시도. 그 UPDATE가 0행이면(사유 다양 — 상한 도달/이미 소비/이미 인증/행 없음) `selectAttemptCount` 읽기 1회로 오류 코드만 재분기(재증가 없음). 즉시 응답 종료(이후 단계 진행 안 함).

2. (비트랜잭션, 읽기) STEP 0 — `MEMBERS`를 `email`/`phone\_norm`으로 조회(`selectMemberIdByEmailOrPhoneNorm`). 존재하면 즉시 응답 종료(이후 단계 진행 안 함).

3. (트랜잭션 시작, `MemberSignupCompletionWriter#completeSignup`) ① `ID\_SEQUENCES` UPSERT + `SELECT LAST\_INSERT\_ID()`로 `member\_id` 채번 ② `MEMBERS` INSERT ③ `MEMBER\_SIGNUP\_VERIFICATIONS.consumed\_at` UPDATE — 세 문장 커밋.

4. (트랜잭션 밖, 커밋 후) `MemberSignedUpEvent(memberId, target)` 발행 — 로그 리스너 1개 (`MemberSignedUpEventLogger`)만 구독, 개인정보는 앞 2자만 노출하고 마스크. 쿠폰 발급 로직은 이 API 범위 밖(별도 프로모션 SR 뒀).

오류 응답

응답 봉투는 모두 `{ "code": "...", "message": "... }` 형태(MBR-4092만 `login\_url` 필드 추가, `MemberRegistrationExceptionHandler`).

코드	HT TP	사유	발생 조건
MBR-4001	400	이름/비밀번호 규칙 위반	`name`이 비었거나 50자 초과, **또는** `password`가 8~64자·영문+숫자 포함 규칙 위반 (두 경우 모두 같은 코드로 응답)
MBR-4091	409	인증이 필요합니다	`target` 형식 오류, 또는 STEP 1 코드 검증 실패(오답·만료·미요청·이미 소비/인증됨) — **target의 가입 여부와 무관하게 항상 이 코드**(존재 오라클 방지)
MBR-4093	409	재발송 필요	STEP 1 진입 시점에 `attempt_count` >= 5(이미 시도 상한 도달, 정답 코드를 내도 거부)
MBR-4092	409	이미 가입된 이메일입니다 (+`login_url` 필드)	STEP 1 성공 후 STEP 0 또는 STEP 2에서 `email` 중복 확인(`uq_members_email`)
MBR-4094	409	이미 가입된 휴대폰번호입니다	STEP 1 성공 후 STEP 0 또는 STEP 2에서 `phone_norm` 중복 확인 (`uq_members_phone_norm`)
MBR-5000	500	일시적인 오류입니다	`DataAccessException`(미분류 PK 충돌 재시도 실패 포함) — 정제된 메시지만 응답, 원본은 서버 로그에만

참조 테이블

`MEMBERS`

`MEMBER\_SIGNUP\_VERIFICATIONS`

`ID\_SEQUENCES`

curl 예시

```
curl -X POST /api/members/signup \
  -H "Content-Type: application/json" \
  -d '{"target": "user@example.com", "code": "123456", "password": "abcd1234", "name": "홍길동", "marketingOptIn": true}'
```

변경 이력

<!-- spec-history: machine-managed -->

일자	SR	항 목	변경 요약	커밋
2026-09-15	SR-295	#1	STEP1/STEP0 로직·조건식은 무변경 — INF-MBR-001의 재발송 리셋으로 선행 데이터 상태가 바뀌어 MBR-4093 회복 경로가 생기고 재발송 후 재시도 시 MBR-4092(+login_url)로 귀결되는 부작용 문서화	shop-api@45e21e8
2026-09-16	SR-298	#1	incrementAttemptCount에 attempt_count<5 조건 추가해 증가+상한판정을 원자 UPDATE로 통합 (병렬 버스트 우회 차단). 0행일 때만 selectAttemptCount로 오류코드(4091/4093)만 재분기, 재증가 없음	shop-api@0ed4f7d

INF-MBR-003 로그인

> [반영: FUNC-member-005] 2026-09-12

INF-MBR-003: POST /api/members/login — 로그인

> ****개요:**** 이메일+비밀번호로 로그인해 세션(리프레시 토큰 30일 + API 키)을 발급한다. 5회 연속 실패 시 10분 잠금.

> ****근거 소스:**** `modules/shop-api/src/main/java/com/sm/lab/shop/controller/MemberLoginController.java:32-35`

요청

Method: POST

Path: /api/members/login

Content-Type: application/json

인증: 없음(무인증) — 로그인 전에는 아직 API 키가 없다. `ApiKeyAuthFilter`의 `isOpenRoute` 화이트리스트(`MEMBER\_LOGIN\_PATH`, 정확 일치)에 등록되어 있다(가입 관련 엔드포인트들과 동일한 이유의 예외).

파라미터	위치	타입	필수	설명
email	Body	string	Y	로그인 이메일(정규화는 `trim()`뿐 — 대소문자 동일시는 DB 콜레이션에 의존)
password	Body	string	Y	비밀번호(평문 전송, 서버에서 BCrypt 대조)

응답 (200 OK)

```
{
  "memberId": "M-0053",
  "memberName": "홍길동",
  "grade": "SILVER",
  "apiKey": "mk_1a2b3c4d5e6f...",
  "refreshToken": "5f2c9e10-.....5f2c9e10-.....",
  "refreshTokenExpiresAt": "2026-10-12T10:00:00"
}
```

`memberId`/`memberName`/`grade`: `MEMBERS` 조회 값(`MemberCredential`을 통해 전달, `passwordHash`는 응답에 절대 포함하지 않는다)

`apiKey`: 이 회원이 이후 `/api/\*\*` 호출에 쓸 키. 최초 로그인이면 신규 발급, 이미 있으면 기존 값 그대로 재조회(재발급 아님) — `MEMBER\_API\_KEYS.member\_id`가 PK라 회원당 항상 1개

`refreshToken`: 30일 유효, **원문이 노출되는 유일한 응답**. DB에는 SHA-256 해시(소문자 hex)만 저장(`MEMBER\_REFRESH\_TOKENS.token\_hash`)

`refreshTokenExpiresAt`: `LocalDateTime` 직렬화(ISO-8601 유사 문자열) — 정확한 포맷 단언은 FUNC-member-004(로그인 화면) 쪽 테스트가 함

비즈니스 규칙

****순서(사람이 직접 확정, 임의 변경 금지)**:** ① 잠금 여부 확인(`MEMBER\_LOGIN\_ATTEMPTS`, 비밀번호 검증 전 — BCrypt 비용 절약) → ② `MEMBERS`를 email로 조회(탈퇴 회원도 포함해서, `del\_yn` 필터 없이) → ③ 비밀번호 대조 → ④ 성공 시 실패카운터 reset → 세션 발급.

****존재 오라클 방지(3경우 완전 동일 응답)**:** "회원 없음" / "탈퇴 회원(`del\_yn`='Y')"/ "비밀번호 불일치" — 이 세 경우는 완전히 동일한 401 `MBR-4011`, 동일한 일반화 문구("이메일 또는 비밀번호가 올바르지 않습니다 (n/5)", 사유를 절대 구분

해 노출하지 않음), 동일한 카운터 증가 의미로 응답한다. 회원이 없거나 탈퇴한 경우에도 더미 BCrypt 해시로 `matches()`를 돌려 응답시간을 비슷하게 만든다(타이밍 사이드채널 완화, should).

****잠금 카운터는 회원 존재와 무관****: `MEMBER\_LOGIN\_ATTEMPTS`는 email 문자열 자체가 PK — 가입되지 않은 이메일도 카운트된다. 이렇게 해야 "이 이메일이 잠겼는가"라는 관찰 자체가 계정 존재의 오라클이 되지 않는다.

****5번째 실패 = 즉시 429(사람 확인 2로 확정)****: 4번째까지는 401(`\n/5` 문구), **5번째 실패는 401이 아니라 곧바로 429 `MBR-4291`**이다. `touchFailure` 한 문장(원자 UPSERT)이 `fail\_count` 증가와 `locked\_until` 세팅을 같은 SET 목록 좌→우 평가로 처리한다(세션변수 미사용). 잠금 만료 후 재실패는 `fail\_count`를 1로 리셋(무한 누적 방지).

****DB 폴백 인증(사람 확인 3으로 확정)****: `ApiKeyAuthFilter`는 정적 `lab.api-keys` 맵(admin-M-0001만 등록됨) 조회가 실패했을 때만(요청당 최대 1회) `MemberApiKeyDao#selectMemberIdByApiKey`로 DB 폴백 조회한다. 이 폴백은 `MEMBERS` 조인 + `del\_yn='N'` + `revoked\_at IS NULL`로 좁혀져 있어(round9 재작업), 탈퇴 회원이나 폐기된 키는 종전과 동일한 401을 받는다. 기존 정적 맵 경로(admin-M-0001)는 이 추가로 코드-동작이 전혀 바뀌지 않는다(정적 맵 히트 시 DB 조회 자체가 일어나지 않음).

****리프레시 토큰 하우스키퍼****: 로그인 성공마다 새 토큰 INSERT 직후 이 회원의 만료·폐기 행을 지우고(`purgeExpiredOrRevoked`), 활성 토큰이 상한(5개)을 넘으면 `issued\_at` 오래된 순으로 초과분을 지운다(`enforceActiveCap`). 정확한 상한 강제가 아니라 하우스키퍼 — 동시 로그인이 6건 이상이면 방금 발급한 토큰이 지워질 수도 있다(사람이 넘긴 low 이슈).

****API 키 동시 최초발급 레이스 방지****: `issuelfAbsent`는 `member\_id` PK에 대한 no-op UPSERT(`ON DUPLICATE KEY UPDATE member\_id = member\_id`)로, 동시에 여러 로그인 요청이 들어와도 실제로 저장되는 키는 하나뿐이다. 호출부는 이 UPSERT 직후 반드시 `selectByMemberId`로 재조회해야 한다(이 요청이 넘긴 후보 키가 실제로 쓰였다는 보장이 없음).

****`@Transactional` 미사용(사람 결정)****: 이 클래스는 트랜잭션을 전혀 쓰지 않는다 — 모든 DB 문장을 개별 autocommit 단일 statement로 처리한다(사례집 SR-231 r2 "카운터 증가가 트랜잭션 롤백에 같이 사라짐" 재발 방지).

****레거시/미해결 주의(QA가 지적, 사람이 waiver 수용 — FUNC-006 착수 시 확정 예정)****: `MEMBER\_API\_KEYS.revoked\_at`을 세팅하는 운영 코드는 아직 없다(FUNC-006 로그아웃 미구현). FUNC-006이 로그아웃에서 `revoked\_at`을 세팅하기 시작하면, 이 로그인 API의 `issuelfAbsent`(no-op UPSERT, `member\_id`가 PK)는 그 폐기 표시를 지우지 않고 이어지는 `selectByMemberId`도 `revoked\_at` 필터가 없어 죽은 키를 그대로 반환한다 — 로그아웃 후 재로그인한 회원이 200 응답과 함께 이미 무효화된 `apiKey`를 받는 ****영구 락아웃**** 경로가 남아 있다. 폐기된 키를 되살리거나 새로 교체하는 재발급 경로가 이 API에는 없다. 사람이 확정한 처리 방향(a: 로그인이 폐기 키를 새 키로 교체 / b: 006이 DELETE로 폐기 / c: apiKey 폐기를 006 범위에서 제외)은 FUNC-006 착수 시점에 결정한다(현재는 FUNC-006이 `Approved`-미구현 상태라 즉시 영향 없음).

트랜잭션 순서

1. (비트랜잭션, 읽기) `MEMBER\_LOGIN\_ATTEMPTS`에서 잠금 여부 확인(`selectAttempt`). 잠겨 있으면 즉시 429 응답, 이후 단계 진행 안 함.
2. (비트랜잭션, 읽기) `MEMBERS`를 email로 조회(`selectAuthByEmail`, `del\_yn` 필터 없음 — 탈퇴 회원도 조회해 동일 401로 처리하기 위함).
3. (비트랜잭션) 비밀번호 대조 실패(회원 없음/탈퇴/비번오류) → `touchFailure`(원자 UPSERT)로 실패 카운터 증가(+임계 도달 시 `locked\_until` 세팅) → 401 또는 429 응답, 이후 단계 진행 안 함.
4. (비트랜잭션) 대조 성공 → `reset`(단일 DELETE)으로 실패 카운터 제거.
5. (비트랜잭션) `MEMBER\_REFRESH\_TOKENS` INSERT(신규 토큰) → 같은 회원의 만료·폐기 행 DELETE(`purgeExpiredOrRevoked`) → 활성 토큰 상한 초과분 DELETE(`enforceActiveCap`).
6. (비트랜잭션) `MEMBER\_API\_KEYS` no-op UPSERT(`issuelfAbsent`) → `selectByMemberId`로 재조회 → 200 응답.

사이드이펙트

로그인 성공마다 `MEMBER\_REFRESH\_TOKENS`에서 ****이 회원의**** 만료·폐기 토큰 삭제 + 활성 토큰 5개 초과분 삭제(로그인 자체의 목적인 "신규 토큰 발급" 외의 하우스키퍼 삭제).

****공유 보안 필터 변경****: `ApiKeyAuthFilter`에 DB 폴백 조회 경로가 이번 FUNC에서 추가되어, 이 로그인 API가 발급한 `apiKey`로 들어오는 ****다른 모든 `/api/\*\*` 요청****의 인증 판정에 `MemberApiKeyDao` 조회가 새로 개입한다(정적 맵 히트 시에는 개입하지 않음 — 기존 admin-M-0001 경로 무변경).

오류 응답

응답 봉투는 모두 `{ "code": "...", "message": "..." }` 형태(MBR-4291만 `retryAfterSeconds` 필드 추가, `MemberLoginExceptionHandler`).

코드	HT TP	사유	발생 조건
MBR-4011	401	이메일 또는 비밀번호가 올바르지 않습니다 (n/5)	회원 없음 / 탈퇴 회원(`del_yn`='Y') / 비밀번호 불일치 — **세 경우 모두 동일 코드·문구** (존재 오라클 방지)
MBR-4291	429	로그인 시도 횟수를 초과했습니다. 잠시 후 다시 시도해 주세요(+`retryAfterSeconds`)	`MEMBER_LOGIN_ATTEMPTS.locked_until`이 현재 시각보다 미래 (5회 실패 도달, 10분 잠금)
MBR-5000	500	일시적인 오류입니다. 잠시 후 다시 시도해 주세요	`DataAccessException` — 정제된 메시지만 응답, 원본은 서버 로그에만(`MemberLoginExceptionHandler`)

참조 테이블

`MEMBER\_LOGIN\_ATTEMPTS`
`MEMBER\_REFRESH\_TOKENS`
`MEMBER\_API\_KEYS`
`MEMBERS`

curl 예시

```
curl -X POST /api/members/login \
-H "Content-Type: application/json" \
-d '{"email": "user@example.com", "password": "abcd1234"}'
```

변경 이력

<!-- spec-history: machine-managed -->

일자	SR	항 목	변경 요약	커 밋
2026-09-15	QUICK-20260915-1	#1	429/5000 오류 문구를 실제 코드 상수와 일치시킴(잠시 후 다시 시도해 주세요 완곡화)	

INF-MBR-004 로그아웃

> [반영: FUNC-member-006] 2026-09-12

INF-MBR-004: POST /api/members/sessions/logout — 로그아웃

> ****개요:**** 로그인 세션을 끝낸다 — 이 회원의 API 키와 리프레시 토큰 전체를 폐기한다. 관련: INF-MBR-005(리프레시) — 같은 세션 생명주기의 반대편.

> ****근거 소스:**** `modules/shop-api/src/main/java/com/sm/lab/shop/controller/MemberSessionController.java:39-43`

요청

Method: POST

Path: /api/members/sessions/logout

Content-Type: (바디 없음)

인증: ****필요****. `X-API-Key` 헤더는 `ApiKeyAuthFilter`가 먼저 검증한다(화이트리스트에 없음 — 인증 필요 경로). 컨트롤러는 그 헤더 값을 다시 읽어 어느 회원의 세션인지 서비스가 해석한다(필터가 memberId를 별도로 실어주지 않음).

파라미터	위치	타입	필수	설명
X-API-Key	Header	string	Y	로그인/리프레시로 발급받은 API 키

응답 (204 No Content)

바디 없음. ****항상 204****(idempotent) — `X-API-Key`가 `MEMBER\_API\_KEYS`에서 조회되지 않아도(admin 키, 이미 폐기된 키 등) 에러가 아니라 "이미 로그아웃된 상태와 동일"로 취급해 204를 반환한다.

비즈니스 규칙

****memberId 해석****: `X-API-Key` 값으로 `MemberApiKeyDao#selectMemberIdByApiKey`(로그인/003이 이미 만든 메서드, `MEMBERS.del\_yn='N'` + `revoked\_at IS NULL` 필터 포함)를 조회한다. 못 찾으면(정적 admin 키, 이미 폐기된 키 등) ****에러 없이 idempotent 204 no-op**** — 두 폐기 DAO 호출 자체가 일어나지 않는다.

****트랜잭션(메서드 한정)****: `MemberSessionService#logout`만 `@Transactional`이다(`refresh()`는 비트랜잭션 유지 — 사례집 SR-231 r2 "실패 경로에서 남아야 하는 카운터를 트랜잭션에 넣지 말라" 원칙은 여기 두 폐기에는 적용되지 않는다. 둘 다 함께 성공/실패해야 하는 단일 논리 동작이기 때문).

****두 폐기 순서(사람 확정, round2 재작업)****: `refreshTokenDao.revokeAllForMember`(리프레시 토큰 전체 폐기) → `apiKeyDao.revokeByMemberId`(API 키 폐기) 순으로 실행한다. 트랜잭션이 있어 중간 실패는 롤백되지만, 혹시 트랜잭션 경계 밖에서 재사용되는 상황까지 대비해 "apiKey가 아직 안 죽었으면 재시도 가능"이 되도록 이 순서로 고정했다(반대 순서였던 round1 구현은 apiKey만 죽고 리프레시 토큰이 살아남아, 그 토큰으로 refresh를 치면 폐기된 apiKey가 새 키로 rotate 되어 로그아웃되지 않은 것과 같은 상태로 세션이 되살아나는 결함이 있었다).

****회원 단위 전체 폐기(개별 세션 아님)****: `MEMBER\_API\_KEYS`가 회원당 1개뿐인 기존 설계와 일관되게, 이 회원의 활성 리프레시 토큰 ****전체****를 폐기한다(디바이스별 개별 로그아웃 모델 아님 — 범위 밖으로 명시적 결정).

****폐기는 DELETE가 아니라 `revoked\_at` 세팅****: `MEMBER\_API\_KEYS`/`MEMBER\_REFRESH\_TOKENS` 모두 행을 지우지 않고 `revoked\_at`을 채운다 — 재조회 시 조건(`revoked\_at IS NULL`)에서만 제외된다.

****세션 종료 시 장바구니 유지(SR 수용기준)****: 이 API는 `CART\_ITEMS`를 건드리지 않는다 — 이 요건은 "손대지 않음"으로 충족된다(통합테스트가 로그아웃 전후 장바구니 수량 불변을 단언).

****레거시/미해결 주의(사람이 확인, 후속 SR 후보)**:** `MEMBER\_REFRESH\_TOKENS`에 `member\_id` 인덱스가 없어 `revokeAllForMember`가 풀스캔이다 — 로그아웃이 트랜잭션이 되면서 그 스캔의 락이 커밋 시점까지 유지된다(랩 규모에서는 실패로 이어지지 않는 low 리스크로 QA가 판단, 후속 SR에서 멍등 인덱스 추가 권고).

****레거시 픽스처 주의**:** 정적 맵 회원(M-0001, `lab-member-0001-key`)은 `MEMBER\_API\_KEYS`에 행이 없어 idempotent no-op 분기를 타 204를 반환하지만 실제로 폐기되는 것이 없다(랩 픽스처 특성 — 실 로그인 회원은 전부 DB 발급 키를 쓰므로 실사용 영향 없음).

트랜잭션 순서

`@Transactional`(메서드 한정)

1. `MEMBER\_API\_KEYS`에서 `X-API-Key`로 memberId 조회(`selectMemberIdByApiKey`). 못 찾으면 여기서 종료 — 204(변경 없음).
2. `MEMBER\_REFRESH\_TOKENS` UPDATE — 이 회원의 활성 리프레시 토큰 전체 폐기(`revokeAllForMember`).
3. `MEMBER\_API\_KEYS` UPDATE — 이 회원의 API 키 폐기(`revokeByMemberId`).
4. 커밋 → 204. (2-3 중 실패 시 전체 롤백 — 부분 폐기 상태가 남지 않는다.)

사이드이펙트

`CART\_ITEMS`/`ORDERS` 등 세션 밖 테이블은 건드리지 않는다.

오류 응답

코드	HT TP	사유	발생 조건
(필터, `{"error": "unauthorized"}`)	401	인증 실패	`X-API-Key` 헤더 없음 — `ApiKeyAuthFilter`가 컨트롤러 도달 전 default-deny로 차단(컨트롤러 레벨 `{code,message}` 봉투가 아니라 필터 전역 포맷)
MBR-5000	500	일시적인 오류입니다	`DataAccessException` — 정제된 메시지만 응답, 원본은 서버 로그에만 (`MemberSessionExceptionHandler`)

참조 테이블

`MEMBER\_API\_KEYS`

`MEMBER\_REFRESH\_TOKENS`

curl 예시

```
curl -X POST /api/members/sessions/logout \
-H "X-API-Key: mk_1a2b3c4d5e6f..."
```

INF-MBR-005 자동 로그인 토큰(리프레시)

> [반영: FUNC-member-006] 2026-09-12

INF-MBR-005: POST /api/members/sessions/refresh — 자동 로그인 토큰(리프레시)

> ****개요:**** 만료 임박한 로그인 정보를 30일 리프레시 토큰으로 자동 재발급한다. 관련: INF-MBR-004(로그아웃) — 같은 세션 생명주기의 반대편.

> ****근거 소스:**** `modules/shop-api/src/main/java/com/sm/lab/shop/controller/MemberSessionController.java:46-49`

요청

Method: POST

Path: /api/members/sessions/refresh

Content-Type: application/json

인증: 없음(무인증) — `ApiKeyAuthFilter`의 `isOpenRoute` 화이트리스트(`MEMBER\_SESSIONS\_REFRESH\_PATH`, 정확 일치)에 등록. 로그아웃 직후 자동 재로그인 시나리오 등 refresh 시점에는 유효한 API 키가 없을 수 있기 때문(로그인·가입 화이트리스트와 동일한 이유).

파라미터	위치	타입	필수	설명
refreshToken	Body	string	Y	로그인/이전 refresh가 발급한 리프레시 토큰 원문. 서버는 SHA-256(소문자 hex)으로 해시해 대조한다 — 원문은 DB에 저장되지 않는다

응답 (200 OK)

```
{
  "memberId": "M-0053",
  "memberName": "홍길동",
  "grade": "SILVER",
  "apiKey": "mk_1a2b3c4d5e6f...",
  "refreshToken": "5f2c9e10-....-....-....-.....5f2c9e10-....-....-....-.....",
  "refreshTokenExpiresAt": "2026-10-12T10:00:00"
}
```

로그인(INF-MBR-003) `LoginResult`와 ****완전히 동일한 필드명·타입·순서****(계약 — 기존 클라이언트 하위호환 유지)의 `SessionResult` 레코드.

`refreshToken`: 이번 호출에서 새로 회전(rotate)된 신규 토큰 — 요청에 실었던 옛 토큰은 이 응답과 동시에 폐기된다(재사용 불가).

`apiKey`: 직전에 로그아웃 이력이 있어 `revoked\_at`이 세팅돼 있었을 때만 실제로 새 값으로 교체된다. 그렇지 않으면(계속 유효했던 키) 기존 값 그대로.

비즈니스 규칙

****4갈래 완전 동일 401(존재 오라클 방지)****: "토큰 미존재" / "토큰 만료" / "토큰 폐기" / "회원 탈퇴(`del\_yn='Y'`)" 이 네 경우는 완전히 동일한 401 `MBR-4012`, 동일한 일반화 문구("유효하지 않거나 만료된 로그인 정보입니다", 사유를 절대 구분

해 노출하지 않는다) — 로그인(003) "3경우 동일 401" 원칙을 refresh에도 그대로 적용(사례집 SR-231 r5/SR-232 r2 계열 재발 방지).

****자격 검문은 반드시 SQL WHERE에 있어야 함****: `MemberRefreshTokenDao#selectActiveByTokenHash`의 `revoked\_at IS NULL AND expires\_at > now`와 `MemberDao#selectById`의 `del\_yn='N'` — 이 두 필터가 빠지면 로그아웃된 세션이 refresh로 영구히 되살아나는 우회 경로가 생긴다. 애플리케이션 레이어에서 재확인하지 않는다(003의 `del\_yn` 필터 관례와 동일).

****토큰 회전(rotation)****: 성공 시 새 토큰을 INSERT하고, 방금 사용한 구 토큰을 `revokeByTokenHash`로 개별 폐기한다(재사용 방지, 이 회원의 다른 활성 토큰에는 영향 없음). 이어서 003의 하우스키퍼(`purgeExpiredOrRevoked`, `enforceActiveCap` 상한 5)을 그대로 재사용한다 — 신규 배치·신규 컬럼 없음.

****apiKey 조건부 rotate(GATE-005 round2 경계표, 사람 확정)****: `issuelfAbsent`는 no-op UPSERT에서 조건부 rotate로 확장됐다 — 행 없으면 INSERT / `revoked\_at IS NOT NULL` (로그아웃 이력)이면 새 후보 문자열로 교체하고 `revoked\_at`을 NULL로 되돌림 / `revoked\_at IS NULL` (계속 유효)이면 완전 no-op. 폐기된 키 문자열 자체는 절대 재사용하지 않는다. 시그니처는 003 그대로라 로그인 API도 이 SQL을 그대로 쓰지만, 활성 키 경로는 완전 no-op이라 로그인 쪽 회귀는 없다.

****`memberApiKey.xml` UPSERT 대입 순서 의존(구현 주의, 사람이 매퍼 주석에 명시)****: `ON DUPLICATE KEY UPDATE`의 세 대입(`api\_key`/`issued\_at`/`revoked\_at`)은 ****revoked_at`이 반드시 마지막\*\***이어야 한다 — MySQL/MariaDB는 SET 절을 좌→우로 평가하며, 앞 컬럼이 이미 갱신된 값을 뒤 컬럼의 `IF(...)`가 보게 되어 있다. 순서가 바뀌면 rotate가 조용히 반쪽(키만 바뀌고 `revoked_at`은 그대로 남거나 그 반대)이 된다.

****레이트리밋 없음(의도된 설계)****: `MEMBER\_LOGIN\_ATTEMPTS` 같은 실패 잠금이 없다 — refreshToken이 UUID×2라 무차별 대입 표면이 사실상 없다는 판단(과설계 금지, SR·확정문답에 요구 없음).

****세션 만료 시 장바구니 유지(SR 수용기준)****: 이 API는 `CART\_ITEMS`를 건드리지 않는다 — 이 요건은 "손대지 않음"으로 충족된다(통합테스트가 refresh 전후 장바구니 수량 불변을 단언).

****레거시/미해결 주의(사람이 확인, 후속 SR 후보)****: `MEMBER\_REFRESH\_TOKENS`에 `member\_id` 인덱스가 없어 `purgeExpiredOrRevoked`/`enforceActiveCap`이 풀스캔이다(랩 규모에서는 실패로 이어지지 않는 low 리스크로 QA가 판단, 후속 SR에서 먹등 인덱스 추가 권고).

트랜잭션 순서

비트랜잭션, 개별 autocommit statement — 003 관례 유지, 각 statement가 `WHERE ... IS NULL`류로 idempotent해 재시도 안전.

1. 입력 `refreshToken` 원문을 SHA-256(소문자 hex)으로 해시.
2. `MEMBER\_REFRESH\_TOKENS`에서 해시로 활성 토큰 조회(`selectActiveByTokenHash`, `revoked\_at IS NULL AND expires\_at > now`). 없으면 401 `MBR-4012`, 이후 단계 없음.
3. `MEMBERS`를 memberId로 조회(`selectById`, `del\_yn='N'` 필터). 결과 없으면(탈퇴) 401 `MBR-4012` (2와 완전 동일 코드·문구), 이후 단계 없음.
4. `MEMBER\_REFRESH\_TOKENS` INSERT — 새 리프레시 토큰 발급(30일).
5. `MEMBER\_REFRESH\_TOKENS` UPDATE — 방금 쓴 구 토큰 개별 폐기(`revokeByTokenHash`, 회전).
6. `MEMBER\_REFRESH\_TOKENS` DELETE — 이 회원의 만료·폐기 행 정리(`purgeExpiredOrRevoked`, 003 하우스키퍼 재사용).
7. `MEMBER\_REFRESH\_TOKENS` DELETE — 활성 토큰 상한(5) 초과분 정리(`enforceActiveCap`, 003 하우스키퍼 재사용).
8. `MEMBER\_API\_KEYS` 조건부 rotate UPSERT(`issuelfAbsent`) → `selectByMemberId` 재조회.
9. 200 응답(`SessionResult`).

사이드이펙트

성공마다 003의 하우스키퍼(만료·폐기 리프레시 토큰 삭제 + 활성 토큰 5개 초과분 삭제)이 부가로 실행된다(리프레시 발급 자체의 목적 외 정리 작업).

apiKey 조건부 rotate('issuelfAbsent' 시맨틱 확장)는 **로그인(INF-MBR-003) API가 호출하는 동일 메서드**에도 영향을 준다 — 로그인 쪽 활성 키 경로는 여전히 완전 no-op이라 회귀는 없다(TC-FUNC-member-006-13 실측).

`CART\_ITEMS`/`ORDERS` 등 세션 밖 테이블은 건드리지 않는다.

오류 응답

코드	HTT P	사유	발생 조건
MBR-4012	401	유효하지 않거나 만료된 로그인 정보입니다	토큰 미존재 / 만료 / 폐기 / 회원 탈퇴 — **네 경우 모두 동일 코드·문구** (존재 오라클 방지)
MBR-5000	500	일시적인 오류입니다	`DataAccessException` — 정제된 메시지만 응답, 원본은 서버 로그에만 (MemberSessionExceptionHandler)

참조 테이블

`MEMBER\_REFRESH\_TOKENS`

`MEMBER\_API\_KEYS`

`MEMBERS`

curl 예시

```
curl -X POST /api/members/sessions/refresh \
-H "Content-Type: application/json" \
-d '{"refreshToken": "5f2c9e10-.....-.....5f2c9e10-.....-.....-....."}'
```

INF-MBR-006 비밀번호 재설정 코드 요청

> [반영: FUNC-member-008] 2026-09-13

INF-MBR-006: POST /api/members/password-resets/codes — 비밀번호 재설정 코드 요청

> ****개요:**** 비밀번호를 잊은 회원이 이메일 또는 휴대폰번호를 제출하면 6자리 코드를 생성해 10분간 유효하게 저장하고 "발송"한다(이 랩은 실제 발송 게이트웨이가 없어 로그로 대체). 코드 확인·새 비밀번호 반영은 별도 API(FUNC-member-009, 범위 밖).

> ****근거 소스:**** `modules/shop-api/src/main/java/com/sm/lab/shop/controller/MemberPasswordResetController.java:39-44`

요청

Method: POST

Path: /api/members/password-resets/codes

Content-Type: application/json

인증: 없음(무인증) — `ApiKeyAuthFilter`의 `isOpenRoute` 화이트리스트(`MEMBER\_PASSWORD\_RESET\_CODE\_PATH`, 정확 일치)에 등록. 로그인 전(비밀번호를 잊은) 사용자가 호출하므로 아직 API 키가 없기 때문(회원가입·로그인·refresh 화이트리스트와 동일한 이유).

파라미터	위치	타입	필수	설명
target	Body	string	Y	이메일 또는 휴대폰번호. 형식으로 채널을 자동 판별한다(정규식 매칭 실패 시 400). 100자 초과·빈 값도 400

응답 (202 Accepted)

```
{
  "channel": "EMAIL",
  "target": "user@example.com",
  "expiresInSeconds": 600
}
```

`channel`: 판별된 채널 — `EMAIL` 또는 `SMS`.

`target`: 정규화된 값(원문 echo가 아님) — 이메일은 `trim().toLowerCase()`, 휴대폰은 숫자만(하이픈·공백 제거, 단 현재 휴대폰 정규식이 하이픈을 애초에 허용하지 않으므로 이 분기는 사실상 도달하지 않는다).

`expiresInSeconds`: 코드 유효기간(고정값 600 = 10분). 실제로 코드가 갱신됐는지(쿨다운 통과 여부), 회원이 존재하는지는 이 응답만으로 절대 구분할 수 없다(오라클 방지 — 형식 오류가 아닌 한 항상 이 모양의 202).

코드 원문은 이 응답에도, 어떤 로그에도 담기지 않는다(SHA-256 해시만 저장).

비즈니스 규칙

****형식 오류만 400, 그 외는 항상 202(존재 오라클·쿨다운 오라클 방지)****: 빈 값 / 100자 초과 / 이메일·휴대폰 형식 불일치만 400 `MBR-4100`. 회원 존재 여부, 쿨다운(60초 이내 재요청) 위반 여부는 응답에 전혀 드러나지 않는다 — 어느 경우든 동일한 202·`VerificationCodeResult` 바디(사례집 SR-231 "존재 판정 오라클"의 일반화 적용).

****회원 테이블을 구조적으로 조회하지 않음****: `MemberPasswordResetService` 생성자는 `MemberDao`를 주입받지 않는다 — 회원 존재/탈퇴 여부와 무관하게 항상 같은 처리 경로를 타도록 클래스 구조 자체로 강제한다.

****target 정규화·형식 정합****: 이메일은 `trim()+toLowerCase()`, 휴대폰은

`MemberRegistrationService.PHONE\_PATTERN`(`^01[016789][0-9]{7,8}\$`, 하이픈 불허)과 문자 단위로 동일한 정규식을 재사용한다(round2, 형제 가입 API와 판정 불일치를 없애기 위한 사람 수정) — 같은 입력이 재설정·가입 두 API에서 다르게 판정되는 것을 방지. PK(target)가 정규화 값이므로 같은 대상의 다른 표기는 같은 행·같은 쿼리문을 공유한다.

****발송 로그는 재조회 대조 후에만 남긴다(round2, QA FAIL 재작업)****: `touchRequest` UPSERT 직후 `selectByTarget`으로 다시 읽어, 저장된 `code\_hash`가 이번 요청이 만든 해시와 같을 때만(=쿨다운에 걸리지 않고 실제로 반영됐을 때만) "발송" 로그를 남긴다. round1은 이 순서 없이 무조건 로그를 남겨, 쿼리문에 걸린 요청도 로그가 남고 정작 저장된 코드는 이전 것이라 발송 로그와 실제 저장 코드가 불일치하는 결함이 있었다(SR-231 round5 재조회 대조와 동일 기법). 응답은 어느 경우든 무조건 202·상수 바디라 이 판정은 로그 여부에만 쓰이고 오라클을 만들지 않는다.

****`memberPasswordReset.xml` UPSERT SET 순서 의존(구현 주의)****: `ON DUPLICATE KEY UPDATE`의 다섯 대입(`code\_hash`/`expires\_at`/`consumed\_at`/`attempt\_count`/`created\_at`) 중 `created\_at`이 반드시 맨 마지막이어야 한다 — MySQL/MariaDB는 SET을 좌→우로 평가하므로, 앞선 네 IF 조건의 `TIMESTAMPDIFF(SECOND, created\_at, #now)` 판정이 항상 갱신 전(원본) `created\_at` 값을 보게 하기 위함이다(`MEMBER\_SIGNUP\_RATE\_LIMITS` round6과 동일 원리). 순서가 바뀌면 쿼리문 판정이 첫 번째 IF에서만 정확하고 이후 컬럼은 이미 갱신된 `created\_at`을 봐서 조용히 깨진다.

****코드값 의미****: `cooldownSeconds=60` (동일 target 재요청 쿼리문), `EXPIRES\_IN\_SECONDS=600` (코드 유효 10분), `CODE\_LENGTH=6` (숫자 코드) — 모두 SR-234 확정 요건.

****레거시/round 이력****: round1은 휴대폰 정규식을 하이픈 허용으로 넓혔다가 형제 가입 API(`MemberRegistrationService`)와 같은 입력에 다른 판정(202 vs 400)을 내는 불일치가 QA에서 지적되어(재작업 지시 low(4)), round2에서 하이픈 불허로 되돌렸다(현재 코드 = round2 상태).

****일일 요청 상한 5회(SR-297 #2)****: 쿼리문 판정과 나란히, target별 일일 상한(`DAILY\_REQUEST\_LIMIT=5`)을 전용 카운터 테이블(`MEMBER\_PASSWORD\_RESET\_RATE\_LIMITS`, SCH-MBR-009)에 대한 원자

UPSERT(`MemberPasswordResetRateLimitDao#touchDailyLimit`)로 판정한다 — 가입

`MemberSignupRateLimitDao#touchRateLimit`과 문자 단위로 동일한 SQL 본문(세션 변수 없이 SET 좌→우 평가만으로 결정적, round6 원리). 이 판정이 코드 테이블 UPSERT(`touchRequest`)보다 ****먼저**** 실행되고, 상한 초과 시 `touchRequest`를 아예 호출하지 않아 발송·코드 갱신 자체를 생략한다(응답은 그래도 항상 동일한 202).

****거부 로그 사유 분기(SR-297 #2, QA CONCERNS round1 재작업)****: 일일 상한 게이트가 거부하면 서버 로그 1줄을 남기 되, 재조회한 `dailyCount >= DAILY\_REQUEST\_LIMIT`이면 "일일 상한 초과", 그 외(쿨다운 미경과)면 "쿨다운"으로 사유를 갈라 적는다(형제 `MemberSignupService#rejectionFor`와 같은 판별 기준). `target`은 `mask()`로 가려 원문을 남기지 않는다. 이 로그는 서버 관측용일 뿐 응답에는 전혀 반영되지 않는다(오라클 방지).

****알려진 동작 — 자정 경계(SR-297 #2, STEP 3-0 사람 확인)****: 자정 직전 요청 뒤 60초 이내에 날짜가 바뀌면, 일일 상한 카운터(SCH-MBR-009, `day\_key` PK)는 새 날짜 행이 생겨 쿼리문 없이 admit(일일 카운트 소비)하지만, 이 코드 테이블의 자체 쿼리문(`created\_at` 기준 60초)은 여전히 걸려 코드가 갱신되지 않을 수 있다 — 일일 카운트만 소비되고 코드는 안 나가는 조합이 가능하다. 응답은 이 경우에도 항상 202·동일 바디라 클라이언트에는 드러나지 않는다(교차-day_key 조회로 이 조합을 막는 것은 범위 밖).

트랜잭션 순서

비트랜잭션, 개별 autocommit statement(`@Transactional` 미사용) — 두 UPSERT(`touchDailyLimit`/`touchRequest`)가 각각 자체 원자 판정이라 응용 레이어의 트랜잭션이 불필요. `MEMBER\_PASSWORD\_RESET\_RATE\_LIMITS`와 `MEMBER\_PASSWORD\_RESETS`는 완전히 분리된 테이블이라 두 UPSERT 사이에 걸치는 락이 없다.

1. target 형식 검증(빈 값 / 100자 초과 / 이메일·휴대폰 정규식). 실패 시 400 `MBR-4100`, 이후 단계 없음.
2. 채널 판별(`EMAIL`/`SMS`) 및 target 정규화.
3. `now`/`dayKey` 계산, 요청 고유 토큰(UUID) 생성.
4. **** (SR-297 #2) **** `MEMBER\_PASSWORD\_RESET\_RATE\_LIMITS` UPSERT(`touchDailyLimit`) — 쿼리문(60초) AND 일일상한(5회) 둘 다 만족할 때만 `daily\_count`/`last\_requested\_at`/`last\_token`을 갱신 → `selectRateLimit` 재조회 → `lastToken`이 내

토큰과 같으면 `admitted`.

5. `admitted=false`(쿨다운 또는 상한 초과)면 **6~9단계를 건너뛰고 바로 10단계**로 — 거부 사유(재조회한 `dailyCount>=5`면 "일일 상한 초과", 아니면 "쿨다운")를 가려 서버 로그 1줄만 남긴다(target 마스킹).

6. 6자리 코드 생성 → SHA-256(소문자 hex) 해시.

7. `MEMBER\_PASSWORD\_RESETS` UPSERT(`touchRequest`) — 코드 테이블 자체 쿨다운(60초) 미만이면 다섯 컬럼 모두 기존 값 유지(no-op), 이상이면 다섯 컬럼 모두 새 값으로 갱신(`attempt\_count`는 0, `consumed\_at`은 NULL로 리셋).

8. `MEMBER\_PASSWORD\_RESETS`를 target으로 재조회(`selectByTarget`).

9. 재조회한 `code\_hash`가 6단계에서 만든 해시와 같으면(=실제로 반영됐으면) 발송 로그 1줄 기록(코드 원문 미포함, target은 마스킹). 다르면(코드 테이블 쿨다운에 걸려 이전 코드가 그대로 남음 — 자정 경계 등) 로그 생략.

10. 202 응답(`VerificationCodeResult`) — 4~9단계 결과와 무관하게 항상 동일한 모양.

사이드이펙트

`MEMBER\_PASSWORD\_RESETS`(코드 테이블) + `MEMBER\_PASSWORD\_RESET\_RATE\_LIMITS`(SR-297 #2, 일일 상한 카운터 테이블) 두 테이블만 다룬다 — 회원 테이블(`MEMBERS`)은 여전히 조회·변경하지 않는다(`MemberPasswordResetService` 생성자가 `MemberDao`를 주입받지 않는 구조적 강제).

"발송"은 실제 게이트웨이 호출이 아니라 로그 1줄 기록으로 대체된다(이 랩 환경의 시뮬레이션, `MemberSignupService`의 가입 인증코드 발송 로그와 동일 모양·수준).

일일 상한 거부 시에도 서버 로그 1줄(사유 포함)을 남긴다(위 "거부 로그 사유 분기" 참고) — 응답에는 반영되지 않는다.

오류 응답

코드	HTTP	사유	발생 조건
MBR-4100	400	이메일 또는 휴대폰번호 형식이 올바르지 않습니다	target이 빈 값 / 100자 초과 / 이메일·휴대폰 정규식 불일치
MBR-5000	500	일시적인 오류입니다	`DataAccessException` — 정제된 메시지만 응답, 원본은 서버 로그에만(`MemberPasswordResetExceptionHandler`)

참조 테이블

`MEMBER\_PASSWORD\_RESETS`

`MEMBER\_PASSWORD\_RESET\_RATE\_LIMITS`(SR-297 #2, SCH-MBR-009)

curl 예시

```
curl -X POST /api/members/password-resets/codes \
  -H "Content-Type: application/json" \
  -d '{"target": "user@example.com"}'
```

변경 이력

<!-- spec-history: machine-managed -->

일자	SR	항 목	변경 요약	커밋
2026-09-16	SR-297	#2	일일 상한 5회 게이트(MEMBER_PASSWORD_RESET_RATE_LIMITS UPSERT)를 코드 UPSERT 앞에 배선, 거부 로그 사유 분기, 자정 경계 알려진 동작 기록	shop-api@b7ae32f

INF-MBR-007 비밀번호 재설정 확정

> [반영: FUNC-member-009] 2026-09-13

INF-MBR-007: POST /api/members/password-resets/confirmations — 비밀번호 재설정 확정

> ****개요:**** 이메일/휴대폰으로 받은 6자리 코드와 새 비밀번호를 제출하면 코드를 확인하고, 맞으면 비밀번호를 반영한 뒤 그 회원의 모든 기기를 로그아웃시킨다(리프레시 토큰 + API 키 폐기). 코드 요청은 별도 API(FUNC-member-008, INF-MBR-006)가 담당한다.

> ****근거 소스:**** `modules/shop-api/src/main/java/com/sm/lab/shop/controller/MemberPasswordResetConfirmationController.java:41-45`

요청

Method: POST

Path: /api/members/password-resets/confirmations

Content-Type: application/json

인증: 없음(무인증) — `ApiKeyAuthFilter`의 `isOpenRoute` 화이트리스트(`MEMBER\_PASSWORD\_RESET\_CONFIRM\_PATH`, 정확 일치)에 등록. 로그인 전(비밀번호를 잊은) 사용자가 호출하므로 아직 API 키가 없기 때문(INF-MBR-006과 동일한 이유). 자격 판정은 필터가 아니라 이 서비스(원자 확정 UPDATE + del_yn 필터 회원 조회)가 한다.

파라미터	위치	타입	필수	설명
target	Body	string	Y	이메일 또는 휴대폰번호. INF-MBR-006이 저장한 것과 같은 정규화 규칙(이메일 trim+소문자, 휴대폰 숫자만)으로 재검증·정규화한다. 형식 오류 시 400
code	Body	string	Y	6자리 인증코드. SHA-256 해시로 저장값과 대조(원문은 저장되지 않음)
newPassword	Body	string	Y	새 비밀번호. 8~64자, 영문+숫자 포함 규칙(가입 API와 동일 검증 재사용). 형식 오류 시 400

응답 (204 No Content)

본문 없음. 아래 어느 경로든(코드 확정 성공 후 회원을 찾은 경우 / 회원을 못 찾은 경우) 동일하게 204를 반환한다 — 회원 존재 여부를 응답으로 절대 구분하지 않는다(`MemberSessionController.logout()`과 동일 형태).

비즈니스 규칙

> 이 섹션은 abstract다. 정본은 앵커(소스)이며, 상세 분기는 file:line을 직접 읽는다.

****존재 오라클 방지(이 API의 핵심 위험)**:** 코드 확정(`confirmIfCodeMatches`)이 회원 조회보다 항상 먼저 실행되고, "행 없음"(요청한 적 없는 target)도 단순 코드 불일치와 완전히 동일한 409 `MBR-4102`로 수렴한다(`handleConfirmFailure`가 두 경우 모두 같은 `mismatchException()` 팩토리 호출). 회원 조회(코드 확정 성공 후 1회) 결과가 있든 없든 응답은 204로 동일 — 이 API의 어떤 응답도 회원 존재 여부를 드러내지 않는다.

****del_yn='N' 이중 필터**:** 회원 매칭 조회(`selectMemberIdByResetTarget`)와 비밀번호 반영

UPDATE(`updatePasswordHash`) 양쪽 모두에 `del\_yn = 'N'`을 건다 — 탈퇴 회원의 비밀번호가 되살아나거나 탈퇴 회원 명

의로 세션이 재발급되는 경로를 원천 차단한다.

****코드 확정이 회원 조회보다 항상 먼저****: ``confirmPasswordReset``은 `target/newPassword` 형식 검증 → 코드 확정 (``confirmIfCodeMatches``) → (성공 시에만) BCrypt → 회원 조회 순서로 진행한다. 회원 조회 이전 어떤 단계도 ``MEMBERS`` 테이블을 참조하지 않는다.

****BCrypt는 확정 성공 시에만, 발견 여부와 무관하게 항상 실행****: 코드 확정이 성공한 직후 새 비밀번호를 BCrypt로 해싱한다 — 이후 회원을 찾든 못 찾든 이 해싱 자체는 항상 실행됐던 상태이므로, 회원 발견 여부가 처리 시간(타이밍)으로 새지 않는다.

****회원 미발견 시 조용히 204****: 코드 확정에 성공했지만 매칭되는 활성 회원이 없으면(가짜 target 등) 로그 없이 그대로 반환한다 — 완료 로그는 실제로 비밀번호가 반영된 경우에만 남긴다.

****트랜잭션 경계(Writer 별도 빈)****: 이 서비스는 ``@Transactional``을 쓰지 않는다 — 코드 확정/시도횟수 증가는 각각 `autocommit` 단일 UPDATE 문(실패 시도 카운터가 트랜잭션 롤백에 함께 사라지는 것을 방지). 비밀번호 반영+전 기기 로그 아웃 별도 스프링 빈(``MemberPasswordResetConfirmationWriter``)의 ``@Transactional`` 메서드로 묶는다 — 서비스 클래스 안에서 자기 메서드를 직접 호출(self-invocation)하면 Spring 프록시가 트랜잭션 어드바이스를 적용하지 못하는 함정을 피하기 위함이다.

****0행이면 세션 폐기 스킵****: ``updatePasswordHash``가 0행(그 사이 회원 탈퇴 등으로 반영 실효 없음)이면 리프레시 토큰 API 키 폐기 두 문장을 건너뛰고 즉시 반환한다.

****전 기기 로그아웃(사이드이펙트)****: 비밀번호 반영에 성공하면 그 회원의 리프레시 토큰 전체와 API 키를 폐기한다(순서: 리프레시 먼저, apiKey 나중 — INF-MBR-005 로그아웃과 동일 순서). 이 트랜잭션이 중간에 실패하면(예: 세션 폐기 UPDATE 예외) 비밀번호 반영까지 함께 롤백된다 — "비밀번호는 바뀌었는데 세션은 살아있는" 위험한 중간 상태를 만들지 않는다.

****1회용(consumed_at)****: ``confirmIfCodeMatches``는 ``consumed_at IS NULL``인 행에만 매치하므로, 같은 코드로 두 번째 확정을 시도하면 이미 소비된 것으로 판정되어 410 ``MBR-4101``이 된다.

****병렬 버스트 우회 차단(SR-298)****: ``incrementAttemptCount``의 ``attempt_count < 5`` 조건절이 단일 PK(``target``) 행 락 위에서 재평가되므로, 동시에 도착한 오답 요청들도 이 한 문장의 짧은 수명 동안만 직렬화된다 — 상한(5)을 넘겨 증가할 수 없다(이전에는 "현재값 읽기 → 상한 미만이면 별도 증가" 순서라 두 문장 사이 레이스로 5를 초과해 증가할 수 있었다). ``handleConfirmFailure``는 이 원자 증가를 ****가장 먼저, 무조건**** 시도하고, 1행이면 그 자체로 "미만료·미소비·상한 미만·단순 오답"이 확정되어 ``selectByTarget`` 없이 바로 409 ``MBR-4102``를 던진다. 0행일 때만(상한 도달 하나만이 아니라 이미 소비·이미 만료·행 없음도 0행이다) ``selectByTarget``로 원인만 재조회해 4갈래로 분류한다(재증가 없음) — 이 읽기는 경합 판정이 아니라 오류 코드(4101/4102/4103) 표시용일 뿐이다.

트랜잭션 순서

1. target 형식 검증·정규화, newPassword 형식 검증(재사용 — INF-MBR-006/가입 API와 동일 규칙). 실패 시 각각 400, 이후 단계 없음(``attempt_count`` 미소모).

2. 코드 SHA-256 해싱.

3. ``MEMBER_PASSWORD_RESETS`` 원자 확정 UPDATE(``confirmIfCodeMatches``, 비트랜잭션 단일 문, ``attempt_count < 5`` 조건 포함). 1행이면 4번으로. 0행이면 ``handleConfirmFailure``:

1. 원자 조건부 UPDATE(``incrementAttemptCount``, ``attempt_count < 5`` 포함 — SR-298 원자화, read-modify-write 아님)를 ****가장 먼저, 무조건**** 시도. 1행이면 그 자체로 "미만료·미소비·상한 미만·단순 오답"이 확정 — ``selectByTarget`` 없이 즉시 409 ``MBR-4102``.

2. 0행이면(사유 다양 — 상한 도달/이미 소비·만료/행 없음) ``selectByTarget`` 읽기 1회로 원인만 재분기(재증가 없음): 행 없음 → 409 ``MBR-4102``, 소비·만료 → 410 ``MBR-4101``, 상한 도달(``attempt_count >= 5``) → 409 ``MBR-4103``, 그 외(방어용 폴백) → 409 ``MBR-4102``.

즉시 응답 종료(이후 단계 진행 안 함).

4. (확정 성공 시에만) BCrypt로 새 비밀번호 해싱.

5. `MEMBERS.selectMemberIdByResetTarget` 조회(이번이 처음 — 이전 어떤 단계도 `MEMBERS`를 조회하지 않음). 못 찾으면 조용히 204(로그 없음).

6. (회원 발견 시) `MemberPasswordResetConfirmationWriter.applyNewPassword`(별도 트랜잭션 빈) 호출:

1. `MEMBERS.updatePasswordHash`(0행이면 이후 두 문장 스킵).

2. `MEMBER\_REFRESH\_TOKENS.revokeAllForMember`(전 기기 로그아웃).

3. `MEMBER\_API\_KEYS.revokeByMemberId`.

7. Writer 트랜잭션 커밋 후에만 완료 로그 1줄(마스킹된 target). 컨트롤러는 204 반환.

사이드이펙트

비밀번호 반영과 함께 그 회원의 전 기기 로그아웃 처리: `MEMBER\_REFRESH\_TOKENS` 전체 폐기 + `MEMBER\_API\_KEYS` 폐기(같은 Writer 트랜잭션, 순서 고정 — 리프레시 먼저, apiKey 나중).

`MEMBERS.updated\_at`(V6 마이그레이션 신규 컬럼) 갱신 — 이 FUNC이 최초로 세팅.

오류 응답

코드	HT TP	사유	발생 조건
MBR-4100	400	이메일 또는 휴대폰번호 형식이 올바르지 않습니다	target 형식 검증 실패(재사용 — INF-MBR-006과 동일 코드)
MBR-4001	400	비밀번호 형식이 올바르지 않습니다	newPassword 형식 검증 실패(재사용 — 가입 API와 동일 코드)
MBR-4101	410	인증코드가 만료되었습니다. 다시 요청해 주세요	코드가 이미 소비됐거나(`consumed_at` 있음) 만료됨(`expires_at` 경과) — 신규
MBR-4102	409	코드가 올바르지 않습니다	코드 불일치, 또는 target에 해당하는 행 자체가 없음(둘을 동일 코드로 통일 — 존재 오라클 방지, 신규)
MBR-4103	409	코드 확인 시도 횟수를 초과했습니다. 다시 요청해 주세요	`attempt_count`가 상한(5회) 이상 — 신규
MBR-5000	500	일시적인 오류입니다	`DataAccessException` — 정제된 메시지만 응답, 원본은 서버 로그에만(`MemberPasswordResetConfirmationExceptionHandler`)

참조 테이블

`MEMBER\_PASSWORD\_RESETS`

curl 예시

```
curl -X POST /api/members/password-resets/confirmations \
-H "Content-Type: application/json" \
-d '{"target": "user@example.com", "code": "123456", "newPassword": "newPass123"}'
```

변경 이력

<!-- spec-history: machine-managed -->

일자	SR	항 목	변경 요약	커밋
2026-09-16	SR-298	#2	incrementAttemptCount에 attempt_count<5 조건 추가해 증가+상한판정을 원자 UPDATE로 통합(병렬 버스트 우회 차단). handleConfirmFailure는 이 증가를 먼저 무조건 시도하고 0행일 때만 selectByTarget로 원인(행없음/소비·만료/상한) 재분기, 재증가 없음	shop-api@0ed4f7d

INF-MBR-008 배송지 CRUD

> [반영: FUNC-member-011] 2026-09-13

INF-MBR-008: 배송지 CRUD API — 회원 배송지 관리

> ****개요:**** 회원이 자신의 배송지를 목록조회·등록·수정·삭제·기본 배송지 설정하는 API 5종. AS-IS 없음 —

> SR-235(FUNC-member-011)로 신설된 완전 신규 계약이며, 코드가 곧 정본이다.

> ****근거 소스:**** `modules/shop-api/src/main/java/com/sm/lab/shop/controller/MemberAddressController.java:36-113`

엔드포인트 목록

Method	Path	설명	성공 코드
GET	/api/members/me/addresses	배송지 목록(최근 사용 순)	200
POST	/api/members/me/addresses	배송지 등록	201
PUT	/api/members/me/addresses/{addressId}	배송지 수정	200
DELETE	/api/members/me/addresses/{addressId}	배송지 삭제(소프트)	204
PUT	/api/members/me/addresses/{addressId}/default	기본 배송지로 설정	200

인증·스코프

전 경로 `X-API-Key` 필요(무인증 화이트리스트 아님).

URL에 `memberId`가 없는 "me" 자원이다 — `ApiKeyAuthFilter#evaluateMemberScope`의

`MEMBER\_ME\_ADDRESSES\_PATH`(`^/api/members/me/addresses(?:/.\*)?\$`) 정규식이 5개 경로 전부(메서드

무관)를 `Decision.SCOPE\_TO\_SELF`로 판정한다 — 기존 `GET /api/orders`(`memberId` 토큰 없는 목록 조회)

강제-자기자신 메커니즘을 재사용한다(새 우회 경로 없음).

admin 키(`\*`)로 호출하면 `evaluateMemberScope` 자체를 타지 않아 이 강제가 일어나지 않는다. 이때

`memberId` 쿼리 파라미터가 비어 있으면 컨트롤러의 `requireMemberId`가 400 `MBR-4200`으로 명시적으로

막는다 — "관리자가 특정 회원 대신 /me를 호출"하는 시나리오는 이 SR에서 정의하지 않는다(사람 확정).

요청

1) GET /api/members/me/addresses — 목록

파라미터	위치	타입	필수	설명
memberId	Query	string	(강제 주입)	클라이언트가 직접 넘기지 않는다 — `ApiKeyAuthFilter`의 `SCOPE_TO_SELF`가 호출자 자신의 memberId로 강제 덮어쓴다

2) POST /api/members/me/addresses — 등록

파라미터	위치	타입	필수	설명
memberId	Query	string	(강제 주입)	목록과 동일
recipient	Body	string	Y	받는사람 이름, 1~50자
phone	Body	string	Y	휴대폰번호 원문, 1~20자, 정규화 후 'MemberRegistrationService.PHONE_PATTERN' 형식 일치 필요
zipcode	Body	string	Y	우편번호, 숫자 5자리 (^[0-9]{5}\$) — ZIPCODES 대조는 이 API 범위 밖(FUNC-member-012 룩), 클라이언트 값을 그대로 저장
roadAddress	Body	string	Y	도로명 주소, 1~200자
detailAddresses	Body	string	Y	상세 주소, 1~200자
entranceMethod	Body	string	N	공동현관 출입방법, 최대 200자
deliveryMemo	Body	string	N	배송 메모, 최대 200자
isDefault	Body	boolean	N	기본 배송지 지정 희망. 첫 등록이면 이 값과 무관하게 무조건 기본 강제

3) PUT /api/members/me/addresses/{addressId} — 수정

파라미터	위치	타입	필수	설명
memberId	Query	string	(강제 주입)	목록과 동일
addressId	Path	long	Y	수정할 배송지 ID
recipient / phone / zipcode / roadAddress / detailAddress / entranceMethod / deliveryMemo	Body	(등록과 동일 타입-검증)	(필수 여부 등록과 동일)	등록과 동일한 검증 규칙 재사용('validateFields')
isDefault	Body	boolean	-	**무시됨** — 'MemberAddressService.update'는 'is_default'를 갱신하지 않는다(기본 지정은 전용 API로만)

4) DELETE /api/members/me/addresses/{addressId} — 삭제

파라미터	위치	타입	필수	설명
memberId	Query	string	(강제 주입)	목록과 동일
addressId	Path	long	Y	삭제할 배송지 ID

본문 없음.

5) PUT /api/members/me/addresses/{addressId}/default — 기본 설정

파라미터	위치	타입	필수	설명
memberId	Query	string	(강제 주입)	목록과 동일
addressId	Path	long	Y	기본으로 지정할 배송지 ID

본문 없음.

응답

배송지 객체 공통 세이프(MemberAddress — 단건 응답은 봉투 없이 그대로, 목록은 {items: [...]} 봉투):

```
{
  "addressId": 12,
  "memberId": "M-0053",
  "recipient": "홍길동",
  "phone": "010-1234-5678",
  "phoneNorm": "01012345678",
  "zipcode": "06236",
  "roadAddress": "서울특별시 강남구 테헤란로 123",
  "detailAddress": "101동 202호",
  "entranceMethod": "공동현관 비밀번호 1234",
  "deliveryMemo": "부재 시 경비실에 맡겨주세요",
  "isDefault": "Y",
  "lastUsedAt": "2026-09-13T10:00:00",
  "createdAt": "2026-09-13T10:00:00",
  "updatedAt": "2026-09-13T10:00:00"
}
```

`isDefault`: 이 프로젝트의 Y/N 플래그 관례(Product.saleYn과 동일) — boolean이 아니라 문자열 `"Y"/"N"` 그대로 노출된다.

`phoneNorm`: 서버가 계산한 정규화 값(숫자만)도 `MemberAddress` getter에 `@JsonIgnore`가 없어 응답에 그대로 포함된다 — 별도 응답 DTO가 없다(컨트롤러가 도메인 객체를 직접 반환).

`lastUsedAt`은 등록 시 NULL이 아니라 `createdAt`과 동일값으로 채워진다(사람 수정 — 목록/승계 정렬이 NULL 흔들림 없이 안정적으로 동작하게 하기 위함). 이 API 범위에서는 등록 이후 갱신되지 않는다(주문 시 갱신은 SR-255 몫).

목록 GET 응답: `{ "items": [ {..위 세이프..}, ... ] }` — 0건이어도 200 + 빈 배열(404 아님).

등록 POST 응답: 201 + 생성된 배송지 객체(봉투 없음).

수정 PUT 응답: 200 + 수정된 배송지 객체(재조회 결과, 봉투 없음).

삭제 DELETE 응답: 204, 본문 없음.

기본설정 PUT 응답: 200 + 배송지 객체. 이미 기본이면 DB를 건드리지 않고 조회한 현재 상태를 그대로 반환(no-op 맥등).

비즈니스 규칙

> 이 섹션은 "완전한 사양"이 아니라 짧은 abstract다. 정보 진실은 frontmatter `anchors:`의 소스 —

> 여기엔 핵심 분기·코드값 의미만 적는다.

****존재+소유 단일 SQL(존재 오라클 방지)**:** `MemberAddressDao#selectOwned`가 `member\_id` + `address\_id`를

한 WHERE절로 묶어 "배송지 없음"과 "남의 배송지"를 원천적으로 같은 결과(null)로 만든다 → 두 경우

모두 동일 404 `MBR-4041`, 동일 문구. 판정을 분리하는 코드는 없다(수정·삭제·기본설정 3곳 공통, 사례집

SR-231 r5 재발 방지).

****회원당 최대 10개****: 등록 시 락(`selectByMemberIdForUpdate`)으로 얻은 목록 크기가 10 이상이면 409 `MBR-4201`. 카운트는 락 이후에만 신뢰한다(동시 등록 레이스 방지).

****기본 배송지 자동 승계****: 삭제 대상이 기본(`is\_default='Y'`)이었으면, 락으로 얻은 목록(대상 제외) 중 `last\_used\_at DESC, created\_at DESC` 1순위가 자동으로 새 기본이 된다. 후보가 없으면(마지막 남은 배송지 삭제) 승계 없이 그대로 기본 없음 상태로 종료.

****첫 등록은 무조건 기본****: `existing.isEmpty()`이면 요청의 `isDefault` 값과 무관하게 기본으로 강제한다(첫 배송지가 기본이 아닌 상태로 남는 경우를 만들지 않음).

****`last\_used\_at`은 NULL 없이 등록****: 등록 시 `created\_at`과 동일값으로 채운다 — 목록/승계 정렬 키가 NULL 여부로 흔들리지 않게 하기 위한 사람 확정 결정. 주문 시점 갱신은 이 FUNC 범위 밖(SR-255).

****락 순서 고정(1곳 공통)****: `register`/`delete`/`setDefault` 세 경로 모두 쓰기 문장 이전에 `selectByMemberIdForUpdate`(`SELECT ... FOR UPDATE`)를 먼저 호출하는 동일 순서를 지킨다 — 두 트랜잭션이 다른 순서로 잠그면 데드락(사례집 SR-231 r3 재발 방지).

`delete`는 이 락 호출 이전에 `selectOwned`로 존재+소유를 먼저 확인한다(싸게 실패). 승계 판정은 락 이후 재조회 없이 이 락 목록(`locked`) 자체를 그대로 쓴다 — MariaDB REPEATABLE READ에서 비잠금 재조회는 락 대기 중 커밋된 변경을 못 볼 수 있어서다(`selectNextDefaultCandidate`는 더 이상 호출되지 않음, 매퍼 테스트용으로만 잔존).

****URL에 memberId 없음****: `/api/members/me/addresses`는 `ApiKeyAuthFilter`의 `SCOPE\_TO\_SELF` 메커니즘이 memberId를 강제한다(위 "인증·스코프" 절). admin 키로는 400으로 막힌다.

****ZIPCODES 대조는 범위 밖****: `zipcode`/`roadAddress`/`detailAddress`는 클라이언트가 보낸 값을 검증(형식·길이)만 하고 그대로 저장한다 — 우편번호 실재 여부 대조는 형제 FUNC-member-012 몫.

****수정은 `isDefault`를 다루지 않음****: 필드 수정과 기본 지정을 분리된 API로 명확히 나눴다 — `update`가 `is\_default`를 건드리면 승계 로직 없이 기본이 조용히 바뀌는 사이드이펙트가 생기므로 의도적으로 배제.

트랜잭션 순서

POST 등록 (`@Transactional`)

1. 휴대폰번호 정규화 + 필드 검증(400, DB 접근 전).
2. `MEMBER\_ADDRESSES` 락 조회(`selectByMemberIdForUpdate`, 첫 문장) — 개수 확인.
3. 10개 이상이면 409, 이후 단계 없음(아직 아무것도 쓰기 전이라 롤백 불필요).
4. 기존 배송지가 있고 이번 등록이 기본이면 `clearDefaultForMember`(기존 기본 해제).

5. `MEMBER\_ADDRESSES` INSERT(`last\_used\_at`= `created\_at`=현재시각, AUTO_INCREMENT로 `addressId` 채번).

DELETE 삭제 (@Transactional)

1. `selectOwned`로 존재+소유 확인(없으면 404, 이후 단계 없음).
2. `MEMBER\_ADDRESSES` 락 조회(`selectByMemberIdForUpdate`, register/setDefault와 동일 지점) — 삭제 대상의 `is\_default`와 승계 후보를 이 잠금 목록에서 판정.
3. `MEMBER\_ADDRESSES` UPDATE(`softDelete`, `del\_yn`='Y').
4. 삭제 대상이 기본이었으면 승계 후보에 `setDefault` UPDATE(후보 없으면 생략).

PUT 기본설정 (@Transactional)

1. `selectOwned`로 존재+소유 확인(없으면 404).
2. 이미 기본이면 즉시 반환(DB 쓰기 없음, no-op 성공).
3. `MEMBER\_ADDRESSES` 락 조회(`selectByMemberIdForUpdate`).
4. `clearDefaultForMember` UPDATE → `setDefault` UPDATE.

PUT 수정 (@Transactional, 단일 UPDATE — 락 불필요)

1. 필드 검증(400).
2. `selectOwned`로 존재+소유 확인(없으면 404).
3. `MEMBER\_ADDRESSES` UPDATE(`updateAddress`, `is\_default` 제외).

GET 목록 (비트랜잭션)

1. `selectList` 단건 SELECT(최근 사용 순, `del\_yn`='N').

사이드이펙트

DELETE가 대상 배송지의 `is\_default`가 `Y`였을 경우, 메인 목적(삭제) 외에 **다른 행**(승계 후보)의 `is\_default`를 `Y`로 UPDATE한다 — 호출자가 명시적으로 요청하지 않은 추가 쓰기.

POST 등록이 기본으로 강제되는 경우(첫 등록 또는 `isDefault=true`), 기존 기본 배송지 행의 `is\_default`를 `N`으로 먼저 UPDATE한다(`clearDefaultForMember`) — 등록 대상 외 행 변경.

그 외 `CART\_ITEMS`/`ORDERS`/`MEMBERS` 등 이 도메인 밖 테이블은 건드리지 않는다(FK 참조만 존재).

오류 응답

코드	HT TP	사유	발생 조건
MBR-4200	400	유효성 실패	`memberId` 누락(admin 키로 /me 호출 등) / 등록·수정 필드 검증 실패(길이·형식·필수값) — 위반 필드명이 message에 포함됨

MBR-4041	404	배송지를 찾을 수 없습니다	수정·삭제·기본설정 대상이 없거나(미존재) 다른 회원 소유(남의 것) — **두 경우 완전 동일 코드·문구**(존재 오라클 방지)
MBR-4201	409	배송지는 최대 10개까지 등록할 수 있습니다	등록 시 랭 조회 결과 기존 배송지 수가 10개 이상
MBR-5000	500	일시적인 오류입니다	`DataAccessException` — 정제된 메시지만 응답, 원본(경로:SQL-커넥션 정보 포함)은 서버 로그에만(`MemberAddressExceptionHandler`, 로그에도 주소 본문·연락처는 남기지 않음)

참조 테이블

`MEMBER\_ADDRESSES`

curl 예시

```
# 1) 목록
curl -X GET "/api/members/me/addresses" \
  -H "X-API-Key: <member-key>"

# 2) 등록
curl -X POST "/api/members/me/addresses" \
  -H "X-API-Key: <member-key>" \
  -H "Content-Type: application/json" \
  -d '{"recipient":"홍길동","phone":"010-1234-5678","zipcode":"06236","roadAddress":"서울특별시 강남구 테헤란로 123","detailAddress":"101동 202호","entranceMethod":"공동현관 비밀번호 1234","deliveryMemo":"부재 시 경비실에 맡겨주세요","isDefault":true}'

# 3) 수정
curl -X PUT "/api/members/me/addresses/12" \
  -H "X-API-Key: <member-key>" \
  -H "Content-Type: application/json" \
  -d '{"recipient":"홍길동","phone":"010-1234-5678","zipcode":"06236","roadAddress":"서울특별시 강남구 테헤란로 123","detailAddress":"103동 501호","entranceMethod":"","deliveryMemo":""}'

# 4) 삭제
curl -X DELETE "/api/members/me/addresses/12" \
  -H "X-API-Key: <member-key>"

# 5) 기본 설정
curl -X PUT "/api/members/me/addresses/12/default" \
  -H "X-API-Key: <member-key>"
```

변경 이력

<!-- spec-history: machine-managed -->

일자	SR	항목	변경 요약	커밋
2026-09-15	SR-235	#2	배송지 CRUD API 구현 — /api/members/me/addresses 목록·등록·수정·삭제·기본 지정(회원당 최대 10개, 기본 1개, 기본 삭제 시 최근 사용 순 승계), 409 MBR-4201·404 MBR-4041(타인 소유 포함)·400 MBR-4200. v4 시절 완료분이라 이력이 빠져 있어 사후 기록(2026-09-19)	shop-api@1a0cece

INF-MBR-010 api members grades

INF-MBR-010: GET /api/members/grades

> 코드(핸들러) 기준 재동기화 골격(zero-LLM) — 요청·응답·오류 계약은 보강 대상

개요

[TBD]

요청

[TBD]

응답

[TBD]

오류

[TBD]

> 보강: `/sl-sync --apply --kind=inf` 또는 `ddd-api-agent`

INF-ORD-001 회원 목록 조회

INF-ORD-001: GET /api/members/ — 회원 목록 조회

> **개요:** 탈퇴하지 않은 회원 전체를 회원ID 순으로 조회한다.

> **근거 소스:** `src/main/java/com/sm/lab/shop/controller/MemberController.java:21-25`

요청

Method: GET

Path: /api/members/

Content-Type: application/json

파라미터	위치	타입	필수	설명
(없음)	-	-	-	-

응답 (200 OK)

```
[
  {
    "memberId": "string",
    "memberName": "string",
    "grade": "BRONZE | SILVER | GOLD | VIP",
    "phone": "string",
    "createdAt": "2026-01-01T00:00:00",
    "recentOrders": []
  }
]
```

비즈니스 규칙

`del\_yn` = 'N' 상시필터 → 탈퇴 회원은 목록에서 제외

`recentOrders`는 이 API에서 채워지지 않고 항상 빈 배열([INF-ORD-002])에서만 채워짐)

오류 응답

코드	사유	발생 조건
400	유효성 실패	필수 필드 누락
401	인증 실패	토큰 없음/만료

참조 테이블

테이블	SCH
MEMBERS	[[SCH-ORD-001]]

curl 예시

```
curl -X GET /api/members/ \  
-H "Content-Type: application/json"
```

INF-ORD-002 회원 상세 조회

INF-ORD-002: GET /api/members/{memberId} — 회원 상세 조회

- > **개요:** 회원 상세 정보와 함께 최근 주문 5건 요약을 반환한다. (linked_func: FUNC-order-004, LAB-103)
- > **근거 소스:** `modules/shop-api/src/main/java/com/sm/lab/shop/controller/MemberController.java:27-31`
- > [반영: FUNC-order-004] SR-207로 신규 화면 `GET /member/{memberId}`(`MemberViewController`)가 이 API의
- > 응답(`MemberService.get` 재사용, HTTP 재호출 없음)을 소비하도록 추가됐다 — 요청/응답/규칙/오류 계약은
- > 변경 없음(AS-IS 유지, qa-agent round2 실측 확인). `screens` 필드는 아직 갱신하지 않았다 — 이 화면의
- > UIS 신규 채번이 아직 안 됐고(STORY-FUNC-order-004 `##` 후속 추적(TODO) `#2-#3`), 존재하지 않는
- > UIS-ID를 여기 채우면 참조가 끊긴다. UIS 채번 시 `screens: ["UIS-ORD-00X"]`로 갱신할 것.

요청

Method: GET

Path: /api/members/{memberId}

Content-Type: application/json

파라미터	위치	타입	필수	설명
memberId	PathVariable	string	Y	회원 ID

응답 (200 OK)

```
{
  "memberId": "string",
  "memberName": "string",
  "grade": "BRONZE | SILVER | GOLD | VIP",
  "phone": "string",
  "createdAt": "2026-01-01T00:00:00",
  "recentOrders": [
    {
      "orderNo": "string",
      "orderState": "PLACED | PAID | SHIPPED | PARTIAL_SHIPPED | CANCELED | DONE",
      "totalAmount": 0,
      "orderedAt": "2026-01-01T00:00:00"
    }
  ]
}
```

비즈니스 규칙

`del\_yn = 'N'` 상시필터 → 탈퇴 회원 조회 시 404

`recentOrders`는 최근 주문 **상위 5건 고정**(`RECENT\_ORDER\_LIMIT`), `ordered\_at` DESC, `order\_no` DESC 순

주문이 없으면 `recentOrders`는 빈 배열(`null` 금지, Member 도메인 필드 기본값)

오류 응답

코드	사유	발생 조건
404	회원 없음	memberId로 조회된 회원이 없거나 탈퇴 회원

참조 테이블

`MEMBERS`

`ORDERS`

curl 예시

```
curl -X GET /api/members/M0001 \  
-H "Content-Type: application/json"
```

INF-ORD-003 주문 목록 조회

INF-ORD-003: GET /api/orders — 주문 목록 조회

> ****개요:**** 회원:주문상태:조회기간으로 필터링한 주문 목록을 페이징 조회한다. (linked_func: FUNC-order-001, LAB-101)

> [반영: FUNC-order-001] SR-205 — 조회 기간(startDate/endDate) 필터 추가.

> ****근거 소스:**** `src/main/java/com/sm/lab/shop/controller/OrderController.java:28-44`

요청

Method: GET

Path: /api/orders

Content-Type: application/json

파라미터	위치	타입	필수	설명
memberId	Query	string	N	회원 ID 필터
orderState	Query	string	N	주문상태 필터
startDate	Query	date(ISO-8601)	N	조회 시작일. 미제시 시 종료일 기준 30일 전(오늘-30일)으로 독립 보장. [반영: FUNC-order-001]
endDate	Query	date(ISO-8601)	N	조회 종료일. 미제시 시 오늘로 독립 보장. `startDate`/`endDate`는 각각 독립적으로 기본값이 적용된다(하나만 제시해도 나머지만 보장). [반영: FUNC-order-001]
page	Query	int	N	페이지 번호(기본 1)
size	Query	int	N	페이지 크기(기본 20)

> ****[반영: SR-300]**** "오늘"은 `OrderService`에 주입되는 `Clock`으로 계산한다(운영 기본값 = 시스템 시계 — 결과는 AS-IS와 동일). 계약:기본값 의미는 변경 없음, 테스트에서만 고정 시계로 재현 가능해졌다.

응답 (200 OK)

```
{
  "totalCount": 0,
  "page": 1,
  "items": [
    {
      "orderNo": "string",
      "memberId": "string",
      "memberName": "string",
      "orderState": "PLACED | PAID | SHIPPED | PARTIAL_SHIPPED | CANCELED | DONE",
      "totalAmount": 0,
      "orderedAt": "2026-01-01T00:00:00",
      "items": null,
    }
  ]
}
```

```

    "deliveries": null
  }
]
}

```

비즈니스 규칙

`del\_yn` = 'N' 상시 필터 → 논리삭제된 주문 제외

`memberId` 파라미터가 있으면 해당 회원 주문만, `orderState` 파라미터가 있으면 해당 상태만 필터(LAB-101 추가 요구사항)

`offset` = $\max(0, \text{page} - 1) * \text{size}$

`ORDERS`와 `MEMBERS`를 조인해 `memberName`을 함께 반환(회원 미탈퇴 여부와 무관하게 조인만 수행)

목록 응답의 각 항목은 `items`/`deliveries`를 채우지 않음(상세는 [[INF-ORD-004]])

[반영: FUNC-order-001] 기간 필터는 `orderState`/`memberId`와 **AND**로 결합되며, `ordered\_at` (DATETIME) 기준

양끝 날짜를 포함한다(`ordered\_at` $\geq$ startDate AND `ordered\_at` $<$ endDate + 1일). 응답 스키마

(`totalCount`/`page`/`items`)는

변경하지 않는다 — 요청 측 파라미터만 추가.

[반영: FUNC-order-001] `startDate`/`endDate`는 각각 독립적으로 기본값이 적용된 뒤 유효 구간 (`effectiveStart`~`effectiveEnd`)이

역전되면(예: `endDate`만 제시해 보정된 `effectiveStart`가 그보다 늦어지는 경우) 조회를 수행하지 않고 400으로 거부한다.

오류 응답

코드	사유	발생 조건
400	유효성 실패	파라미터 타입 오류(날짜 형식 포함) 또는 유효 조회기간 역전(`effectiveStart` > `effectiveEnd`) [반영: FUNC-order-001]
401	인증 실패	토큰 없음/만료

인증·인가 (SR-204 — 전역 정책)

> 이 오류 계약은 `/api/\*\*` 전역에 적용되는 횡단 정책이다(구현: `ApiKeyAuthFilter`). 화면

> 라우트(/product/**/cart/order/**)는 무인증 유지. 개별 엔드포인트 INF는 이 정책을 상속한다.

조건	상태	본문
`X-API-Key` 미제시/무효	401	`{"error": "unauthorized"}`
member 스코프 키로 타 회원 자원 접근(소유권 불일치)	403	`{"error": "forbidden"}`
회원 목록·주문 export 등 관리 자원에 member 키	403	admin 키 전용

키 매핑은 앱 설정 `lab.api-keys` (admin→`, member→회원ID). 경로 판정은 서블릿 정규화 경로

기준(퍼센트 인코딩·matrix 파라미터 우회 차단, default-deny). 소유권은 자원의 실제 member_id를

DB 대조(orders/{orderNo}) — 토큰 문자열 대조가 아니다(SR-204 D15).

참조 테이블

테이블	SCH
ORDERS	[[SCH-ORD-004]]
MEMBERS	[[SCH-ORD-001]]

curl 예시

```
curl -X GET "/api/orders?memberId=M0001&orderState=PLACED&page=1&size=20" \  
-H "Content-Type: application/json"
```

변경 이력

<!-- spec-history: machine-managed -->

일자	SR	항 목	변경 요약	커밋
2026-09-19	SR-300	#1	'오늘' 계산을 주입 가능한 Clock으로 변경(운영 기본값=시스템 시계, 계약-응답 불변) — anchors 줄번호 OrderService.java:76-90으로 갱신	shop-api@416b260

INF-ORD-004 주문 상세 조회

INF-ORD-004: GET /api/orders/{orderNo} — 주문 상세 조회

> ****개요:**** 주문 1건을 라인(아이템)·배송 정보를 포함해 조회한다.

> ****근거 소스:**** `src/main/java/com/sm/lab/shop/controller/OrderController.java:46-50`

요청

Method: GET

Path: /api/orders/{orderNo}

Content-Type: application/json

파라미터	위치	타입	필수	설명
orderNo	PathVariable	string	Y	주문번호

응답 (200 OK)

```
{
  "orderNo": "string",
  "memberId": "string",
  "memberName": "string",
  "orderState": "PLACED | PAID | SHIPPED | PARTIAL_SHIPPED | CANCELED | DONE",
  "totalAmount": 0,
  "orderedAt": "2026-01-01T00:00:00",
  "items": [
    {
      "orderNo": "string",
      "lineNo": 1,
      "sku": "string",
      "productName": "string",
      "qty": 0,
      "unitPrice": 0
    }
  ],
  "deliveries": [
    {
      "deliveryNo": "string",
      "orderNo": "string",
      "deliveryState": "READY | SHIPPED | DELIVERED | CANCELED",
      "invoiceNo": "string",
      "shippedAt": "2026-01-01T00:00:00"
    }
  ]
}
```

비즈니스 규칙

`del\_yn = 'N'` 상시필터 → 논리삭제된 주문 조회 시 404

`items`는 `ORDER\_ITEMS`와 `PRODUCTS`를 조인해 `productName`을 함께 반환
 `deliveries`는 배송이 없으면 빈 배열

오류 응답

코드	사유	발생 조건
404	주문 없음	orderNo로 조회된 주문이 없거나 삭제됨
401	인증 실패	토큰 없음/만료
403	인가 실패	member 스코프 키가 타 회원 주문에 접근

인증·인가 (SR-204 — 전역 정책)

> 이 오류 계약은 `/api/\*\*` 전역에 적용되는 횡단 정책이다(구현: `ApiKeyAuthFilter`). 화면

> 라우트(`/product/\*\*/cart/order/\*\*`)는 무인증 유지. 개별 엔드포인트 INF는 이 정책을 상속한다.

조건	상태	본문
`X-API-Key` 미제시/무효	401	`{"error": "unauthorized"}`
member 스코프 키로 타 회원 주문 접근(소유권 불일치)	403	`{"error": "forbidden"}`

소유권 판정은 `orderNo`로 `OrderDao.selectByOrderNo`를 조회한 실제 `member\_id`와 요청 키의

스코프를 대조한다(토큰 문자열 대조가 아님, SR-204 D15). admin 스코프(`\*`) 키는 이 필터를

통과해 컨트롤러의 404 분기까지 도달하지만, member 스코프 키는 주문이 없어도(조회 결과 null)

소유권을 확인할 수 없으므로 컨트롤러 도달 전에 403으로 거부된다(즉 member 키로는 존재하지

않는 orderNo도 404가 아니라 403으로 응답됨).

참조 테이블

테이블	SCH
ORDERS	[[SCH-ORD-004]]
MEMBERS	[[SCH-ORD-001]]
ORDER_ITEMS	[[SCH-ORD-003]]
PRODUCTS	[[SCH-ORD-005]]
ORDER_DELIVERY	[[SCH-ORD-002]]

curl 예시

```
curl -X GET /api/orders/20260101-0101 \
  -H "Content-Type: application/json"
```

INF-ORD-005 주문 생성

INF-ORD-005: POST /api/orders — 주문 생성

> **개요:** 요청된 상품 라인들을 검증·재고 차감한 뒤 신규 주문을 생성한다.

> **근거 소스:** `modules/shop-api/src/main/java/com/sm/lab/shop/controller/OrderController.java:52-56`

요청

Method: POST

Path: /api/orders

Content-Type: application/json

```
{
  "memberId": "string",
  "items": [
    { "sku": "string", "qty": 0 }
  ]
}
```

파라미터	위치	타입	필수	설명
memberId	Body	string	Y	회원 ID
items	Body	array	Y	주문 라인 목록(sku, qty)

응답 (200 OK)

```
{
  "orderNo": "string",
  "memberId": "string",
  "memberName": "string",
  "orderState": "PLACED",
  "totalAmount": 0,
  "orderedAt": "2026-01-01T00:00:00",
  "items": [ { "orderNo": "string", "lineNo": 1, "sku": "string", "productName": "string", "qty": 0, "unitPrice": 0 } ],
  "deliveries": []
}
```

비즈니스 규칙

각 라인의 상품이 `sale\_yn` = 'Y'(판매중)가 아니면 400

재고 차감(`decreaseStock`)이 0행이면 재고 부족으로 409 — `WHERE stock\_qty >= qty` 조건부 UPDATE로 동시성 가드
주문번호(`orderNo`)는 `yyyymmdd-{자리 시퀀스}` 형식으로 채번(`AtomicInteger`, 시작값 100 — 인스턴스 내 메모리 채
번, 재기동 시 초기화)

신규 주문 상태는 항상 `PLACED`

`totalAmount`는 각 라인 `단가 × 수량`의 합

트랜잭션 순서

1. 라인별로 `PRODUCTS` 조회 → 판매상태 검증 → 재고 조건부 차감(`UPDATE ... WHERE stock\_qty >= qty`)
2. `ORDERS` INSERT (order_state='PLACED')
3. 라인별 `ORDER\_ITEMS` INSERT
4. 생성된 주문을 [[INF-ORD-004]]와 동일한 조회(`selectByOrderNo` + `selectItems` + `selectDeliveries`)로 재조회하여 반환

오류 응답

코드	사유	발생 조건
400	판매 중인 상품이 아님	상품 미존재 또는 `sale_yn != 'Y'`
409	재고 부족	차감 대상 수량이 현재 재고보다 많음

참조 테이블

테이블	SCH
PRODUCTS	[[SCH-ORD-005]]
ORDERS	[[SCH-ORD-004]]
ORDER_ITEMS	[[SCH-ORD-003]]
MEMBERS	[[SCH-ORD-001]]
ORDER_DELIVERY	[[SCH-ORD-002]]

curl 예시

```
curl -X POST /api/orders \
  -H "Content-Type: application/json" \
  -d '{"memberId":"M0001","items":[{"sku":"SKU001","qty":2}]}'
```

INF-ORD-006 주문 취소

INF-ORD-006: PATCH /api/orders/{orderNo}/cancel — 주문 취소

> ****개요:**** 주문을 취소 상태로 바꾸고, 이미 취소 완료 상태이거나 출고 완료된 배송이 있으면 거부하며, 취소 성공 시 라인별 재고를 원복한다.

> ****근거 소스:**** `modules/shop-api/src/main/java/com/sm/lab/shop/controller/OrderController.java:59-61`

요청

Method: PATCH

Path: /api/orders/{orderNo}/cancel

Content-Type: application/json

파라미터	위치	타입	필수	설명
orderNo	PathVariable	String	Y	취소할 주문번호

응답 (200 OK)

```
{
  "orderNo": "20260907-0101",
  "memberId": "M0001",
  "memberName": "홍길동",
  "orderState": "CANCELED",
  "totalAmount": 45000,
  "orderedAt": "2026-09-01T10:20:30",
  "items": [
    {
      "orderNo": "20260907-0101",
      "lineNo": 1,
      "sku": "SKU-001",
      "productName": "상품A",
      "qty": 2,
      "unitPrice": 15000
    }
  ],
  "deliveries": [
    {
      "deliveryNo": "D0001",
      "orderNo": "20260907-0101",
      "deliveryState": "READY",
      "invoiceNo": null,
      "shippedAt": null
    }
  ]
}
```

취소 완료 후의 주문 상세(items-deliveries 포함)를 그대로 반환한다. `deliveries`가 없는 주문은 빈 배열.

비즈니스 규칙

주문 상태가 `CANCELED` 또는 `DONE`이면 → 409 거부(이미 종결된 주문)

배송 중 하나라도 `SHIPPED` 또는 `DELIVERED` 상태이면 → 409 거부(코드 `ORD-4001`, 출고 완료 배송은 취소 불가)

상태 전이(UPDATE)는 `order\_state` NOT IN (`CANCELED`,`DONE`) 조건부 쿼리로 실행 — 동시 요청 중 하나가 먼저 선택하면 0행이 되어 409로 거부(레이스 가드, 재고는 원복하지 않음)

상태 전이가 성공한 뒤에만 라인별 재고를 원복한다 (linked_func: FUNC-order-002)

트랜잭션 순서

1. `ORDERS`+`MEMBERS` 조회(주문 존재·현재 상태 확인) — 없으면 404
2. `ORDER\_ITEMS`+`PRODUCTS`, `ORDER\_DELIVERY` 조회(라인·배송 상태 확인)
3. `ORDERS.order\_state`를 `CANCELED`로 조건부 UPDATE (미종결 상태일 때만, 레이스 가드)
4. 주문 라인마다 `PRODUCTS.stock\_qty` 증가(재고 원복)
5. `ORDERS`+`MEMBERS`, `ORDER\_ITEMS`+`PRODUCTS`, `ORDER\_DELIVERY` 재조회 후 응답

사이드이펙트

취소 성공 시: 주문에 포함된 각 라인의 `sku`만큼 `PRODUCTS.stock\_qty`를 증가시킨다(메인 목적인 `ORDERS` 상태 변경 외 추가 처리).

오류 응답

코드	사유	발생 조건
404	주문 없음	`orderNo`에 해당하는 주문이 없음
409	취소 불가 상태	주문 상태가 이미 `CANCELED` 또는 `DONE`
409	출고 완료 배송 존재 (ORD-4001)	배송 중 `SHIPPED`/`DELIVERED` 상태가 하나라도 있음
409	취소 불가 상태(동시 처리됨)	상태 전이 UPDATE가 0행 — 동시 요청이 먼저 취소·완료 처리함

참조 테이블

`ORDERS`

`MEMBERS`

`ORDER\_ITEMS`

`PRODUCTS`

`ORDER\_DELIVERY`

curl 예시

```
curl -X PATCH /api/orders/20260907-0101/cancel \
-H "Content-Type: application/json"
```

INF-ORD-007 주문 배송 목록 조회

INF-ORD-007: GET /api/orders/{orderNo}/deliveries — 주문 배송 목록 조회

> **개요:** 주문 1건의 배송 목록만 반환한다.

> **근거 소스:** `modules/shop-api/src/main/java/com/sm/lab/shop/controller/OrderController.java:64-68`

요청

Method: GET

Path: /api/orders/{orderNo}/deliveries

Content-Type: application/json

파라미터	위치	타입	필수	설명
orderNo	PathVariable	string	Y	주문번호

응답 (200 OK)

```
[
  {
    "deliveryNo": "string",
    "orderNo": "string",
    "deliveryState": "READY | SHIPPED | DELIVERED | CANCELED",
    "invoiceNo": "string",
    "shippedAt": "2026-01-01T00:00:00"
  }
]
```

비즈니스 규칙

내부적으로 [[INF-ORD-004]]와 동일한 `OrderService.detail()`을 호출한다 — `ORDER\_ITEMS`(`PRODUCTS` 조인 포함)도

함께 조회되지만 응답에는 `deliveries`만 반환된다(불필요한 내부 조회이나 기능상 영향 없음)

배송 목록은 `shipped\_at DESC, delivery\_no DESC`로 정렬(미발송 `READY` 건은 `shipped\_at`이 NULL이라 하단 — LAB-113)
배송이 없으면 빈 배열

오류 응답

코드	사유	발생 조건
404	주문 없음	orderNo로 조회된 주문이 없음

참조 테이블

테이블	SCH
-----	-----

ORDERS	[[SCH-ORD-004]]
MEMBERS	[[SCH-ORD-001]]
ORDER_ITEMS	[[SCH-ORD-003]]
PRODUCTS	[[SCH-ORD-005]]
ORDER_DELIVERY	[[SCH-ORD-002]]

curl 예시

```
curl -X GET /api/orders/20260101-0101/deliveries \
-H "Content-Type: application/json"
```

변경 이력

날짜	변경 내용	변경자
2026-08-23	배송 이력 정렬을 최신순으로(shipped_at DESC, 미발송 null은 하단 · tie는 delivery_no DESC) — LAB-113, linked_func: FUNC-order-006	/sl-change --quick (auto)

INF-ORD-008 판매중 상품 목록 조회

> [변경: SR-201] 2026-08-22

INF-ORD-008: GET /api/products — 판매중 상품 목록 조회

> ****개요:**** 판매중 상태인 상품 전체를 SKU 순으로 조회한다.

> ****근거 소스:**** `src/main/java/com/sm/lab/shop/controller/ProductController.java:24-31`

요청

Method: GET

Path: /api/products

Content-Type: application/json

파라미터	위치	타입	필수	설명
keyword	Query	string	N	상품명 부분 일치 검색어. 미지정 시 전체(판매중) 목록 반환(하위 호환) — [반영: FUNC-order-007] 2026-08-22
inStock	Query	boolean	N	true면 재고 1 이상(`stock_qty` >= 1)인 상품만 반환. 미지정/false는 기존 동작과 동일(하위 호환) — [반영: FUNC-order-007](SR-220) 2026-09-09. non-boolean 값은 Spring 바인딩 실패로 400

응답 (200 OK)

```
[
  {
    "sku": "string",
    "productName": "string",
    "price": 0,
    "stockQty": 0,
    "saleYn": "Y",
    "listPrice": 0,
    "imageUrl": "string"
  }
]
```

> [변경: SR-306] 2026-09-17 — `listPrice`(정가, number|null)·`imageUrl`(대표 이미지 경로, string|null) 필드

> 추가(필드 추가만, 기존 5필드 이름·순서·타입 불변, 하위호환). 값이 없는 상품은 `null`로 내려간다 —

> 화면은 이때 정가 취소선·할인 배지를 그리지 않는다(할인율은 저장하지 않고 화면이 `price`와 비교 계산).

> `imageUrl`은 앱이 서빙하는 정적 경로(`/images/products/...`)이며 외부 URL이 아니다.

비즈니스 규칙

`sale\_yn` = 'Y' 상시필터 → 판매중 상품만 반환(단건 조회인 [[INF-ORD-009]]는 이 필터가 없음)

[반영: FUNC-order-007] `keyword` 파라미터가 있으면 `product\_name` 부분 일치(LIKE) 검색 —

`sale\_yn` = 'Y' 상시필터와 AND 결합. `keyword` 미지정 시 기존과 동일하게 전체(판매중) 목록 반환(하위 호환),

기존 소비처 무영향). 응답 스키마(`sku`/`productName`/`price`/`stockQty`/`saleYn`) 변경 없음 —

"품질" 판정은 화면(UIS-ORD-003) 몹으로 `stockQty` 값을 그대로 사용.

[반영: FUNC-order-007](SR-220) `inStock=true`면 `stock\_qty >= 1` 조건을 `sale\_yn = 'Y'` 상시필터와

AND 결합해 품질 상품을 제외한다. `inStock` 미지정 또는 `false`는 기존 동작 그대로(하위 호환).

별도 count 쿼리·페이징이 없어 목록 결과와 총 건수가 항상 일치한다. 응답 스키마 변경 없음.

[반영: SR-306] `listPrice`·`imageUrl`은 표시 전용 — 주문·장바구니 금액 계산은 종전대로 `price`만 쓴다.

인증

> [변경: SR-307] 2026-09-17 — 무키(X-API-Key 헤더 자체 없음) GET 요청은 인증 없이 공개된다(`ApiKeyAuthFilter`가

> `isOpenRoute` 화이트리스트가 아니라 `doFilterInternal`에서 "무키일 때만" 예외 처리 — 필터는 항상 실행된다).

> **키를 보낸 요청(무효 키 포함)은 이 예외를 타지 않고 종전과 동일한 인증·인가 판정을 그대로 거친다** —

> 예: member 키로 `?memberId=타인`을 보내면 종전과 동일하게 403. GET 외 메서드(POST/PATCH/DELETE)는

> 이 예외 대상이 아니라 무키 시 401 그대로.

오류 응답

코드	사유	발생 조건
400	유효성 실패	필수 필드 누락
401	인증 실패	키가 있으나 무효, 또는 GET 외 메서드로 무키 요청(위 "인증" 절 참고)

참조 테이블

테이블	SCH
PRODUCTS	[[SCH-ORD-005]]

curl 예시

```
curl -X GET /api/products \
-H "Content-Type: application/json"
```

[반영: FUNC-order-007] `keyword` 검색 예시:

```
curl -X GET "/api/products?keyword=셔츠" \
-H "Content-Type: application/json"
```

[반영: FUNC-order-007](SR-220) `inStock` 필터 예시(품질 제외):

```
curl -X GET "/api/products?inStock=true" \
```

-H "Content-Type: application/json"

변경 이력(이전 기록)

날짜	변경 내용	변경자
2026-08-22	SR-201: keyword 부분일치 검색 파라미터 추가(구현 반영 — FUNC-order-007). path 표기를 실측 라우트 (/api/products — 트레일링 슬래시 404)로 정정	sl-aidd STEP 5.5 재동기화
2026-09-09	SR-220: inStock 쿼리 파라미터 추가(재고 1 이상만 조회, 미지정 시 하위호환) — 구현 반영(FUNC-order-007)	sl-aidd STEP 5.5 재동기화
2026-09-17	SR-307: 무키 GET 공개(앱 서버 /shop 401 해소) — 쿼리·응답 스키마 불변, "인증" 절 신설	sl-aidd STEP 5.5 재동기화

변경 이력

<!-- spec-history: machine-managed -->

일자	SR	항목	변경 요약	커밋
2026-09-17	SR-307	#1	무키 GET 공개(앱 서버 /shop 401 해소) — 인증 절 신설, 쿼리·응답 스키마 불변	shop-api@cd859a1
2026-09-17	SR-306	#2	응답에 listPrice·imageUrl 필드 추가(필드 추가만, 하위호환, 값 없으면 null)	shop-api, shop-api@7eebc9f, shop-api@e1d18da, shop-web@cbe9816

INF-ORD-009 상품 단건 조회

INF-ORD-009: GET /api/products/{sku} — 상품 단건 조회

> ****개요:**** SKU로 상품 1건을 조회한다.

> ****근거 소스:**** `src/main/java/com/sm/lab/shop/controller/ProductController.java:27-31`

요청

Method: GET

Path: /api/products/{sku}

Content-Type: application/json

파라미터	위치	타입	필수	설명
sku	PathVariable	string	Y	상품 SKU

응답 (200 OK)

```
{
  "sku": "string",
  "productName": "string",
  "price": 0,
  "stockQty": 0,
  "saleYn": "Y",
  "listPrice": 0,
  "imageUrl": "string"
}
```

> [변경: SR-306] 2026-09-17 — `listPrice`(정가, number|null)·`imageUrl`(대표 이미지 경로, string|null) 필드

> 추가(필드 추가만, 기존 5필드 이름·순서·타입 불변, 하위호환). 값이 없으면 `null`. [[INF-ORD-008]]과 동일 계약.

비즈니스 규칙

목록 조회([[INF-ORD-008]])와 달리 `sale\_yn` 필터가 없다 — 판매종료 상품도 SKU를 알면 조회 가능

[반영: SR-306] `listPrice`·`imageUrl`은 표시 전용 — 주문·장바구니 금액 계산은 종전대로 `price`만 쓴다.

인증

> [변경: SR-307] 2026-09-17 — [[INF-ORD-008]]과 동일: 무키 GET은 공개(`ApiKeyAuthFilter` doFilterInternal

> 예외), 키를 보낸 요청은 종전 판정 그대로. ****공개 대상 SKU 형식은 `SKU-` 접두사 + 영숫자·하이픈****

> (`^/api/products/(SKU-[A-Za-z0-9-]+)\$`)으로 한정된다 — 이 형식과 맞지 않는 세그먼트(예: 미래에 추가될

> `/api/products/export` 같은 서브리소스)는 이 경로 패턴에 매치되지 않아 무키 401(fail-closed)로 남는다.

오류 응답

코드	사유	발생 조건
401	인증 실패	키가 있으나 무효, 또는 GET 외 메서드로 무키 요청, 또는 sku가 `SKU-` 형식이 아니어서 공개 판정에 매치되지 않음(위 "인증" 절 참고)
404	상품 없음	sku로 조회된 상품이 없음

참조 테이블

테이블	SCH
PRODUCTS	[[SCH-ORD-005]]

curl 예시

```
curl -X GET /api/products/SKU001 \
-H "Content-Type: application/json"
```

변경 이력(이전 기록)

날짜	변경 내용	변경자
2026-09-17	SR-307: 무키 GET 공개(앱 서빙 /shop 401 해소) + 공개 대상 SKU 형식(`SKU-` 접두사) 명시 — 쿼리·응답 스키마 불변, "인증" 절 신설	sl-aidd STEP 5.5 재 동기화

변경 이력

<!-- spec-history: machine-managed -->

일자	SR	항목	변경 요약	커밋
2026-09-17	SR-307	#1	무키 GET 공개 + 공개 대상 SKU 형식(SKU- 접두사) 명시 — 쿼리·응답 스키마 불변	shop-api@cd859a1
2026-09-17	SR-306	#2	응답에 listPrice.imageUrl 필드 추가(필드 추가만, 하위호환, 값 없으면 null)	shop-api, shop-api@7eebc9f, shop-api@e1d18da, shop-web@cbe9816

INF-ORD-010 장바구니 담기

INF-ORD-010: POST /api/cart/items — 장바구니 담기

> ****개요:**** 회원 장바구니에 상품을 담는다. 같은 상품을 재담기하면 기존 수량에 합산된다.

> ****근거 소스:**** `src/main/java/com/sm/lab/shop/controller/CartController.java:22-26`

요청

Method: POST

Path: /api/cart/items

Content-Type: application/json

파라미터	위치	타입	필수	설명
memberId	Body	string	Y	회원 ID
sku	Body	string	Y	상품 SKU
qty	Body	int	Y	담을 수량 (1 이상)

응답 (200 OK)

```
{
  "memberId": "string",
  "sku": "string",
  "productName": "string",
  "price": 0,
  "qty": 0,
  "addedAt": "2026-08-23T10:00:00",
  "lineTotal": 0
}
```

비즈니스 규칙

같은 상품 재담기는 DB 원자 UPSERT('INSERT ... ON DUPLICATE KEY UPDATE qty = qty + VALUES(qty)')로 수량 합산(PK `member\_id,sku`) — 동시 담기에도 lost update-PK 중복 500 없음

판매중지('sale_yn='N') 상품은 담기 거부(409)

품절('stock_qty=0')은 UPSERT 이전에 조기 거부(409) — 합산·원복 사이클 자체가 불필요

UPSERT 직후 최종(합산) qty가 재고를 초과하면, 방금 더한 만큼만 되돌리고(원복값 0 이하면 행 삭제) 409 — UPSERT가 잡은 행 잠금이 트랜잭션 종료까지 유지되어 판정·원복 사이 다른 트랜잭션 개입 불가

담기는 재고를 차감하지 않는다(보관 전용)

트랜잭션 순서

1. CART_ITEMS UPSERT(수량 합산)
2. CART_ITEMS 병합 결과 SELECT(상품명·현재 단가 조인)
3. (재고 초과 시) CART_ITEMS 원복 — 복원값 > 0이면 UPDATE, 0 이하면 DELETE

오류 응답

코드	사유	발생 조건
400	유효성 실패	qty < 1
404	회원 없음	memberId로 조회된 회원 없음
404	상품 없음	sku로 조회된 상품 없음
409	판매중지 상품	상품의 sale_yn = 'N'
409	품절 상품	상품의 stock_qty = 0
409	재고 초과	합산된 qty가 가용 재고 초과

참조 테이블

테이블	
CART_ITEMS	[[SCH-ORD-006]]
PRODUCTS	[[SCH-ORD-005]]
MEMBERS	[[SCH-ORD-001]]

curl 예시

```
curl -X POST /api/cart/items \
  -H "Content-Type: application/json" \
  -d '{"memberId":"M001","sku":"SKU001","qty":2}'
```

INF-ORD-011 장바구니 조회

INF-ORD-011: GET /api/cart/ — 장바구니 조회

> **개요:** 회원의 장바구니 품목 전체와 총액을 조회한다.

> **근거 소스:** `src/main/java/com/sm/lab/shop/controller/CartController.java:28-32`

요청

Method: GET

Path: /api/cart/

Content-Type: application/json

파라미터	위치	타입	필수	설명
memberId	Query	string	Y	회원 ID

응답 (200 OK)

```
{
  "items": [
    {
      "memberId": "string",
      "sku": "string",
      "productName": "string",
      "price": 0,
      "qty": 0,
      "addedAt": "2026-08-23T10:00:00",
      "lineTotal": 0
    }
  ],
  "totalAmount": 0
}
```

비즈니스 규칙

품목은 담은 순(`added\_at`, `sku`)으로 정렬된다

`totalAmount`는 각 품목의 `price × qty`(`lineTotal`) 합계로, 조회 시점에 애플리케이션에서 계산한다(저장 컬럼 아님)

오류 응답

코드	사유	발생 조건
404	회원 없음	memberId로 조회된 회원 없음

참조 테이블

테이블	SCH
CART_ITEMS	[[SCH-ORD-006]]

PRODUCTS	[[SCH-ORD-005]]
MEMBERS	[[SCH-ORD-001]]

curl 예시

```
curl -X GET "/api/cart/?memberId=M001" \  
-H "Content-Type: application/json"
```

INF-ORD-012 장바구니 수량 변경

INF-ORD-012: PATCH /api/cart/items/{sku} — 장바구니 수량 변경

> **개요:** 장바구니에 담긴 품목의 수량을 변경한다.

> **근거 소스:** `src/main/java/com/sm/lab/shop/controller/CartController.java:34-39`

요청

Method: PATCH

Path: /api/cart/items/{sku}

Content-Type: application/json

파라미터	위치	타입	필수	설명
sku	PathVariable	string	Y	상품 SKU
memberId	Query	string	Y	회원 ID
qty	Body	int	Y	변경할 수량 (1 이상)

응답 (200 OK)

```
{
  "memberId": "string",
  "sku": "string",
  "productName": "string",
  "price": 0,
  "qty": 0,
  "addedAt": "2026-08-23T10:00:00",
  "lineTotal": 0
}
```

비즈니스 규칙

qty < 1은 400으로 거부한다 — 삭제는 명시적 DELETE로만 수행(실수 삭제 방지)

요청 qty가 상품 재고를 초과하거나 상품이 품절(stock_qty=0)이면 409

트랜잭션 순서

- CART_ITEMS UPDATE(qty 변경)
- CART_ITEMS SELECT(변경 결과 반환용, 상품명·현재 단가 조인)

오류 응답

코드	사유	발생 조건
400	유효성 실패	qty < 1

404	장바구니 품목 없음	memberId+sku 조합의 품목 없음
404	상품 없음	sku로 조회된 상품 없음
409	재고 초과/품절	요청 qty가 가용 재고 초과 또는 상품 품절

참조 테이블

테이블	SCH
CART_ITEMS	[[SCH-ORD-006]]
PRODUCTS	[[SCH-ORD-005]]

curl 예시

```
curl -X PATCH "/api/cart/items/SKU001?memberId=M001" \
  -H "Content-Type: application/json" \
  -d '{"qty":3}'
```

INF-ORD-013 장바구니 품목 삭제

INF-ORD-013: DELETE /api/cart/items/{sku} — 장바구니 품목 삭제

> ****개요:**** 장바구니에서 품목 1건을 삭제한다.

> ****근거 소스:**** `src/main/java/com/sm/lab/shop/controller/CartController.java:41-46`

요청

Method: DELETE

Path: /api/cart/items/{sku}

Content-Type: application/json

파라미터	위치	타입	필수	설명
sku	PathVariable	string	Y	상품 SKU
memberId	Query	string	Y	회원 ID

응답 (204 No Content)

응답 본문 없음.

오류 응답

코드	사유	발생 조건
404	장바구니 품목 없음	memberId+sku 조합의 품목 없음

참조 테이블

테이블	
CART_ITEMS	[[SCH-ORD-006]]
PRODUCTS	[[SCH-ORD-005]]

curl 예시

```
curl -X DELETE "/api/cart/items/SKU001?memberId=M001" \
-H "Content-Type: application/json"
```

INF-ORD-014 장바구니 체크아웃

INF-ORD-014: POST /api/cart/checkout — 장바구니 체크아웃

> **개요:** 장바구니 전체 품목을 기존 주문 생성 규칙으로 전환해 주문을 만들고 장바구니를 비운다.

> **근거 소스:** `src/main/java/com/sm/lab/shop/controller/CartController.java:53-56`

요청

Method: POST

Path: /api/cart/checkout

Content-Type: application/json

파라미터	위치	타입	필수	설명
memberId	Body	string	Y	회원 ID

```
{
  "memberId": "M001"
}
```

응답 (200 OK)

```
{
  "orderNo": "20260823-0101",
  "totalAmount": 0,
  "itemCount": 0
}
```

비즈니스 규칙

주문 생성은 새 규칙을 만들지 않고 `OrderService.create`를 그대로 재사용한다(채번·PLACED 상태·재고 차감 — 검증/차감 로직 미복제)

장바구니 조회는 `SELECT ... FOR UPDATE`로 잠근다 — 동일 회원의 동시 체크아웃에서 스냅샷이 아닌 최신 커밋 데이터를 잠그므로, 중복 주문이 발생하지 않는다

잠금 직후 전 품목을 사전 스왑하여 상품없음·판매중지(`sale\_yn`='N')·재고부족을 한 번에 모아 409 사유 목록 하나로 응답한다(품목별 사유, 부분 주문 없이 전량 거부)

사전 스왑을 통과해도 `OrderService.create`의 실제 재고 차감(`decreaseStock`) 실패가 최종 판정이다(스왑·차감 사이 시점 차 대비)

장바구니는 주문 생성이 트랜잭션 내에서 확정된 뒤에만 비운다(실패 시 품목 보존)

트랜잭션 순서

1. `CART\_ITEMS` 잠금 조회(`FOR UPDATE`) — 비어 있으면 400
2. 품목별 `PRODUCTS` 사전 스왑(상품없음·판매중지/재고부족) — 이슈 있으면 409

3. `OrderService.create` 위임: `PRODUCTS.stock\_qty` 차감 → `ORDERS` 1건 INSERT → `ORDER\_ITEMS` N건 INSERT (동일 트랜잭션 참여)

4. `CART\_ITEMS` 전체 삭제 (주문 확정 후)

사이드이펙트

체크아웃 성공 시 품목별로 `PRODUCTS.stock\_qty`가 차감된다(`OrderService.create` → `decreaseStock`)

체크아웃 성공 시 `CART\_ITEMS`에서 해당 회원의 품목이 전부 삭제된다(`deleteAllItems`)

오류 응답

코드	사유	발생 조건
400	빈 장바구니	memberId의 장바구니에 품목 없음
400	판매 중인 상품이 아님	`OrderService.create` 내부 판정 — 사전 스왑 통과 후 시점 차로 발생 가능
404	회원 없음	memberId로 조회된 회원 없음
409	재고 부족/판매중지/상품없음	사전 스왑에서 품목별 사유 목록으로 응답(가용재고/요청수량 포함)
409	재고 부족(차감 실패)	`OrderService.create`의 `decreaseStock` 0건 — 스왑 이후 경합

참조 테이블

`CART\_ITEMS`

`MEMBERS`

`PRODUCTS`

`ORDERS`

`ORDER\_ITEMS`

curl 예시

```
curl -X POST /api/cart/checkout \
  -H "Content-Type: application/json" \
  -d '{"memberId": "M001"}'
```

INF-ORD-015 주문 목록 CSV 내보내기

INF-ORD-015: GET /api/orders/export — 주문 목록 CSV 내보내기

> ****개요:**** [[INF-ORD-003]]과 동일한 필터·정렬·상시필터로 조회한 주문 목록을 CSV 파일로 내려받는다. (linked_func: FUNC-order-013, LAB-104)

> ****근거 소스:**** `src/main/java/com/sm/lab/shop/controller/OrderController.java:79-90`

요청

Method: GET

Path: /api/orders/export

Content-Type: application/json

파라미터	위치	타입	필수	설명
memberId	Query	string	N	회원 ID 필터
orderState	Query	string	N	주문상태 필터(정본)
status	Query	string	N	주문상태 필터 별칭 — `orderState`와 동시에 오면 `orderState` 우선(D11, QA r1 FAIL 정정)

응답 (200 OK)

`Content-Type: text/csv; charset=UTF-8`, `Content-Disposition: attachment; filename="orders\_{yyyyMMdd\_HHmss}.csv"`

UTF-8 BOM(‘EF BB BF’)으로 시작하는 CSV 바이너리. 헤더 행 + [[INF-ORD-003]] 목록 조회 결과를 행 단위로 직렬화(페이징 미적용 — 전체 건).

```
주문번호,회원,상태,금액,주문일시
2026082300001-0101,M0001,PLACED,15000,2026-08-23T10:00:00
```

> ****헤더 라벨(SR-223)**:** 데이터 컬럼 순서·필드명·타입·의미(`orderNo,memberId,status,totalAmount,orderedAt`)는

> 그대로다. 헤더 텍스트만 같은 위치의 한글 라벨로 1:1 치환했다(주문번호↔orderNo, 회원↔memberId,

> 상태↔status, 금액↔totalAmount, 주문일시↔orderedAt). 근거: `docs/변경관리/SR-223/00\_요구사항.md`

> 확정 문답 `header\_label\_order`.

비즈니스 규칙

조회 규칙은 [[INF-ORD-003]]과 완전히 동일(`OrderDao.selectOrders` 재사용) — 별도 조회 규칙 없음. `del\_yn` = 'N' 상시 필터 포함

페이징 미적용: `offset=0`, `size=Integer.MAX\_VALUE`로 고정 호출(전체 건 CSV화)

`orderedAt`은 `ISO\_LOCAL\_DATE\_TIME` 고정 패턴으로 직렬화(가변폭 금지)

CSV 필드는 RFC 4180 인용 규칙 적용(콤마·큰따옴표·개행 포함 시 큰따옴표로 감싸고 내부 큰따옴표는 이중화)

CSV 수식 인젝션 방어: 필드 선두 문자가 `=`,`+`,`-`,`@`,`이면 작은따옴표(`"`)를 앞에 붙여 Excel이 수식으로 해석하지 않게 함 (D11)

파일 본문은 UTF-8 BOM 프리픽스 포함(Excel에서 한글 깨짐 방지)

오류 응답

코드	사유	발생 조건
400	유효성 실패	파라미터 타입 오류
401	인증 실패	토큰 없음/만료

인증·인가 (SR-204 — 전역 정책)

> 이 오류 계약은 `/api/\*\*` 전역에 적용되는 횡단 정책이다(구현: `ApiKeyAuthFilter`). 화면

> 라우트(`/product/\*\*/cart/order/\*\*`)는 무인증 유지. 개별 엔드포인트 INF는 이 정책을 상속한다.

조건	상태	본문
`X-API-Key` 미제시/무효	401	`{"error":"unauthorized"}`
member 스코프 키로 타 회원 자원 접근(소유권 불일치)	403	`{"error":"forbidden"}`
회원 목록·주문 export 등 관리 자원에 member 키	403	admin 키 전용

키 매핑은 앱 설정 `lab.api-keys`(admin→`, member→회원ID). 경로 판정은 서블릿 정규화 경로

기준(퍼센트 인코딩·matrix 파라미터 우회 차단, default-deny). 소유권은 자원의 실제 member_id를

DB 대조(orders/{orderNo}) — 토큰 문자열 대조가 아니다(SR-204 D15).

반출 상한 (SR-204 R-3)

결과 1000행 초과 → **413** `{"error":"payload\_too\_large","limit":1000}` (부분 반출 위장 없이 전량 거부, CSV 조립 전 판정). 이 엔드포인트는 **admin 키 전용**.

참조 테이블

ORDERS

MEMBERS

curl 예시

```
curl -X GET "/api/orders/export?memberId=M0001&orderState=PLACED" \
-o orders.csv
```

INF-ORD-016 api zipcodes

INF-ORD-016: GET /api/zipcodes

> 코드(핸들러) 기준 재동기화 골격(zero-LLM) — 요청·응답·오류 계약은 보장 대상

개요

[TBD]

요청

[TBD]

응답

[TBD]

오류

[TBD]

> 보장: `sl-sync --apply --kind=inf` 또는 ddd-api-agent

변경 이력

<!-- spec-history: machine-managed -->

일자	SR	항 목	변경 요약	커밋
2026-09-19	SR-235	#3	GET /api/zipcodes 우편번호(도로명) 검색 API 구현 — 200 {items: [{zipcode,roadAddress,sido,sigungu}]}(0건 포함), 400 MBR-4202(검색어 정규화 후 2~50자 범위 밖), 숫자 검색어는 zipcode 전방일치도 포함(OR)	shop-api@1c82e8c, shop-web@8e1e3e3

INF-ORD-017 shop spa static serving

INF-ORD-017: GET /shop/** — 쇼핑 SPA 정적 서빙 + 딥링크 폴백

- > ****개요:**** shop-web(React/Vite) 빌드 산출물(`dist`)을 shop-api가 `/shop/\*\*` 경로로 직접 서빙한다.
- > 별도 개발 서버(:5273) 재기동 없이, shop-api 재기동 한 번으로 화면 변경이 반영되게 하는 것이 목적(SR-301).
- > 새 REST 엔드포인트가 아니라 정적 리소스 매핑 + SPA 딥링크 폴백이며, 새 오류 코드·인증 계약 변경은 없다.
- > ****근거 소스:**** `modules/shop-api/src/main/java/com/sm/lab/shop/web/ShopStaticResourceConfig.java:75-95`,
`modules/shop-api/src/main/java/com/sm/lab/shop/controller/ShopIndexController.java:46-56`

요청

Method: GET

Path 패턴 2종(경로 특이도로 자동 우선순위 결정, 등록 순서 무관):

`/shop/assets/\*\*` — Vite 빌드 산출물 중 해시 붙은 JS/CSS(장기 캐시 대상)

`/shop/\*\*` — 그 외 전부(정적 파일이 실재하면 그 파일, 없으면 SPA 딥링크로 간주)

`/shop`·`/shop/` (경로 세그먼트 없음, 정확 일치) — `ShopIndexController`가 전담(아래 "구현 세부" 참고)

인증: 무인증(`ApiKeyAuthFilter.isOpenRoute`에 `/shop`·`/shop/\*\*` 화이트리스트, `modules/shop-api/.../ApiKeyAuthFilter.java:230`). 실제 데이터 접근은 여전히 `/api/\*\*` (불변)에서만 일어나므로 이 화이트리스트가 자격 판정을 우회하지 않는다.

응답

요청 형태	응답
`/shop/assets/{해시파일}` (실재)	200, 원본 Content-Type, `Cache-Control: max-age=31536000, public, immutable`
`/shop`, `/shop/`, `/shop/index.html`	200, `text/html`, `Cache-Control: no-cache`, index.html 본문
`/shop/{미매칭 딥링크}` (예: `/shop/products/123`)	200, `text/html`, `Cache-Control: no-cache`, index.html 본문(SPA 클라이언트 라우팅 위임)
빌드 산출물 자체가 없음(`index.html` 부재)	404(Spring 표준, `NoResourceFoundException` 계열) — 200으로 위장하지 않는다
`/shop/assets/{존재하지 않는 파일}`	404(자산은 폴백하지 않는다 — MIME 오류를 index.html로 감추지 않는다)

비즈니스 규칙

정적 자산(`/shop/assets/\*\*`)과 SPA 폴백(`/shop/\*\*`)은 서로 다른 캐시 정책을 쓴다: 해시 붙은 자산은 불변으로 간주(1년, `immutable`), `index.html`은 배포 직후 옛 화면이 남지 않도록 `no-cache`.

SPA 폴백은 `PathResourceResolver`를 확장해 `super.getResource()` (Spring 내장 위치-이탈 방지)로 실제 파일을 먼저 찾고, 없을 때만 `index.html`로 대체한다 — 새 경로 순회(traversal) 표면을 만들지 않는다.

`ResourceHttpRequestHandler`는 매핑 패턴을 벗어난 나머지 경로가 빈 문자열이면(정확히 `/shop`·`/shop/`) 리졸버 체인 진입 전에 자체 가드로 404를 내는 실측 제약이 있다 — 이 두 정확 경로만

`ShopIndexController`(`@GetMapping("/{shop}"/"/shop/")`)가 별도로 index.html을 직접 읽어 반환한다. 하위 미매칭 경로

(`/shop/products/123` 등)는 리소스 경로가 비어있지 않으므로 `ShopStaticResourceConfig`의 폴백 리졸버가 정상 처리한다.

빌드 산출물이 없어도 shop-api 기동은 성공한다 — 기동 시 1회(`@PostConstruct`) `index.html` 존재를 확인해 없으면 WARN 로그("...해결: modules/shop-web에서 npm run build 실행 후 shop-api 재빌드-재기동.")를 남기고, 요청마다 반복 로깅하지 않는다.

`shop.static-location` 프로퍼티(기본값 `classpath:/static/shop/`)는 후행 슬래시 유무와 무관하게 정규화되어 동작한다 (`ShopStaticResourceConfig`·`ShopIndexController` 양쪽 생성자에서 동일하게 보정).

`/shop`으로 시작하지만 `/shop/`이 아닌 경로(예: `/shopkeeper`)는 화이트리스트 오매칭 없이 여전히 401(`ApiKeyAuthFilter`가 `equals("/shop") || startsWith("/shop/")` 정확 조합 사용).

클라이언트 진입 규칙(SR-302.1, 서버 계약 불변): 이 엔드포인트가 `index.html`을 반환한 뒤, 그 안의 SPA가 해시 없이 최초 로드되면 쇼핑 홈(`UIS-ORD-008`)으로 1회 이동하고 이후 `#/`는 주문 목록이다 — 자세한 내용은 `UIS-ORD-008`의 "진입/라우팅" 참고. 이 문서의 요청-응답-앵커 줄 범위는 변경되지 않았다(클라이언트 라우팅은 이 서버 엔드포인트 밖의 동작).

빌드-배포 절차 (AC "빌드 산출물 위치-갱신 방법" 확정값)

1. `modules/shop-web`에서 `npm run build` 실행 → `modules/shop-web/dist` 생성. 이때 `vite.config.ts`의 `base`가 `command===`'build`일 때만 `/shop/`으로 설정되어(개발 서버 `npm run dev`, :5273은 `base:`/` 그대로 무변경) 산출물의 자산 참조 경로가 `/shop/assets/...`로 나온다.
2. `modules/shop-api`에서 Maven 빌드(`mvnw` 등) 시 `generate-resources` 단계에서 `maven-resources-plugin`(`pom.xml:93-119`, 실행 id `copy-shop-spa-dist`)이 `./shop-web/dist`를 `target/classes/static/shop`으로 복사한다. `src/main/resources`(소스 트리)는 건드리지 않으므로 `.gitignore` 변경 불필요.
3. `shop-web/dist`가 없는 환경(예: 아직 한 번도 빌드하지 않음)에서도 이 복사 단계는 조용히 건너뛰고 빌드-기동이 실패하지 않는다.
4. 화면을 갱신하려면: shop-web에서 `npm run build` 재실행 → shop-api 재빌드-재기동 1회. (SR-301의 목적 — 별도 개발 서버 재기동 불필요)

오류 응답

새 오류 코드는 만들지 않는다(`{code,message}` 오류 계약 대상 아님 — 정적 리소스 표준 응답만 사용).

상태	조건
404	요청 경로에 대응하는 정적 파일·`index.html` 모두 없음(빌드 산출물 없음)
404	`/shop/assets/**` 하위 자산 요청인데 실제 파일 없음(SPA 폴백 대상 아님)
401	`/shop`·`/shop/**` 밖의 경로(`/api/**` 등)를 인증 헤더 없이 요청(기존 계약 불변)

범위 밖

빌드된 SPA가 실제로 `/api/\*\*`를 호출해 정상 동작하는지(런타임 API 연동)는 이 SR 범위 밖 — 후속 SR 후보.

`npm run build` + Maven 빌드를 한 명령으로 묶는 배포 자동화는 이 SR 범위 밖.

개발 서버(:5273)의 `server.proxy` 등 구성 변경 없음 — `vite.config.ts`는 `base`(빌드 전용) 한 줄만 조건부 변경.

스펙 ID 참고

`INF-ORD-017`은 SR-212(`docs/00\_FUNC/stories/STORY-FUNC-order-016.md`, "상품 리뷰 API")가 과거 예약했던 것과 같은 번호이나, SR-212는 원장에서 SUPERSEDED(대체)되어 그 예약이 살아있는 점유가 아니고(claim 해제됨, v5.1부터 실행

불가한 레거시 FUNC STORY로 읽기 전용 이력), 실제 `INF-ORD-017.md` 스펙 파일 자체가 이 SR 이전까지 존재하지 않았다(사람 확인·승인 — SR-301.1 STORY `## Dev 기록`).

curl 예시

```
curl -i http://localhost:8087/shop/
curl -i http://localhost:8087/shop/products/123
curl -i http://localhost:8087/shop/assets/index-9yozd6aq.js
```

변경 이력

<!-- spec-history: machine-managed -->

일자	SR	항 목	변경 요약	커밋
2026-09-17	SR-301	#1	신규: shop-web 빌드 산출물을 /shop/**로 정적 서빙 + SPA 딥링크 폴백 (ShopStaticResourceConfig·ShopIndexController), /shop 인증 화이트리스트, maven-resources-plugin dist 복사, vite base 조건부 변경	shop-api@0f75df7, shop-web@7feb578
2026-09-17	SR-302	#1	클라이언트 진입 규칙 각주 추가(서버 계약·앵커 불변) — index.html 반환 후 SPA가 쇼핑 홈으로 이동하는 규칙을 UIS-ORD-008에 위임	shop-web@efd0e7c

데이터 설계서

SCH-MBR-001 ID_SEQUENCES

SCH-MBR-001: ID_SEQUENCES

> **FUNC-ID:** [TBD] | **SRS-F:** [TBD] | **API:** INF-MBR-002 | **화면:** [TBD]

근거 소스: `modules/shop-api/src/main/resources/db/V3\_\_members\_signup.sql` — CREATE TABLE 문에서 컬럼·타입·NULL·기본값·키를 그대로 옮겼다(DB 실측이 아니라 DDL 기준).

컬럼 설명

컬럼명	타입	NUL L	키	기본 값	설명
name	VARCHAR(50)	N	P K	—	시퀀스 이름(예: MEMBER_ID) — 이 SR은 MEMBER_ID만 사용
next_val	BIGINT	N		—	직전에 채번된 값(다음 채번은 INSERT..ON DUPLICATE KEY UPDATE로 이 값+1을 원자적으로 반환)

인덱스

인덱스명	컬럼	타입	목적
—	—	—	—

코드값

[TBD]

관계 (FK / 관찰된 조인)

> 출처 `DB FK`=DB 선언 제약, `쿼리관찰(N)`=소스 SQL에서 N회 등장한 등가조인(논리 FK).

자식 컬럼	참조 테이블	참조 컬럼	출처	ON DELETE
—	—	—	—	—

mini-ERD

ID_SEQUENCES

비즈니스 주의사항

[TBD] — 보강: `/sl-sync --apply --kind=sch` 또는 ddd-db-agent

SCH-MBR-002 MEMBER_ADDRESSES

SCH-MBR-002: MEMBER_ADDRESSES

> **FUNC-ID:** [TBD] | **SRS-F:** [TBD] | **API:** INF-MBR-008 | **화면:** [TBD]

근거 소스: `modules/shop-api/src/main/resources/db/V7\_\_member\_addresses.sql` — CREATE TABLE 문에서 컬럼·타입·NULL·기본값·키를 그대로 옮겼다(DB 실측이 아니라 DDL 기준).

컬럼 설명

컬럼명	타입	NULL	키	기본값	설명
address_id	BIGINT	N	PK	NULL	[TBD]
member_id	VARCHAR(20)	N		—	[TBD]
recipient	VARCHAR(50)	N		—	[TBD]
phone	VARCHAR(20)	N		—	[TBD]
phone_norm	VARCHAR(20)	N		—	[TBD]
zipcode	VARCHAR(10)	N		—	[TBD]
road_address	VARCHAR(200)	N		—	[TBD]
detail_address	VARCHAR(200)	N		—	[TBD]
entrance_method	VARCHAR(200)	Y		NULL	[TBD]
delivery_memo	VARCHAR(200)	Y		NULL	[TBD]
is_default	CHAR(1)	N		N	[TBD]
last_used_at	DATETIME	Y		NULL	[TBD]
created_at	DATETIME	N		—	[TBD]
updated_at	DATETIME	N		—	[TBD]
del_yn	CHAR(1)	N		N	[TBD]

인덱스

인덱스명	컬럼	타입	목적
—	—	—	—

코드값

[TBD]

관계 (FK / 관찰된 조인)

> 출처 `DB FK`=DB 선언 제약, `쿼리관찰(N)`=소스 SQL에서 N회 등장한 등가조인(논리 FK).

자식 컬럼	참조 테이블	참조 컬럼	출처	ON DELETE
member_id	MEMBERS	member_id	DB FK(DDL)	—

mini-ERD

```
MEMBER_ADDRESSES -member_id → MEMBERS.member_id
```

비즈니스 주의사항

[TBD] — 보강: `/sl-sync --apply --kind=sch` 또는 ddd-db-agent

SCH-MBR-003 MEMBER_API_KEYS

SCH-MBR-003: MEMBER_API_KEYS

> **FUNC-ID:** [TBD] | **SRS-F:** [TBD] | **API:** INF-MBR-003, INF-MBR-004, INF-MBR-005 | **화면:** [TBD]

근거 소스: `modules/shop-api/src/main/resources/db/V4\_\_members\_login.sql` — CREATE TABLE 문에서 컬럼·타입·NULL·기본값·키를 그대로 옮겼다(DB 실측이 아니라 DDL 기준).

컬럼 설명

컬럼명	타입	NULL	키	기본값	설명
member_id	VARC HAR(20)	N	PK	—	MEMBERS.member_id
api_key	VARC HAR(64)	N		—	발급된 API 키(정적 lab.api-keys 맵과 별개 — DB 폴백 조회 전용)
issued_at	DATETIME(3)	N		—	최초 발급 시각
revoked_at	DATETIME(3)	Y		NULL	폐기 시각 — round 9(재작업 지시 2) 추가. 로그인(이 FUNC)은 항상 NULL로만 INSERT하고 세팅하지 않는다. 폐기는 FUNC-006(로그아웃) 소관 — 006이 로그아웃 시 이 컬럼을 세팅하는 인터페이스로 못박는다(STORY "006과의 인터페이스 가정" 참고).

인덱스

인덱스명	컬럼	타입	목적
—	—	—	—

코드값

[TBD]

관계 (FK / 관찰된 조인)

> 출처 `DB FK`=DB 선언 제약, `쿼리관찰(N)`=소스 SQL에서 N회 등장한 등가조인(논리 FK).

자식 컬럼	참조 테이블	참조 컬럼	출처	ON DELETE
—	—	—	—	—

mini-ERD

--

MEMBER_API_KEYS

비즈니스 주의사항

[TBD] — 보강: `/sl-sync --apply --kind=sch` 또는 ddd-db-agent

SCH-MBR-004 MEMBER_LOGIN_ATTEMPTS

SCH-MBR-004: MEMBER_LOGIN_ATTEMPTS

> **FUNC-ID:** [TBD] | **SRS-F:** [TBD] | **API:** INF-MBR-003 | **화면:** [TBD]

근거 소스: `modules/shop-api/src/main/resources/db/V4\_\_members\_login.sql` — CREATE TABLE 문에서 컬럼·타입·NULL·기본값·키를 그대로 옮겼다(DB 실측이 아니라 DDL 기준).

컬럼 설명

컬럼명	타입	NULL	키	기본값	설명
email	VARCHAR(255)	N	PK	—	로그인 시도 대상 이메일(회원 미존재도 포함) — FUNC-member-005
fail_count	INT	N		0	연속 실패 횟수(잠금 만료 후 재실패 시 1로 리셋 — 무한 누적 방지)
locked_until	DATETIME(3)	Y		NULL	이 시각까지 잠김(NULL이면 미잠김) — 5회 도달 시 세팅
last_failed_at	DATETIME(3)	Y		NULL	마지막 실패 시각

인덱스

인덱스명	컬럼	타입	목적
—	—	—	—

코드값

[TBD]

관계 (FK / 관찰된 조인)

> 출처 `DB FK`=DB 선언 제약, `쿼리관찰(N)`=소스 SQL에서 N회 등장한 등가조인(논리 FK).

자식 컬럼	참조 테이블	참조 컬럼	출처	ON DELETE
—	—	—	—	—

mini-ERD

MEMBER_LOGIN_ATTEMPTS

비즈니스 주의사항

[TBD] — 보강: `/sl-sync --apply --kind=sch` 또는 ddd-db-agent

SCH-MBR-005 MEMBER_PASSWORD_RESETS

SCH-MBR-005: MEMBER_PASSWORD_RESETS

> **FUNC-ID:** [TBD] | **SRS-F:** [TBD] | **API:** INF-MBR-006, INF-MBR-007 | **화면:** [TBD]

근거 소스: `modules/shop-api/src/main/resources/db/V5\_\_member\_password\_resets.sql` — CREATE TABLE 문에서 컬럼·타입·NULL·기본값·키를 그대로 옮겼다(DB 실측이 아니라 DDL 기준).

컬럼 설명

컬럼명	타입	NULL	키	기본값	설명
target	VARCHAR(100)	N	PK	—	재설정 대상(이메일 또는 휴대폰번호, 정규화된 값) — 단일 컬럼(email trim+소문자 또는 phone 숫자만)
code_hash	CHAR(64)	N		—	SHA-256 hex(소문자) — 코드 원문은 저장하지 않음
expires_at	DATETIME(3)	N		—	코드 만료 시각(발급+10분)
consumed_at	DATETIME(3)	Y		NULL	코드 소비(확정 완료) 시각 — 이 FUNC(008)은 항상 NULL로만 쓰고 세팅하지 않음. FUNC-009(확정 API) 소관
attempt_count	INT	N		0	확정 시도 횟수 — 이 FUNC(008)은 항상 0으로만 두고 증가시키지 않음. FUNC-009(확정 API) 소관
created_at	DATETIME(3)	N		—	이 코드가 마지막으로 발급된 시각(쿨다운 60초 판정 기준) — ON DUPLICATE KEY UPDATE SET 목록의 맨 뒤에서만 갱신되어 앞선 IF 조건들이 항상 갱신 전 값을 참조한다 (MEMBER_SIGNUP_RATE_LIMITS round6과 동일 원리)

인덱스

인덱스명	컬럼	타입	목적
idx_mpr_expires_at	expires_at	INDEX	정리 배치(BAT-MBR-001)가 `expires_at < now - 보존기간`으로 만료 행을 조건부 DELETE할 때 풀스캔을 피한다(SR-297 #1, `V9__member_password_reset_rate_limits.sql`에서 추가 — DDL 원본 `V5__member_password_resets.sql`은 무변경)

코드값

없음 — 코드성(열거값) 컬럼이 없다. 상태는 `consumed\_at`(NULL=미소비)·`expires\_at`(현재 시각 비교)·`attempt\_count`(확정 오답 누적, 상한 5)로 판정한다.

관계 (FK / 관찰된 조인)

> 출처 `DB FK`=DB 선언 제약, `쿼리관찰(N)`=소스 SQL에서 N회 등장한 등가조인(논리 FK).

자식 컬럼	참조 테이블	참조 컬럼	출처	ON DELETE
—	—	—	—	—

mini-ERD

MEMBER_PASSWORD_RESETS

비즈니스 주의사항

target당 1행(PK) — 재요청은 같은 행을 `INSERT ... ON DUPLICATE KEY UPDATE`로 덮어쓴다(새 코드·만료
·`consumed\_at`/`attempt\_count` 초기화). `created\_at`은 SET 목록 맨 뒤에서만 갱신돼 쿨다운(60초) 판정이 갱신 전 값을 본
다.

코드 원문은 저장하지 않는다(`code\_hash` = SHA-256 hex).

만료된 행은 곧바로 지워지지 않는다 — 정리 배치(BAT-MBR-001, `MemberPasswordResetMaintenanceScheduler`)가
`expires\_at` < now - 보존기간(member.password-reset.purge-retention, 기본 P7D)인 행만 매일 조건부 DELETE한다(SR-297
#1). 보존기간 안에서는 행이 남아 있어도 확정 API가 `expires\_at`으로 계속 410(MBR-4101)을 낸다 — 보존기간을 넘겨 행
이 지워지면 확정 시도는 409(MBR-4102, 응답과 동일 코드)로 바뀐다(알려진 동작, INF-MBR-007 참고).

`attempt\_count` 증가는 확정 API(INF-MBR-007)가 조건부 원자 UPDATE(`attempt\_count` < 상한)로만 한다(SR-298 #2) —
읽고 비교한 뒤 쓰지 않는다.

근거: DDL `V5\_member\_password\_resets.sql` 주석 + 매퍼 `memberPasswordReset.xml`(소스 기준, DB 실측 아님).

변경 이력

<!-- spec-history: machine-managed -->

일자	SR	항 목	변경 요약	커밋
2026-09-16	SR-297	#1	expires_at 인덱스(idx_mpr_expires_at) 추가 + 만료 행 보존기간(P7D) 정리 배치 반영 — 확정 API 410→409 전환 시점 명시	shop-api@b7ae32f
2026-09-16	SR-297	#2	스키마 무변경 확인(컬럼·타입·NULL·기본값·PK 그대로) — 이 항목은 이 테이블을 읽거나 쓰지 않음, AC로 검증됨	shop-api@b7ae32f

SCH-MBR-006 MEMBER_REFRESH_TOKENS

SCH-MBR-006: MEMBER_REFRESH_TOKENS

> **FUNC-ID:** [TBD] | **SRS-F:** [TBD] | **API:** INF-MBR-003, INF-MBR-004, INF-MBR-005 | **화면:** [TBD]

근거 소스: `modules/shop-api/src/main/resources/db/V4\_\_members\_login.sql` — CREATE TABLE 문에서 컬럼·타입·NULL·기본값·키를 그대로 옮겼다(DB 실측이 아니라 DDL 기준).

컬럼 설명

컬럼명	타입	NULL	키	기본값	설명
token_hash	VARCHAR(64)	N	PK	—	SHA-256 hex(소문자) — 원문 미저장
member_id	VARCHAR(20)	N		—	MEMBERS.member_id
issued_at	DATETIME(3)	N		—	발급 시각
expires_at	DATETIME(3)	N		—	만료 시각(발급+30일)
revoked_at	DATETIME(3)	Y		NULL	폐기 시각(로그아웃) — FUNC-006 전용, 이 FUNC(005)은 항상 NULL로만 INSERT

인덱스

인덱스명	컬럼	타입	목적
—	—	—	—

코드값

[TBD]

관계 (FK / 관찰된 조인)

> 출처 `DB FK`=DB 선언 제약, `쿼리관찰(N)`=소스 SQL에서 N회 등장한 등가조인(논리 FK).

자식 컬럼	참조 테이블	참조 컬럼	출처	ON DELETE
—	—	—	—	—

mini-ERD

MEMBER_REFRESH_TOKENS

비즈니스 주의사항

[TBD] — 보강: `sl-sync --apply --kind=sch` 또는 ddd-db-agent

SCH-MBR-007 MEMBER_SIGNUP_RATE_LIMITS

SCH-MBR-007: MEMBER_SIGNUP_RATE_LIMITS

> **FUNC-ID:** [TBD] | **SRS-F:** [TBD] | **API:** INF-MBR-001 | **화면:** [TBD]

근거 소스: `modules/shop-api/src/main/resources/db/member\_signup\_rate\_limits.sql` — CREATE TABLE 문에서 컬럼·타입·NULL·기본값·키를 그대로 옮겼다(DB 실측이 아니라 DDL 기준).

컬럼 설명

컬럼명	타입	NU LL	키	기본 값	설명
target	VARCHAR(100)	N	P K	—	인증 대상(이메일 또는 휴대폰번호 원문) — 채널 무관, target 단위로 레이트리밋
day_key	DATE	N	P K	—	요청 날짜(앱 시계 기준 로컬 날짜) — daily_count의 하루 경계
daily_count	INT	N		1	이 target의 이 날짜에 허용된 요청 횟수(거부된 요청은 세지 않음)
last_requested_at	DATETIME(3)	N		—	이 target의 이 날짜 마지막으로 허용된 요청 시각(쿨다운 60초 판정 기준)
last_token	VARCHAR(36)	Y		NUL L	round5 — 마지막으로 "허용"을 기록한 요청의 UUID 토큰. touchRateLimit이 조건부로 갱신하고, selectRateLimit이 재조회한 값이 호출자 자신의 토큰과 같으면 허용(200)으로 판정

인덱스

인덱스명	컬럼	타입	목적
idx_msrl_day_key	day_key	INDEX	[TBD]

코드값

[TBD]

관계 (FK / 관찰된 조인)

> 출처 `DB FK`=DB 선언 제약, `쿼리관찰(N)`=소스 SQL에서 N회 등장한 등가조인(논리 FK).

자식 컬럼	참조 테이블	참조 컬럼	출처	ON DELETE
—	—	—	—	—

mini-ERD

MEMBER_SIGNUP_RATE_LIMITS

비즈니스 주의사항

[TBD] — 보장: `/sl-sync --apply --kind=sch` 또는 ddd-db-agent

SCH-MBR-008 MEMBER_SIGNUP_VERIFICATIONS

SCH-MBR-008: MEMBER_SIGNUP_VERIFICATIONS

> **FUNC-ID:** [TBD] | **SRS-F:** [TBD] | **API:** INF-MBR-001, INF-MBR-002 | **화면:** [TBD]

근거 소스: `modules/shop-api/src/main/resources/db/member\_signup\_verifications.sql` — CREATE TABLE 문에서 컬럼·타입·NULL·기본값·키를 그대로 옮겼다(DB 실측이 아니라 DDL 기준).

컬럼 설명

컬럼명	타입	NU LL	키	기 본 값	설명
channel	VARCH AR(10)	N	P K	—	인증 채널: EMAIL/SMS
target	VARCH AR(100)	N	P K	—	인증 대상(이메일 주소 또는 휴대폰번호 원문)
code	CHAR(6)	Y		NU LL	6자리 인증코드 — 레이트리밋(별도 테이블) 판정을 통과한 요청에만 채워짐
expires_at	DATETI ME	Y		NU LL	만료 시각(코드 발급 시점 + 5분, 앱 시계 기준)
verified_at	DATETI ME	Y		NU LL	인증 확인 시각(가입완료 단계, FUNC-member-003에서 기록 예정)
requested_at	DATETI ME	Y		NU LL	(deprecated, round3) 과거 버전 컬럼 — 더 이상 앱이 쓰지 않는다
last_reque sted_at	DATETI ME	Y		NU LL	round4 — 의미 변경: 이 채널·타겟으로 코드가 마지막으로 실제 발송된 시각(정보용, writeCode가 갱신). round3까지는 레이트리밋 판정 기준이었으나 round4부터 그 역할은 MEMBER_SIGNUP_RATE_LIMITS로 이전
previous_r equested_ at	DATETI ME	Y		NU LL	(deprecated, round4) round3 레이트리밋 판정용 컬럼 — 카운터가 전용 테이블로 이전되어 더 이상 앱이 쓰지 않는다
daily_coun t	INT	N		1	(deprecated, round4) round3 레이트리밋 판정용 컬럼 — MEMBER_SIGNUP_RATE_LIMITS.daily_count로 대체
attempt_c ount	INT	N		0	인증코드 대입 시도 횟수 — 이 FUNC은 컬럼만 소유하고 쓰지 않음, FUNC-member-003(가입완료 API)이 코드 검증 시 사용

인덱스

인덱스명	컬럼	타입	목적
—	—	—	—

코드값

[TBD]

관계 (FK / 관찰된 조인)

> 출처 `DB FK`=DB 선언 제약, `쿼리관찰(N)`=소스 SQL에서 N회 등장한 등가조인(논리 FK).

자식 컬럼	참조 테이블	참조 컬럼	출처	ON DELETE
—	—	—	—	—

mini-ERD

MEMBER_SIGNUP_VERIFICATIONS

비즈니스 주의사항

[TBD] — 보강: `/sl-sync --apply --kind=sch` 또는 ddd-db-agent

SCH-MBR-009 MEMBER_PASSWORD_RESET_RATE_LIMITS

SCH-MBR-009: MEMBER_PASSWORD_RESET_RATE_LIMITS

> [반영: SR-297.2] — SR-297.1이 만든 DDL 골격에 SR-297.2가 일일 상한 원자 판정(UPSERT)을 배선. 아래 컬럼 설명·비즈니스 주의사항을 실제 사용 상태로 갱신.

> **FUNC-ID:** [TBD] | **SRS-F:** [TBD] | **API:** INF-MBR-006 | **화면:** [TBD]

근거 소스: `modules/shop-api/src/main/resources/db/V9\_\_member\_password\_reset\_rate\_limits.sql` — CREATE TABLE 문에서 컬럼·타입·NULL·기본값·키를 그대로 옮겼다(DB 실측이 아니라 DDL 기준).

컬럼 설명

컬럼명	타입	NULL	키	기본값	설명
target	VARCHAR(100)	N	PK	—	재설정 코드 요청 대상(이메일 또는 휴대폰번호, 정규화된 값) — target 단위로 일일 상한
day_key	DATE	N	PK	—	요청 날짜(앱 시계 기준 로컬 날짜) — daily_count의 하루 경계. 날짜가 바뀌면 새 행이 INSERT되어 daily_count가 1부터 재시작
daily_count	INT	N		1	이 target의 이 날짜에 허용된 요청 횟수 — `MemberPasswordResetRateLimitDao#touchDailyLimit` (SR-297.2)이 쿼다운·일일상한(5회) 둘 다 만족할 때만 원자 UPSERT로 갱신
last_requested_at	DATETIME(3)	N		—	이 target의 이 날짜 마지막으로 허용된 요청 시각 — `touchDailyLimit` (SR-297.2)의 쿼다운 (60초) 판정 기준
last_token	VARCHAR(36)	Y		NULL	`touchDailyLimit` (SR-297.2)이 조건부로 갱신하는 요청 토큰(UUID) — affected-rows 대신 `selectRateLimit` 재조회로 "내 토큰이 저장돼 있으면 허용" 판정

인덱스

인덱스명	컬럼	타입	목적
idx_mprl_day_key	day_key	INDEX	정리 배치(BAT-MBR-001)가 `day_key < 오늘`로 지난 날짜 행을 조건부 DELETE할 때 스캔 범위를 좁힌다

코드값

없음 — 코드성(열거값) 컬럼이 없다.

관계 (FK / 관찰된 조인)

> 출처 `DB FK`=DB 선언 제약, `쿼리관찰(N)`=소스 SQL에서 N회 등장한 등가조인(논리 FK).

자식 컬럼	참조 테이블	참조 컬럼	출처	ON DELETE
—	—	—	—	—

mini-ERD

MEMBER_PASSWORD_RESET_RATE_LIMITS

비즈니스 주의사항

(target, day_key) PK — target별 하루 단위 카운터 행. 가입 도메인 'MEMBER_SIGNUP_RATE_LIMITS'와 동형 스키마이자 문자 단위로 동일한 UPSERT 본문(round6 원리, 세션 변수 없이 SET 좌→우 평가만으로 결정적).

****SR-297.1(DDL·정리)과 SR-297.2(일일 상한 판정)가 이 테이블을 나눠 쓴다****: SR-297.1은 정리 배치('purgeOldRows', 'day_key < 오늘' 조건부 DELETE)만, SR-297.2는 'daily_count'/'last_requested_at'/'last_token' 세 컬럼을 원자 UPSERT('touchDailyLimit')로 갱신하는 일일 상한 판정을 맡는다 — 요청 경로(SR-297.2)에는 정리 로직이 없고, 배치(SR-297.1)에는 판정 로직이 없다.

****자정 경계(day_key 롤오버) 알려진 동작****: 'day_key'가 PK 일부라 날짜가 바뀌면 새 행이 생겨 쿼다운 없이 admit되지만, 코드 테이블('MEMBER_PASSWORD_RESETS.created_at')의 자체 쿼다운은 어제 시각 기준으로 여전히 거부할 수 있다 — 이 경우 'daily_count'만 소비되고 코드는 갱신되지 않는다(INF-MBR-006 "알려진 동작" 참고, 응답은 항상 202로 동일해 오라 클 없음).

정리 배치(BAT-MBR-001, 'MemberPasswordResetMaintenanceScheduler')가 'day_key < 오늘'인 행을 매일 조건부 DELETE(먹등, 재실행해도 결과 동일) — 신규 요청 트래픽과 무관한 시각에 실행되고 '@Transactional' 없이 독립 autocommit.

근거: DDL 'V9_member_password_reset_rate_limits.sql'(소스 기준, DB 실측 아님) +

'MemberPasswordResetRateLimitDao'/'memberPasswordResetRateLimit.xml'(SR-297.2 실 사용 코드).

변경 이력

<!-- spec-history: machine-managed -->

일자	SR	항목	변경 요약	커밋
2026-09-16	SR-297	#1	코드값·인덱스 목적·비즈니스 주의사항 보강(daily_count 등은 이 항목 미사용, #2 몫 명시)	shop-api@b7ae32f
2026-09-16	SR-297	#2	daily_count/last_requested_at/last_token 실사용 시작(touchDailyLimit) — 컬럼 설명·비즈니스 주의사항을 실사용 상태로 갱신, inf 연결(INF-MBR-006)	shop-api@b7ae32f

SCH-ORD-001 members

SCH-ORD-001: members

> **FUNC-ID:** [TBD] | **SRS-F:** [TBD] | **API:** [TBD] | **화면:** [TBD]

****근거 소스:**** `DB 실측(db-main MCP describe)` — 컬럼 타입·NULL·기본값은 MariaDB(sl_lab) 실스키마 조회로 사실화 (P1/SR-202 enrichment). [미확인] 헤더 잔존은 자기모순이라 정정(OBS-018, 2026-08-23)

컬럼 설명

컬럼명	타입	NULL	키	기본값	설명
member_id	VARCHAR(20)	N	PK	—	회원 ID (M-접두)
member_name	VARCHAR(50)	N		—	회원명
grade	VARCHAR(10)	N		BRONZE	등급 (BRONZE/SILVER/GOLD/VIP)
phone	VARCHAR(20)	Y		NULL	휴대폰
del_yn	CHAR(1)	N		N	탈퇴 여부 (soft delete)
created_at	DATETIME	N		CURRENT_TIMESTAMP	가입일시

인덱스

인덱스명	컬럼	타입	목적
—	—	—	—

코드값

****grade (등급)****

값	의미	비고
BRONZE	일반 회원	INF-ORD-001 응답 예시
SILVER	실버 회원	INF-ORD-001 응답 예시
GOLD	골드 회원	INF-ORD-001 응답 예시
VIP	VIP 회원	INF-ORD-001 응답 예시

관계 (FK / 관찰된 조인)

자식 컬럼	참조 테이블	참조 컬럼	출처	ON DELETE
member_id	ORDERS	member_id	쿼리관찰(4)	—

> 출처 `DB FK`=DB 선언 제약, `쿼리관찰(N)`=소스 SQL에서 N회 등장한 등가조인(논리 FK).

<details>

<summary> 📌 쿼리 작성 가이드 — 상시 필터 (관찰 1건)</summary>

> 이 테이블 조회 시 코드에서 반복 관찰된 술어. AIDD 쿼리 생성 시 누락하면 결과가 틀어진다.

컬럼	조건	빈도	의미	출처
DEL_YN	= 'N'	4	탈퇴하지 않은 회원만 조회(soft-delete 행 제외)	member.xml

</details>

mini-ERD

```
erDiagram
    members {
        varchar(20) member_id PK
        varchar(50) member_name
        varchar(10) grade
        varchar(20) phone
        char(1) del_yn
        datetime created_at
    }
    orders {
        varchar(20) order_no PK
        varchar(20) member_id FK
    }
    members ||--o{ orders : "has"
```

비즈니스 주의사항

탈퇴 회원(`del\_yn` = 'Y')은 조회 시 모든 API에서 자동으로 필터됨([[INF-ORD-001]], [[INF-ORD-002]])
모든 조회 쿼리에서 `del\_yn` = 'N' 상시필터 적용 — 누락 시 탈퇴 회원도 노출

SCH-ORD-002 order_delivery

SCH-ORD-002: order_delivery

> **FUNC-ID:** [TBD] | **SRS-F:** [TBD] | **API:** [TBD] | **화면:** [TBD]

근거 소스: `DB 실측(db-main MCP describe)` — 컬럼 타입·NULL·기본값은 MariaDB(sl_lab) 실스키마 조회로 사실화 (P1/SR-202 enrichment). [미확인] 헤더 잔존은 자기모순이라 정정(OBS-018, 2026-08-23)

컬럼 설명

컬럼명	타입	NULL	키	기본값	설명
delivery_no	VARCHAR(20)	N	PK	—	배송번호
order_no	VARCHAR(20)	N		—	주문번호
delivery_state	VARCHAR(20)	N		READY	배송 상태 (READY/SHIPPED/DELIVERED/CANCELED)
invoice_no	VARCHAR(30)	Y		NULL	송장번호
shipped_at	DATETIME	Y		NULL	출고일시

인덱스

인덱스명	컬럼	타입	목적
—	—	—	—

코드값

delivery_state (배송 상태)

값	의미	비고
READY	배송 준비	초기 상태, 송장 미할당
SHIPPED	배송중	송장번호 할당, shipped_at 기록
DELIVERED	배송 완료	고객 수령
CANCELED	배송 취소	주문 취소 시

관계 (FK / 관찰된 조인)

자식 컬럼	참조 테이블	참조 컬럼	출처	ON DELETE
order_no	ORDERS	order_no	INF-ORD-004	—

> 출처 `DB FK`=DB 선언 제약, `쿼리관찰(N)`=소스 SQL에서 N회 등장한 등가조인(논리 FK).

mini-ERD

```

erDiagram
    orders {
        varchar(20) order_no PK
    }
    order_delivery {
        varchar(20) delivery_no PK
        varchar(20) order_no FK
        varchar(20) delivery_state
        varchar(30) invoice_no
        datetime shipped_at
    }
    orders ||--o{ order_delivery : "ships"

```

비즈니스 주의사항

배송은 주문 생성 후 자동으로 생성되지 않음([INF-ORD-004] 응답에서 `deliveries`는 존재하지 않는 경우가 대부분)

`delivery\_state` = 'READY'일 때는 `invoice\_no`와 `shipped\_at`이 NULL

배송이 시작되면 `invoice\_no` 할당 및 `shipped\_at` 기록 — AIDD 쿼리 생성 시 `shipped\_at` NULL 체크 필요

SCH-ORD-003 order_items

SCH-ORD-003: order_items

> **FUNC-ID:** [TBD] | **SRS-F:** [TBD] | **API:** [TBD] | **화면:** [TBD]

****근거 소스:**** `DB 실측(db-main MCP describe)` — 컬럼 타입·NULL·기본값은 MariaDB(sl_lab) 실스키마 조회로 사실화 (P1/SR-202 enrichment). [미확인] 헤더 잔존은 자기모순이라 정정(OBS-018, 2026-08-23)

컬럼 설명

컬럼명	타입	NULL	키	기본값	설명
order_no	VARCHAR(20)	N	PK	—	주문번호
line_no	INT(11)	N	PK	—	주문 라인 번호
sku	VARCHAR(20)	N		—	상품 SKU
qty	INT(11)	N		—	수량
unit_price	BIGINT(20)	N		—	주문 시점 단가(원)

인덱스

인덱스명	컬럼	타입	목적
—	—	—	—

관계 (FK / 관찰된 조인)

자식 컬럼	참조 테이블	참조 컬럼	출처	ON DELETE
order_no	ORDERS	order_no	INF-ORD-004	—
sku	PRODUCTS	sku	쿼리관찰(2)	—

> 출처 `DB FK`=DB 선언 제약, `쿼리관찰(N)`=소스 SQL에서 N회 등장한 등가조인(논리 FK).

mini-ERD

```
erDiagram
    orders {
        varchar(20) order_no PK
    }
    order_items {
        varchar(20) order_no PK
        int line_no PK
        varchar(20) sku FK
        int qty
        bigint unit_price
    }
```

```
products {
  varchar(20) sku PK
  varchar(100) product_name
}
orders ||--o{ order_items : "contains"
products ||--o{ order_items : "references"
```

비즈니스 주의사항

복합 PK: `(order\_no, line\_no)` — 같은 주문 내에서 라인 번호로 중복 방지([INF-ORD-004] 응답의 `items` 배열에서 순서 유지)

`unit\_price`는 **주문 시점의 상품 가격**을 고정 기록 — 현재 `PRODUCTS.price`와 다를 수 있음([INF-ORD-005] 트랜잭션 순서에서 "라인별로 PRODUCTS 조회 → 재고 차감")

주문 생성 후 라인별 `productName` 조회는 `JOIN PRODUCTS ON ORDER\_ITEMS.sku = PRODUCTS.sku`([INF-ORD-004])

SCH-ORD-004 orders

SCH-ORD-004: orders

> **FUNC-ID:** [TBD] | **SRS-F:** [TBD] | **API:** [TBD] | **화면:** [TBD]

****근거 소스:**** `DB 실측(db-main MCP describe)` — 컬럼 타입·NULL·기본값은 MariaDB(sl_lab) 실스키마 조회로 사실화 (P1/SR-202 enrichment). [미확인] 헤더 잔존은 자기모순이라 정정(OBS-018, 2026-08-23)

컬럼 설명

컬럼명	타입	NULL	키	기본값	설명
order_no	VARCHAR(20)	N	PK	—	주문번호 (yyyymmdd+seq)
member_id	VARCHAR(20)	N		—	주문 회원
order_state	VARCHAR(20)	N		PLACED	상태 (PLACED/PAID/SHIPPED/PARTIAL_SHIPPED/CANCELED/DONE)
total_amount	BIGINT(20)	N		—	주문 총액(원)
del_yn	CHAR(1)	N		N	논리 삭제 (soft delete)
ordered_at	DATETIME	N		CURRENT_TIMESTAMP	주문일시

인덱스

인덱스명	컬럼	타입	목적
—	—	—	—

코드값

****order_state (주문 상태)****

값	의미	비고
PLACED	주문 접수	신규 주문 초기 상태([INF-ORD-005] 생성 시)
PAID	결제 완료	결제 단계([INF-ORD-003] 상태 필터)
SHIPPED	배송중	전체 아이템 배송 시작
PARTIAL_SHIPPED	부분 배송	일부 아이템만 배송([INF-ORD-003] 상태 필터)
CANCELED	주문 취소	취소 완료
DONE	주문 완료	모든 배송 완료

관계 (FK / 관찰된 조인)

자식 컬럼	참조 테이블	참조 컬럼	출처	ON DELETE
member_id	MEMBERS	member_id	쿼리관찰(4)	—

> 출처 `DB FK`=DB 선언 제약, `쿼리관찰(N)`=소스 SQL에서 N회 등장한 등가조인(논리 FK).

<details>

<summary> 🐿 쿼리 작성 가이드 — 상시 필터 (관찰 1건)</summary>

> 이 테이블 조회 시 코드에서 반복 관찰된 술어. AIDD 쿼리 생성 시 누락하면 결과가 틀어진다.

컬럼	조건	빈도	의미	출처
DEL_YN	= 'N'	8	논리삭제된 주문 제외(soft-delete 행 제외)	order.xml

</details>

mini-ERD

```
erDiagram
    members {
        varchar(20) member_id PK
    }
    orders {
        varchar(20) order_no PK
        varchar(20) member_id FK
        varchar(20) order_state
        bigint total_amount
        char(1) del_yn
        datetime ordered_at
    }
    order_items {
        varchar(20) order_no FK
        int line_no PK
    }
    members ||--o{ orders : "places"
    orders ||--o{ order_items : "contains"
```

비즈니스 주의사항

모든 조회 쿼리에서 `del\_yn = 'N'` 상시필터 적용([INF-ORD-003], [INF-ORD-004]) — 논리삭제된 주문 조회 시 404

`order\_no` 채번: `yyyyMMdd-{4자리 시퀀스}` 형식, 시작값 100([INF-ORD-005]) — AIDD 쿼리 생성 시 시퀀스 직접

UPDATE 대신 애플리케이션 로직에 의존

`total\_amount`는 각 라인 `unit\_price × qty`의 합([INF-ORD-005]) — DB에서 직접 계산하지 말고 애플리케이션에서 계산해 INSERT

`member\_id` 조인 시 회원이 탈퇴했어도 주문 기록은 유지([INF-ORD-003]) "회원 미탈퇴 여부와 무관하게 조인만 수행")

SCH-ORD-005 products

SCH-ORD-005: products

> **FUNC-ID:** [TBD] | **SRS-F:** [TBD] | **API:** [TBD] | **화면:** [TBD]

****근거 소스:**** `DB 실측(db-main MCP describe)` — 컬럼 타입·NULL·기본값은 MariaDB(sl_lab) 실스키마 조회로 사실화 (P1/SR-202 enrichment). [미확인] 헤더 잔존은 자기모순이라 정정(OBS-018, 2026-08-23)

컬럼 설명

컬럼명	타입	NULL	키	기본값	설명
sku	VARCHAR(20)	N	PK	—	상품 SKU
product_name	VARCHAR(100)	N		—	상품명
price	BIGINT(20)	N		—	판매가(원)
stock_qty	INT(11)	N		0	가용 재고
sale_yn	CHAR(1)	N		Y	판매 여부
list_price	BIGINT(20)	Y		NULL	[변경: SR-306] 정가(원, 표시 전용) — `price`와 무관, NULL=정가 없음(화면은 이때 할인 표시를 하지 않는다). 할인율은 저장하지 않고 화면이 `list_price`·`price`로 계산
image_url	VARCHAR(300)	Y		NULL	[변경: SR-306] 대표 이미지 경로(앱이 서빙하는 정적 경로, 예: `/images/products/sku-1001.svg`) — 외부 URL 금지, NULL=이미지 없음(화면은 이니셜 대체 영역 표시)

인덱스

인덱스명	컬럼	타입	목적
—	—	—	—

코드값

****sale_yn (판매 여부)****

값	의미	비고
Y	판매중	주문 생성 시 필수([INF-ORD-005]) 검증 "sale_yn = 'Y'가 아니면 400"
N	판매 중단	주문 생성 불가

관계 (FK / 관찰된 조인)

자식 컬럼	참조 테이블	참조 컬럼	출처	ON DELETE
-------	--------	-------	----	-----------

sku	ORDER_ITEMS	sku	쿼리관찰(2)	—
-----	-------------	-----	---------	---

> 출처 `DB FK`=DB 선언 제약, `쿼리관찰(N)`=소스 SQL에서 N회 등장한 등가조인(논리 FK).

<details>

<summary> 🐛 쿼리 작성 가이드 — 상시 필터 (관찰 1건)</summary>

> 이 테이블 조회 시 코드에서 반복 관찰된 술어. AIDD 쿼리 생성 시 누락하면 결과가 틀어진다.

컬럼	조건	빈도	의미	출처
SALE_YN	= 'Y'	2	판매중인 상품만 조회(판매 중단 상품 제외)	product.xml

</details>

mini-ERD

```
erDiagram
    products {
        varchar(20) sku PK
        varchar(100) product_name
        bigint price
        int stock_qty
        char(1) sale_yn
        bigint list_price "NULL 허용, 표시 전용"
        varchar(300) image_url "NULL 허용, 표시 전용"
    }
    order_items {
        varchar(20) sku FK
        int qty
        bigint unit_price
    }
    products ||--o{ order_items : "has"
```

비즈니스 주의사항

주문 생성 시 상품 검증: `sale\_yn = 'Y'`가 아니면 400 오류([INF-ORD-005]) — AIDD 쿼리 생성 시 필수 필터

재고 차감: 조건부 UPDATE `WHERE stock\_qty >= qty` 사용 — 동시 주문 시 데이터베이스 레벨 동시성 가드([INF-ORD-005]) "재고 부족으로 409")

`price`는 현재 판매가 — `ORDER\_ITEMS.unit\_price`는 주문 시점의 가격을 별도 기록([INF-ORD-004]) 상세 조회 시 과거 단가 표시)

`stock\_qty` UPDATE는 AIDD 쿼리 생성 시 직접 위임 안 함 — 동시성 문제 방지 위해 애플리케이션이 주도

[변경: SR-306] `list\_price`·`image\_url`은 런타임 쓰기 경로가 없다(시드로만 채워짐) — 주문·정산·통계 등 타 도메인 조인·집계는 이 두 컬럼을 읽지 않고 `price`만 쓴다(표시 전용, 파급 없음)

변경 이력

<!-- spec-history: machine-managed -->

일자	SR	항목	변경 요약	커밋
----	----	----	-------	----

2026-09-17	SR-306	#2	list_price-image_url 컬럼 추가(둘 다 NULL 허용, 표시 전용, 런타임 쓰기 없음)	shop-api@7eebc9f, shop-api@e1d18da, shop-web@cbe9816
------------	--------	----	---	--

SCH-ORD-006 cart_items

SCH-ORD-006: cart_items

> **FUNC-ID:** [TBD] | **SRS-F:** [TBD] | **API:** [INF-ORD-010](../INF/INF-ORD-010.md), [INF-ORD-011](../INF/INF-ORD-011.md), [INF-ORD-012](../INF/INF-ORD-012.md), [INF-ORD-013](../INF/INF-ORD-013.md) | **화면:** [TBD]

근거 소스: `src/main/java/com/sm/lab/shop/controller/CartController.java` + `src/main/resources/mapper/cart.xml`

컬럼 설명

컬럼명	타입	NULL	키	기본값	설명
member_id	VARCHAR(20)	N	PK	—	회원 ID (M-접두)
sku	VARCHAR(20)	N	PK	—	상품 SKU
qty	INT(11)	N		—	장바구니 수량
added_at	DATETIME	N		CURRENT_TIMESTAMP	장바구니 담은 날짜시간

인덱스

인덱스명	컬럼	타입	목적
idx_cart_items_added_at	added_at	—	담은 순 정렬([INF-ORD-011] 응답에서 `added_at, sku` 순 정렬)

관계 (FK / 관찰된 조인)

자식 컬럼	참조 테이블	참조 컬럼	출처	ON DELETE
member_id	MEMBERS	member_id	INF-ORD-010	—
sku	PRODUCTS	sku	쿼리관찰(4)	—

> 출처 `DB FK`=DB 선언 제약, `쿼리관찰(N)`=소스 SQL에서 N회 등장한 등가조인(논리 FK).

mini-ERD

```
erDiagram
    members {
        varchar(20) member_id PK
    }
    products {
        varchar(20) sku PK
        varchar(100) product_name
        bigint price
        int stock_qty
    }
    cart_items {
        varchar(20) member_id PK
        varchar(20) sku PK
    }
```

```

    int qty
    datetime added_at
}
members ||--o{ cart_items : "has"
products ||--o{ cart_items : "referenced"

```

비즈니스 주의사항

복합 PK: `(member\_id, sku)` — 같은 상품의 재담기는 UPSERT로 수량 합산([[INF-ORD-010]] "INSERT ... ON DUPLICATE KEY UPDATE qty = qty + VALUES(qty)")

응답의 `productName`, `price`, `lineTotal`은 테이블 컬럼이 아니라 애플리케이션이 JOIN PRODUCTS로 조회 후 계산해서 제공([[INF-ORD-011]] "`totalAmount`는 각 품목의 `price × qty` 합계로, 조회 시점에 애플리케이션에서 계산")

판매중지(`PRODUCTS.sale\_yn='N'`) 상품은 담기 거부(409 — [[INF-ORD-010]])

품절(`PRODUCTS.stock\_qty=0`)은 담기 전 조기 거부(409 — [[INF-ORD-010]])

담기는 상품 재고를 직접 차감하지 않음([[INF-ORD-010]] "담기는 재고를 차감하지 않는다(보관 전용)")

품목 삭제는 명시적 DELETE만 가능, qty를 0으로 변경할 수 없음([[INF-ORD-012]] "qty < 1은 400으로 거부한다 — 삭제는 명시적 DELETE로만 수행")

SCH-ORD-007 ZIPCODES

SCH-ORD-007: ZIPCODES

> **FUNC-ID:** [TBD] | **SRS-F:** [TBD] | **API:** [TBD] | **화면:** [TBD]

근거 소스: `modules/shop-api/src/main/resources/db/V8\_zipcodes.sql` — CREATE TABLE 문에서 컬럼·타입·NULL·기본값·키를 그대로 옮겼다(DB 실측이 아니라 DDL 기준).

컬럼 설명

컬럼명	타입	NULL	키	기본값	설명
zipcode_id	BIGINT	N	PK	NULL	[TBD]
zipcode	VARCHAR(10)	N		—	[TBD]
road_address	VARCHAR(200)	N		—	[TBD]
sido	VARCHAR(50)	N		—	[TBD]
sigungu	VARCHAR(50)	N		—	[TBD]

인덱스

인덱스명	컬럼	타입	목적
uq_zipcodes_zipcode	zipcode	UNIQUE	[TBD]

코드값

[TBD]

관계 (FK / 관찰된 조인)

> 출처 `DB FK`=DB 선언 제약, `쿼리관찰(N)`=소스 SQL에서 N회 등장한 등가조인(논리 FK).

자식 컬럼	참조 테이블	참조 컬럼	출처	ON DELETE
—	—	—	—	—

mini-ERD

ZIPCODES

비즈니스 주의사항

[TBD] — 보강: `/sl-sync --apply --kind=sch` 또는 ddd-db-agent

변경 이력

<!-- spec-history: machine-managed -->

일자	SR	항 목	변경 요약	커밋
2026-09-19	SR-235	#3	ZIPCODES 테이블 신설(V8_zipcodes.sql) — 시드 100건(도로명 검색용), round2 인코딩 수정 + round3 COMMENT 정본화	shop-api@1c82e8c, shop-web@8e1e3e3

배치 설계서

BAT-MBR-001 비밀번호 재설정 만료 코드 및 상한 카운터 정리

BAT-MBR-001: 비밀번호 재설정 만료 코드 및 상한 카운터 정리

> **근거 소스:** `modules/shop-api/src/main/java/com/sm/lab/shop/service/MemberPasswordResetMaintenanceScheduler.java:74-84` (Scheduler 메서드)

개요

항목	내용
목적	비밀번호 재설정 시스템의 정기 유지보수 — 보존기간을 초과한 만료 코드 행과 지난 날짜의 일일 요청 상한 카운터를 데이터베이스에서 정리(DELETE)한다. 요청 경로(API)와 완전히 분리된 독립 배치로 실행되어 트랜잭션.락이 섞이지 않는다.
트리거	매일 스케줄 기반(Cron)
스케줄	`0 0 3 * * *` (매일 새벽 3시 UTC 기준) — 설정값 `\${lab.batch.password-reset-cleanup.cron}`으로 조정 가능
처리 단위	배치 단일 실행당 조건부 DELETE 2개(코드 행 + 카운터 행)

입력 파라미터

파라미터	타입	출처	설명
`member.password-reset.purge-retention`	Duration(ISO-8601)	환경변수 / application.yml	만료 코드 보존 기간. 기본값 `P7D`(7일). 이 기간을 초과한 `expires_at` 미만 행만 DELETE 대상
현재 시각 (`now`)	LocalDateTime	시스템 시계 (Clock.systemDefaultZone())	보존기간 기준 시각 계산 및 지난 날짜 결정. 테스트 시에만 주입 가능(생성자 오버로드)

처리 흐름

1. ****시스템 시간 조회**** — Clock에서 현재 LocalDateTime 획득
2. ****보존기간 계산**** — `beforeExpiresAt = now - purgeRetention` (기본 now - 7일)
3. ****만료 코드 정리**** — `MemberPasswordResetDao.purgeExpiredCodes(beforeExpiresAt)` 호출

SQL: `DELETE FROM MEMBER\_PASSWORD\_RESETS WHERE expires\_at < #{beforeExpiresAt}`

조건: 보존기간을 초과해 만료된 코드 행만 대상 (명등)

4. ****카운터 행 정리**** — `MemberPasswordResetRateLimitDao.purgeOldRows(now.toLocalDate())` 호출

SQL: `DELETE FROM MEMBER\_PASSWORD\_RESET\_RATE\_LIMITS WHERE day\_key < #{beforeDay}` (오늘 날짜)

조건: 어제 이전 날짜의 카운터 행 즉시 DELETE (명등, 보존기간 미적용)

5. ****로깅**** — 두 DELETE가 모두 완료된 뒤에만 삭제 건수를 slf4j info로 기록 (부수효과는 작업 후 마지막에)

데이터 흐름

방향	시스템/테이블	설명
삭제 대상	MEMBER_PASSWORD_RESETS	`expires_at` < (now - 7일) 인 만료 코드 행
삭제 대상	MEMBER_PASSWORD_RESET_RATE_LIMITS	`day_key` < 오늘 날짜의 요청 상한 카운터 행

비즈니스 규칙

만료 코드 정리 규칙

****보존 기간 의도****: 비밀번호 재설정 확정 API(INF-MBR-007)의 오류 계약 안정성 확보

만료된 코드 행이 존재하면 410(MBR-4101, "만료됨") 응답

만료된 코드 행이 없으면 409(MBR-4102, "오답") 응답

보존기간 없이 즉시 삭제하면, "어제 받은 코드로 오늘 재확정 시도" 같은 정상 사용 시나리오에서 410→409로 오류 코드가 변경되어, 클라이언트 화면(FUNC-member-007)의 "재요청" 전이 로직이 작동하지 않음

****조건부 DELETE 특성****: 재실행해도 결과가 동일 (멥등성 보장)

배치 실패 후 재실행 가능

스케줄 중복 실행 안전

카운터 행 정리 규칙

****즉시 삭제****: 지난 날짜(`day\_key` < 오늘) 행은 보존기간 없이 매일 정리

일일 요청 상한 카운터(`daily\_count`)는 day별로 격리되므로 즉시 삭제 가능

다음 작업항목(#2, INF-MBR-006)이 필요한 원자 UPSERT 메서드(`touchDailyLimit`, `selectRateLimit`) 추가 예정 — 그 시점까지 이 테이블은 대부분 빈 상태

****조건부 DELETE****: 오늘 카운터는 절대 건드리지 않음 (`day\_key` < 오늘 조건)

재실행 안전 (멥등성)

오류 처리

조건	처리 방식
DAO 호출 실패 (DB 접속 오류, 권한 부족 등)	예외 발생 → Spring 전역 예외 핸들러로 위임 → 로그 기록 후 배치 중단
0행 DELETE	정상 동작 (조건 만족 행이 없는 경우). 로그는 `purgedCodes > 0    purgedRateLimits > 0` 조건에서만 출력 (로그 스팸 방지)
부분 실패 (코드 정리 성공, 카운터 정리 실패)	코드는 이미 DELETE됨 (원자 autocommit). 예외 발생 시점 이후 작업은 미실행

재처리 방법

항목	내용
멥등성	**안전** — 두 DELETE 문 모두 조건부이므로, 같은 파라미터(같은 시각 + 보존기간)로 재실행해도 이미 지워진 행은 다시 지워지지 않음 (0행). 스케줄 중복 실행이나 수동 재시도 안전.
재실행 방법	Spring 기동 후 자동 스케줄 실행. 또는 테스트/수동 트리거: `MemberPasswordResetMaintenanceScheduler.purgeExpiredPasswordResetData()` 메서드 직접 호출(생성자에 Clock·Duration 주입으로 테스트 가능). 같은 LocalDateTime·Duration 파라미터를 사용하면 결과 동일 보장.

부분 실패 시	각 DELETE가 독립 autocommit이므로 둘이 분리. 코드 정리는 성공했지만 카운터 정리가 실패한 경우, 코드는 이미 지워짐(원복 불가). 다음 실행 주기에 카운터 정리 재시도. 데이터 일관성: 코드와 카운터는 별도 테이블이고 FK 없으므로 부분 삭제는 테이블 무결성 위반 아님.
데이터 복구	만료된 코드 행이 실수로 지워진 경우 복구 불가(이미 보존기간 경과). 테스트 DB에서는 `MemberPasswordResetDao.deleteByTarget()` 등의 정리 메서드가 별도 존재하므로 테스트 데이터 초기화 시 사용. 프로덕션 오류 시 재발급 요청이 유일한 사용자 대응 경로.

선행 Job 의존성

없음 — 이 배치는 독립 실행. 다른 배치나 API에 대한 완료 대기 조건 없음.

모니터링 / 로그

성공 로그

비밀번호 재설정 데이터 정리(배치) - 만료 후 보존기간 지난 코드 N건, 지난 날짜 일일상한 카운터 M건 삭제

로그 레벨: INFO

출력 조건: `purgedCodes > 0 || purgedRateLimits > 0` (0건이면 로그 없음)

기록 내용: 삭제 건수 숫자만 (타겟 이메일/휴대폰 원문 미포함, 개인정보 보호)

성공 판단 기준

DAO 호출이 정상 반환 (예외 없음)

삭제 건수가 0 이상 정수 (음수는 기술적으로 불가)

실패 판단 기준

데이터베이스 연결 오류

매퍼 쿼리 실행 오류 (권한, 테이블 미존재 등)

예외 발생 후 Spring 로그에 스택 트레이스 기록

관련 테스트케이스

`MemberPasswordResetMaintenanceSchedulerTest` (단위 테스트, Mockito)

호출 순서 검증: 코드 정리 → 카운터 정리

호출 횟수 검증: 각각 1회

파라미터 검증: `beforeExpiresAt`(now - 7일), `beforeDay`(오늘)

`MemberPasswordResetDaoTest` (실 DB DAO 테스트)

조건부 DELETE 시맨틱 검증 (`expires\_at < beforeExpiresAt`)

`MemberPasswordResetRateLimitDaoTest` (실 DB DAO 테스트)

조건부 DELETE 시맨틱 검증 (`day\_key < beforeDay`)

`MemberPasswordResetConfirmationFlowTest` (통합 흐름 테스트)

보존기간 내 만료 코드도 여전히 410 응답 검증

알림/임계값

모니터링 시스템 미구성 (기본)

실패 시: 앱 로그 확인 필수

큰 DELETE 건수 이상 징후: 쿨다운 설정·코드 유효시간 등 비즈니스 파라미터 변경 여부 확인

관계 정보

관계	항목	설명
테이블	SCH-MBR-005	MEMBER_PASSWORD_RESETS (만료 코드 정리 대상)
테이블	SCH-MBR-009	MEMBER_PASSWORD_RESET_RATE_LIMITS (카운터 행 정리 대상)
연결 API	INF-MBR-006	POST /api/members/password-resets/codes (코드 요청) — 이 배치가 정리하는 테이블 사용
연결 API	INF-MBR-007	POST /api/members/password-resets/confirmations (코드 확정) — 이 배치가 정리하는 테이블 사용. 보존기간 내 만료 행 존재 시 410 응답 로직 의존
SR	SR-297 #1	이 배치의 신규 구현 항목

버전 이력

일자	SR	변경 내용
2026-09-16	SR-297 #1	신규 생성 — MemberPasswordResetMaintenanceScheduler 배치 명세서

변경 이력

<!-- spec-history: machine-managed -->

일자	SR	항목	변경 요약	커밋
2026-09-16	SR-297	#1	신규 배치 명세서 생성 — 만료 코드 보존기간(P7D) 정리 + 카운터 즉시 정리	shop-api@b7ae32f